"The purpose of learning mathematics is not merely to find answers, but to understand the ideas behind them."



Numerical Analysis

And Computational Methods

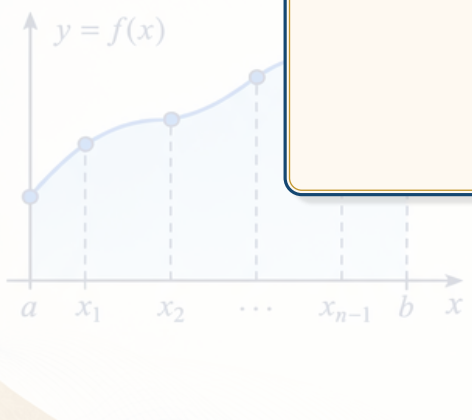
Lecture Notes in Undergraduate Mathematics

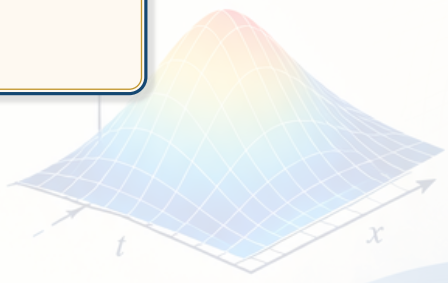
Prepared and Written by

Dr. Aijaz

Assistant Professor of Mathematics

JK Higher Education Department





Study Materials for UG Mathematics, PG Mathematics and Entrance Examinations -

NET–JRF / SET / GATE / PhD / and others

Website: https://aijaz.vercel.app

Downloaded from – **https://aijaz.vercel.app**

CHAPTER

1

Errors, Nonlinear Equations, and Interpolation

1.1 Introduction to Numerical Methods and Error Analysis

In physical sciences, engineering, and applied mathematics, many mathematical problems—such as high-degree algebraic equations, complex transcendental equations, non-separable differential equations, and multidimensional integrals—cannot be solved in closed form using standard analytical techniques. Numerical analysis provides a collection of algorithms to approximate the solutions to such problems to any desired degree of accuracy.

Because numerical procedures deal with finite representations of numbers and approximate mathematical operations, the computed result almost invariably differs from the exact mathematical solution. Understanding, quantifying, and bounding these errors is the primary objective of error analysis.

1.1.1 Basic Error Definitions and Significant Digits

Definition 1.1.1 (Absolute, Relative, and Percentage Errors)**.** Let x denote the true (exact) value of a quantity, and let $\hat{x}$ denote its approximate value.

(i) The **Absolute Error**, denoted by E_a, is the magnitude of the difference between the exact value and the approximate value:

$$E_a = |x - \hat{x}|$$

(ii) The **Relative Error**, denoted by E_r, is the ratio of the absolute error to the absolute value of the exact quantity, provided $x \neq 0$:

$$E_r = \frac{E_a}{|x|} = \frac{|x - \hat{x}|}{|x|}$$

(iii) The **Percentage Error**, denoted by E_p, expresses the relative error as a percentage:

$$E_p = E_r \times 100\% = \frac{|x - \hat{x}|}{|x|} \times 100\%$$

Remark 1.1.2. When the exact value x is unknown (which is usually the case in real-world numerical problems), the relative error is frequently approximated by replacing x with its computed estimate $\hat{x}$, i.e., $\hat{E}_r = \frac{|x-\hat{x}|}{|\hat{x}|}$, provided $\hat{x} \neq 0$.

Downloaded from – https://aijaz.vercel.app

Definition 1.1.3 (Significant Digits). A number $\hat{x}$ is said to approximate x to d **significant digits** (or significant figures) if d is the largest non-negative integer for which:

$$\frac{|x-\hat{x}|}{|x|} \leq 5 \times 10^{-d}$$

Theorem 1.1.4 (Rounding Errors and Significant Digits). *If a real number x is rounded to d digits after the decimal point, the absolute error satisfies $|x-\hat{x}| \leq \frac{1}{2}10^{-d}$. If instead $x \neq 0$ is rounded to d significant digits and $x = \pm(0.d_1d_2\cdots)10^m$, where $d_1 = k \in \{1,\ldots,9\}$, then*

$$|x-\hat{x}| \leq \frac{1}{2}10^{m-d}, \qquad E_r = \frac{|x-\hat{x}|}{|x|} \leq \frac{1}{2k}10^{-(d-1)} \leq 5 \times 10^{-d}.$$

In particular, the rounded value satisfies the preceding criterion for at least d significant digits.

Proof. Rounding to d decimal places changes the number by no more than half a unit in its d-th decimal place, giving $|x-\hat{x}| \leq \frac{1}{2}10^{-d}$. For d significant digits, the place value of the last retained digit in $x = \pm(0.d_1d_2\cdots)10^m$ is 10^{m-d}, so $|x-\hat{x}| \leq \frac{1}{2}10^{m-d}$. Since $|x| \geq k10^{m-1}$,

$$E_r \leq \frac{\frac{1}{2}10^{m-d}}{k10^{m-1}} = \frac{1}{2k}10^{-(d-1)} \leq 5 \times 10^{-d}.$$

The last inequality follows from $k \geq 1$. ■

1.1.2 Propagation of Errors

When mathematical operations are performed on approximate quantities containing inherent errors, these errors propagate through the calculations.

Theorem 1.1.5 (General Error Propagation Formula). *Let $u = f(x_1,\ldots,x_n)$, where f is continuously differentiable on an open convex set containing the line segment joining $\hat{\mathbf{x}} = (\hat{x}_1,\ldots,\hat{x}_n)$ and $\mathbf{x} = (x_1,\ldots,x_n)$. Set $\Delta x_i = x_i - \hat{x}_i$ and $\Delta u = f(\mathbf{x}) - f(\hat{\mathbf{x}})$. For small perturbations,*

$$\Delta u = \sum_{i=1}^{n} \frac{\partial f}{\partial x_i}(\hat{\mathbf{x}})\Delta x_i + o(\|\Delta\mathbf{x}\|).$$

If M_i bounds $|\partial f/\partial x_i|$ throughout that segment, a rigorous bound is

$$|\Delta u| \leq \sum_{i=1}^{n} M_i|\Delta x_i|.$$

In first-order calculations, $|\Delta u| \lesssim \sum_i |\partial f/\partial x_i(\hat{\mathbf{x}})|\,|\Delta x_i|$ is an approximation rather than an exact upper bound.

Proof. Differentiability at $\hat{\mathbf{x}}$ gives the first-order expansion with a remainder $o(\|\Delta\mathbf{x}\|)$. For the exact estimate define $g(t) = f(\hat{\mathbf{x}} + t\Delta\mathbf{x})$, $0 \leq t \leq 1$. By the fundamental theorem of calculus and the chain rule,

$$\Delta u = \int_0^1 g'(t)\,dt = \int_0^1 \sum_{i=1}^{n} \frac{\partial f}{\partial x_i}(\hat{\mathbf{x}} + t\Delta\mathbf{x})\Delta x_i\,dt.$$

Taking absolute values and applying the derivative bounds proves $|\Delta u| \leq \sum_i M_i|\Delta x_i|$. ■

Downloaded from – https://aijaz.vercel.app

Example 1.1.6. Find the absolute error, relative error, and percentage error when the exact value $x = \frac{1}{3}$ is approximated by $\hat{x} = 0.3333$. Determine the number of correct significant digits in $\hat{x}$.

Solution. The exact value is $x = \frac{1}{3} = 0.3333333\ldots$, and the approximate value is $\hat{x} = 0.3333 = \frac{3333}{10000}$.

(i) **Absolute Error:**

$$E_a = |x - \hat{x}| = \left|\frac{1}{3} - \frac{3333}{10000}\right| = \left|\frac{10000 - 9999}{30000}\right| = \frac{1}{30000} \approx 3.3333 \times 10^{-5}$$

(ii) **Relative Error:**

$$E_r = \frac{E_a}{|x|} = \frac{1/30000}{1/3} = \frac{3}{30000} = \frac{1}{10000} = 10^{-4}$$

(iii) **Percentage Error:**

$$E_p = E_r \times 100\% = 10^{-4} \times 100\% = 0.01\%$$

To find the number of significant digits d, we check the condition $E_r \leq 5 \times 10^{-d}$:

$$10^{-4} \leq 5 \times 10^{-d} \implies 10^{d-4} \leq 5$$

For $d = 4$, we have $10^0 = 1 \leq 5$, which holds true. For $d = 5$, $10^1 = 10 \not\leq 5$. Hence, $\hat{x}$ is correct to 4 significant digits. ■

Example 1.1.7. The dimensions of a rectangular solid are measured as $x = 5.0$ cm, $y = 4.0$ cm, and $z = 3.0$ cm, with maximum possible measurement errors $|\Delta x| = 0.05$ cm, $|\Delta y| = 0.02$ cm, and $|\Delta z| = 0.01$ cm. Compute the maximum absolute error and relative error in calculating the volume $V = xyz$.

Solution. The volume function is $V(x, y, z) = xyz$. The nominal volume is:

$$V = 5.0 \times 4.0 \times 3.0 = 60.0 \text{ cm}^3$$

The partial derivatives of V are:

$$\frac{\partial V}{\partial x} = yz, \quad \frac{\partial V}{\partial y} = xz, \quad \frac{\partial V}{\partial z} = xy$$

Evaluating these at the given values:

$$\frac{\partial V}{\partial x} = 4.0 \times 3.0 = 12, \quad \frac{\partial V}{\partial y} = 5.0 \times 3.0 = 15, \quad \frac{\partial V}{\partial z} = 5.0 \times 4.0 = 20$$

Using the first-order error propagation formula, the estimated maximum absolute error $|\Delta V|$ is:

$$|\Delta V| \lesssim \left|\frac{\partial V}{\partial x}\right| |\Delta x| + \left|\frac{\partial V}{\partial y}\right| |\Delta y| + \left|\frac{\partial V}{\partial z}\right| |\Delta z|$$

$$|\Delta V| \lesssim (12)(0.05) + (15)(0.02) + (20)(0.01) = 0.60 + 0.30 + 0.20 = 1.10 \text{ cm}^3$$

The relative error $E_r(V)$ is:

$$E_r(V) \approx \frac{|\Delta V|}{V} \lesssim \frac{1.10}{60.0} \approx 0.01833 \quad (1.83\%)$$

Downloaded from – https://aijaz.vercel.app

Alternatively, using logarithmic differentiation for $V = xyz$:

$$\ln V = \ln x + \ln y + \ln z \implies \frac{\Delta V}{V} \approx \frac{\Delta x}{x} + \frac{\Delta y}{y} + \frac{\Delta z}{z}$$

$$E_r(V) \lesssim \frac{0.05}{5.0} + \frac{0.02}{4.0} + \frac{0.01}{3.0} = 0.01 + 0.005 + 0.00333 = 0.01833$$

Both first-order methods yield the same estimated error bound. For a rigorous bound, derivative maxima over the full measurement intervals should be used. ■

1.1.3 Sources of Errors

Numerical solutions deviate from exact mathematical solutions due to several distinct error mechanisms. These sources are broadly classified as follows:

1. **Inherent Errors (Input Errors):** Errors present in the input data prior to numerical processing. They stem from physical measurement limitations, empirical approximations, or previous computational inaccuracies.

2. **Round-off Errors:** Errors generated because computers represent continuous real numbers using a finite number of binary bits (floating-point representation). When an arithmetic operation produces a number requiring more digits than the computer's word length allows, the low-order bits are either dropped (*chopping*) or rounded (*rounding*).

3. **Truncation Errors:** Errors created when an infinite mathematical procedure is approximated by a finite sequence of operations. Common examples include approximating an infinite series by a finite sum, or approximating a derivative by a finite difference quotient.

Theorem 1.1.8 (Taylor's Remainder Theorem for Truncation Error). *Let $f \in C^{n+1}[a,b]$ and $x_0, x \in [a,b]$. Then $f(x) = P_n(x) + R_n(x)$, where*

$$P_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k, \qquad R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_0)^{n+1}$$

for some ξ strictly between x_0 and x when $x \neq x_0$ (and $R_n(x_0) = 0$).

Proof. Fix $x \neq x_0$ and define

$$g(t) = f(x) - \sum_{k=0}^{n} \frac{f^{(k)}(t)}{k!}(x - t)^k - K(x - t)^{n+1}, \quad K = \frac{f(x) - P_n(x)}{(x - x_0)^{n+1}}.$$

Then $g(x_0) = g(x) = 0$. Differentiating the finite sum, with the $k = 0$ term treated separately, gives the telescoping identity

$$\frac{d}{dt} \sum_{k=0}^{n} \frac{f^{(k)}(t)}{k!}(x - t)^k = \frac{f^{(n+1)}(t)}{n!}(x - t)^n.$$

Indeed, for $k \geq 1$ the derivative of the k-th summand is $f^{(k+1)}(t)(x-t)^k/k! - f^{(k)}(t)(x-t)^{k-1}/(k-1)!$, and the $k = 0$ derivative is $f'(t)$. Thus $g'(t) = \big((n+1)K - f^{(n+1)}(t)/n!\big)(x-t)^n$. Rolle's theorem supplies ξ strictly between x_0 and x with $g'(\xi) = 0$. Since $\xi \neq x$, it follows that $K = f^{(n+1)}(\xi)/(n+1)!$, proving the result. ■

Downloaded from – https://aijaz.vercel.app

Example 1.1.9. Approximate $f(x) = \cos x$ around $x_0 = 0$ using a 4-th degree Taylor polynomial $P_4(x)$.

(i) Find $P_4(x)$ and determine the truncation error formula $R_4(x)$.

(ii) Estimate $\cos(0.5)$ using $P_4(0.5)$ and compute an upper bound for the absolute truncation error.

Solution. **Part (i):** Compute the derivatives of $f(x) = \cos x$ at $x_0 = 0$:

$$\begin{aligned} f(x) &= \cos x \implies f(0) = 1, & f'(x) &= -\sin x \implies f'(0) = 0 \\ f''(x) &= -\cos x \implies f''(0) = -1, & f'''(x) &= \sin x \implies f'''(0) = 0 \\ f^{(4)}(x) &= \cos x \implies f^{(4)}(0) = 1, & f^{(5)}(x) &= -\sin x \end{aligned}$$

The 4th-degree Taylor polynomial is:

$$P_4(x) = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 + \frac{f^{(4)}(0)}{4!}x^4 = 1 - \frac{x^2}{2} + \frac{x^4}{24}$$

By Taylor's theorem, the truncation error $R_4(x)$ for $n = 4$ is:

$$R_4(x) = \frac{f^{(5)}(\xi)}{5!}x^5 = \frac{-\sin\xi}{120}x^5 \quad \text{for some } \xi \text{ between } 0 \text{ and } x$$

Part (ii): Evaluating $P_4(x)$ at $x = 0.5$:

$$P_4(0.5) = 1 - \frac{(0.5)^2}{2} + \frac{(0.5)^4}{24} = 1 - \frac{0.25}{2} + \frac{0.0625}{24} = 1 - 0.125 + 0.00260417 = 0.87760417$$

To bound the truncation error $|R_4(0.5)|$, note that $|\sin\xi| \leq \sin(0.5) < 0.5$ for $\xi \in (0, 0.5)$. Using the worst-case bound $|\sin\xi| \leq 1$:

$$|R_4(0.5)| = \frac{|\sin\xi|}{120}(0.5)^5 \leq \frac{1}{120} \times 0.03125 = \frac{0.03125}{120} \approx 2.6042 \times 10^{-4}$$

Comparing with the exact value $\cos(0.5) \approx 0.87758256$:

$$E_a = |0.87758256 - 0.87760417| = 2.161 \times 10^{-5}$$

The actual error 2.161×10^{-5} is well within the derived maximum truncation bound of 2.6042×10^{-4}. ■

Example 1.1.10. Approximate the exponential function $f(x) = e^x$ about $x_0 = 0$ using a 3rd-degree Taylor polynomial $P_3(x)$.

(i) Derive $P_3(x)$ and the Lagrange form of the truncation error $R_3(x)$.

(ii) Use $P_3(x)$ to approximate $e^{0.2}$ and establish an upper bound for the absolute truncation error. Compare this bound with the actual error.

Solution. **Part (i):** The function is $f(x) = e^x$. Since $\frac{d^k}{dx^k}(e^x) = e^x$ for all $k \geq 0$, evaluating the derivatives at $x_0 = 0$ yields:

$$f(0) = 1, \quad f'(0) = 1, \quad f''(0) = 1, \quad f'''(0) = 1, \quad f^{(4)}(x) = e^x$$

Downloaded from – https://aijaz.vercel.app

The 3rd-degree Taylor polynomial $P_3(x)$ is:

$$P_3(x) = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 = 1 + x + \frac{x^2}{2} + \frac{x^3}{6}$$

By Taylor's theorem, the truncation error $R_3(x)$ for $n = 3$ is given by:

$$R_3(x) = \frac{f^{(4)}(\xi)}{4!}x^4 = \frac{e^\xi}{24}x^4 \quad \text{for some } \xi \text{ strictly between 0 and } x$$

Part (ii): Evaluating $P_3(x)$ at $x = 0.2$:

$$P_3(0.2) = 1 + 0.2 + \frac{(0.2)^2}{2} + \frac{(0.2)^3}{6} = 1 + 0.2 + \frac{0.04}{2} + \frac{0.008}{6}$$

$$P_3(0.2) = 1 + 0.2 + 0.02 + 0.00133333 = 1.22133333$$

To bound the truncation error $|R_3(0.2)|$, note that $\xi \in (0, 0.2)$. Since e^x is a strictly increasing function:

$$e^0 < e^\xi < e^{0.2} < 3$$

Using the conservative upper bound $e^\xi < 3$:

$$|R_3(0.2)| = \frac{e^\xi}{24}(0.2)^4 < \frac{3}{24}(0.0016) = \frac{0.0016}{8} = 2.0 \times 10^{-4}$$

(A tighter bound using $e^{0.2} < 1.3$ yields $|R_3(0.2)| \leq \frac{1.3 \times 0.0016}{24} \approx 8.6667 \times 10^{-5}$).

Comparing with the true value $e^{0.2} \approx 1.22140276$:

$$E_a = |1.22140276 - 1.22133333| = 6.943 \times 10^{-5}$$

The actual truncation error 6.943×10^{-5} is well within the calculated upper bound of 2.0×10^{-4}. ■

Example 1.1.11. Consider the natural logarithm function $f(x) = \ln(1 + x)$ defined for $x > -1$.

(i) Expand $f(x)$ in a 3rd-degree Taylor polynomial $P_3(x)$ about $x_0 = 0$, and state the expression for the truncation error $R_3(x)$.

(ii) Approximate $\ln(1.1)$ using $P_3(0.1)$, determine a theoretical maximum bound on the truncation error, and verify it against the exact error.

Solution. **Part (i):** Compute the first four derivatives of $f(x) = \ln(1 + x)$ and evaluate them at $x_0 = 0$:

$$\begin{aligned} f(x) &= \ln(1+x) &&\implies f(0) = 0, & f'(x) &= (1+x)^{-1} &&\implies f'(0) = 1 \\ f''(x) &= -(1+x)^{-2} &&\implies f''(0) = -1, & f'''(x) &= 2(1+x)^{-3} &&\implies f'''(0) = 2 \\ f^{(4)}(x) &= -6(1+x)^{-4} \end{aligned}$$

The 3rd-degree Taylor polynomial $P_3(x)$ is:

$$P_3(x) = 0 + (1)x + \frac{-1}{2!}x^2 + \frac{2}{3!}x^3 = x - \frac{x^2}{2} + \frac{x^3}{3}$$

The truncation error $R_3(x)$ in Lagrange remainder form is:

$$R_3(x) = \frac{f^{(4)}(\xi)}{4!}x^4 = \frac{-6(1+\xi)^{-4}}{24}x^4 = -\frac{x^4}{4(1+\xi)^4} \quad \text{for some } \xi \text{ between 0 and } x$$

Downloaded from – https://aijaz.vercel.app

Part (ii): Setting $x = 0.1$ to evaluate $\ln(1+0.1) = \ln(1.1)$:

$$P_3(0.1) = (0.1) - \frac{(0.1)^2}{2} + \frac{(0.1)^3}{3} = 0.1 - 0.005 + 0.00033333 = 0.09533333$$

To obtain an upper bound for $|R_3(0.1)|$, observe that for $\xi \in (0, 0.1)$:

$$1 + \xi > 1 \implies (1+\xi)^4 > 1 \implies \frac{1}{(1+\xi)^4} < 1$$

Therefore, the truncation error bound is:

$$|R_3(0.1)| = \frac{(0.1)^4}{4(1+\xi)^4} < \frac{0.0001}{4(1)^4} = 2.5 \times 10^{-5}$$

Comparing with the true value $\ln(1.1) \approx 0.09531018$:

$$E_a = |0.09531018 - 0.09533333| = 2.315 \times 10^{-5}$$

The actual error 2.315×10^{-5} strictly satisfies the theoretical upper bound $E_a \leq 2.5 \times 10^{-5}$. ■

1.1.4 Concept of Approximation, Convergence, and Rate of Convergence

In numerical analysis, an algorithm generates a sequence of successive approximations $\{x_k\}_{k=0}^{\infty}$ that idealizes an exact solution α. For an algorithm to be useful, this sequence must converge to α, and do so efficiently.

Definition 1.1.12 (Convergence of a Numerical Sequence)**.** Let $\{x_k\}_{k=0}^{\infty}$ be a sequence of real numbers generated by an iterative numerical algorithm, and let $\alpha \in \mathbb{R}$ represent the exact target solution. The sequence $\{x_k\}$ is said to **converge** to α if:

$$\lim_{k\to\infty} |x_k - \alpha| = 0$$

Defining the error at the k-th iteration as $e_k = x_k - \alpha$, convergence is equivalent to stating that the sequence of error magnitudes satisfies:

$$\lim_{k\to\infty} |e_k| = 0$$

For any pre-assigned error tolerance $\epsilon > 0$, there exists a positive integer $K(\epsilon)$ such that $|x_k - \alpha| < \epsilon$ for all $k \geq K(\epsilon)$.

Example 1.1.13. Consider the sequence $\{x_k\}_{k=0}^{\infty}$ defined by the recurrence relation:

$$x_0 = 2, \quad x_{k+1} = 1 + \frac{1}{2}(x_k - 1) \quad \text{for } k \geq 0$$

Show that the sequence converges to the limit $\alpha = 1$, and find the iteration index K such that the error $|e_k| < 10^{-4}$.

Solution. Subtracting the target limit $\alpha = 1$ from both sides of the recurrence relation:

$$x_{k+1} - 1 = \left(1 + \frac{1}{2}(x_k - 1)\right) - 1 = \frac{1}{2}(x_k - 1)$$

Downloaded from – https://aijaz.vercel.app

In terms of the error $e_k = x_k - 1$, we have $e_{k+1} = \frac{1}{2}e_k$. Unrolling this linear recurrence from $e_0 = x_0 - 1 = 2 - 1 = 1$:

$$e_k = \left(\frac{1}{2}\right)^k e_0 = \left(\frac{1}{2}\right)^k$$

Taking the limit as $k \to \infty$:

$$\lim_{k\to\infty} |e_k| = \lim_{k\to\infty} \left(\frac{1}{2}\right)^k = 0$$

Hence, the sequence $\{x_k\}$ converges to $\alpha = 1$.

To find the index K such that $|e_k| < 10^{-4}$:

$$\left(\frac{1}{2}\right)^k < 10^{-4} \implies 2^k > 10^4 = 10000$$

Taking logarithms with base 10:

$$k \ln 2 > 4 \ln 10 \implies k > \frac{4 \ln 10}{\ln 2} \approx \frac{4 \times 2.30258}{0.69315} \approx 13.287$$

Thus, for $k \geq 14$, the error $|e_k|$ is guaranteed to be strictly less than 10^{-4}. ■

Example 1.1.14. An iterative root-finding process generates a sequence of approximations $\{x_k\}_{k=0}^{\infty}$ defined by the recurrence relation:

$$x_0 = 1, \quad x_{k+1} = \frac{1}{4}x_k + 3 \quad \text{for } k \geq 0$$

(i) Find the exact limiting value α of the sequence.

(ii) Derive an explicit formula for the k-th error term $e_k = x_k - \alpha$ and prove that the sequence converges to α.

(iii) Determine the minimum number of iterations K required to ensure that the absolute error $|e_k| < 10^{-5}$.

Solution. **Part (i):** If the sequence converges to a limit α, taking the limit as $k \to \infty$ on both sides of the recurrence relation gives:

$$\alpha = \frac{1}{4}\alpha + 3 \implies \frac{3}{4}\alpha = 3 \implies \alpha = 4$$

Part (ii): Define the error at the k-th iteration as $e_k = x_k - \alpha = x_k - 4$. Subtracting $\alpha = 4$ from both sides of $x_{k+1} = \frac{1}{4}x_k + 3$:

$$e_{k+1} = x_{k+1} - 4 = \left(\frac{1}{4}x_k + 3\right) - 4 = \frac{1}{4}(x_k - 4) = \frac{1}{4}e_k$$

This forms a geometric sequence of errors with initial error $e_0 = x_0 - 4 = 1 - 4 = -3$. Thus, the explicit formula for e_k is:

$$e_k = e_0 \left(\frac{1}{4}\right)^k = -3\left(\frac{1}{4}\right)^k$$

Taking the limit of the absolute error as $k \to \infty$:

$$\lim_{k\to\infty} |e_k| = \lim_{k\to\infty} \left|-3\left(\frac{1}{4}\right)^k\right| = 3 \lim_{k\to\infty} \left(\frac{1}{4}\right)^k = 0$$

Since $\lim_{k\to\infty} |e_k| = 0$, the sequence $\{x_k\}$ converges to $\alpha = 4$.

Part (iii): To guarantee that $|e_k| < 10^{-5}$, we set up the inequality:

$$3\left(\frac{1}{4}\right)^k < 10^{-5} \implies 4^k > 3 \times 10^5 = 300000$$

Taking natural logarithms on both sides:

$$k \ln 4 > \ln(300000) \implies k > \frac{\ln(300000)}{\ln 4} \approx \frac{12.61154}{1.38629} \approx 9.097$$

Since k must be an integer, at least $K = 10$ iterations are required to satisfy the tolerance $|e_k| < 10^{-5}$. ■

Example 1.1.15. Suppose the Bisection method is applied to locate a root α of a continuous function $f(x)$ on the initial interval $[a_0, b_0] = [1, 3]$. Let x_k denote the midpoint of the interval at the k-th step.

(i) Derive the absolute error bound for the sequence of midpoints $\{x_k\}_{k=0}^{\infty}$ and show that $\{x_k\}$ converges to α.

(ii) Compute the minimum number of bisection steps K needed to guarantee an absolute error $|x_k - \alpha| \leq 10^{-6}$.

Solution. **Part (i):** At the initial step $k = 0$, the length of the interval $[a_0, b_0]$ is $L_0 = b_0 - a_0 = 3 - 1 = 2$. Since the root α lies within $[a_0, b_0]$ and the first approximation x_0 is the midpoint, the maximum distance between x_0 and α is half the interval length:

$$|e_0| = |x_0 - \alpha| \leq \frac{b_0 - a_0}{2} = 1$$

At each subsequent step k, the interval length is halved, so $L_k = \frac{b_0 - a_0}{2^k}$. Thus, the error bound for x_k is:

$$|e_k| = |x_k - \alpha| \leq \frac{b_0 - a_0}{2^{k+1}} = \frac{2}{2^{k+1}} = \frac{1}{2^k}$$

Taking the limit as $k \to \infty$:

$$0 \leq \lim_{k\to\infty} |x_k - \alpha| \leq \lim_{k\to\infty} \frac{1}{2^k} = 0$$

By the Squeeze Theorem, $\lim_{k\to\infty} |x_k - \alpha| = 0$, which proves that the sequence of midpoints $\{x_k\}$ converges to the exact root α.

Part (ii): To ensure an absolute error of at most 10^{-6}, we require:

$$\frac{1}{2^k} \leq 10^{-6} \implies 2^k \geq 10^6 = 1000000$$

Taking logarithms base 10:

$$k \log_{10}(2) \geq 6 \implies k \geq \frac{6}{\log_{10}(2)} \approx \frac{6}{0.30103} \approx 19.93$$

Rounding up to the nearest integer, a minimum of $K = 20$ bisection iterations guarantees that the error $|x_k - \alpha| \leq 10^{-6}$. ■

Downloaded from – https://aijaz.vercel.app

Definition 1.1.16 (Order and Rate of Convergence)**.** Suppose the sequence $\{x_k\}_{k=0}^{\infty}$ converges to the exact limit α. Let $e_k = x_k - \alpha$ denote the error at the k-th iteration. The sequence is said to converge with **order of convergence** $p \geq 1$ if there exists a positive finite constant $C > 0$ (called the **asymptotic error constant**) such that:

$$\lim_{k\to\infty} \frac{|x_{k+1} - \alpha|}{|x_k - \alpha|^p} = \lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^p} = C$$

The value of p classifies the speed of convergence as follows:

(i) **Linear Convergence:** $p = 1$, provided the asymptotic error constant satisfies $C < 1$.

(ii) **Superlinear Convergence:** $1 < p < 2$ (e.g., the Secant method has $p = \frac{1+\sqrt{5}}{2} \approx 1.618$).

(iii) **Quadratic Convergence:** $p = 2$ (e.g., Newton–Raphson method for simple roots).

(iv) **Cubic Convergence:** $p = 3$ (e.g., Halley's method).

Remark 1.1.17. Under linear convergence ($p = 1$), the error decreases by a factor of approximately C per iteration. Under quadratic convergence ($p = 2$), the number of correct decimal places approximately doubles with each iteration once e_k becomes small.

Example 1.1.18. Analyze the convergence rate of the following two iterative sequences converging to $\alpha = 0$:

(i) $x_{k+1} = \frac{1}{3}x_k$ with $x_0 = 1$.

(ii) $y_{k+1} = 5y_k^2$ with $y_0 = 0.1$.

Solution. **Case (i):** Here $\alpha = 0$, so $e_k = x_k - 0 = x_k$. The relation is $e_{k+1} = \frac{1}{3}e_k$. Setting $p = 1$:

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^1} = \lim_{k\to\infty} \frac{\frac{1}{3}|e_k|}{|e_k|} = \frac{1}{3} = C$$

Since $p = 1$ and $C = \frac{1}{3} < 1$, sequence $\{x_k\}$ exhibits **linear convergence**.

Case (ii): Again $\alpha = 0$, so error $\tilde{e}_k = y_k$. The error relation is $\tilde{e}_{k+1} = 5\tilde{e}_k^2$. Setting $p = 2$:

$$\lim_{k\to\infty} \frac{|\tilde{e}_{k+1}|}{|\tilde{e}_k|^2} = \lim_{k\to\infty} \frac{5|\tilde{e}_k|^2}{|\tilde{e}_k|^2} = 5 = C$$

Since $p = 2$ and $C = 5 > 0$, sequence $\{y_k\}$ exhibits **quadratic convergence**. ■

Example 1.1.19. The Newton–Raphson method for finding the square root of a positive real number $a > 0$ generates a sequence $\{x_k\}_{k=0}^{\infty}$ via the iteration:

$$x_{k+1} = \frac{1}{2}\left(x_k + \frac{a}{x_k}\right)$$

(i) Express the error $e_{k+1} = x_{k+1} - \sqrt{a}$ in terms of $e_k = x_k - \sqrt{a}$.

(ii) Determine the order of convergence p and the asymptotic error constant C.

Downloaded from – https://aijaz.vercel.app

Solution. **Part (i):** The exact target solution is $\alpha = \sqrt{a}$. Subtracting $\sqrt{a}$ from both sides of the iteration formula:

$$e_{k+1} = x_{k+1} - \sqrt{a} = \frac{1}{2}\left(x_k + \frac{a}{x_k}\right) - \sqrt{a} = \frac{x_k^2 + a - 2\sqrt{a}x_k}{2x_k}$$

Noting that $x_k^2 - 2\sqrt{a}x_k + a = (x_k - \sqrt{a})^2 = e_k^2$, we obtain:

$$e_{k+1} = \frac{e_k^2}{2x_k}$$

Part (ii): Dividing both sides by e_k^2 and taking the limit as $k \to \infty$:

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^2} = \lim_{k\to\infty} \frac{1}{2x_k}$$

Since $x_k \to \sqrt{a}$ as $k \to \infty$, we have:

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^2} = \frac{1}{2\sqrt{a}}$$

Comparing with the definition $\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^p} = C$, we conclude that:

- The order of convergence is **quadratic** ($p = 2$).
- The asymptotic error constant is $C = \frac{1}{2\sqrt{a}}$.

■

Example 1.1.20. The Secant method for finding a simple root α of a smooth function $f(x) = 0$ satisfies the error relation:

$$e_{k+1} = A\, e_k\, e_{k-1}$$

where $A = \frac{f''(\alpha)}{2f'(\alpha)}$ is a non-zero constant. Derive the order of convergence p for the Secant method.

Solution. Assume an asymptotic error relationship of the form $|e_{k+1}| \approx C|e_k|^p$, where $p \geq 1$ and $C > 0$. This implies that for the previous step:

$$|e_k| \approx C|e_{k-1}|^p \implies |e_{k-1}| \approx C^{-1/p}|e_k|^{1/p}$$

Substituting the expressions for $|e_{k+1}|$ and $|e_{k-1}|$ into the relation $|e_{k+1}| = |A||e_k||e_{k-1}|$:

$$C|e_k|^p = |A||e_k|\left(C^{-1/p}|e_k|^{1/p}\right) = |A|C^{-1/p}|e_k|^{1+\frac{1}{p}}$$

For this asymptotic equation to hold for all sufficiently small values of $|e_k|$, the exponents of $|e_k|$ on both sides must be equal:

$$p = 1 + \frac{1}{p} \implies p^2 - p - 1 = 0$$

Solving this quadratic equation using the quadratic formula:

$$p = \frac{1 \pm \sqrt{1 - 4(1)(-1)}}{2} = \frac{1 \pm \sqrt{5}}{2}$$

Since the order of convergence must be positive ($p > 0$), we take the positive root:

$$p = \frac{1 + \sqrt{5}}{2} \approx 1.618$$

Thus, the Secant method exhibits **superlinear convergence** with an order $p = \frac{1+\sqrt{5}}{2} \approx 1.618$ (the Golden Ratio). ■

Downloaded from – https://aijaz.vercel.app

Example 1.1.21. Suppose Newton's method $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$ is applied to find a root α of multiplicity $m \geq 2$, so that $f(x) = (x-\alpha)^m g(x)$ with $g(\alpha) \neq 0$. Prove that the order of convergence reduces from quadratic ($p = 2$) to linear ($p = 1$), and find the asymptotic error constant C.

Solution. Differentiating $f(x) = (x-\alpha)^m g(x)$ using the product rule:

$$f'(x) = m(x-\alpha)^{m-1} g(x) + (x-\alpha)^m g'(x) = (x-\alpha)^{m-1} \left[m g(x) + (x-\alpha) g'(x) \right]$$

Substituting $f(x_k)$ and $f'(x_k)$ into the Newton–Raphson formula:

$$x_{k+1} = x_k - \frac{(x_k-\alpha)^m g(x_k)}{(x_k-\alpha)^{m-1} \left[m g(x_k) + (x_k-\alpha) g'(x_k) \right]} = x_k - (x_k-\alpha) \frac{g(x_k)}{m g(x_k) + (x_k-\alpha) g'(x_k)}$$

Subtracting α from both sides yields the error $e_{k+1} = x_{k+1} - \alpha$:

$$e_{k+1} = e_k - e_k \frac{g(x_k)}{m g(x_k) + e_k g'(x_k)} = e_k \left[1 - \frac{g(x_k)}{m g(x_k) + e_k g'(x_k)} \right]$$

Dividing both sides by e_k:

$$\frac{e_{k+1}}{e_k} = 1 - \frac{g(x_k)}{m g(x_k) + e_k g'(x_k)}$$

Taking the limit as $k \to \infty$ (noting that $x_k \to \alpha$ and $e_k \to 0$):

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|} = \left| 1 - \frac{g(\alpha)}{m g(\alpha) + 0} \right| = \left| 1 - \frac{1}{m} \right| = \frac{m-1}{m}$$

Since $m \geq 2$, we have $0 < \frac{m-1}{m} < 1$. Comparing with $\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^p} = C$:

- The convergence is **linear** ($p = 1$).
- The asymptotic error constant is $C = \frac{m-1}{m}$.

Remark 1.1.22. For a double root ($m = 2$), $C = \frac{2-1}{2} = 0.5$. Thus, Newton's method loses its characteristic quadratic convergence when applied to multiple roots unless modified.

■

Definition 1.1.23 (Big-$\mathcal{O}$ Notation for Continuous Approximations)**.** Let $h > 0$ represent a continuous discretization step size (such as interval width or grid spacing). A numerical function $F(h)$ approximates a limiting physical or mathematical value L with an **order of accuracy** $\mathcal{O}(h^p)$ if there exist positive constants $C > 0$ and $h_0 > 0$ such that:

$$|F(h) - L| \leq C h^p \quad \text{for all } 0 < h \leq h_0$$

In asymptotic notation, we write:

$$F(h) = L + \mathcal{O}(h^p) \quad \text{as } h \to 0$$

Here, $p > 0$ specifies the power of h governing the asymptotic decay of the discretization error.

Downloaded from – https://aijaz.vercel.app

Example 1.1.24. Analyze the order of accuracy for the forward difference formula:

$$D_h f(x) = \frac{f(x+h) - f(x)}{h}$$

used to approximate the first derivative $f'(x)$ of a sufficiently smooth function $f(x)$.

Solution. By Taylor's Theorem with Lagrange remainder, $f(x+h)$ expanded around x is:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(\xi) \quad \text{for some } \xi \in (x, x+h)$$

Substituting this expansion into the forward difference operator:

$$D_h f(x) = \frac{\left(f(x) + hf'(x) + \frac{h^2}{2}f''(\xi)\right) - f(x)}{h} = \frac{hf'(x) + \frac{h^2}{2}f''(\xi)}{h} = f'(x) + \frac{h}{2}f''(\xi)$$

Subtracting the exact value $f'(x)$:

$$|D_h f(x) - f'(x)| = \left|\frac{h}{2}f''(\xi)\right|$$

Assuming $|f''(t)| \leq M_2$ for all t in a neighborhood of x:

$$|D_h f(x) - f'(x)| \leq \left(\frac{M_2}{2}\right) h^1$$

Choosing $C = \frac{M_2}{2}$, we have $|D_h f(x) - f'(x)| \leq Ch^1$, which proves:

$$D_h f(x) = f'(x) + \mathcal{O}(h)$$

Thus, the forward difference approximation is **first-order accurate** ($\mathcal{O}(h)$). Reducing the step size h by half reduces the truncation error by approximately a factor of 2. ■

Theorem 1.1.25 (Sufficient Condition for Linear Convergence of Fixed-Point Iteration)**.** *Let $g \in C^1[a,b]$ and suppose $g(x) \in [a,b]$ for all $x \in [a,b]$. If the fixed-point iteration $x_{k+1} = g(x_k)$ converges to a fixed point $\alpha = g(\alpha)$, and if $g'(\alpha) \neq 0$ with $|g'(\alpha)| < 1$, then the iteration converges linearly with asymptotic error constant $C = |g'(\alpha)|$.*

Proof. Subtracting $\alpha = g(\alpha)$ from $x_{k+1} = g(x_k)$ yields the error relation:

$$e_{k+1} = x_{k+1} - \alpha = g(x_k) - g(\alpha)$$

Since g is continuously differentiable, applying the Mean Value Theorem on the interval between x_k and α guarantees the existence of a point ξ_k between x_k and α such that:

$$g(x_k) - g(\alpha) = g'(\xi_k)(x_k - \alpha) \implies e_{k+1} = g'(\xi_k)e_k$$

Taking absolute values and dividing by $|e_k|$ (for $e_k \neq 0$):

$$\frac{|e_{k+1}|}{|e_k|} = |g'(\xi_k)|$$

Since $x_k \to \alpha$ as $k \to \infty$, ξ_k is squeezed between x_k and α, implying $\lim_{k\to\infty} \xi_k = \alpha$. Taking the limit as $k \to \infty$ on both sides:

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|} = \lim_{k\to\infty} |g'(\xi_k)| = |g'(\alpha)| = C$$

Since $p = 1$ and $0 < C < 1$, the sequence converges linearly with asymptotic constant $|g'(\alpha)|$. ■

Downloaded from – https://aijaz.vercel.app

Example 1.1.26. Consider two numerical methods producing sequences $\{x_k\}$ and $\{y_k\}$ that converge to $\alpha = 0$:

$$x_k = \frac{1}{2^k}, \quad y_k = \frac{1}{2^{2^k}} \quad (k \geq 0)$$

Determine the order of convergence p and the asymptotic error constant C for both sequences.

Solution. **For sequence $\{x_k\}$:** The exact limit is $\alpha = 0$, so error $e_k = x_k - 0 = \frac{1}{2^k}$. Compute the ratio of successive errors for $p = 1$:

$$\frac{|e_{k+1}|}{|e_k|^1} = \frac{1/2^{k+1}}{1/2^k} = \frac{2^k}{2^{k+1}} = \frac{1}{2}$$

Taking the limit as $k \to \infty$:

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|} = \frac{1}{2}$$

Thus, $\{x_k\}$ converges **linearly** ($p = 1$) with asymptotic error constant $C = \frac{1}{2}$.

For sequence $\{y_k\}$: The error is $e_k = y_k - 0 = \frac{1}{2^{2^k}} = 2^{-2^k}$. Check for quadratic convergence ($p = 2$):

$$|e_k|^2 = \left(2^{-2^k}\right)^2 = 2^{-2\cdot 2^k} = 2^{-2^{k+1}}$$

Notice that $e_{k+1} = 2^{-2^{k+1}}$. Taking the ratio:

$$\frac{|e_{k+1}|}{|e_k|^2} = \frac{2^{-2^{k+1}}}{2^{-2^{k+1}}} = 1$$

Taking the limit as $k \to \infty$:

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^2} = 1$$

Thus, $\{y_k\}$ converges **quadratically** ($p = 2$) with asymptotic error constant $C = 1$. ■

Example 1.1.27. Consider the centered finite difference formula for approximating the first derivative of a smooth function $f(x)$:

$$f'(x) \approx D_h f(x) = \frac{f(x+h) - f(x-h)}{2h}$$

Derive the expression for the truncation error and establish its order of accuracy in Big-$\mathcal{O}$ notation.

Solution. Expand $f(x+h)$ and $f(x-h)$ using Taylor series about x:

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + \frac{h^4}{24}f^{(4)}(\xi_1)$$

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(x) + \frac{h^4}{24}f^{(4)}(\xi_2)$$

Subtracting the second expansion from the first:

$$f(x+h) - f(x-h) = 2hf'(x) + \frac{h^3}{3}f'''(x) + \frac{h^4}{24}\left[f^{(4)}(\xi_1) - f^{(4)}(\xi_2)\right]$$

Dividing both sides by $2h$:

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6}f'''(x) + \mathcal{O}(h^3)$$

Downloaded from – https://aijaz.vercel.app

Rearranging terms yields the error formula:

$$D_h f(x) - f'(x) = \frac{h^2}{6} f'''(x) + \mathcal{O}(h^3)$$

Taking the absolute error bound, assuming $|f'''(x)| \leq M_3$ on the local domain:

$$|D_h f(x) - f'(x)| \leq \frac{M_3}{6} h^2 + \mathcal{O}(h^3)$$

Therefore, the centered difference approximation is **second-order accurate**, written as:

$$D_h f(x) = f'(x) + \mathcal{O}(h^2)$$

Halving the step size $h \to h/2$ reduces the truncation error by a factor of approximately 4. ■

1.2 Numerical Solution of Nonlinear Equations

1.2.1 Bisection Method

The **Bisection Method** (also known as the *Binary Chopping Method* or *Interval Halving Method*) is one of the simplest, most robust, and reliable numerical procedures for finding a real root of a continuous non-linear equation $f(x) = 0$. It is classified as a *bracketing method* because it starts with an interval known to contain at least one root and systematically shrinks the interval at each step.

The method is rooted directly in the **Intermediate Value Theorem (IVT)**.

Theorem 1.2.1 (Intermediate Value Theorem). *If $f(x)$ is a real-valued function continuous on the closed interval $[a, b]$, and if $f(a)$ and $f(b)$ have opposite signs (i.e., $f(a) \cdot f(b) < 0$), then there exists at least one real number $\alpha \in (a, b)$ such that $f(\alpha) = 0$.*

Given an initial bracket $[a_0, b_0]$ where $f(a_0)f(b_0) < 0$, the Bisection Method divides the interval into two equal subintervals by computing the midpoint:

$$c_0 = \frac{a_0 + b_0}{2}$$

Evaluating $f(c_0)$ gives three possible outcomes:

(i) If $f(c_0) = 0$, then c_0 is the exact root α, and the algorithm terminates.

(ii) If $f(a_0) \cdot f(c_0) < 0$, the root lies in the left subinterval $[a_0, c_0]$. The interval for the next iteration is set to $[a_1, b_1] = [a_0, c_0]$.

(iii) If $f(c_0) \cdot f(b_0) < 0$, the root lies in the right subinterval $[c_0, b_0]$. The interval for the next iteration is set to $[a_1, b_1] = [c_0, b_0]$.

This process is repeated iteratively to generate a sequence of nested intervals $[a_0, b_0] \supset [a_1, b_1] \supset [a_2, b_2] \ldots$ whose lengths approach zero.

Algorithmic Procedure

Step 1: Choose an initial interval $[a_0, b_0]$ such that $f(a_0) \cdot f(b_0) < 0$. Specify the desired error tolerance $\epsilon > 0$ or maximum iterations $N_{\max}$.

Step 2: For $k = 0, 1, 2, \ldots, N_{\max}$:

(a) Compute the midpoint $c_k = \frac{a_k + b_k}{2}$.

Downloaded from – https://aijaz.vercel.app

(b) Evaluate $f(c_k)$.

(c) Check stopping criterion: If $|f(c_k)| < \epsilon$ or $\frac{b_k - a_k}{2} < \epsilon$, terminate and output c_k as the approximate root.

(d) Update the bracket:

$$\text{If } f(a_k) \cdot f(c_k) < 0, \quad \text{set } a_{k+1} = a_k,\ b_{k+1} = c_k$$

$$\text{If } f(a_k) \cdot f(c_k) > 0, \quad \text{set } a_{k+1} = c_k,\ b_{k+1} = b_k$$

Convergence Analysis

Theorem 1.2.2 (Bisection Method Convergence and Error Bound). *Let $f \in C[a_0, b_0]$ with $f(a_0)f(b_0) < 0$. The midpoints c_k produced by bisection converge to a root $\alpha \in (a_0, b_0)$ and satisfy*

$$|c_k - \alpha| \leq \frac{b_0 - a_0}{2^{k+1}}, \qquad k \geq 0.$$

Thus the method has a geometrically decreasing error bound (R-linear convergence with rate at most $1/2$). The actual errors need not have an asymptotic ratio $|c_{k+1} - \alpha|/|c_k - \alpha|$ converging to $1/2$.

Proof. Each step preserves a bracket containing a root and halves the interval length, so $b_k - a_k = (b_0 - a_0)/2^k$. Because $c_k = (a_k + b_k)/2$ and $\alpha \in [a_k, b_k]$,

$$0 \leq |c_k - \alpha| \leq \frac{b_k - a_k}{2} = \frac{b_0 - a_0}{2^{k+1}} \longrightarrow 0.$$

This proves convergence and the asserted geometric majorant. The ratio $E_{k+1}/E_k = 1/2$ holds for the bounds $E_k = (b_0 - a_0)/2^{k+1}$, not necessarily for the actual errors. ■

Corollary 1.2.3 (Iteration Bound Formula). *To guarantee that the absolute error $|c_k - \alpha|$ is less than or equal to a specified tolerance ϵ, the minimum number of bisection iterations N required is:*

$$N \geq \frac{\ln(b_0 - a_0) - \ln \epsilon}{\ln 2} - 1 = \frac{\log_{10}(b_0 - a_0) - \log_{10} \epsilon}{\log_{10} 2} - 1$$

Remark 1.2.4 (Properties of Bisection Method).

- **Advantages:** Always globally convergent provided $f(x)$ is continuous and changes sign on $[a_0, b_0]$; error bound is known *a priori*.
- **Disadvantages:** Slow rate of convergence compared to open methods (e.g., Newton–Raphson); cannot find roots of even multiplicity where $f(x)$ touches the x-axis without crossing.

Example 1.2.5. Find a real root of the polynomial equation $f(x) = x^3 - x - 1 = 0$ in the interval $[1, 2]$ using 4 iterations of the Bisection Method.

Solution. First, verify continuity and sign change on $[a_0, b_0] = [1, 2]$:

$$f(1) = (1)^3 - 1 - 1 = -1 < 0, \quad f(2) = (2)^3 - 2 - 1 = 5 > 0$$

Since $f(1) \cdot f(2) < 0$, a root exists in $[1, 2]$.

- **Iteration 1 ($k = 0$):**

$$c_0 = \frac{1+2}{2} = 1.5, \quad f(1.5) = (1.5)^3 - 1.5 - 1 = 3.375 - 2.5 = +0.875 > 0$$

Since $f(1) \cdot f(1.5) < 0$, the root lies in $[1, 1.5]$. Set $a_1 = 1.0, b_1 = 1.5$.

- **Iteration 2 ($k = 1$):**

$$c_1 = \frac{1.0+1.5}{2} = 1.25, \quad f(1.25) = (1.25)^3 - 1.25 - 1 = 1.953125 - 2.25 = -0.296875 < 0$$

Since $f(1.25) \cdot f(1.5) < 0$, the root lies in $[1.25, 1.5]$. Set $a_2 = 1.25, b_2 = 1.5$.

- **Iteration 3 ($k = 2$):**

$$c_2 = \frac{1.25+1.5}{2} = 1.375, \quad f(1.375) = (1.375)^3 - 1.375 - 1 = 2.599609 - 2.375 = +0.224609 > 0$$

Since $f(1.25) \cdot f(1.375) < 0$, the root lies in $[1.25, 1.375]$. Set $a_3 = 1.25, b_3 = 1.375$.

- **Iteration 4 ($k = 3$):**

$$c_3 = \frac{1.25+1.375}{2} = 1.3125, \quad f(1.3125) = (1.3125)^3 - 1.3125 - 1 = -0.051270 < 0$$

Since $f(1.3125) \cdot f(1.375) < 0$, the root lies in $[1.3125, 1.375]$.

The computation is summarized in the following table:

k	a_k	b_k	$c_k = \frac{a_k+b_k}{2}$	$f(c_k)$	New Interval
0	1.000000	2.000000	1.500000	+0.875000	[1.000000, 1.500000]
1	1.000000	1.500000	1.250000	−0.296875	[1.250000, 1.500000]
2	1.250000	1.500000	1.375000	+0.224609	[1.250000, 1.375000]
3	1.250000	1.375000	1.312500	−0.051270	[1.312500, 1.375000]

After 4 iterations, the approximate root is $\alpha \approx 1.3125$ with maximum error bound $\frac{2-1}{2^4} = 0.0625$. ■

Example 1.2.6. Find a real root of the transcendental equation $f(x) = xe^x - 1 = 0$ in the interval $[0, 1]$ correct to 3 decimal places using the Bisection Method.

Solution. Evaluate $f(x)$ at the initial interval bounds $a_0 = 0, b_0 = 1$:

$$f(0) = 0 \cdot e^0 - 1 = -1 < 0, \quad f(1) = 1 \cdot e^1 - 1 = e - 1 \approx 1.718282 > 0$$

Since $f(0) \cdot f(1) < 0$, a root exists in $[0, 1]$. We proceed with bisection iterations until $|b_k - a_k| < 0.001$:

- **Iteration 1:** $c_0 = \frac{0+1}{2} = 0.5$.

$$f(0.5) = 0.5e^{0.5} - 1 = 0.5(1.648721) - 1 = -0.175639 < 0$$

Root in $[0.5, 1.0]$. Set $a_1 = 0.5, b_1 = 1.0$.

- **Iteration 2:** $c_1 = \frac{0.5+1.0}{2} = 0.75$.

$$f(0.75) = 0.75e^{0.75} - 1 = 0.75(2.117000) - 1 = +0.587750 > 0$$

Root in $[0.5, 0.75]$. Set $a_2 = 0.5, b_2 = 0.75$.

- **Iteration 3:** $c_2 = \frac{0.5+0.75}{2} = 0.625$.

$$f(0.625) = 0.625e^{0.625} - 1 = 0.625(1.868246) - 1 = +0.167654 > 0$$

Root in $[0.5, 0.625]$. Set $a_3 = 0.5, b_3 = 0.625$.

- **Iteration 4:** $c_3 = \frac{0.5+0.625}{2} = 0.5625$.

$$f(0.5625) = 0.5625e^{0.5625} - 1 = 0.5625(1.755055) - 1 = -0.012781 < 0$$

Root in $[0.5625, 0.625]$. Set $a_4 = 0.5625, b_4 = 0.625$.

- **Iteration 5:** $c_4 = \frac{0.5625+0.625}{2} = 0.59375$.

$$f(0.59375) = 0.59375e^{0.59375} - 1 = 0.59375(1.810780) - 1 = +0.075151 > 0$$

Root in $[0.5625, 0.59375]$. Set $a_5 = 0.5625, b_5 = 0.59375$.

- **Iteration 6:** $c_5 = \frac{0.5625+0.59375}{2} = 0.578125$.

$$f(0.578125) = 0.578125e^{0.578125} - 1 = 0.578125(1.782692) - 1 = +0.030623 > 0$$

Root in $[0.5625, 0.578125]$. Set $a_6 = 0.5625, b_6 = 0.578125$.

- **Iteration 7:** $c_6 = \frac{0.5625+0.578125}{2} = 0.570313$.

$$f(0.570313) = 0.570313e^{0.570313} - 1 = 0.570313(1.768820) - 1 = +0.008779 > 0$$

Root in $[0.5625, 0.570313]$. Set $a_7 = 0.5625, b_7 = 0.570313$.

- **Iteration 8:** $c_7 = \frac{0.5625+0.570313}{2} = 0.566406$.

$$f(0.566406) = 0.566406e^{0.566406} - 1 = -0.002035 < 0$$

Root in $[0.566406, 0.570313]$. Interval length $b_8 - a_8 = 0.003907$.

- **Iteration 9:** $c_8 = \frac{0.566406+0.570313}{2} = 0.568359$.

$$f(0.568359) = +0.003363 > 0$$

Root in $[0.566406, 0.568359]$. Interval length $b_9 - a_9 = 0.001953$.

- **Iteration 10:** $c_9 = \frac{0.566406+0.568359}{2} = 0.567383$.

Since $a_{10} = 0.566406$ and $b_{10} = 0.568359$, both bounds round to 0.567 correct to 3 decimal places. Hence, the root is $\alpha \approx 0.567$. ■

Example 1.2.7. Solve the equation $f(x) = \cos x - x = 0$ in the interval $[0, 1]$ using 4 iterations of the Bisection Method (where x is in radians).

Solution. Check endpoints $a_0 = 0, b_0 = 1$:

$$f(0) = \cos(0) - 0 = 1.0 > 0, \quad f(1) = \cos(1) - 1 = 0.540302 - 1 = -0.459698 < 0$$

Since $f(0) \cdot f(1) < 0$, a unique root exists in $[0, 1]$.

- **Iteration 1 ($k = 0$):**

$$c_0 = \frac{0+1}{2} = 0.5, \quad f(0.5) = \cos(0.5) - 0.5 = 0.877583 - 0.5 = +0.377583 > 0$$

Since $f(0.5) \cdot f(1) < 0$, root in $[0.5, 1.0]$. Set $a_1 = 0.5, b_1 = 1.0$.

- **Iteration 2 ($k = 1$):**

$$c_1 = \frac{0.5+1.0}{2} = 0.75, \quad f(0.75) = \cos(0.75) - 0.75 = 0.731689 - 0.75 = -0.018311 < 0$$

Since $f(0.5) \cdot f(0.75) < 0$, root in $[0.5, 0.75]$. Set $a_2 = 0.5, b_2 = 0.75$.

- **Iteration 3 ($k = 2$):**

$$c_2 = \frac{0.5+0.75}{2} = 0.625, \quad f(0.625) = \cos(0.625) - 0.625 = 0.810965 - 0.625 = +0.185965 > 0$$

Since $f(0.625) \cdot f(0.75) < 0$, root in $[0.625, 0.75]$. Set $a_3 = 0.625, b_3 = 0.75$.

- **Iteration 4 ($k = 3$):**

$$c_3 = \frac{0.625+0.75}{2} = 0.6875, \quad f(0.6875) = \cos(0.6875) - 0.6875 = 0.772882 - 0.6875 = +0.085382 > 0$$

Since $f(0.6875) \cdot f(0.75) < 0$, root in $[0.6875, 0.75]$.

Summary of results:

k	a_k	b_k	c_k	$f(c_k)$	Updated Interval
0	0.0000	1.0000	0.5000	$+0.377583$	$[0.5000, 1.0000]$
1	0.5000	1.0000	0.7500	-0.018311	$[0.5000, 0.7500]$
2	0.5000	0.7500	0.6250	$+0.185965$	$[0.6250, 0.7500]$
3	0.6250	0.7500	0.6875	$+0.085382$	$[0.6875, 0.7500]$

After 4 iterations, the approximate root is $c_3 = 0.6875$ with error bound $\leq \frac{1-0}{2^4} = 0.0625$. ■

Example 1.2.8. Consider finding the real root of $f(x) = x^3 - 4x - 9 = 0$ in the interval $[2, 3]$.

(i) Determine the theoretical minimum number of bisection iterations required to guarantee an absolute error $|c_k - \alpha| \leq 10^{-4}$.

(ii) Perform the first 3 iterations of the Bisection Method.

Solution. **Part (i):** The initial bracket is $[a_0, b_0] = [2, 3]$, so $b_0 - a_0 = 3 - 2 = 1$. The required tolerance is $\epsilon = 10^{-4}$. Using the error bound formula $|c_N - \alpha| \leq \frac{b_0 - a_0}{2^{N+1}} \leq \epsilon$:

$$\frac{1}{2^{N+1}} \leq 10^{-4} \implies 2^{N+1} \geq 10^4 = 10000$$

Taking logarithms base 10 on both sides:

$$(N+1)\log_{10}(2) \geq 4 \implies N+1 \geq \frac{4}{\log_{10}(2)} \approx \frac{4}{0.301030} \approx 13.2877$$

$$N \geq 13.2877 - 1 = 12.2877$$

Rounding up to the next integer gives $N = 13$. Thus, at least **13 iterations** are required to guarantee an accuracy of 10^{-4}.

Part (ii): Evaluate endpoints:

$$f(2) = (2)^3 - 4(2) - 9 = 8 - 8 - 9 = -9 < 0$$

$$f(3) = (3)^3 - 4(3) - 9 = 27 - 12 - 9 = +6 > 0$$

- **Iteration 1 ($k = 0$):**

$$c_0 = \frac{2+3}{2} = 2.5, \quad f(2.5) = (2.5)^3 - 4(2.5) - 9 = 15.625 - 10 - 9 = -3.375 < 0$$

Since $f(2.5) \cdot f(3) < 0$, root in $[2.5, 3.0]$. Set $a_1 = 2.5, b_1 = 3.0$.

- **Iteration 2 ($k = 1$):**

$$c_1 = \frac{2.5+3.0}{2} = 2.75, \quad f(2.75) = (2.75)^3 - 4(2.75) - 9 = 20.796875 - 11 - 9 = +0.796875 > 0$$

Since $f(2.5) \cdot f(2.75) < 0$, root in $[2.5, 2.75]$. Set $a_2 = 2.5, b_2 = 2.75$.

- **Iteration 3 ($k = 2$):**

$$c_2 = \frac{2.5+2.75}{2} = 2.625, \quad f(2.625) = (2.625)^3 - 4(2.625) - 9 = 18.087891 - 10.5 - 9 = -1.412109 < 0$$

Since $f(2.625) \cdot f(2.75) < 0$, root in $[2.625, 2.75]$.

After 3 iterations, the root is estimated as $c_2 = 2.625$ with an interval bracket $[2.625, 2.75]$. ■

1.2.2 Fixed-Point Iteration Method

The **Fixed-Point Iteration Method** (also known as the *Simple Iteration Method* or the *Method of Successive Approximations*) is a fundamental open iterative method for solving non-linear algebraic and transcendental equations.

To find a real root of the non-linear equation $f(x) = 0$, the equation is algebraically rearranged into an equivalent **fixed-point form**:

$$x = g(x)$$

Definition 1.2.9 (Fixed Point)**.** A point $\alpha \in \mathbb{R}$ is called a **fixed point** of a function $g(x)$ if:

$$g(\alpha) = \alpha$$

Geometrically, a fixed point α corresponds to the x-coordinate of the intersection between the curve $y = g(x)$ and the line $y = x$.

Definition 1.2.10 (Fixed-Point Iterative Scheme)**.** Given an initial guess $x_0 \approx \alpha$, the **fixed-point iteration** constructs a sequence of approximations $\{x_k\}_{k=0}^{\infty}$ defined by the recurrence relation:

$$x_{k+1} = g(x_k), \quad k = 0, 1, 2, \ldots$$

If $g(x)$ is continuous and the sequence $\{x_k\}$ converges to a limit α, then taking the limit on both sides yields:

$$\alpha = \lim_{k\to\infty} x_{k+1} = \lim_{k\to\infty} g(x_k) = g\left(\lim_{k\to\infty} x_k\right) = g(\alpha)$$

Downloaded from – https://aijaz.vercel.app

Hence, the limit α is a fixed point of $g(x)$ and a root of $f(x) = 0$.

Not every choice of $g(x)$ guarantees convergence. The conditions for existence, uniqueness, and convergence of a fixed point are established by the following theorem.

Theorem 1.2.11 (Fixed-Point Theorem / Contraction Mapping Theorem). *Let $g \in C[a,b]$ be a continuous function mapping the closed interval $[a,b]$ into itself, i.e., $g(x) \in [a,b]$ for all $x \in [a,b]$.*

(i) ***Existence:*** *$g(x)$ has at least one fixed point $\alpha \in [a,b]$.*

(ii) ***Uniqueness and Convergence:*** *If, in addition, $g'(x)$ exists on (a,b) and there exists a constant $k \in (0,1)$ such that:*

$$|g'(x)| \leq k < 1 \quad \text{for all } x \in (a,b)$$

then the fixed point $\alpha \in [a,b]$ is unique. Furthermore, for any initial choice $x_0 \in [a,b]$, the sequence $x_{k+1} = g(x_k)$ converges to α, and the error satisfies:

$$|x_k - \alpha| \leq \frac{k^k}{1-k}|x_1 - x_0|$$

Proof. **(i) Existence:** Define $h(x) = g(x) - x$. Since $g(x) \in [a,b]$ for all $x \in [a,b]$, we have $a \leq g(a) \leq b$ and $a \leq g(b) \leq b$. Thus:

$$h(a) = g(a) - a \geq 0, \quad \text{and} \quad h(b) = g(b) - b \leq 0$$

If $h(a) = 0$ or $h(b) = 0$, then a or b is a fixed point. If $h(a) > 0$ and $h(b) < 0$, then by the Intermediate Value Theorem, there exists at least one point $\alpha \in (a,b)$ such that $h(\alpha) = 0 \implies g(\alpha) = \alpha$.

(ii) Uniqueness: Suppose there exist two distinct fixed points $\alpha, \beta \in [a,b]$ with $\alpha \neq \beta$. By the Mean Value Theorem applied to g on the interval between α and β:

$$g(\alpha) - g(\beta) = g'(\xi)(\alpha - \beta) \quad \text{for some } \xi \text{ between } \alpha \text{ and } \beta$$

Since $g(\alpha) = \alpha$ and $g(\beta) = \beta$, we have:

$$\alpha - \beta = g'(\xi)(\alpha - \beta) \implies |\alpha - \beta| = |g'(\xi)||\alpha - \beta| \leq k|\alpha - \beta|$$

Since $k < 1$, this yields $|\alpha - \beta|(1-k) \leq 0$, which implies $|\alpha - \beta| = 0 \implies \alpha = \beta$.

Convergence: Subtracting $\alpha = g(\alpha)$ from $x_{k+1} = g(x_k)$:

$$e_{k+1} = x_{k+1} - \alpha = g(x_k) - g(\alpha) = g'(\xi_k)(x_k - \alpha) = g'(\xi_k)e_k$$

Taking absolute values:

$$|e_{k+1}| \leq k|e_k| \leq k^2|e_{k-1}| \leq \cdots \leq k^{k+1}|e_0|$$

Since $0 < k < 1$, $\lim_{k\to\infty} k^{k+1} = 0$, which proves $\lim_{k\to\infty} |e_k| = 0$, i.e., $x_k \to \alpha$.

Error Bound: Observe that for any $m \geq 1$:

$$|x_{k+m} - x_k| \leq \sum_{j=1}^{m} |x_{k+j} - x_{k+j-1}| \leq \sum_{j=1}^{m} k^{k+j-1}|x_1 - x_0| = k^k|x_1 - x_0| \sum_{j=0}^{m-1} k^j$$

Taking the limit as $m \to \infty$ and using the sum of an infinite geometric series $\sum_{j=0}^{\infty} k^j = \frac{1}{1-k}$:

$$|\alpha - x_k| \leq \frac{k^k}{1-k}|x_1 - x_0|$$

■

Downloaded from – https://aijaz.vercel.app

Theorem 1.2.12 (Order of Convergence of Fixed-Point Iteration). *Let $\alpha = g(\alpha)$ be a fixed point.*

*(i) If $g'(\alpha) \neq 0$ and $|g'(\alpha)| < 1$, the fixed-point iteration converges **linearly** ($p = 1$) with asymptotic error constant $C = |g'(\alpha)|$.*

*(ii) If $g'(\alpha) = 0$ and $g''(\alpha) \neq 0$, the fixed-point iteration converges **quadratically** ($p = 2$) with asymptotic error constant $C = \frac{|g''(\alpha)|}{2}$.*

Example 1.2.13. Consider the equation $f(x) = x^3 + x - 1 = 0$ in the interval $[0.5, 1.0]$.

(i) Rearrange $f(x) = 0$ into two different fixed-point forms $x = g_1(x)$ and $x = g_2(x)$, and analyze their convergence behavior on $[0.5, 1.0]$.

(ii) Perform 5 iterations using the convergent form starting with $x_0 = 0.5$.

Solution. **Part (i):**

- **Formulation 1:** $x = 1 - x^3 \implies g_1(x) = 1 - x^3$. Differentiating: $g_1'(x) = -3x^2$. On $[0.5, 1.0]$, $|g_1'(x)| \in [0.75, 3.0]$. Since $|g_1'(x)| > 1$ over most of the domain, the iteration $x_{k+1} = g_1(x_k)$ **diverges**.
- **Formulation 2:** $x(x^2 + 1) = 1 \implies x = \frac{1}{x^2+1} \implies g_2(x) = (1 + x^2)^{-1}$. Differentiating:

$$g_2'(x) = \frac{-2x}{(1 + x^2)^2}$$

On $[0.5, 1.0]$, the maximum value of $|g_2'(x)|$ occurs near $x \approx 0.58$:

$$|g_2'(0.5)| = \frac{1.0}{(1.25)^2} = 0.64 < 1, \quad |g_2'(1.0)| = \frac{2.0}{4.0} = 0.50 < 1$$

Since $|g_2'(x)| \leq k = 0.64 < 1$ for all $x \in [0.5, 1.0]$, the iteration $x_{k+1} = g_2(x_k)$ is guaranteed to **converge**.

Part (ii): Using $x_{k+1} = \frac{1}{1+x_k^2}$ with $x_0 = 0.5$:

- $k = 0$: $x_1 = \frac{1}{1+(0.5)^2} = \frac{1}{1.25} = 0.800000$
- $k = 1$: $x_2 = \frac{1}{1+(0.800000)^2} = \frac{1}{1.64} = 0.609756$
- $k = 2$: $x_3 = \frac{1}{1+(0.609756)^2} = \frac{1}{1.371803} = 0.728967$
- $k = 3$: $x_4 = \frac{1}{1+(0.728967)^2} = \frac{1}{1.531393} = 0.653001$
- $k = 4$: $x_5 = \frac{1}{1+(0.653001)^2} = \frac{1}{1.426410} = 0.701061$

The iterations oscillate around the exact root $\alpha \approx 0.682328$. ■

Example 1.2.14. Find a real root of $f(x) = \cos x - x = 0$ using the Fixed-Point Iteration Method starting with $x_0 = 0.5$. Continue until the solution is correct to 4 decimal places.

Downloaded from – https://aijaz.vercel.app

Solution. Rewrite the equation as $x = \cos x$, so $g(x) = \cos x$. Check the convergence condition on $I = [0, 1]$:

$$g'(x) = -\sin x \implies |g'(x)| = \sin x$$

On $[0, 1]$, $\max_{x\in[0,1]} |g'(x)| = \sin(1) \approx 0.841471 < 1$. Thus, the iteration $x_{k+1} = \cos(x_k)$ is guaranteed to converge.

Executing the iterations (with x in radians):

k	x_k	$x_{k+1} = \cos(x_k)$	$\lvert x_{k+1} - x_k \rvert$
0	0.500000	0.877583	0.377583
1	0.877583	0.639012	0.238571
2	0.639012	0.802685	0.163673
3	0.802685	0.694778	0.107907
4	0.694778	0.768196	0.073418
5	0.768196	0.719165	0.049031
6	0.719165	0.752352	0.033187
...	...	...	...
13	0.738722	0.739329	0.000607
14	0.739329	0.738920	0.000409
15	0.738920	0.739195	0.000275
16	0.739195	0.739010	0.000185
17	0.739010	0.739135	0.000125
18	0.739135	0.739051	0.000084

Rounding to 4 decimal places, $x_{18} \approx 0.7391$. ■

Example 1.2.15. Consider $g(x) = \frac{1}{3}\cos x + \frac{1}{2}$ defined on the interval $[0, 1]$.

(i) Prove that $g(x)$ has a unique fixed point in $[0, 1]$.

(ii) Using $x_0 = 0.5$, compute the theoretical number of iterations N required to achieve an error bound $|x_N - \alpha| \leq 10^{-4}$.

(iii) Compute x_1, x_2, and x_3.

Solution. **Part (i):**

- **Self-mapping $g([0, 1]) \subseteq [0, 1]$:** Since $\cos x$ is strictly decreasing on $[0, 1]$:

$$g(1) = \frac{1}{3}\cos(1) + \frac{1}{2} \approx \frac{0.540302}{3} + 0.5 = 0.680101 \geq 0$$

$$g(0) = \frac{1}{3}\cos(0) + \frac{1}{2} = \frac{1}{3} + \frac{1}{2} = \frac{5}{6} \approx 0.833333 \leq 1$$

  Thus, $g(x) \in [0.6801, 0.8333] \subset [0, 1]$ for all $x \in [0, 1]$. By the Fixed-Point Theorem, at least one fixed point exists.

- **Contraction Condition:** $g'(x) = -\frac{1}{3}\sin x$. On $[0, 1]$:

$$|g'(x)| = \frac{1}{3}\sin x \leq \frac{1}{3}\sin(1) \approx \frac{0.841471}{3} = 0.280490 < 1$$

  Set $k = 0.280490$. Since $k < 1$, $g(x)$ has a **unique fixed point** $\alpha \in [0, 1]$.

Part (ii): Compute x_1 with $x_0 = 0.5$:

$$x_1 = g(0.5) = \frac{1}{3}\cos(0.5) + \frac{1}{2} = \frac{0.877583}{3} + 0.5 = 0.792528$$

The initial step size is $|x_1 - x_0| = |0.792528 - 0.5| = 0.292528$. Using the theoretical error bound formula $|x_N - \alpha| \leq \frac{k^N}{1-k}|x_1 - x_0| \leq 10^{-4}$:

$$\frac{(0.280490)^N}{1 - 0.280490}(0.292528) \leq 10^{-4} \implies \frac{(0.280490)^N}{0.719510}(0.292528) \leq 10^{-4}$$

$$(0.280490)^N(0.406566) \leq 10^{-4} \implies (0.280490)^N \leq \frac{10^{-4}}{0.406566} \approx 2.45962 \times 10^{-4}$$

Taking natural logarithms:

$$N\ln(0.280490) \leq \ln(2.45962 \times 10^{-4}) \implies N(-1.271217) \leq -8.310419$$

$$N \geq \frac{8.310419}{1.271217} \approx 6.537$$

Hence, $N = 7$ iterations guarantee an error less than 10^{-4}.

Part (iii): First 3 iterations:

- $x_1 = 0.792528$
- $x_2 = g(0.792528) = \frac{1}{3}\cos(0.792528) + 0.5 = \frac{0.702082}{3} + 0.5 = 0.734027$
- $x_3 = g(0.734027) = \frac{1}{3}\cos(0.734027) + 0.5 = \frac{0.742468}{3} + 0.5 = 0.747489$

■

1.2.3 Newton–Raphson Method

The **Newton–Raphson Method** (often referred to simply as *Newton's Method*) is one of the most powerful, widely used, and computationally efficient open iterative techniques for finding real roots of differentiable non-linear equations $f(x) = 0$. Unlike bracketing methods (e.g., Bisection), it requires only a single initial guess x_0 and utilizes local derivative information to achieve rapid convergence.

The Newton–Raphson formula can be derived through two distinct mathematical approaches: geometrically via tangent line approximation and analytically via Taylor series expansion.

1. Geometric Derivation: Let x_k be a current approximation to the true root α of $f(x) = 0$. Consider the point $(x_k, f(x_k))$ on the curve $y = f(x)$. Assuming $f(x)$ is differentiable, the equation of the tangent line to the curve at this point is given by point-slope form:

$$y - f(x_k) = f'(x_k)(x - x_k)$$

The next approximation x_{k+1} is defined as the x-intercept of this tangent line (where $y = 0$):

$$0 - f(x_k) = f'(x_k)(x_{k+1} - x_k)$$

Assuming $f'(x_k) \neq 0$, solving for x_{k+1} yields the famous **Newton–Raphson Iterative Formula**:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \quad k = 0, 1, 2, \ldots$$

Downloaded from – https://aijaz.vercel.app

2. Analytic Derivation via Taylor Series: Let x_k be an approximation to the exact root $\alpha = x_k + h$, where h is a small correction step such that $f(\alpha) = f(x_k + h) = 0$. Expanding $f(x_k + h)$ in a Taylor series about x_k:

$$f(x_k + h) = f(x_k) + hf'(x_k) + \frac{h^2}{2!}f''(\xi) = 0 \quad \text{for some } \xi \text{ between } x_k \text{ and } x_k + h$$

Assuming h is sufficiently small, the higher-order term $\mathcal{O}(h^2)$ can be neglected:

$$f(x_k) + hf'(x_k) \approx 0 \implies h \approx -\frac{f(x_k)}{f'(x_k)} \quad (f'(x_k) \neq 0)$$

Adding the correction step h to x_k gives the updated approximation $x_{k+1} = x_k + h$:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Theorem 1.2.16 (Quadratic Convergence of Newton–Raphson Method)**.** *Let $f \in C^2[a, b]$ and let $\alpha \in (a, b)$ be a simple root such that $f(\alpha) = 0$ and $f'(\alpha) \neq 0$. Then there exists a neighborhood $I = [\alpha - \delta, \alpha + \delta]$ such that for any initial guess $x_0 \in I$, the sequence $\{x_k\}_{k=0}^{\infty}$ generated by Newton's method converges **quadratically** ($p = 2$) to α. The error $e_k = x_k - \alpha$ satisfies:*

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^2} = \frac{|f''(\alpha)|}{2|f'(\alpha)|} = C$$

where C is the asymptotic error constant.

Proof. By definition of the Newton–Raphson formula, $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$. Subtracting the exact root α from both sides gives the error relation:

$$e_{k+1} = x_{k+1} - \alpha = x_k - \alpha - \frac{f(x_k)}{f'(x_k)} = e_k - \frac{f(x_k)}{f'(x_k)} = \frac{e_k f'(x_k) - f(x_k)}{f'(x_k)}$$

Expanding $f(\alpha) = f(x_k - e_k)$ in a Taylor series expansion about x_k:

$$f(\alpha) = f(x_k) - e_k f'(x_k) + \frac{e_k^2}{2}f''(\xi_k) = 0 \quad \text{for some } \xi_k \text{ between } x_k \text{ and } \alpha$$

Rearranging terms:

$$e_k f'(x_k) - f(x_k) = \frac{e_k^2}{2}f''(\xi_k)$$

Substituting this identity into the numerator of e_{k+1}:

$$e_{k+1} = \frac{\frac{e_k^2}{2}f''(\xi_k)}{f'(x_k)} = \frac{f''(\xi_k)}{2f'(x_k)}e_k^2$$

Dividing by e_k^2 and taking absolute values:

$$\frac{|e_{k+1}|}{|e_k|^2} = \frac{|f''(\xi_k)|}{2|f'(x_k)|}$$

Taking the limit as $k \to \infty$ (noting $x_k \to \alpha$ and $\xi_k \to \alpha$):

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^2} = \frac{|f''(\alpha)|}{2|f'(\alpha)|} = C$$

Since $p = 2$ and $C > 0$ (for $f''(\alpha) \neq 0$), the Newton–Raphson method converges **quadratically**. ■

Downloaded from – https://aijaz.vercel.app

1.2.4 Limitations and Pitfalls of Newton's Method

Despite its rapid quadratic convergence, Newton's method can fail or perform poorly under specific conditions:

1. **Zero Derivative ($f'(x_k) = 0$):** If $f'(x_k) = 0$ at any iteration point, the formula involves division by zero, causing the algorithm to crash geometrically (tangent line is parallel to the x-axis).
2. **Poor Initial Guess x_0:** If x_0 is chosen far from the true root α, the method may diverge or jump to an entirely different root.
3. **Inflection Points ($f''(x_k) = 0$):** Near inflection points, the tangent line can shoot the next approximation far outside the region of interest.
4. **Cycles and Oscillations:** The sequence may get trapped in a non-convergent periodic loop, e.g., $x_0 \to x_1 \to x_0 \to x_1 \dots$
5. **Multiple Roots ($f'(\alpha) = 0$):** If α is a root of multiplicity $m > 1$, the convergence rate degenerates from quadratic ($p = 2$) to linear ($p = 1$).

Example 1.2.17. Find the positive real root of the algebraic polynomial equation $f(x) = x^3 - 2x - 5 = 0$ near $x_0 = 2$ correct to 4 decimal places using the Newton–Raphson method.

Solution. Given $f(x) = x^3 - 2x - 5$. Differentiating gives:

$$f'(x) = 3x^2 - 2$$

The Newton–Raphson iterative formula is:

$$x_{k+1} = x_k - \frac{x_k^3 - 2x_k - 5}{3x_k^2 - 2} = \frac{2x_k^3 + 5}{3x_k^2 - 2}$$

Starting with $x_0 = 2$:

- **Iteration 1 ($k = 0$):**

$$f(2) = 2^3 - 2(2) - 5 = -1, \quad f'(2) = 3(2)^2 - 2 = 10$$

$$x_1 = 2 - \frac{-1}{10} = 2 + 0.1 = 2.100000$$

- **Iteration 2 ($k = 1$):**

$$f(2.1) = (2.1)^3 - 2(2.1) - 5 = 9.261 - 4.2 - 5 = +0.061000$$

$$f'(2.1) = 3(2.1)^2 - 2 = 13.23 - 2 = 11.230000$$

$$x_2 = 2.1 - \frac{0.061000}{11.230000} = 2.1 - 0.005432 = 2.094568$$

- **Iteration 3 ($k = 2$):**

$$f(2.094568) = (2.094568)^3 - 2(2.094568) - 5 = +0.000185$$

$$f'(2.094568) = 3(2.094568)^2 - 2 = 11.161706$$

$$x_3 = 2.094568 - \frac{0.000185}{11.161706} = 2.094568 - 0.000017 = 2.094551$$

- **Iteration 4 ($k = 3$):**

$$f(2.094551) = (2.094551)^3 - 2(2.094551) - 5 \approx 0.000000$$

$$x_4 = 2.094551$$

Since x_3 and x_4 agree to 5 decimal places, the root correct to 4 decimal places is $\alpha \approx 2.0946$. ■

Example 1.2.18. Find the real root of the transcendental equation $f(x) = xe^x - 1 = 0$ starting with $x_0 = 0.5$ correct to 5 decimal places using the Newton–Raphson method.

Solution. The function is $f(x) = xe^x - 1$. Differentiating using the product rule:

$$f'(x) = e^x + xe^x = (x+1)e^x$$

The Newton–Raphson iteration formula is:

$$x_{k+1} = x_k - \frac{x_k e^{x_k} - 1}{(x_k+1)e^{x_k}} = \frac{x_k^2 e^{x_k} + 1}{(x_k+1)e^{x_k}}$$

Starting with $x_0 = 0.5$:

- **Iteration 1 ($k = 0$):**

$$f(0.5) = 0.5e^{0.5} - 1 = 0.5(1.648721) - 1 = -0.175639$$

$$f'(0.5) = (0.5+1)e^{0.5} = 1.5(1.648721) = 2.473082$$

$$x_1 = 0.5 - \frac{-0.175639}{2.473082} = 0.5 + 0.071020 = 0.571020$$

- **Iteration 2 ($k = 1$):**

$$f(0.571020) = 0.571020e^{0.571020} - 1 = 0.571020(1.770072) - 1 = +0.010747$$

$$f'(0.571020) = (1.571020)e^{0.571020} = (1.571020)(1.770072) = 2.780819$$

$$x_2 = 0.571020 - \frac{0.010747}{2.780819} = 0.571020 - 0.003865 = 0.567155$$

- **Iteration 3 ($k = 2$):**

$$f(0.567155) = 0.567155e^{0.567155} - 1 = 0.567155(1.763240) - 1 = +0.000020$$

$$f'(0.567155) = (1.567155)(1.763240) = 2.763283$$

$$x_3 = 0.567155 - \frac{0.000020}{2.763283} = 0.567155 - 0.000007 = 0.567148$$

- **Iteration 4 ($k = 3$):**

$$f(0.567148) \approx 0.000000 \implies x_4 = 0.567143$$

Rounding to 5 decimal places, the root is $\alpha \approx 0.56714$. ■

Example 1.2.19. (i) Derive a division-free Newton–Raphson iterative scheme for computing the reciprocal $\frac{1}{N}$ of a positive real number $N > 0$.

Downloaded from – https://aijaz.vercel.app

(ii) Use this scheme to evaluate $\frac{1}{17}$ correct to 6 decimal places starting with $x_0 = 0.05$.

Solution. **Part (i):** To find $\frac{1}{N}$ without performing division, define the function:

$$f(x) = \frac{1}{x} - N = 0$$

Differentiating with respect to x:

$$f'(x) = -\frac{1}{x^2}$$

Substituting $f(x)$ and $f'(x)$ into Newton's iterative formula:

$$x_{k+1} = x_k - \frac{\frac{1}{x_k} - N}{-\frac{1}{x_k^2}} = x_k + x_k^2\left(\frac{1}{x_k} - N\right) = x_k + x_k - Nx_k^2$$

Factoring out x_k gives the **Division-Free Reciprocal Formula**:

$$x_{k+1} = x_k(2 - Nx_k)$$

Notice that this iteration uses only multiplications and subtractions, making it ideal for computer arithmetic hardware.

Part (ii): Compute $\frac{1}{17}$ with $N = 17$ and $x_0 = 0.05$:

- **Iteration 1 ($k = 0$):**

$$x_1 = 0.05\,(2 - 17 \times 0.05) = 0.05(2 - 0.85) = 0.05(1.15) = 0.057500$$

- **Iteration 2 ($k = 1$):**

$$x_2 = 0.0575\,(2 - 17 \times 0.0575) = 0.0575(2 - 0.9775) = 0.0575(1.0225) = 0.058794$$

- **Iteration 3 ($k = 2$):**

$$x_3 = 0.05879375\,(2 - 17 \times 0.05879375) = 0.05879375(1.00050625) = 0.05882352$$

- **Iteration 4 ($k = 3$):**

$$x_4 = 0.058823529\,(2 - 17 \times 0.058823529) = 0.058823529$$

Comparing with the exact quotient $\frac{1}{17} = 0.058823529\ldots$, x_4 is accurate to 8 decimal places in just 4 steps. Correct to 6 decimal places: $\frac{1}{17} \approx 0.058824$. ■

Example 1.2.20. Demonstrate a failure mode of the Newton–Raphson method by applying it to $f(x) = x^3 - 5x = 0$ with an initial guess of $x_0 = 1$. Explain the observed behavior.

Solution. Given $f(x) = x^3 - 5x$, the exact roots are $x = 0, \sqrt{5} \approx 2.236, -\sqrt{5} \approx -2.236$. Differentiating $f(x)$:

$$f'(x) = 3x^2 - 5$$

The Newton–Raphson iterative formula is:

$$x_{k+1} = x_k - \frac{x_k^3 - 5x_k}{3x_k^2 - 5} = \frac{3x_k^3 - 5x_k - x_k^3 + 5x_k}{3x_k^2 - 5} = \frac{2x_k^3}{3x_k^2 - 5}$$

Starting with initial guess $x_0 = 1$:

Downloaded from – https://aijaz.vercel.app

- **Iteration 1 ($k = 0$):**
$$x_1 = \frac{2(1)^3}{3(1)^2 - 5} = \frac{2}{-2} = -1$$
- **Iteration 2 ($k = 1$):**
$$x_2 = \frac{2(-1)^3}{3(-1)^2 - 5} = \frac{-2}{-2} = 1$$
- **Iteration 3 ($k = 2$):**
$$x_3 = \frac{2(1)^3}{3(1)^2 - 5} = -1$$

Conclusion: The algorithm gets trapped in an infinite non-convergent 2-cycle $1 \to -1 \to 1 \to -1 \dots$ indefinitely. Geometrically, the tangent line at $x_0 = 1$ intersects the x-axis at $x_1 = -1$, and the tangent line at $x_1 = -1$ intersects the x-axis back at $x_2 = 1$. To break the oscillation and achieve convergence to a root, a different initial guess (e.g., $x_0 = 2$) must be chosen. ■

1.3 Polynomial Interpolation

In scientific computing and data analysis, functions are often known only at a finite set of discrete points (e.g., from experimental measurements or numerical simulations). **Polynomial Interpolation** is the process of constructing a continuous polynomial of minimal degree that passes exactly through these given data points. Interpolating polynomials serve as the foundational building blocks for numerical differentiation, numerical integration, and root-finding algorithms.

Definition 1.3.1 (Interpolation Problem)**.** Given $n+1$ distinct data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ where $x_0 < x_1 < \cdots < x_n$ are distinct nodes, the polynomial interpolation problem seeks a polynomial $P_n(x)$ of degree at most n belonging to the vector space $\mathcal{P}_n$ such that:

$$P_n(x_i) = y_i = f(x_i) \quad \text{for all } i = 0, 1, 2, \dots, n$$

Theorem 1.3.2 (Existence and Uniqueness of Interpolating Polynomial)**.** *Let $x_0, x_1, \dots, x_n$ be $n+1$ distinct real numbers, and let $y_0, y_1, \dots, y_n$ be arbitrary real numbers. Then there exists a **unique** polynomial $P_n(x)$ of degree at most n satisfying $P_n(x_i) = y_i$ for each $i = 0, 1, \dots, n$.*

Proof. Express the general n-th degree polynomial as $P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$. Imposing the $n+1$ interpolation conditions $P_n(x_i) = y_i$ yields a linear system of $n+1$ equations in $n+1$ unknown coefficients $a_0, a_1, \dots, a_n$:

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ 1 & x_2 & x_2^2 & \dots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

The coefficient matrix V is the famous **Vandermonde Matrix**. Its determinant is given by:

$$\det(V) = \prod_{0 \le j < i \le n} (x_i - x_j)$$

Since all nodes x_i are distinct ($x_i \neq x_j$ for $i \neq j$), $\det(V) \neq 0$. Matrix V is non-singular and invertible, guaranteeing that the linear system has a **unique solution** for the coefficient vector $(a_0, a_1, \dots, a_n)^T$. ■

Downloaded from – https://aijaz.vercel.app

1.3.1 Lagrange Interpolation

The **Lagrange Interpolating Polynomial** constructs the unique interpolating polynomial $P_n(x)$ explicitly without solving the computationally expensive Vandermonde linear system.

Lagrange Basis Polynomials

Definition 1.3.3 (Lagrange Basis Polynomials)**.** For $n+1$ distinct nodes $x_0, x_1, \ldots, x_n$, the k-th **Lagrange basis polynomial** $L_{n,k}(x)$ (or simply $L_k(x)$) of degree n is defined by:

$$L_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^{n} \frac{x - x_j}{x_k - x_j} = \frac{(x-x_0)(x-x_1)\ldots(x-x_{k-1})(x-x_{k+1})\ldots(x-x_n)}{(x_k-x_0)(x_k-x_1)\ldots(x_k-x_{k-1})(x_k-x_{k+1})\ldots(x_k-x_n)}$$

Theorem 1.3.4 (Properties of Lagrange Basis Polynomials)**.** *The Lagrange basis polynomials $L_k(x)$ satisfy the following fundamental properties:*

(i) ***Kronecker Delta Property:***

$$L_k(x_i) = \delta_{ki} = \begin{cases} 1 & \text{if } i = k \\ 0 & \text{if } i \neq k \end{cases}$$

(ii) ***Partition of Unity (Sum of Weights Property):*** *For any $x \in \mathbb{R}$:*

$$\sum_{k=0}^{n} L_k(x) = 1$$

Definition 1.3.5 (Lagrange Interpolating Formula)**.** The **Lagrange Interpolating Polynomial** $P_n(x)$ of degree at most n passing through the data points $(x_i, y_i)_{i=0}^{n}$ is given by:

$$P_n(x) = \sum_{k=0}^{n} y_k L_k(x) = \sum_{k=0}^{n} f(x_k) L_k(x)$$

Evaluating $P_n(x)$ at $x = x_i$ verifies the interpolation property:

$$P_n(x_i) = \sum_{k=0}^{n} y_k L_k(x_i) = \sum_{k=0}^{n} y_k \delta_{ki} = y_i$$

Theorem 1.3.6 (Lagrange Interpolation Error Formula)**.** *Let $f \in C^{n+1}[a,b]$ and let $x_0, x_1, \ldots, x_n$ be $n+1$ distinct nodes in $[a,b]$. If $P_n(x)$ is the polynomial interpolating $f(x)$ at these nodes, then for any $x \in [a,b]$, there exists a point $\xi_x \in (a,b)$ such that the interpolation error $E_n(x) = f(x) - P_n(x)$ is given by:*

$$E_n(x) = f(x) - P_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} W_{n+1}(x)$$

where $W_{n+1}(x) = \prod_{j=0}^{n}(x - x_j) = (x-x_0)(x-x_1)\ldots(x-x_n)$ is the nodal polynomial.

Proof. If $x = x_i$ for any $i \in \{0, 1, \ldots, n\}$, then $f(x_i) - P_n(x_i) = 0$ and $W_{n+1}(x_i) = 0$, so the formula holds trivially for any choice of ξ_x.

Downloaded from – https://aijaz.vercel.app

Now fix $x \in [a,b]$ such that $x \neq x_i$ for all i. Define an auxiliary function $g(t)$ for $t \in [a,b]$ by:

$$g(t) = f(t) - P_n(t) - KW_{n+1}(t)$$

where K is a constant chosen such that $g(x) = 0$. That is:

$$K = \frac{f(x) - P_n(x)}{W_{n+1}(x)}$$

Since $f \in C^{n+1}[a,b]$ and P_n, W_{n+1} are polynomials, $g(t)$ is $(n+1)$-times continuously differentiable on $[a,b]$. Notice that $g(t) = 0$ at the $n+2$ distinct points $x_0, x_1, \ldots, x_n$, and x.

Applying **Rolle's Theorem** successively:

- $g(t)$ has $n+2$ zeros in $[a,b] \implies g'(t)$ has at least $n+1$ distinct zeros in (a,b).
- $g'(t)$ has $n+1$ zeros $\implies g''(t)$ has at least n distinct zeros in (a,b).
- Repeating this process $n+1$ times, $g^{(n+1)}(t)$ has at least one zero $\xi_x \in (a,b)$.

Differentiating $g(t)$ $(n+1)$ times with respect to t:

$$g^{(n+1)}(t) = f^{(n+1)}(t) - P_n^{(n+1)}(t) - KW_{n+1}^{(n+1)}(t)$$

Since $P_n(t)$ is a polynomial of degree $\leq n$, its $(n+1)$-th derivative is zero ($P_n^{(n+1)}(t) = 0$). Since $W_{n+1}(t) = t^{n+1} + \mathcal{O}(t^n)$, its $(n+1)$-th derivative is $(n+1)!$. Thus:

$$g^{(n+1)}(t) = f^{(n+1)}(t) - 0 - K(n+1)!$$

Evaluating at $t = \xi_x$ where $g^{(n+1)}(\xi_x) = 0$:

$$f^{(n+1)}(\xi_x) - K(n+1)! = 0 \implies K = \frac{f^{(n+1)}(\xi_x)}{(n+1)!}$$

Substituting K back into $f(x) - P_n(x) = KW_{n+1}(x)$ gives:

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{j=0}^{n} (x - x_j)$$

■

Example 1.3.7. Construct the quadratic Lagrange interpolating polynomial $P_2(x)$ for the following dataset with unequally spaced nodes:

x_i	0	1	3
y_i	1	3	25

Hence, estimate $P_2(2)$.

Solution. Here $n = 2$, with nodes $x_0 = 0, x_1 = 1, x_2 = 3$ and values $y_0 = 1, y_1 = 3, y_2 = 25$.

Compute the Lagrange basis polynomials $L_k(x)$:

$$L_0(x) = \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} = \frac{(x-1)(x-3)}{(0-1)(0-3)} = \frac{x^2 - 4x + 3}{3}$$

$$L_1(x) = \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} = \frac{(x-0)(x-3)}{(1-0)(1-3)} = \frac{x(x-3)}{-2} = \frac{-x^2 + 3x}{2}$$

Downloaded from – https://aijaz.vercel.app

$$L_2(x) = \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)} = \frac{(x-0)(x-1)}{(3-0)(3-1)} = \frac{x(x-1)}{6} = \frac{x^2-x}{6}$$

Construct $P_2(x) = y_0L_0(x) + y_1L_1(x) + y_2L_2(x)$:

$$P_2(x) = 1 \cdot \left(\frac{x^2-4x+3}{3}\right) + 3 \cdot \left(\frac{-x^2+3x}{2}\right) + 25 \cdot \left(\frac{x^2-x}{6}\right)$$

Taking a common denominator of 6:

$$P_2(x) = \frac{2(x^2-4x+3) - 9(x^2-3x) + 25(x^2-x)}{6}$$

$$P_2(x) = \frac{2x^2-8x+6-9x^2+27x+25x^2-25x}{6} = \frac{18x^2-6x+6}{6} = 3x^2-x+1$$

Evaluating at $x = 2$:

$$P_2(2) = 3(2)^2 - (2) + 1 = 12 - 2 + 1 = 11$$

■

Example 1.3.8. Given the function values $y_i = f(x_i)$ at four nodes:

x_i	0	1	2	3
$f(x_i)$	-1	1	9	29

Use Lagrange interpolation to construct $P_3(x)$ and estimate $f(1.5)$.

Solution. Here $n = 3$, with $x_0 = 0, x_1 = 1, x_2 = 2, x_3 = 3$. We calculate $P_3(1.5)$ directly by evaluating basis functions at $x = 1.5$:

$$L_0(1.5) = \frac{(1.5-1)(1.5-2)(1.5-3)}{(0-1)(0-2)(0-3)} = \frac{(0.5)(-0.5)(-1.5)}{-6} = \frac{0.375}{-6} = -0.0625$$

$$L_1(1.5) = \frac{(1.5-0)(1.5-2)(1.5-3)}{(1-0)(1-2)(1-3)} = \frac{(1.5)(-0.5)(-1.5)}{2} = \frac{1.125}{2} = 0.5625$$

$$L_2(1.5) = \frac{(1.5-0)(1.5-1)(1.5-3)}{(2-0)(2-1)(2-3)} = \frac{(1.5)(0.5)(-1.5)}{-2} = \frac{-1.125}{-2} = 0.5625$$

$$L_3(1.5) = \frac{(1.5-0)(1.5-1)(1.5-2)}{(3-0)(3-1)(3-2)} = \frac{(1.5)(0.5)(-0.5)}{6} = \frac{-0.375}{6} = -0.0625$$

Substituting into $P_3(1.5) = \sum_{k=0}^{3} y_k L_k(1.5)$:

$$P_3(1.5) = (-1)(-0.0625) + (1)(0.5625) + (9)(0.5625) + (29)(-0.0625)$$

$$P_3(1.5) = 0.0625 + 0.5625 + 5.0625 - 1.8125 = 3.875$$

■

Example 1.3.9. Consider the function $f(x) = \ln x$ interpolated at $x_0 = 1, x_1 = 2, x_2 = 4$.

(i) Construct $P_2(x)$ and estimate $\ln(3)$.

(ii) Compute an upper bound for the absolute interpolation error $|f(3) - P_2(3)|$ using the error theorem, and compare it with the actual error.

Downloaded from – https://aijaz.vercel.app

Solution. **Part (i):** Data points are $(1,0),(2,\ln 2 \approx 0.693147),(4,\ln 4 \approx 1.386294)$. Evaluate $L_k(3)$ at $x=3$:

$$L_0(3) = \frac{(3-2)(3-4)}{(1-2)(1-4)} = \frac{(1)(-1)}{3} = -\frac{1}{3}$$

$$L_1(3) = \frac{(3-1)(3-4)}{(2-1)(2-4)} = \frac{(2)(-1)}{-2} = 1$$

$$L_2(3) = \frac{(3-1)(3-2)}{(4-1)(4-2)} = \frac{(2)(1)}{6} = \frac{1}{3}$$

Interpolated value for $\ln(3)$:

$$P_2(3) = 0\left(-\frac{1}{3}\right) + (\ln 2)(1) + (\ln 4)\left(\frac{1}{3}\right) = \ln 2 + \frac{2}{3}\ln 2 = \frac{5}{3}\ln 2 \approx \frac{5}{3}(0.693147) = 1.155245$$

Part (ii): By the Interpolation Error Theorem for $n=2$:

$$|f(3) - P_2(3)| = \left|\frac{f'''(\xi)}{3!}(3-1)(3-2)(3-4)\right| = \frac{|f'''(\xi)|}{6}|(2)(1)(-1)| = \frac{|f'''(\xi)|}{3}$$

For $f(x) = \ln x$:

$$f'(x) = x^{-1}, \quad f''(x) = -x^{-2}, \quad f'''(x) = 2x^{-3}$$

For $\xi \in [1,4]$, the third derivative is bounded by:

$$\max_{\xi\in[1,4]} |f'''(\xi)| = \max_{\xi\in[1,4]} \frac{2}{\xi^3} = \frac{2}{1^3} = 2$$

Hence, the maximum theoretical error bound is:

$$|E_2(3)| \leq \frac{2}{3} \approx 0.666667$$

Comparing with the exact value $\ln(3) \approx 1.098612$:

$$\text{Actual Error } = |1.098612 - 1.155245| = 0.056633$$

The actual error 0.056633 is well within the derived maximum bound of 0.666667. ■

Example 1.3.10. Inverse Lagrange Interpolation: Given the tabular data:

x	1	2	4
$y = f(x)$	2	5	17

Use Inverse Lagrange Interpolation to estimate the value of x corresponding to $y = 10$.

Solution. In **Inverse Interpolation**, we treat y as the independent variable and x as the dependent variable ($x = g(y)$). The target point is $y = 10$ using nodes $y_0 = 2, y_1 = 5, y_2 = 17$ with values $x_0 = 1, x_1 = 2, x_2 = 4$.

Construct the inverse Lagrange basis functions $L_k(y)$ evaluated at $y = 10$:

$$L_0(10) = \frac{(10-y_1)(10-y_2)}{(y_0-y_1)(y_0-y_2)} = \frac{(10-5)(10-17)}{(2-5)(2-17)} = \frac{(5)(-7)}{(-3)(-15)} = \frac{-35}{45} = -\frac{7}{9}$$

$$L_1(10) = \frac{(10-y_0)(10-y_2)}{(y_1-y_0)(y_1-y_2)} = \frac{(10-2)(10-17)}{(5-2)(5-17)} = \frac{(8)(-7)}{(3)(-12)} = \frac{-56}{-36} = \frac{14}{9}$$

Downloaded from – https://aijaz.vercel.app

$$L_2(10) = \frac{(10-y_0)(10-y_1)}{(y_2-y_0)(y_2-y_1)} = \frac{(10-2)(10-5)}{(17-2)(17-5)} = \frac{(8)(5)}{(15)(12)} = \frac{40}{180} = \frac{2}{9}$$

Estimate $x(10)$ using $x(y) = \sum_{k=0}^{2} x_k L_k(y)$:

$$x(10) = 1 \cdot \left(-\frac{7}{9}\right) + 2 \cdot \left(\frac{14}{9}\right) + 4 \cdot \left(\frac{2}{9}\right) = \frac{-7+28+8}{9} = \frac{29}{9} \approx 3.2222$$

■

1.3.2 Newton's Divided Difference Interpolation

While the Lagrange interpolation formula provides an explicit expression for the interpolating polynomial $P_n(x)$, it has a major computational drawback: if additional data points are added to refine an existing approximation, all Lagrange basis polynomials $L_k(x)$ must be recomputed from scratch. **Newton's Divided Difference Interpolation** overcomes this limitation by expressing $P_n(x)$ in a recursive, incremental form where adding a new node simply appends a higher-degree term without altering the previously calculated coefficients.

Let $x_0, x_1, \ldots, x_n$ be $n+1$ distinct nodes with corresponding function values $f(x_0), f(x_1), \ldots, f(x_n)$.

Definition 1.3.11 (Divided Differences)**.** Divided differences of various orders are defined recursively as follows:

(i) **Zeroth-Order Divided Difference:**

$$f[x_i] = f(x_i)$$

(ii) **First-Order Divided Difference:**

$$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}$$

(iii) **Second-Order Divided Difference:**

$$f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}$$

(iv) ***k*-th Order Divided Difference:** In general, for $k \geq 1$:

$$f[x_i, x_{i+1}, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}$$

Theorem 1.3.12 (Symmetry Property)**.** *Divided differences are symmetric functions of their arguments. That is, the value of $f[x_0, x_1, \ldots, x_k]$ is completely independent of the order or permutation of the nodes $x_0, x_1, \ldots, x_k$.*

Theorem 1.3.13 (Relationship with Derivatives)**.** *Let $f \in C^k[a,b]$ and let $x_0, x_1, \ldots, x_k$ be $k+1$ distinct nodes in $[a,b]$. Then there exists a point $\xi \in (a,b)$ such that:*

$$f[x_0, x_1, \ldots, x_k] = \frac{f^{(k)}(\xi)}{k!}$$

Downloaded from – https://aijaz.vercel.app

Proof. Let $P_k(x)$ be the polynomial of degree $\leq k$ interpolating $f(x)$ at $x_0, x_1, \ldots, x_k$. By the Interpolation Error Theorem, for any $x \in [a, b]$:

$$f(x) - P_k(x) = \frac{f^{(k+1)}(\xi_x)}{(k+1)!} \prod_{j=0}^{k} (x - x_j)$$

Comparing this error term with the Newton polynomial expansion $f(x) - P_k(x) = f[x, x_0, x_1, \ldots, x_k] \prod_{j=0}^{k} (x - x_j)$ yields:

$$f[x, x_0, x_1, \ldots, x_k] = \frac{f^{(k+1)}(\xi_x)}{(k+1)!}$$

Replacing $k+1$ by k gives $f[x_0, x_1, \ldots, x_k] = \frac{f^{(k)}(\xi)}{k!}$. ■

Corollary 1.3.14 (Divided Differences of Polynomials). *If $f(x)$ is a polynomial of degree n, then:*

(i) Its n-th divided difference $f[x_0, x_1, \ldots, x_n]$ is a constant equal to the leading coefficient a_n.

(ii) All higher-order divided differences $f[x_0, x_1, \ldots, x_k]$ for $k > n$ are identically zero.

Newton's Divided Difference Formula

From the definition of first-order divided difference, $f[x, x_0] = \frac{f(x) - f(x_0)}{x - x_0}$. Rearranging terms gives:

$$f(x) = f(x_0) + (x - x_0) f[x, x_0]$$

Similarly, from the second-order divided difference $f[x, x_0, x_1] = \frac{f[x, x_0] - f[x_0, x_1]}{x - x_1}$:

$$f[x, x_0] = f[x_0, x_1] + (x - x_1) f[x, x_0, x_1]$$

Substituting $f[x, x_0]$ back into the equation for $f(x)$:

$$f(x) = f(x_0) + f[x_0, x_1](x - x_0) + f[x, x_0, x_1](x - x_0)(x - x_1)$$

Continuing this process recursively n times yields **Newton's Divided Difference Formula**:

$$f(x) = P_n(x) + E_n(x)$$

where $P_n(x)$ is the **Newton Divided Difference Interpolating Polynomial**:

$$P_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots + f[x_0, x_1, \ldots, x_n] \prod_{j=0}^{n-1} (x - x_j)$$

and the truncation error $E_n(x)$ is expressed concisely as:

$$E_n(x) = f[x, x_0, x_1, \ldots, x_n] \prod_{j=0}^{n} (x - x_j)$$

Structure of Divided Difference Table

The computation of coefficients is systematically organized in a triangular **Divided Difference Table**:

x_i	$f[x_i]$	1st Difference	2nd Difference	3rd Difference
x_0	$\mathbf{f[x_0]}$			
		$\mathbf{f[x_0, x_1]}$		
x_1	$f[x_1]$		$\mathbf{f[x_0, x_1, x_2]}$	
		$f[x_1, x_2]$		$\mathbf{f[x_0, x_1, x_2, x_3]}$
x_2	$f[x_2]$		$f[x_1, x_2, x_3]$	
		$f[x_2, x_3]$		
x_3	$f[x_3]$			

The top diagonal coefficients (in bold) form the coefficients of $P_n(x)$.

Example 1.3.15. Construct the Newton divided difference table for the dataset:

x_i	-1	0	1	3
$f(x_i)$	3	-1	1	35

Find the interpolating polynomial $P_3(x)$ and evaluate $P_3(2)$.

Solution. Construct the divided difference table step-by-step:

- **1st Divided Differences:**

$$f[x_0, x_1] = \frac{-1-3}{0-(-1)} = \frac{-4}{1} = -4$$

$$f[x_1, x_2] = \frac{1-(-1)}{1-0} = \frac{2}{1} = 2$$

$$f[x_2, x_3] = \frac{35-1}{3-1} = \frac{34}{2} = 17$$

- **2nd Divided Differences:**

$$f[x_0, x_1, x_2] = \frac{2-(-4)}{1-(-1)} = \frac{6}{2} = 3$$

$$f[x_1, x_2, x_3] = \frac{17-2}{3-0} = \frac{15}{3} = 5$$

- **3rd Divided Difference:**

$$f[x_0, x_1, x_2, x_3] = \frac{5-3}{3-(-1)} = \frac{2}{4} = 0.5$$

Divided Difference Table:

x_i	$f[x_i]$	1st Diff	2nd Diff	3rd Diff
-1	**3**			
		$\mathbf{-4}$		
0	-1		**3**	
		2		**0.5**
1	1		5	
		17		
3	35			

Construct $P_3(x)$ using the top diagonal coefficients $f[x_0] = 3, f[x_0, x_1] = -4, f[x_0, x_1, x_2] = 3, f[x_0, x_1, x_2, x_3] = 0.5$:

$$P_3(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2)$$

Downloaded from – https://aijaz.vercel.app

$$P_3(x) = 3 - 4(x+1) + 3(x+1)(x-0) + 0.5(x+1)(x-0)(x-1)$$
$$P_3(x) = 3 - 4x - 4 + 3x^2 + 3x + 0.5x(x^2-1)$$
$$P_3(x) = 3x^2 - x - 1 + 0.5x^3 - 0.5x = 0.5x^3 + 3x^2 - 1.5x - 1$$

Evaluating at $x = 2$:

$$P_3(2) = 0.5(2)^3 + 3(2)^2 - 1.5(2) - 1 = 0.5(8) + 3(4) - 3 - 1 = 4 + 12 - 3 - 1 = 12$$

■

Example 1.3.16. Incremental Property of Newton Interpolation: Given the data points $(0,1), (1,2), (2,9)$, the corresponding interpolating polynomial is $P_2(x) = 3x^2 - 2x + 1$. Suppose a fourth point $(3,28)$ is added to the dataset. Construct $P_3(x)$ efficiently without recomputing $P_2(x)$.

Solution. Construct the extended divided difference table incorporating the new node $x_3 = 3, y_3 = 28$:

x_i	$f[x_i]$	1st Diff	2nd Diff	3rd Diff
0	**1**			
		1		
1	2		**3**	
		7		**1**
2	9		6	
		19		
3	28			

Where:

$$f[2,3] = \frac{28-9}{3-2} = 19, \quad f[1,2,3] = \frac{19-7}{3-1} = 6, \quad f[0,1,2,3] = \frac{6-3}{3-0} = 1$$

Using the Newton nested formula $P_3(x) = P_2(x) + f[x_0, x_1, x_2, x_3](x - x_0)(x - x_1)(x - x_2)$:

$$P_3(x) = (3x^2 - 2x + 1) + 1 \cdot (x-0)(x-1)(x-2)$$
$$P_3(x) = 3x^2 - 2x + 1 + x(x^2 - 3x + 2) = 3x^2 - 2x + 1 + x^3 - 3x^2 + 2x = x^3 + 1$$

Notice how $P_3(x)$ was constructed simply by adding one new term to $P_2(x)$. ■

Example 1.3.17. Construct a Newton divided difference polynomial $P_3(x)$ to approximate $f(x) = e^x$ using the node values:

$$x_0 = 0, \quad x_1 = 0.2, \quad x_2 = 0.4, \quad x_3 = 0.6$$

Estimate $e^{0.3}$ and compute the absolute error against the true value $e^{0.3} \approx 1.349859$.

Solution. Evaluate $f(x_i) = e^{x_i}$:

$$f(0) = 1.000000, \quad f(0.2) = 1.221403, \quad f(0.4) = 1.491825, \quad f(0.6) = 1.822119$$

Build the divided difference table:

x_i	$f[x_i]$	1st Diff	2nd Diff	3rd Diff
0.0	**1.000000**			
		1.107015		
0.2	1.221403		**0.612738**	
		1.352110		**0.226103**
0.4	1.491825		0.748400	
		1.651470		
0.6	1.822119			

Downloaded from – https://aijaz.vercel.app

Calculations:

$$f[x_0, x_1] = \frac{1.221403 - 1.000000}{0.2} = 1.107015, \quad f[x_1, x_2] = \frac{1.491825 - 1.221403}{0.2} = 1.352110$$

$$f[x_2, x_3] = \frac{1.822119 - 1.491825}{0.2} = 1.651470$$

$$f[x_0, x_1, x_2] = \frac{1.352110 - 1.107015}{0.4 - 0.0} = 0.612738, \quad f[x_1, x_2, x_3] = \frac{1.651470 - 1.352110}{0.6 - 0.2} = 0.748400$$

$$f[x_0, x_1, x_2, x_3] = \frac{0.748400 - 0.612738}{0.6 - 0.0} = 0.226103$$

Construct $P_3(x)$:

$$P_3(x) = 1.0 + 1.107015(x) + 0.612738x(x - 0.2) + 0.226103x(x - 0.2)(x - 0.4)$$

Evaluating $P_3(0.3)$:

$$P_3(0.3) = 1.0 + 1.107015(0.3) + 0.612738(0.3)(0.1) + 0.226103(0.3)(0.1)(-0.1)$$

$$P_3(0.3) = 1.0 + 0.332105 + 0.018382 - 0.000678 = 1.349809$$

Comparing with true value $e^{0.3} \approx 1.349859$:

$$\text{Absolute Error } = |1.349859 - 1.349809| = 0.000050 \quad (5 \times 10^{-5})$$

■

Example 1.3.18. A dataset generated by a polynomial $P(x)$ of degree 2 contains an unknown missing value M at $x = 3$:

x	0	1	2	3	4
y	1	2	5	M	17

Use the properties of divided differences to determine the value of M.

Solution. Since the data is generated by a polynomial of degree 2, its 3rd-order divided differences must be identically zero ($f[x_0, x_1, x_2, x_3] = 0$).

Compute the divided differences in terms of M:

- **1st Differences:**

$$f[0, 1] = \frac{2 - 1}{1 - 0} = 1, \quad f[1, 2] = \frac{5 - 2}{2 - 1} = 3, \quad f[2, 3] = \frac{M - 5}{3 - 2} = M - 5$$

- **2nd Differences:**

$$f[0, 1, 2] = \frac{3 - 1}{2 - 0} = 1$$

$$f[1, 2, 3] = \frac{(M - 5) - 3}{3 - 1} = \frac{M - 8}{2}$$

- **3rd Difference:**

$$f[0, 1, 2, 3] = \frac{f[1, 2, 3] - f[0, 1, 2]}{3 - 0} = \frac{\frac{M-8}{2} - 1}{3} = \frac{M - 8 - 2}{6} = \frac{M - 10}{6}$$

Setting $f[0, 1, 2, 3] = 0$:

$$\frac{M - 10}{6} = 0 \implies M - 10 = 0 \implies M = 10$$

(Verification: The underlying polynomial is $P_2(x) = x^2 + 1$, which gives $P_2(3) = 3^2 + 1 = 10$). ■

Downloaded from – https://aijaz.vercel.app

Exercise 1.3.19. Solve the following exercises.

1. Find the absolute error, relative error, and percentage error when the rational number $x = \frac{22}{7}$ is approximated by $\hat{x} = 3.1416$. Determine the number of correct significant digits in $\hat{x}$.

2. A physical quantity u is given by the expression:
$$u = \frac{x^2 y^3}{z^4}$$
If the measured values are $x = 2.0 \pm 0.01$, $y = 3.0 \pm 0.02$, and $z = 1.0 \pm 0.005$, calculate the nominal value of u, the maximum possible absolute error $|\Delta u|$, and the maximum relative error $E_r(u)$.

3. Derive the Taylor polynomial $P_3(x)$ of degree 3 for $f(x) = e^{-x}$ expanded about $x_0 = 0$. Use $P_3(0.2)$ to estimate $e^{-0.2}$, and determine a theoretical upper bound for the absolute truncation error using Lagrange's remainder formula.

4. Show that if an approximate number $\hat{x}$ has a relative error $E_r \leq \frac{1}{2k} \times 10^{-(d-1)}$, where $k \in \{1, 2, \ldots, 9\}$ is the leading non-zero digit of x, then $\hat{x}$ is correct to at least d significant digits.

5. An iterative method generates an error sequence $\{e_k\}$ satisfying $e_{k+1} = \frac{1}{4} e_k^3$ for $k \geq 0$. Determine the order of convergence p and the asymptotic error constant C of this method.

6. Using Taylor series expansions about x, prove that the second-order centered difference formula:
$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$
has a truncation error of order $\mathcal{O}(h^2)$. Find the leading term of the truncation error.

7. Show that the equation $f(x) = x^3 - 4x - 9 = 0$ has a unique real root in the interval $[2, 3]$. Perform 4 iterations of the Bisection Method to find the approximate root c_3, and state the maximum possible error bound.

8. Determine the minimum number of bisection iterations required to locate the root of $f(x) = xe^x - 2 = 0$ on the interval $[0, 1]$ with an absolute accuracy of $\epsilon = 10^{-5}$.

9. Consider the equation $x^3 + x - 1 = 0$ on the interval $[0.5, 1.0]$.
   (a) Test the convergence behavior of $g_1(x) = 1 - x^3$ and $g_2(x) = \frac{1}{1+x^2}$ by evaluating $|g'(x)|$ on $[0.5, 1.0]$.
   (b) Using the convergent formulation, perform 4 iterations starting with $x_0 = 0.5$.

10. Prove that the function $g(x) = \frac{1}{2} + \frac{1}{3} \sin x$ has a unique fixed point in $[0, 1]$. Perform 3 iterations of the scheme $x_{k+1} = g(x_k)$ starting with $x_0 = 0.5$.

11. Find a real root of $x = \cos x + 0.1$ correct to 3 decimal places using the Fixed-Point Iteration Method with $x_0 = 0.5$.

12. Use the Newton–Raphson method to find the positive real root of $f(x) = x^3 - 3x - 5 = 0$ near $x_0 = 2$ correct to 4 decimal places.

13. (a) Derive the Newton–Raphson iterative formula for finding the cube root $\sqrt[3]{N}$ of a positive real number $N > 0$.

Downloaded from – https://aijaz.vercel.app

(b) Use this formula to compute $\sqrt[3]{25}$ correct to 5 decimal places starting with $x_0 = 3.0$.

14. Apply Newton's method to find the root of $f(x) = (x-1)^2 e^x = 0$ starting at $x_0 = 2.0$. Perform 3 iterations and explain why the convergence rate is linear ($p = 1$) instead of quadratic ($p = 2$).

15. Construct the Lagrange interpolating polynomial $P_2(x)$ for the given dataset:

x_i	0	2	3
y_i	1	5	10

Evaluate $P_2(1)$ and confirm that $\sum_{k=0}^{2} L_k(1) = 1$.

16. Let $f(x) = \sqrt{x}$ be interpolated by a polynomial $P_2(x)$ at nodes $x_0 = 1, x_1 = 4, x_2 = 9$.

(a) Estimate $\sqrt{5} \approx P_2(5)$ using Lagrange interpolation.

(b) Use the Interpolation Error Theorem to compute a theoretical maximum bound for the error $|\sqrt{5} - P_2(5)|$, and compare it with the actual error.

17. **Inverse Interpolation:** Given the tabular data:

x	1	2	4
$y = f(x)$	3	7	19

Use Inverse Lagrange Interpolation to estimate the value of x when $y = 12$.

18. Construct the Newton divided difference table for the dataset:

x_i	-1	0	2	5
$f(x_i)$	10	3	7	220

Find the interpolating polynomial $P_3(x)$ and evaluate $P_3(1)$.

19. Given the dataset $(1, 2), (2, 5), (4, 17)$, the corresponding Newton polynomial is $P_2(x) = x^2 + 1$. A fourth data point $(5, 26)$ is added. Construct the new polynomial $P_3(x)$ using the incremental property of Newton's divided difference formula.

20. A second-degree polynomial $P_2(x)$ generates the following table with one missing entry M:

x	0	1	2	3	4
y	1	4	M	22	37

Using the property that the 3rd-order divided difference of a quadratic polynomial is identically zero ($f[x_0, x_1, x_2, x_3] = 0$), determine the exact value of M.

CHAPTER

2

Finite Differences, Splines, and Numerical Integration

2.1 Finite Difference Operators

In numerical analysis, finite differences form the foundational mathematical language for interpolating discrete data, approximating derivatives, and constructing numerical schemes for ordinary and partial differential equations. Throughout this chapter, we consider a discrete set of equispaced nodes:

$$x_i = x_0 + ih, \quad i = 0, 1, 2, \ldots, n$$

where x_0 is the initial node and $h > 0$ is the constant grid spacing or step size. The corresponding function values are denoted by $f_i = f(x_i)$.

2.1.1 Forward, Backward, Central, Shift, and Averaging Operators

To manipulate discrete function values systematically, we define six fundamental operators acting on a real-valued function $f(x)$.

Definition 2.1.1 (Forward Difference Operator Δ)**.** The **forward difference operator** Δ with step size h is defined by:

$$\Delta f(x) = f(x + h) - f(x)$$

For a discrete grid, $\Delta f_i = f_{i+1} - f_i$. Higher-order forward differences are defined recursively by:

$$\Delta^0 f(x) = f(x), \quad \Delta^k f(x) = \Delta\left(\Delta^{k-1} f(x)\right), \quad k \geq 1$$

Definition 2.1.2 (Backward Difference Operator ∇)**.** The **backward difference operator** ∇ with step size h is defined by:

$$\nabla f(x) = f(x) - f(x - h)$$

For a discrete grid, $\nabla f_i = f_i - f_{i-1}$. Higher-order backward differences are defined recursively by:

$$\nabla^0 f(x) = f(x), \quad \nabla^k f(x) = \nabla\left(\nabla^{k-1} f(x)\right), \quad k \geq 1$$

Definition 2.1.3 (Central Difference Operator δ)**.** The **central difference operator** δ with step

Downloaded from – https://aijaz.vercel.app

size h is defined by:

$$\delta f(x) = f\left(x + \frac{h}{2}\right) - f\left(x - \frac{h}{2}\right)$$

For a discrete grid, $\delta f_i = f_{i+1/2} - f_{i-1/2}$.

Definition 2.1.4 (Shift Operator E)**.** The **shift operator** E advances the argument of a function by one step size h:

$$Ef(x) = f(x + h)$$

In general, for any real power $n \in \mathbb{R}$:

$$E^n f(x) = f(x + nh)$$

Definition 2.1.5 (Averaging Operator μ)**.** The **averaging operator** μ computes the mean of a function at half-step shifts:

$$\mu f(x) = \frac{1}{2}\left[f\left(x + \frac{h}{2}\right) + f\left(x - \frac{h}{2}\right)\right]$$

Definition 2.1.6 (Differential Operator D)**.** The **differential operator** D represents the exact first derivative with respect to x:

$$Df(x) = \frac{d}{dx}f(x) = f'(x), \quad D^k f(x) = \frac{d^k}{dx^k}f(x)$$

2.1.2 Fundamental Operator Identities and Relations

The finite difference operators satisfy linear algebraic properties and can be expressed in terms of one another through operator identities.

Theorem 2.1.7 (Relations Between $E, \Delta, \nabla, \delta, \mu$)**.** *The finite difference operators satisfy the following fundamental operator equivalences:*

(i) $\Delta = E - 1 \iff E = 1 + \Delta$

(ii) $\nabla = 1 - E^{-1} \iff E = (1 - \nabla)^{-1}$

(iii) $\Delta E^{-1} = \nabla = E^{-1}\Delta$

(iv) $\delta = E^{1/2} - E^{-1/2} = \Delta E^{-1/2} = \nabla E^{1/2}$

(v) $\mu = \frac{1}{2}\left(E^{1/2} + E^{-1/2}\right)$

(vi) $\mu^2 = 1 + \frac{1}{4}\delta^2$

(vii) $\mu\delta = \frac{1}{2}(\Delta + \nabla)$

Proof. We prove parts (iii), (vi), and (vii):

Proof of (iii): Applying ΔE^{-1} to $f(x)$:

$$(\Delta E^{-1})f(x) = \Delta f(x - h) = f(x) - f(x - h) = \nabla f(x)$$

Downloaded from – https://aijaz.vercel.app

Similarly, $(E^{-1}\Delta)f(x) = E^{-1}[f(x+h) - f(x)] = f(x) - f(x-h) = \nabla f(x)$. Thus, $\Delta E^{-1} = \nabla = E^{-1}\Delta$.

Proof of (vi): Express μ and δ in terms of $E^{1/2}$ and $E^{-1/2}$:

$$\mu^2 = \left[\frac{1}{2}\left(E^{1/2} + E^{-1/2}\right)\right]^2 = \frac{1}{4}\left(E + 2 + E^{-1}\right)$$

$$1 + \frac{1}{4}\delta^2 = 1 + \frac{1}{4}\left(E^{1/2} - E^{-1/2}\right)^2 = 1 + \frac{1}{4}\left(E - 2 + E^{-1}\right) = \frac{4 + E - 2 + E^{-1}}{4} = \frac{E + 2 + E^{-1}}{4}$$

Hence, $\mu^2 = 1 + \frac{1}{4}\delta^2$.

Proof of (vii): Multiplying μ and δ:

$$\mu\delta = \frac{1}{2}\left(E^{1/2} + E^{-1/2}\right)\left(E^{1/2} - E^{-1/2}\right) = \frac{1}{2}\left(E - E^{-1}\right)$$

On the other hand, $\Delta + \nabla = (E-1) + (1 - E^{-1}) = E - E^{-1}$. Thus, $\mu\delta = \frac{1}{2}(\Delta + \nabla)$. ■

Theorem 2.1.8 (Connections with the Differential Operator D)**.** *Let $f(x)$ be an infinitely differentiable function. The shift operator E and the differential operator D are related exponentially by:*

$$E = e^{hD}$$

Consequently, the difference operators satisfy:

(i) $hD = \ln(1 + \Delta) = \Delta - \frac{\Delta^2}{2} + \frac{\Delta^3}{3} - \frac{\Delta^4}{4} + \dots$

(ii) $hD = -\ln(1 - \nabla) = \nabla + \frac{\nabla^2}{2} + \frac{\nabla^3}{3} + \frac{\nabla^4}{4} + \dots$

(iii) $hD = 2\sinh^{-1}\left(\frac{\delta}{2}\right)$

Proof. Applying Taylor's series expansion to $Ef(x) = f(x+h)$:

$$Ef(x) = f(x) + hf'(x) + \frac{h^2}{2!}f''(x) + \frac{h^3}{3!}f'''(x) + \cdots = \left(1 + hD + \frac{h^2D^2}{2!} + \frac{h^3D^3}{3!} + \dots\right) f(x)$$

Recognizing the power series definition of the exponential function $e^z = \sum_{k=0}^{\infty} \frac{z^k}{k!}$, we obtain:

$$Ef(x) = e^{hD}f(x) \implies E = e^{hD}$$

Since $E = 1 + \Delta$, we have $e^{hD} = 1 + \Delta$. Taking the natural logarithm:

$$hD = \ln(1 + \Delta)$$

Expanding $\ln(1 + \Delta)$ in a Maclaurin series gives $hD = \Delta - \frac{\Delta^2}{2} + \frac{\Delta^3}{3} - \frac{\Delta^4}{4} + \dots$ ■

2.1.3 Difference Tables and Polynomial Differences

Difference operations on discrete tabular data are organized systematically into **Difference Tables**.

Downloaded from – https://aijaz.vercel.app

Forward Difference Table Structure

For $n+1$ data points $(x_i, y_i)_{i=0}^{n}$, the forward difference table is formatted as follows:

x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$
x_0	y_0			
		$\Delta y_0 = y_1 - y_0$		
x_1	y_1		$\Delta^2 y_0 = \Delta y_1 - \Delta y_0$	
		$\Delta y_1 = y_2 - y_1$		$\Delta^3 y_0 = \Delta^2 y_1 - \Delta^2 y_0$
x_2	y_2		$\Delta^2 y_1 = \Delta y_2 - \Delta y_1$	
		$\Delta y_2 = y_3 - y_2$		
x_3	y_3			

Theorem 2.1.9 (Fundamental Theorem of Finite Differences). *If $P_n(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$ is a polynomial of degree n with leading coefficient $a_n \neq 0$, and the grid spacing $h > 0$ is constant, then:*

*(i) The n-th forward difference $\Delta^n P_n(x)$ is a **constant** given by:*

$$\Delta^n P_n(x) = a_n\, n!\, h^n$$

(ii) All higher-order differences are identically zero:

$$\Delta^k P_n(x) = 0 \quad \text{for all } k > n$$

Proof. Consider the action of Δ on a monomial x^k:

$$\Delta(x^k) = (x+h)^k - x^k = \left(x^k + khx^{k-1} + \binom{k}{2} h^2 x^{k-2} + \cdots + h^k\right) - x^k = khx^{k-1} + \mathcal{O}(x^{k-2})$$

Thus, the forward difference operator reduces the degree of a polynomial by exactly 1. Applying Δ n times to $P_n(x) = a_n x^n + \mathcal{O}(x^{n-1})$:

$$\Delta P_n(x) = a_n(nhx^{n-1}) + \mathcal{O}(x^{n-2})$$

$$\Delta^2 P_n(x) = a_n n(n-1)h^2 x^{n-2} + \mathcal{O}(x^{n-3})$$

$$\vdots$$

$$\Delta^n P_n(x) = a_n\, n(n-1)(n-2)\ldots(1)\, h^n x^0 = a_n\, n!\, h^n$$

Since a_n, $n!$, and h^n are constants, $\Delta^n P_n(x)$ is constant. Taking one more difference:

$$\Delta^{n+1} P_n(x) = \Delta\left(a_n n! h^n\right) = 0$$

■

Example 2.1.10. Prove the following operator identities:

(i) $\mu^2 = 1 + \frac{1}{4}\delta^2$

(ii) $\Delta - \nabla = \delta^2$

Downloaded from – https://aijaz.vercel.app

Solution. **Part (i):** Using operator definitions in terms of shift operator E:

$$\mu = \frac{1}{2}\left(E^{1/2} + E^{-1/2}\right), \quad \delta = E^{1/2} - E^{-1/2}$$

Squaring δ:

$$\delta^2 = \left(E^{1/2} - E^{-1/2}\right)^2 = E - 2 + E^{-1}$$

Squaring μ:

$$\mu^2 = \left[\frac{1}{2}\left(E^{1/2} + E^{-1/2}\right)\right]^2 = \frac{1}{4}\left(E + 2 + E^{-1}\right)$$

Evaluate the right-hand side $1 + \frac{1}{4}\delta^2$:

$$1 + \frac{1}{4}\delta^2 = 1 + \frac{1}{4}(E - 2 + E^{-1}) = \frac{4 + E - 2 + E^{-1}}{4} = \frac{E + 2 + E^{-1}}{4} = \mu^2$$

Hence, $\mu^2 = 1 + \frac{1}{4}\delta^2$.

Part (ii): Express Δ and ∇ in terms of E:

$$\Delta = E - 1, \quad \nabla = 1 - E^{-1}$$

Subtracting ∇ from Δ:

$$\Delta - \nabla = (E - 1) - (1 - E^{-1}) = E - 2 + E^{-1}$$

Since $\delta^2 = (E^{1/2} - E^{-1/2})^2 = E - 2 + E^{-1}$, we have:

$$\Delta - \nabla = \delta^2$$

■

Example 2.1.11. (i) Express $\Delta^n f(x)$ explicitly in terms of function values $f(x + kh)$ for $k = 0, 1, \ldots, n$.

(ii) Evaluate $\Delta^3(x^3 - 2x^2 + 5x - 7)$ with step size $h = 1$ both algebraically and using the Fundamental Theorem of Finite Differences.

Solution. **Part (i):** Use the operator relation $\Delta = E - 1$:

$$\Delta^n f(x) = (E - 1)^n f(x)$$

Expanding $(E - 1)^n$ using the Binomial Theorem:

$$(E - 1)^n = \sum_{k=0}^{n} \binom{n}{k} E^k (-1)^{n-k} = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} E^k$$

Applying this expansion to $f(x)$ (noting $E^k f(x) = f(x + kh)$):

$$\Delta^n f(x) = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} f(x + kh)$$

Part (ii): Let $P_3(x) = x^3 - 2x^2 + 5x - 7$ with $h = 1$.

- **Method 1 (Fundamental Theorem):** $P_3(x)$ is a polynomial of degree $n = 3$ with leading coefficient $a_3 = 1$. By the Fundamental Theorem of Finite Differences:

$$\Delta^3 P_3(x) = a_3\, 3!\, h^3 = (1)(6)(1)^3 = 6$$

Downloaded from – https://aijaz.vercel.app

- **Method 2 (Algebraic Verification):** Compute $\Delta P_3(x) = P_3(x+1) - P_3(x)$:

  $P_3(x+1) = (x+1)^3 - 2(x+1)^2 + 5(x+1) - 7 = x^3 + 3x^2 + 3x + 1 - 2x^2 - 4x - 2 + 5x + 5 - 7 = x^3 + x^2 + 4x$

  $$\Delta P_3(x) = (x^3 + x^2 + 4x - 3) - (x^3 - 2x^2 + 5x - 7) = 3x^2 - x + 4$$

  Compute $\Delta^2 P_3(x) = \Delta(3x^2 - x + 4)$:

  $\Delta^2 P_3(x) = [3(x+1)^2 - (x+1) + 4] - [3x^2 - x + 4] = (3x^2 + 6x + 3 - x - 1 + 4) - (3x^2 - x + 4) = 6x + 2$

  Compute $\Delta^3 P_3(x) = \Delta(6x + 2)$:

  $$\Delta^3 P_3(x) = [6(x+1) + 2] - [6x + 2] = 6x + 8 - 6x - 2 = 6$$

Both methods yield $\Delta^3 P_3(x) = 6$. ■

Example 2.1.12. Construct the forward difference table for the following data:

x_i	0	1	2	3	4
y_i	1	2	11	34	77

Show that the third-order differences are constant and find the underlying polynomial $y = f(x)$.

Solution. Compute forward differences step-by-step:

- **1st Differences (Δy_i):**

  $$\Delta y_0 = 2 - 1 = 1, \quad \Delta y_1 = 11 - 2 = 9, \quad \Delta y_2 = 34 - 11 = 23, \quad \Delta y_3 = 77 - 34 = 43$$

- **2nd Differences ($\Delta^2 y_i$):**

  $$\Delta^2 y_0 = 9 - 1 = 8, \quad \Delta^2 y_1 = 23 - 9 = 14, \quad \Delta^2 y_2 = 43 - 23 = 20$$

- **3rd Differences ($\Delta^3 y_i$):**

  $$\Delta^3 y_0 = 14 - 8 = 6, \quad \Delta^3 y_1 = 20 - 14 = 6$$

- **4th Difference ($\Delta^4 y_i$):**

  $$\Delta^4 y_0 = 6 - 6 = 0$$

Forward Difference Table:

x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$
0	**1**				
		1			
1	2		**8**		
		9		**6**	
2	11		14		**0**
		23		6	
3	34		20		
		43			
4	77				

Since $\Delta^3 y_i = 6$ is constant, the underlying function is a polynomial of degree $n = 3$.

Reconstruct $f(x)$ using $y(x) = E^x y_0 = (1+\Delta)^x y_0$ with $h = 1, x_0 = 0$:

$$f(x) = y_0 + x\Delta y_0 + \frac{x(x-1)}{2!}\Delta^2 y_0 + \frac{x(x-1)(x-2)}{3!}\Delta^3 y_0$$

$$f(x) = 1 + x(1) + \frac{x(x-1)}{2}(8) + \frac{x(x-1)(x-2)}{6}(6)$$

$$f(x) = 1 + x + 4x(x-1) + x(x-1)(x-2) = 1 + x + 4x^2 - 4x + x(x^2 - 3x + 2)$$

$$f(x) = 1 - 3x + 4x^2 + x^3 - 3x^2 + 2x = x^3 + x^2 - x + 1$$

■

Example 2.1.13. (i) Derive the finite difference expression for the first derivative $f'(x_0)$ in terms of forward differences up to Δ^3.

(ii) Estimate $f'(1)$ for the dataset:

x_i	1	2	3	4
$f(x_i)$	2	9	28	65

Solution. **Part (i):** From the operator identity $hD = \ln(1+\Delta)$:

$$D = \frac{1}{h}\ln(1+\Delta) = \frac{1}{h}\left(\Delta - \frac{\Delta^2}{2} + \frac{\Delta^3}{3} - \frac{\Delta^4}{4} + \dots\right)$$

Truncating after third differences gives the forward difference approximation for the derivative:

$$f'(x_0) = Df(x_0) \approx \frac{1}{h}\left(\Delta y_0 - \frac{\Delta^2 y_0}{2} + \frac{\Delta^3 y_0}{3}\right)$$

Part (ii): Step size $h = 2 - 1 = 1$. Compute forward differences at $x_0 = 1$:

$$\begin{aligned} \Delta y_0 &= 9 - 2 = 7, & \Delta y_1 &= 28 - 9 = 19, & \Delta y_2 &= 65 - 28 = 37 \\ \Delta^2 y_0 &= 19 - 7 = 12, & \Delta^2 y_1 &= 37 - 19 = 18 \\ \Delta^3 y_0 &= 18 - 12 = 6 \end{aligned}$$

Substituting into the derivative formula:

$$f'(1) \approx \frac{1}{1}\left(7 - \frac{12}{2} + \frac{6}{3}\right) = 7 - 6 + 2 = 3$$

(Verification: The underlying function is $f(x) = x^3 + 1$. Its exact derivative is $f'(x) = 3x^2 \implies f'(1) = 3(1)^2 = 3$). ■

2.2 Difference Interpolation Formulas

When data nodes $x_0, x_1, \dots, x_n$ are equally spaced with a constant step size $h = x_{i+1} - x_i$, the general Lagrange and Newton divided difference polynomials simplify significantly into **Difference Interpolation Formulas**. Depending on the position of the desired interpolation point x within the tabular dataset, specific formulas are preferred:

- **Newton's Forward Difference Formula:** Ideal for interpolating near the *beginning* of a tabular dataset.
- **Newton's Backward Difference Formula:** Ideal for interpolating near the *end* of a tabular dataset.
- **Central Difference Formulas (Gauss, Stirling, Bessel):** Ideal for interpolating near the *middle* of a tabular dataset.

Downloaded from – https://aijaz.vercel.app

2.2.1 Newton's Forward Difference Interpolation

Let $y = f(x)$ be a function given at $n+1$ equispaced nodes $x_0, x_1, \ldots, x_n$ with constant step size $h > 0$. Any arbitrary interpolation point x can be parameterized in terms of a dimensionless variable u defined by:

$$x = x_0 + uh \implies u = \frac{x - x_0}{h}$$

where $u \in \mathbb{R}$ represents the number of step sizes x lies away from the initial node x_0.

Derivation Using Shift Operator Algebra

Using the shift operator E and its equivalence $E = 1 + \Delta$:

$$f(x) = f(x_0 + uh) = E^u f(x_0) = (1 + \Delta)^u y_0$$

Expanding $(1 + \Delta)^u$ using Newton's binomial series expansion for any real power u:

$$(1 + \Delta)^u = 1 + u\Delta + \frac{u(u-1)}{2!}\Delta^2 + \frac{u(u-1)(u-2)}{3!}\Delta^3 + \cdots + \frac{u(u-1)\ldots(u-k+1)}{k!}\Delta^k + \ldots$$

Truncating this infinite series after the n-th difference term gives the n-th degree **Newton Forward Difference Interpolating Polynomial** $P_n(x)$:

Definition 2.2.1 (Newton's Forward Difference Interpolation Formula)**.** Let $y_0, y_1, \ldots, y_n$ be function values at equispaced nodes $x_i = x_0 + ih$. The **Newton forward difference polynomial** $P_n(x)$ is given by:

$$P_n(x_0 + uh) = y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \frac{u(u-1)(u-2)}{3!}\Delta^3 y_0 + \cdots + \frac{\prod_{j=0}^{n-1}(u-j)}{n!}\Delta^n y_0$$

where $u = \frac{x - x_0}{h}$ and $\Delta^k y_0$ are the leading forward differences taken from the top diagonal of the forward difference table.

Theorem 2.2.2 (Newton's Forward Interpolation Error)**.** *Let $f \in C^{n+1}[a, b]$ and let $P_n(x)$ be the Newton forward difference polynomial interpolating $f(x)$ at $n+1$ equispaced nodes $x_i = x_0 + ih \in [a, b]$. Then the truncation error $E_n(x) = f(x) - P_n(x)$ at $x = x_0 + uh$ is given by:*

$$E_n(x) = f(x) - P_n(x) = \frac{u(u-1)(u-2)\ldots(u-n)}{(n+1)!} h^{n+1} f^{(n+1)}(\xi)$$

for some point $\xi \in (a, b)$ depending on x.

Proof. By the General Interpolation Error Theorem, for $n+1$ nodes $x_0, x_1, \ldots, x_n$:

$$E_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{j=0}^{n} (x - x_j)$$

Substitute $x = x_0 + uh$ and $x_j = x_0 + jh$:

$$x - x_j = (x_0 + uh) - (x_0 + jh) = (u - j)h$$

Taking the product over $j = 0, 1, \ldots, n$:

$$\prod_{j=0}^{n} (x - x_j) = \prod_{j=0}^{n} (u - j)h = h^{n+1} u(u-1)(u-2)\ldots(u-n)$$

Downloaded from – https://aijaz.vercel.app

Substituting this product into the error expression yields:

$$E_n(x) = \frac{u(u-1)(u-2)\dots(u-n)}{(n+1)!}h^{n+1}f^{(n+1)}(\xi)$$

■

Example 2.2.3. The following table gives values of $y = \sin(x^\circ)$ for equispaced angles x:

x°	10°	20°	30°	40°
y	0.1736	0.3420	0.5000	0.6428

Use Newton's Forward Difference Interpolation Formula to estimate $\sin(15^\circ)$.

Solution. Here $x_0 = 10^\circ$, step size $h = 10^\circ$, and target $x = 15^\circ$. Calculate the parameter u:

$$u = \frac{x - x_0}{h} = \frac{15 - 10}{10} = \frac{5}{10} = 0.5$$

Construct the forward difference table:

x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$
10°	**0.1736**			
		0.1684		
20°	0.3420		$-$**0.0104**	
		0.1580		$-$**0.0048**
30°	0.5000		-0.0152	
		0.1428		
40°	0.6428			

Leading differences are:

$$y_0 = 0.1736, \quad \Delta y_0 = 0.1684, \quad \Delta^2 y_0 = -0.0104, \quad \Delta^3 y_0 = -0.0048$$

Apply Newton's forward difference formula for $n = 3$:

$$P_3(15^\circ) = y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \frac{u(u-1)(u-2)}{3!}\Delta^3 y_0$$

Compute coefficient products with $u = 0.5$:

- $u = 0.5$
- $\frac{u(u-1)}{2} = \frac{(0.5)(-0.5)}{2} = -0.125$
- $\frac{u(u-1)(u-2)}{6} = \frac{(0.5)(-0.5)(-1.5)}{6} = \frac{0.375}{6} = +0.0625$

Substitute back into $P_3(15^\circ)$:

$$P_3(15^\circ) = 0.1736 + (0.5)(0.1684) + (-0.125)(-0.0104) + (0.0625)(-0.0048)$$

$$P_3(15^\circ) = 0.1736 + 0.0842 + 0.00130 - 0.00030 = 0.2588$$

(Comparing with exact value $\sin(15^\circ) \approx 0.258819$, the estimate is accurate to 4 decimal places). ■

Downloaded from – https://aijaz.vercel.app

Example 2.2.4. Given the tabular values of a function $f(x)$:

x	0	1	2	3	4
$f(x)$	1	3	7	13	21

(i) Construct the forward difference table and find the explicit polynomial $P_2(x)$.

(ii) Evaluate $f(0.5)$.

Solution. **Part (i):** Step size $h = 1$ and $x_0 = 0$. Construct the forward difference table:

x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$
0	**1**			
		2		
1	3		**2**	
		4		**0**
2	7		2	
		6		
3	13			
		8		
4	21			

Since $\Delta^2 y_i = 2$ is constant and $\Delta^3 y_0 = 0$, $f(x)$ is a 2nd-degree polynomial.

Set $u = \frac{x - x_0}{h} = \frac{x-0}{1} = x$. Substituting into Newton's forward formula:

$$P_2(x) = y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 = 1 + x(2) + \frac{x(x-1)}{2}(2)$$

$$P_2(x) = 1 + 2x + x^2 - x = x^2 + x + 1$$

Part (ii): Evaluate at $x = 0.5$:

$$P_2(0.5) = (0.5)^2 + (0.5) + 1 = 0.25 + 0.5 + 1 = 1.75$$

■

Example 2.2.5. The population P of a city (in thousands) recorded during decennial census years is given below:

Year (x)	1991	2001	2011	2021
Population (y)	46	66	81	93

Estimate the population of the city in the year 1995 using Newton's Forward Difference Interpolation.

Solution. Here $x_0 = 1991$, step size $h = 10$ years, and target year $x = 1995$. Compute u:

$$u = \frac{1995 - 1991}{10} = \frac{4}{10} = 0.4$$

Construct the forward difference table:

x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$
1991	**46**			
		20		
2001	66		$-$**5**	
		15		**2**
2011	81		-3	
		12		
2021	93			

Leading differences: $y_0 = 46, \Delta y_0 = 20, \Delta^2 y_0 = -5, \Delta^3 y_0 = 2$. Substitute into $P_3(x)$:

$$P_3(1995) = y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \frac{u(u-1)(u-2)}{3!}\Delta^3 y_0$$

Compute coefficient terms for $u = 0.4$:

- $u = 0.4$
- $\frac{u(u-1)}{2} = \frac{(0.4)(-0.6)}{2} = -0.12$
- $\frac{u(u-1)(u-2)}{6} = \frac{(0.4)(-0.6)(-1.6)}{6} = \frac{0.384}{6} = 0.064$

Calculate $P_3(1995)$:

$$P_3(1995) = 46 + (0.4)(20) + (-0.12)(-5) + (0.064)(2)$$

$$P_3(1995) = 46 + 8.0 + 0.60 + 0.128 = 54.728 \text{ thousand}$$

The estimated population in 1995 is approximately 54,728 inhabitants. ■

Example 2.2.6. Consider $f(x) = \ln x$ tabulated at equispaced nodes $x = 10, 11, 12, 13$:

x_i	10	11	12	13
$\ln x_i$	2.302585	2.397895	2.484907	2.564949

(i) Estimate $\ln(10.4)$ using Newton's forward difference polynomial of degree 3.

(ii) Find a theoretical bound for the truncation error $|E_3(10.4)|$ and compare it with the actual error.

Solution. **Part (i):** Here $x_0 = 10, h = 1$, and $x = 10.4 \implies u = \frac{10.4-10}{1} = 0.4$.

Construct the forward difference table:

x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$
10	**2.302585**			
		0.095310		
11	2.397895		$-$**0.008298**	
		0.087012		**0.001328**
12	2.484907		-0.006970	
		0.080042		
13	2.564949			

Substitute $u = 0.4$ and differences into $P_3(10.4)$:

$$P_3(10.4) = 2.302585 + (0.4)(0.095310) + \frac{(0.4)(-0.6)}{2}(-0.008298) + \frac{(0.4)(-0.6)(-1.6)}{6}(0.001328)$$

Downloaded from – https://aijaz.vercel.app

$$P_3(10.4) = 2.302585 + 0.038124 + 0.000996 + 0.000085 = 2.341790$$

Part (ii): The truncation error formula for $n = 3$ is:

$$E_3(10.4) = \frac{u(u-1)(u-2)(u-3)}{4!} h^4 f^{(4)}(\xi)$$

For $u = 0.4$:

$$u(u-1)(u-2)(u-3) = (0.4)(-0.6)(-1.6)(-2.6) = -0.9984$$

For $f(x) = \ln x$, its 4th derivative is $f^{(4)}(x) = -6x^{-4}$. On $\xi \in [10, 13]$, the maximum bound on $|f^{(4)}(\xi)|$ occurs at $\xi = 10$:

$$\max_{\xi \in [10,13]} |f^{(4)}(\xi)| = \frac{6}{10^4} = 0.0006$$

Computing the error bound (with $h = 1$):

$$|E_3(10.4)| \leq \frac{0.9984}{24}(1)^4(0.0006) = (0.0416)(0.0006) \approx 0.000025$$

Comparing with true value $\ln(10.4) \approx 2.341806$:

$$\text{Actual Error } = |2.341806 - 2.341790| = 0.000016$$

The actual error 0.000016 is strictly within the derived maximum bound of 0.000025. ■

2.2.2 Newton's Backward Difference Interpolation

When an interpolation point x lies near the *end* of a tabular dataset of equispaced nodes $x_0, x_1, \ldots, x_n$, using forward differences becomes inconvenient because forward differences would require unavailable data beyond the boundary. In such cases, **Newton's Backward Difference Interpolation** is the preferred method.

Let $y_0, y_1, \ldots, y_n$ be function values at equispaced nodes $x_i = x_0 + ih$ with constant step size $h > 0$. We parameterize any interpolation point x relative to the final node x_n using a dimensionless variable v:

$$x = x_n + vh \implies v = \frac{x - x_n}{h}$$

Notice that for points x located within the tabular interval $[x_0, x_n]$ near x_n, the parameter v takes on negative values (e.g., $x \in [x_{n-1}, x_n] \implies v \in [-1, 0]$).

Derivation Using Shift Operator Algebra

Using the shift operator E and its backward difference relation $E^{-1} = 1 - \nabla \implies E = (1 - \nabla)^{-1}$:

$$f(x) = f(x_n + vh) = E^v f(x_n) = (1 - \nabla)^{-v} y_n$$

Expanding $(1 - \nabla)^{-v}$ using Newton's binomial expansion for negative powers:

$$(1 - \nabla)^{-v} = 1 + (-v)(-\nabla) + \frac{(-v)(-v-1)}{2!}(-\nabla)^2 + \frac{(-v)(-v-1)(-v-2)}{3!}(-\nabla)^3 + \ldots$$

Simplifying the minus signs in each term yields:

$$(1 - \nabla)^{-v} = 1 + v\nabla + \frac{v(v+1)}{2!}\nabla^2 + \frac{v(v+1)(v+2)}{3!}\nabla^3 + \cdots + \frac{\prod_{j=0}^{k-1}(v+j)}{k!}\nabla^k + \ldots$$

Truncating this infinite series after the n-th backward difference gives the n-th degree **Newton Backward Difference Interpolating Polynomial** $P_n(x)$:

Downloaded from – https://aijaz.vercel.app

Definition 2.2.7 (Newton's Backward Difference Interpolation Formula)**.** Let $y_0, y_1, \ldots, y_n$ be function values at equispaced nodes $x_i = x_0 + ih$. The **Newton backward difference polynomial** $P_n(x)$ evaluated at $x = x_n + vh$ is given by:

$$P_n(x_n + vh) = y_n + v\nabla y_n + \frac{v(v+1)}{2!}\nabla^2 y_n + \frac{v(v+1)(v+2)}{3!}\nabla^3 y_n + \cdots + \frac{\prod_{j=0}^{n-1}(v+j)}{n!}\nabla^n y_n$$

where $v = \frac{x - x_n}{h}$ and $\nabla^k y_n$ are the bottom diagonal backward differences taken from the backward difference table.

Theorem 2.2.8 (Newton's Backward Interpolation Error)**.** *Let $f \in C^{n+1}[a,b]$ and let $P_n(x)$ be the Newton backward difference polynomial interpolating $f(x)$ at $n+1$ equispaced nodes $x_i = x_0 + ih \in [a,b]$. Then the truncation error $E_n(x) = f(x) - P_n(x)$ at $x = x_n + vh$ is given by:*

$$E_n(x) = f(x) - P_n(x) = \frac{v(v+1)(v+2)\ldots(v+n)}{(n+1)!}h^{n+1}f^{(n+1)}(\xi)$$

for some point $\xi \in (a,b)$ depending on x.

Proof. By the General Interpolation Error Theorem, for $n+1$ nodes $x_0, x_1, \ldots, x_n$:

$$E_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}\prod_{j=0}^{n}(x - x_j)$$

Expressing x_j in terms of x_n: $x_j = x_n - (n-j)h$. Substituting $x = x_n + vh$:

$$x - x_j = (x_n + vh) - (x_n - (n-j)h) = (v + n - j)h$$

Taking the product over $j = 0, 1, \ldots, n$:

$$\prod_{j=0}^{n}(x - x_j) = \prod_{j=0}^{n}(v+n-j)h = h^{n+1}v(v+1)(v+2)\ldots(v+n)$$

Substituting this product into the error formula yields:

$$E_n(x) = \frac{v(v+1)(v+2)\ldots(v+n)}{(n+1)!}h^{n+1}f^{(n+1)}(\xi)$$

■

Structure of Backward Difference Table

Backward differences are calculated identically to forward differences, but are indexed relative to the trailing node x_n:

x_i	y_i	∇y_i	$\nabla^2 y_i$	$\nabla^3 y_i$
x_0	y_0			
		$\nabla y_1 = y_1 - y_0$		
x_1	y_1		$\nabla^2 y_2 = \nabla y_2 - \nabla y_1$	
		$\nabla y_2 = y_2 - y_1$		$\nabla^3 y_3 = \nabla^2 y_3 - \nabla^2 y_2$
x_2	y_2		$\nabla^2 \mathbf{y_3} = \nabla \mathbf{y_3} - \nabla \mathbf{y_2}$	
		$\nabla \mathbf{y_3} = \mathbf{y_3} - \mathbf{y_2}$		$\nabla^3 \mathbf{y_3}$
x_3	$\mathbf{y_3}$			

The bottom diagonal entries (in bold) supply the coefficients for Newton's backward difference formula.

Downloaded from – https://aijaz.vercel.app

Example 2.2.9. The following table provides values of $y = \sin(x^\circ)$ for equispaced angles x:

x°	10°	20°	30°	40°
y	0.1736	0.3420	0.5000	0.6428

Use Newton's Backward Difference Interpolation Formula to estimate $\sin(38^\circ)$.

Solution. Since $x = 38^\circ$ lies near the end of the table ($x_n = 40^\circ$), we use Newton's backward formula with $x_n = 40^\circ$ and step size $h = 10^\circ$. Compute the dimensionless variable v:

$$v = \frac{x - x_n}{h} = \frac{38 - 40}{10} = \frac{-2}{10} = -0.2$$

Construct the backward difference table:

x_i	y_i	∇y_i	$\nabla^2 y_i$	$\nabla^3 y_i$
10°	0.1736			
		0.1684		
20°	0.3420		-0.0104	
		0.1580		$\mathbf{-0.0048}$
30°	0.5000		$\mathbf{-0.0152}$	
		0.1428		
40°	**0.6428**			

Trailing differences from the bottom diagonal are:

$$y_3 = 0.6428, \quad \nabla y_3 = 0.1428, \quad \nabla^2 y_3 = -0.0152, \quad \nabla^3 y_3 = -0.0048$$

Substitute into Newton's backward difference formula for $n = 3$:

$$P_3(38^\circ) = y_3 + v\nabla y_3 + \frac{v(v+1)}{2!}\nabla^2 y_3 + \frac{v(v+1)(v+2)}{3!}\nabla^3 y_3$$

Calculate coefficients with $v = -0.2$:

- $v = -0.2$
- $\frac{v(v+1)}{2} = \frac{(-0.2)(0.8)}{2} = -0.08$
- $\frac{v(v+1)(v+2)}{6} = \frac{(-0.2)(0.8)(1.8)}{6} = \frac{-0.288}{6} = -0.048$

Substitute coefficients into $P_3(38^\circ)$:

$$P_3(38^\circ) = 0.6428 + (-0.2)(0.1428) + (-0.08)(-0.0152) + (-0.048)(-0.0048)$$

$$P_3(38^\circ) = 0.6428 - 0.02856 + 0.001216 + 0.000230 = 0.615686 \approx 0.6157$$

(Comparing with exact value $\sin(38^\circ) \approx 0.615661$, the result is accurate to 4 decimal places). ■

Example 2.2.10. Given tabular data for $y = x^3$:

x	1	2	3	4	5
y	1	8	27	64	125

Use Newton's Backward Difference Interpolation Formula to construct the polynomial $P_3(x)$ using the last four nodes, and evaluate $P_3(4.5)$.

Downloaded from – https://aijaz.vercel.app

Solution. Here $x_n = 5$, step size $h = 1$, and target $x = 4.5 \implies v = \frac{4.5-5}{1} = -0.5$.

Construct the backward difference table:

x_i	y_i	∇y_i	$\nabla^2 y_i$	$\nabla^3 y_i$	$\nabla^4 y_i$
1	1				
		7			
2	8		12		
		19		6	
3	27		18		**0**
		37		**6**	
4	64		**24**		
		61			
5	**125**				

Trailing differences at $x_4 = 5$:

$$y_4 = 125, \quad \nabla y_4 = 61, \quad \nabla^2 y_4 = 24, \quad \nabla^3 y_4 = 6$$

Substitute into Newton's backward difference formula for $n = 3$:

$$P_3(x_n + vh) = y_4 + v\nabla y_4 + \frac{v(v+1)}{2!}\nabla^2 y_4 + \frac{v(v+1)(v+2)}{3!}\nabla^3 y_4$$

Evaluate $P_3(4.5)$ with $v = -0.5$:

- $v = -0.5$
- $\frac{v(v+1)}{2} = \frac{(-0.5)(0.5)}{2} = -0.125$
- $\frac{v(v+1)(v+2)}{6} = \frac{(-0.5)(0.5)(1.5)}{6} = \frac{-0.375}{6} = -0.0625$

Calculate $P_3(4.5)$:

$$P_3(4.5) = 125 + (-0.5)(61) + (-0.125)(24) + (-0.0625)(6)$$

$$P_3(4.5) = 125 - 30.5 - 3.0 - 0.375 = 91.125$$

(Verification: Exact value $(4.5)^3 = 91.125$. The result is exact because $y = x^3$ is a 3rd-degree polynomial). ■

Example 2.2.11. The population P of a country (in millions) recorded during decennial censuses is given below:

Year (x)	1981	1991	2001	2011	2021
Population (y)	68	84	102	121	142

Estimate the population of the country in the year 2018 using Newton's Backward Difference Interpolation Formula.

Solution. Target year $x = 2018$ lies near the end of the dataset ($x_n = 2021$). Step size $h = 10$ years. Compute v:

$$v = \frac{2018 - 2021}{10} = \frac{-3}{10} = -0.3$$

Downloaded from – https://aijaz.vercel.app

Construct the backward difference table:

x_i	y_i	∇y_i	$\nabla^2 y_i$	$\nabla^3 y_i$	$\nabla^4 y_i$
1981	68				
		16			
1991	84		2		
		18		-1	
2001	102		1		**2**
		19		**1**	
2011	121		**2**		
		21			
2021	**142**				

Trailing differences from the bottom diagonal:

$$y_4 = 142, \quad \nabla y_4 = 21, \quad \nabla^2 y_4 = 2, \quad \nabla^3 y_4 = 1, \quad \nabla^4 y_4 = 2$$

Substitute into Newton's backward difference formula for $n = 4$:

$$P_4(2018) = y_4 + v\nabla y_4 + \frac{v(v+1)}{2!}\nabla^2 y_4 + \frac{v(v+1)(v+2)}{3!}\nabla^3 y_4 + \frac{v(v+1)(v+2)(v+3)}{4!}\nabla^4 y_4$$

Calculate coefficients with $v = -0.3$:

- $v = -0.3$
- $\frac{v(v+1)}{2} = \frac{(-0.3)(0.7)}{2} = -0.105$
- $\frac{v(v+1)(v+2)}{6} = \frac{(-0.3)(0.7)(1.7)}{6} = \frac{-0.357}{6} = -0.0595$
- $\frac{v(v+1)(v+2)(v+3)}{24} = \frac{(-0.3)(0.7)(1.7)(2.7)}{24} = \frac{-0.9639}{24} = -0.0401625$

Substitute coefficients into $P_4(2018)$:

$$P_4(2018) = 142 + (-0.3)(21) + (-0.105)(2) + (-0.0595)(1) + (-0.0401625)(2)$$

$$P_4(2018) = 142 - 6.3 - 0.21 - 0.0595 - 0.080325 = 135.350175 \text{ million}$$

The estimated population in 2018 is approximately 135.35 million. ■

Example 2.2.12. Consider $f(x) = \sqrt{x}$ tabulated at equispaced nodes $x = 10, 11, 12, 13$:

x_i	10	11	12	13
$\sqrt{x_i}$	3.162278	3.316625	3.464102	3.605551

(i) Estimate $\sqrt{12.8}$ using Newton's backward difference polynomial of degree 3.

(ii) Find a theoretical upper bound for the truncation error $|E_3(12.8)|$ and compare it with the actual error.

Solution. **Part (i):** Here $x_n = 13, h = 1$, and target $x = 12.8 \implies v = \frac{12.8-13}{1} = -0.2$.

Construct the backward difference table:

x_i	y_i	∇y_i	$\nabla^2 y_i$	$\nabla^3 y_i$
10	3.162278			
		0.154347		
11	3.316625		-0.006870	
		0.147477		**0.000842**
12	3.464102		$-$**0.006028**	
		0.141449		
13	**3.605551**			

Substitute $v = -0.2$ and trailing differences into $P_3(12.8)$:

$$P_3(12.8) = 3.605551 + (-0.2)(0.141449) + \frac{(-0.2)(0.8)}{2}(-0.006028) + \frac{(-0.2)(0.8)(1.8)}{6}(0.000842)$$

$$P_3(12.8) = 3.605551 - 0.028290 + (-0.08)(-0.006028) + (-0.048)(0.000842)$$

$$P_3(12.8) = 3.605551 - 0.028290 + 0.000482 - 0.000040 = 3.577703$$

Part (ii): The truncation error formula for $n = 3$ is:

$$E_3(12.8) = \frac{v(v+1)(v+2)(v+3)}{4!}h^4 f^{(4)}(\xi)$$

For $v = -0.2$:

$$v(v+1)(v+2)(v+3) = (-0.2)(0.8)(1.8)(2.8) = -0.8064$$

For $f(x) = x^{1/2}$, higher-order derivatives are:

$$f'(x) = \frac{1}{2}x^{-1/2}, \quad f''(x) = -\frac{1}{4}x^{-3/2}, \quad f'''(x) = \frac{3}{8}x^{-5/2}, \quad f^{(4)}(x) = -\frac{15}{16}x^{-7/2}$$

On $\xi \in [10, 13]$, the maximum bound on $|f^{(4)}(\xi)|$ occurs at $\xi = 10$:

$$\max_{\xi \in [10,13]} |f^{(4)}(\xi)| = \frac{15}{16}(10)^{-7/2} = 0.9375 \times 0.0003162 \approx 0.0002964$$

Computing the error bound (with $h = 1$):

$$|E_3(12.8)| \leq \frac{0.8064}{24}(1)^4(0.0002964) = (0.0336)(0.0002964) \approx 0.000010$$

Comparing with the exact value $\sqrt{12.8} \approx 3.577709$:

$$\text{Actual Error } = |3.577709 - 3.577703| = 0.000006$$

The actual error 0.000006 strictly satisfies the theoretical maximum bound of 0.000010. ■

2.3 Spline Interpolation

When high-degree global polynomials are used to interpolate a large number of data points, the resulting interpolating polynomials often exhibit severe unphysical oscillations, particularly near the domain boundaries. This instability is known as **Runge's Phenomenon**.

To circumvent this drawback, **Piecewise Polynomial Interpolation** divides the domain into smaller subintervals and fits low-degree polynomials on each subinterval. A **Spline** is a piecewise polynomial function that enforces high-order smoothness (continuity of function values and derivatives) across the subinterval joints, known as **knots**.

2.3.1 Mathematical Definition of Cubic Splines

Among all piecewise polynomials, **Cubic Splines** are the most widely used in numerical analysis and computer-aided design because they provide C^2-continuity (continuous function values, first derivatives, and second derivatives) while maintaining computational efficiency and minimizing visual curvature.

Downloaded from – https://aijaz.vercel.app

Definition 2.3.1 (Cubic Spline). Let $a = x_0 < x_1 < x_2 < \cdots < x_n = b$ be $n+1$ distinct knots partitioning the interval $[a, b]$ into n subintervals $[x_j, x_{j+1}]$ of width $h_j = x_{j+1} - x_j$ for $j = 0, 1, \ldots, n-1$. Given corresponding function values $y_j = f(x_j)$, a function $S(x)$ is a **Cubic Spline Interpolant** if:

(i) On each subinterval $[x_j, x_{j+1}]$, $S(x)$ is represented by a cubic polynomial $S_j(x)$:

$$S(x) = S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3 \quad \text{for } x \in [x_j, x_{j+1}]$$

(ii) $S(x_j) = y_j$ for all $j = 0, 1, \ldots, n$ (Interpolation Property).

(iii) $S(x)$, $S'(x)$, and $S''(x)$ are continuous on the entire interval $[a, b]$ (C^2-Smoothness Property).

Derivation of the Tridiagonal System for Cubic Splines

A cubic spline on n subintervals possesses $4n$ unknown coefficients (a_j, b_j, c_j, d_j for $j = 0, \ldots, n-1$). Let $M_j = S''(x_j)$ denote the second derivative of $S(x)$ at knot x_j for $j = 0, 1, \ldots, n$.

Since $S_j(x)$ is cubic on $[x_j, x_{j+1}]$, its second derivative $S_j''(x)$ is linear. Applying Lagrange linear interpolation to $S_j''(x)$ at the endpoints x_j and x_{j+1}:

$$S_j''(x) = M_j \frac{x_{j+1} - x}{h_j} + M_{j+1} \frac{x - x_j}{h_j}$$

Integrating $S_j''(x)$ twice with respect to x:

$$S_j(x) = \frac{M_j}{6h_j}(x_{j+1} - x)^3 + \frac{M_{j+1}}{6h_j}(x - x_j)^3 + C_j(x - x_j) + D_j(x_{j+1} - x)$$

Imposing the interpolation conditions $S_j(x_j) = y_j$ and $S_j(x_{j+1}) = y_{j+1}$ allows us to solve for the integration constants C_j and D_j:

$$D_j = \frac{y_j}{h_j} - \frac{M_j h_j}{6}, \quad C_j = \frac{y_{j+1}}{h_j} - \frac{M_{j+1} h_j}{6}$$

Substituting C_j and D_j yields the explicit formula for $S_j(x)$ in terms of M_j and M_{j+1}:

$$S_j(x) = \frac{M_j}{6h_j}(x_{j+1} - x)^3 + \frac{M_{j+1}}{6h_j}(x - x_j)^3 + \left(\frac{y_{j+1}}{h_j} - \frac{M_{j+1} h_j}{6}\right)(x - x_j) + \left(\frac{y_j}{h_j} - \frac{M_j h_j}{6}\right)(x_{j+1} - x)$$

To enforce C^1-continuity, we equate the left and right first derivatives at the interior knot x_j:

$$S_{j-1}'(x_j) = S_j'(x_j) \quad \text{for } j = 1, 2, \ldots, n-1$$

Differentiating $S_j(x)$ and simplifying leads to the linear system relating the unknown second derivatives M_j:

Theorem 2.3.2 (Tridiagonal System for Cubic Splines). *The second derivatives $M_j = S''(x_j)$ at the interior knots $x_1, x_2, \ldots, x_{n-1}$ satisfy the symmetric **tridiagonal linear system**:*

$$h_{j-1} M_{j-1} + 2(h_{j-1} + h_j) M_j + h_j M_{j+1} = 6\left(\frac{y_{j+1} - y_j}{h_j} - \frac{y_j - y_{j-1}}{h_{j-1}}\right) \quad \text{for } j = 1, 2, \ldots, n-1$$

Downloaded from – https://aijaz.vercel.app

For equispaced knots where $h_{j-1} = h_j = h$, the tridiagonal system simplifies to:

$$M_{j-1} + 4M_j + M_{j+1} = \frac{6}{h^2}\left(y_{j+1} - 2y_j + y_{j-1}\right) = \frac{6}{h^2}\Delta^2 y_{j-1}$$

Types of Boundary Conditions

The tridiagonal system provides $n-1$ equations for $n+1$ unknown second derivatives $(M_0, M_1, \ldots, M_n)$. Two additional boundary conditions must be specified to close the system uniquely:

1. **Natural Cubic Spline:** Zero curvature at the boundaries:

$$M_0 = S''(x_0) = 0 \quad \text{and} \quad M_n = S''(x_n) = 0$$

2. **Clamped (Complete) Cubic Spline:** First derivatives specified at boundaries:

$$S'(x_0) = f'(x_0) = f'_0 \quad \text{and} \quad S'(x_n) = f'(x_n) = f'_n$$

   This yields two supplementary boundary equations:

$$2h_0M_0 + h_0M_1 = 6\left(\frac{y_1 - y_0}{h_0} - f'_0\right)$$

$$h_{n-1}M_{n-1} + 2h_{n-1}M_n = 6\left(f'_n - \frac{y_n - y_{n-1}}{h_{n-1}}\right)$$

3. **Periodic Cubic Spline:** Used when $f(x)$ is periodic with period $L = b - a$:

$$S'(x_0) = S'(x_n) \quad \text{and} \quad S''(x_0) = S''(x_n) \implies M_0 = M_n$$

Example 2.3.3. Construct a **Natural Cubic Spline** $S(x)$ to interpolate the dataset:

x_j	0	1	2
y_j	1	2	9

Hence, estimate $S(0.5)$.

Solution. Here $n = 2$ subintervals with knots $x_0 = 0, x_1 = 1, x_2 = 2$. Subinterval step sizes: $h_0 = x_1 - x_0 = 1$ and $h_1 = x_2 - x_1 = 1$. Function values: $y_0 = 1, y_1 = 2, y_2 = 9$.

Since $S(x)$ is a Natural Cubic Spline, boundary curvatures are zero:

$$M_0 = 0 \quad \text{and} \quad M_2 = 0$$

We need to determine M_1 at the single interior knot $j = 1$. Applying the tridiagonal equation for $j = 1$:

$$h_0M_0 + 2(h_0 + h_1)M_1 + h_1M_2 = 6\left(\frac{y_2 - y_1}{h_1} - \frac{y_1 - y_0}{h_0}\right)$$

Substitute $M_0 = 0, M_2 = 0, h_0 = 1, h_1 = 1$:

$$(1)(0) + 2(1+1)M_1 + (1)(0) = 6\left(\frac{9-2}{1} - \frac{2-1}{1}\right)$$

$$4M_1 = 6(7-1) = 6(6) = 36 \implies M_1 = 9$$

Downloaded from – https://aijaz.vercel.app

Second derivatives: $M_0 = 0, M_1 = 9, M_2 = 0$.
Construct the spline $S_0(x)$ on the first subinterval $[0,1]$ ($j = 0$):

$$S_0(x) = \frac{M_0}{6h_0}(x_1 - x)^3 + \frac{M_1}{6h_0}(x - x_0)^3 + \left(\frac{y_1}{h_0} - \frac{M_1 h_0}{6}\right)(x - x_0) + \left(\frac{y_0}{h_0} - \frac{M_0 h_0}{6}\right)(x_1 - x)$$

$$S_0(x) = 0 + \frac{9}{6(1)}(x - 0)^3 + \left(\frac{2}{1} - \frac{9(1)}{6}\right)x + \left(\frac{1}{1} - 0\right)(1 - x)$$

$$S_0(x) = \frac{3}{2}x^3 + \left(2 - \frac{3}{2}\right)x + (1 - x) = \frac{3}{2}x^3 + \frac{1}{2}x + 1 - x = \frac{3}{2}x^3 - \frac{1}{2}x + 1$$

Construct the spline $S_1(x)$ on the second subinterval $[1,2]$ ($j = 1$):

$$S_1(x) = \frac{9}{6(1)}(2 - x)^3 + 0 + \left(\frac{9}{1} - 0\right)(x - 1) + \left(\frac{2}{1} - \frac{9(1)}{6}\right)(2 - x)$$

$$S_1(x) = \frac{3}{2}(2 - x)^3 + 9(x - 1) + \frac{1}{2}(2 - x)$$

Evaluating $S(0.5)$ using $S_0(x)$ on $[0,1]$:

$$S(0.5) = S_0(0.5) = \frac{3}{2}(0.5)^3 - \frac{1}{2}(0.5) + 1 = \frac{3}{2}(0.125) - 0.25 + 1 = 0.1875 + 0.75 = 0.9375$$

■

Example 2.3.4. Construct a **Natural Cubic Spline** for four equispaced data points:

x_j	0	1	2	3
y_j	0	1	0	1

Set up and solve the system for second derivatives M_j.

Solution. Here $n = 3$ subintervals with $h_0 = h_1 = h_2 = 1$. Knots: $x_0 = 0, x_1 = 1, x_2 = 2, x_3 = 3$. Values: $y_0 = 0, y_1 = 1, y_2 = 0, y_3 = 1$.

Natural spline boundary conditions:

$$M_0 = 0 \quad \text{and} \quad M_3 = 0$$

We need to determine M_1 and M_2 at interior knots $j = 1$ and $j = 2$. Using $M_{j-1} + 4M_j + M_{j+1} = 6(y_{j+1} - 2y_j + y_{j-1})$:

- **For $j = 1$:**

$$M_0 + 4M_1 + M_2 = 6(y_2 - 2y_1 + y_0) \implies 0 + 4M_1 + M_2 = 6(0 - 2(1) + 0) = -12$$

$$4M_1 + M_2 = -12 \quad \text{— (Equation 1)}$$

- **For $j = 2$:**

$$M_1 + 4M_2 + M_3 = 6(y_3 - 2y_2 + y_1) \implies M_1 + 4M_2 + 0 = 6(1 - 2(0) + 1) = 12$$

$$M_1 + 4M_2 = 12 \quad \text{— (Equation 2)}$$

Solving the 2×2 system: From Eq (1), $M_2 = -12 - 4M_1$. Substitute into Eq (2):

$$M_1 + 4(-12 - 4M_1) = 12 \implies M_1 - 48 - 16M_1 = 12 \implies -15M_1 = 60 \implies M_1 = -4$$

Then $M_2 = -12 - 4(-4) = -12 + 16 = 4$.

The second derivatives are:

$$M_0 = 0, \quad M_1 = -4, \quad M_2 = 4, \quad M_3 = 0$$

The cubic splines on subintervals $[0,1], [1,2], [2,3]$ are readily obtained by substituting M_j into the standard spline expression. ■

Example 2.3.5. Construct a **Clamped Cubic Spline** on the single interval $[0,1]$ ($n = 1$) for $y_0 = 1, y_1 = 3$ with specified boundary derivative conditions $f'(0) = 0$ and $f'(1) = 3$.

Solution. Here $n = 1$, $x_0 = 0, x_1 = 1, h_0 = 1$, with $y_0 = 1, y_1 = 3$. We need to determine M_0 and M_1. The clamped boundary equations are:

$$2h_0M_0 + h_0M_1 = 6\left(\frac{y_1 - y_0}{h_0} - f'(0)\right)$$

$$h_0M_0 + 2h_0M_1 = 6\left(f'(1) - \frac{y_1 - y_0}{h_0}\right)$$

Substitute $h_0 = 1, y_0 = 1, y_1 = 3, f'(0) = 0, f'(1) = 3$:

$$2M_0 + M_1 = 6\left(\frac{3-1}{1} - 0\right) = 6(2) = 12 \quad \text{— (Equation 1)}$$

$$M_0 + 2M_1 = 6\left(3 - \frac{3-1}{1}\right) = 6(3-2) = 6 \quad \text{— (Equation 2)}$$

Multiply Eq (2) by 2 and subtract Eq (1):

$$(2M_0 + 4M_1) - (2M_0 + M_1) = 12 - 12 \implies 3M_1 = 0 \implies M_1 = 0$$

Substituting $M_1 = 0$ into Eq (1) gives $2M_0 = 12 \implies M_0 = 6$.

Construct $S_0(x)$ on $[0,1]$:

$$S_0(x) = \frac{6}{6(1)}(1-x)^3 + 0 + \left(\frac{3}{1} - 0\right)x + \left(\frac{1}{1} - \frac{6(1)}{6}\right)(1-x)$$

$$S_0(x) = (1-x)^3 + 3x + 0 = 1 - 3x + 3x^2 - x^3 + 3x = -x^3 + 3x^2 + 1$$

Verify boundary constraints:

$$S_0(0) = 1, \quad S_0(1) = -1 + 3 + 1 = 3$$

$$S_0'(x) = -3x^2 + 6x \implies S_0'(0) = 0, \quad S_0'(1) = -3 + 6 = 3$$

All clamped conditions are satisfied perfectly. ■

Example 2.3.6. Explain how Cubic Spline Interpolation resolves **Runge's Phenomenon** when interpolating Runge's function $f(x) = \frac{1}{1+25x^2}$ on $[-1,1]$.

Downloaded from – https://aijaz.vercel.app

Solution. When interpolating Runge's function $f(x) = \frac{1}{1+25x^2}$ using a single high-degree global polynomial $P_n(x)$ at $n+1$ equispaced nodes:

- As $n \to \infty$, the maximum error near the domain boundaries $x = \pm 1$ increases without bound:

$$\lim_{n\to\infty} \max_{x\in[-1,1]} |f(x) - P_n(x)| = \infty$$

  This instability occurs because high-degree polynomial derivatives grow exponentially near boundaries.

In contrast, **Cubic Spline Interpolation** partitions $[-1,1]$ into n subintervals of length $h = \frac{2}{n}$ and fits piece-wise cubic polynomials $S_j(x)$. By the Cubic Spline Error Theorem, if $f \in C^4[-1,1]$:

$$\max_{x\in[-1,1]} |f(x) - S(x)| \leq Ch^4 \max_{x\in[-1,1]} |f^{(4)}(x)| = \mathcal{O}(h^4)$$

As $n \to \infty$ ($h \to 0$), the maximum error decreases rapidly to zero at a 4th-order rate $\mathcal{O}(h^4)$, completely eliminating edge oscillations and providing a smooth, stable fit. ■

2.4 Numerical Integration

Numerical Integration (traditionally called **Numerical Quadrature**) is the process of computing the numerical value of a definite integral:

$$I = \int_a^b f(x)\,dx$$

Numerical quadrature is essential when the integrand $f(x)$ cannot be integrated analytically in terms of elementary functions (e.g., e^{-x^2}, $\frac{\sin x}{x}$), or when $f(x)$ is specified only at discrete tabular points.

Most quadrature rules approximate I using a weighted linear combination of function values evaluated at $n+1$ discrete nodes $x_0, x_1, \ldots, x_n \in [a,b]$:

$$I = \int_a^b f(x)\,dx \approx \sum_{i=0}^{n} w_i f(x_i)$$

where w_i are predetermined **quadrature weights**.

2.4.1 Basic Quadrature Rules

Before examining composite Newton–Cotes formulas (such as Trapezoidal and Simpson's rules), we establish the foundational principles of **Numerical Quadrature**. A numerical quadrature rule approximates a definite integral:

$$I(f) = \int_a^b f(x)\,dx$$

by a weighted finite sum of function evaluations:

$$Q(f) = \sum_{i=0}^{n} w_i f(x_i)$$

where $x_0, x_1, \ldots, x_n \in [a,b]$ are the **quadrature nodes** and $w_0, w_1, \ldots, w_n$ are the **quadrature weights**.

Elementary Quadrature Rules

The simplest basic quadrature rules approximate $f(x)$ on a single interval $[a,b]$ using constant functions based on endpoint or midpoint evaluations:

Downloaded from – https://aijaz.vercel.app

Definition 2.4.1 (Left and Right Rectangle Rules)**.** Let $h = b - a$.

(i) **Left Rectangle Rule:** Approximates $f(x) \approx f(a)$:

$$I_{\text{Left}} = (b-a)f(a) = hf(a)$$

(ii) **Right Rectangle Rule:** Approximates $f(x) \approx f(b)$:

$$I_{\text{Right}} = (b-a)f(b) = hf(b)$$

Definition 2.4.2 (Midpoint Rule)**.** The **Midpoint Rule** evaluates $f(x)$ at the midpoint $m = \frac{a+b}{2}$ of the interval $[a, b]$:

$$I_M = (b-a)f\left(\frac{a+b}{2}\right) = hf(m)$$

For a composite grid with n subintervals $[x_{i-1}, x_i]$ of width $h = \frac{b-a}{n}$ and midpoints $m_i = \frac{x_{i-1}+x_i}{2}$:

$$I_{\text{Composite Midpoint}} = h\sum_{i=1}^{n} f(m_i)$$

Theorem 2.4.3 (Midpoint Rule Error Bound)**.** *Let $f \in C^2[a, b]$. The truncation error E_M of the Composite Midpoint Rule is given by:*

$$E_M = \int_a^b f(x)\,dx - h\sum_{i=1}^{n} f(m_i) = \frac{(b-a)h^2}{24}f''(\xi) = \frac{(b-a)^3}{24n^2}f''(\xi)$$

for some point $\xi \in (a, b)$.

Proof. Expanding $f(x)$ in a Taylor series about the midpoint m_i on a single subinterval $[x_{i-1}, x_i]$:

$$f(x) = f(m_i) + (x - m_i)f'(m_i) + \frac{(x-m_i)^2}{2}f''(m_i) + \mathcal{O}((x-m_i)^3)$$

Integrating $f(x)$ over $[x_{i-1}, x_i] = \left[m_i - \frac{h}{2}, m_i + \frac{h}{2}\right]$:

$$\int_{x_{i-1}}^{x_i} f(x)\,dx = \int_{m_i-h/2}^{m_i+h/2} \left[f(m_i) + (x-m_i)f'(m_i) + \frac{(x-m_i)^2}{2}f''(m_i)\right] dx$$

Evaluating each integral term:

$$\int_{m_i-h/2}^{m_i+h/2} f(m_i)\,dx = hf(m_i)$$

$$\int_{m_i-h/2}^{m_i+h/2} (x-m_i)f'(m_i)\,dx = 0 \quad \text{(by odd symmetry about } m_i\text{)}$$

$$\int_{m_i-h/2}^{m_i+h/2} \frac{(x-m_i)^2}{2}f''(m_i)\,dx = \frac{f''(m_i)}{2}\left[\frac{(x-m_i)^3}{3}\right]_{-h/2}^{h/2} = \frac{f''(m_i)}{6}\left(\frac{h^3}{8} - \left(-\frac{h^3}{8}\right)\right) = \frac{h^3}{24}f''(m_i)$$

Thus, the single-panel error is $E_{M,i} = \frac{h^3}{24}f''(m_i)$. Summing over n subintervals and using the Mean Value Theorem gives $E_M = \frac{nh^3}{24}f''(\xi) = \frac{(b-a)h^2}{24}f''(\xi)$. ∎

Downloaded from – https://aijaz.vercel.app

Degree of Exactness and Method of Undetermined Coefficients

Definition 2.4.4 (Degree of Exactness / Precision). A quadrature rule $Q(f) = \sum_{i=0}^{n} w_i f(x_i)$ is said to have a **degree of exactness** (or precision) equal to d if $Q(P) = \int_a^b P(x)\,dx$ holds **exactly** with zero truncation error for all polynomials $P(x)$ of degree $\leq d$, but fails to hold for at least one polynomial of degree $d+1$.

Theorem 2.4.5 (Method of Undetermined Coefficients). *Quadrature weights $w_0, w_1, \ldots, w_n$ for pre-assigned nodes $x_0, x_1, \ldots, x_n$ can be uniquely determined by forcing the quadrature rule to be exact for the monomial basis functions $\{1, x, x^2, \ldots, x^n\}$:*

$$\sum_{i=0}^{n} w_i x_i^k = \int_a^b x^k\,dx = \frac{b^{k+1} - a^{k+1}}{k+1} \quad \text{for } k = 0, 1, 2, \ldots, n$$

Example 2.4.6. Evaluate $I = \int_0^2 x^3\,dx$ using the **Composite Midpoint Rule** with $n = 4$ subintervals. Compare the result with the exact analytical value $I = 4.0$.

Solution. Here $a = 0, b = 2, n = 4 \implies h = \frac{2-0}{4} = 0.5$. The subintervals are $[0, 0.5]$, $[0.5, 1.0]$, $[1.0, 1.5]$, $[1.5, 2.0]$.
Calculate midpoints $m_i = \frac{x_{i-1}+x_i}{2}$ and function evaluations $f(m_i) = m_i^3$:

- Subinterval 1: $m_1 = 0.25 \implies f(m_1) = (0.25)^3 = 0.015625$
- Subinterval 2: $m_2 = 0.75 \implies f(m_2) = (0.75)^3 = 0.421875$
- Subinterval 3: $m_3 = 1.25 \implies f(m_3) = (1.25)^3 = 1.953125$
- Subinterval 4: $m_4 = 1.75 \implies f(m_4) = (1.75)^3 = 5.359375$

Sum of function values at midpoints:

$$\sum_{i=1}^{4} f(m_i) = 0.015625 + 0.421875 + 1.953125 + 5.359375 = 7.750000$$

Apply Composite Midpoint Rule:

$$I_{\text{Midpoint}} = h \sum_{i=1}^{4} f(m_i) = 0.5 \times 7.750000 = 3.875$$

Comparing with exact value $I_{\text{exact}} = \left[\frac{x^4}{4}\right]_0^2 = \frac{16}{4} = 4.0$:

$$\text{Absolute Error } = |4.0 - 3.875| = 0.125$$

■

Example 2.4.7. Use the **Method of Undetermined Coefficients** to derive a symmetric two-node quadrature formula of the form:

$$\int_{-1}^{1} f(x)\,dx \approx w_0 f\left(-\frac{1}{2}\right) + w_1 f\left(\frac{1}{2}\right)$$

Determine its degree of exactness.

Downloaded from – https://aijaz.vercel.app

Solution. The quadrature rule has fixed nodes $x_0 = -1/2$ and $x_1 = 1/2$. We determine weights w_0, w_1 by forcing exactness for monomials $f(x) = 1$ and $f(x) = x$:

- **For $f(x) = 1$:**

$$\int_{-1}^{1} 1\,dx = 2 \implies w_0(1) + w_1(1) = 2 \implies w_0 + w_1 = 2 \quad \text{— (Equation 1)}$$

- **For $f(x) = x$:**

$$\int_{-1}^{1} x\,dx = 0 \implies w_0\left(-\frac{1}{2}\right) + w_1\left(\frac{1}{2}\right) = 0 \implies -\frac{1}{2}w_0 + \frac{1}{2}w_1 = 0 \implies w_0 = w_1$$

Solving $w_0 + w_1 = 2$ with $w_0 = w_1$ gives:

$$w_0 = 1 \quad \text{and} \quad w_1 = 1$$

The derived quadrature formula is:

$$\int_{-1}^{1} f(x)\,dx \approx f\left(-\frac{1}{2}\right) + f\left(\frac{1}{2}\right)$$

Test Degree of Exactness:

- **For $f(x) = x^2$:**

$$I_{\text{exact}} = \int_{-1}^{1} x^2\,dx = \left[\frac{x^3}{3}\right]_{-1}^{1} = \frac{2}{3} \approx 0.666667$$

$$I_{\text{approx}} = \left(-\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2 = \frac{1}{4} + \frac{1}{4} = \frac{1}{2} = 0.500000$$

Since $I_{\text{exact}} \neq I_{\text{approx}}$ for $f(x) = x^2$, the formula fails for degree 2. Thus, the degree of exactness of this formula is $d = 1$. ■

Example 2.4.8. Find the weights w_0, w_1, w_2 such that the quadrature formula:

$$\int_{0}^{1} f(x)\,dx \approx w_0 f(0) + w_1 f\left(\frac{1}{2}\right) + w_2 f(1)$$

has maximum degree of exactness. Verify that this formula is Simpson's $\frac{1}{3}$ Rule on $[0, 1]$.

Solution. Force exactness for basis monomials $f(x) = 1, x, x^2$:

- **For $f(x) = 1$ ($k = 0$):**

$$\int_{0}^{1} 1\,dx = 1 \implies w_0 + w_1 + w_2 = 1 \quad \text{— (Eq 1)}$$

- **For $f(x) = x$ ($k = 1$):**

$$\int_{0}^{1} x\,dx = \frac{1}{2} \implies w_0(0) + w_1\left(\frac{1}{2}\right) + w_2(1) = \frac{1}{2} \implies \frac{1}{2}w_1 + w_2 = \frac{1}{2} \implies w_1 + 2w_2 = 1 \quad \text{— (Eq 2)}$$

- **For $f(x) = x^2$ ($k = 2$):**

$$\int_{0}^{1} x^2\,dx = \frac{1}{3} \implies w_0(0)^2 + w_1\left(\frac{1}{2}\right)^2 + w_2(1)^2 = \frac{1}{3} \implies \frac{1}{4}w_1 + w_2 = \frac{1}{3} \implies w_1 + 4w_2 = \frac{4}{3} \quad \text{— (Eq 3)}$$

Downloaded from – https://aijaz.vercel.app

Subtracting Eq 2 from Eq 3:

$$(w_1 + 4w_2) - (w_1 + 2w_2) = \frac{4}{3} - 1 \implies 2w_2 = \frac{1}{3} \implies w_2 = \frac{1}{6}$$

Substitute $w_2 = 1/6$ into Eq 2:

$$w_1 + 2\left(\frac{1}{6}\right) = 1 \implies w_1 + \frac{1}{3} = 1 \implies w_1 = \frac{2}{3} = \frac{4}{6}$$

Substitute $w_1 = 4/6$ and $w_2 = 1/6$ into Eq 1:

$$w_0 + \frac{4}{6} + \frac{1}{6} = 1 \implies w_0 + \frac{5}{6} = 1 \implies w_0 = \frac{1}{6}$$

The resulting weights are $w_0 = 1/6, w_1 = 4/6, w_2 = 1/6$. The derived quadrature formula is:

$$\int_0^1 f(x)\,dx \approx \frac{1}{6}f(0) + \frac{4}{6}f\left(\frac{1}{2}\right) + \frac{1}{6}f(1) = \frac{1/2}{3}\left[f(0) + 4f\left(\frac{1}{2}\right) + f(1)\right]$$

With step size $h = 1/2$, this formula matches Simpson's $\frac{1}{3}$ Rule on $[0, 1]$ exactly. ■

2.4.2 Newton–Cotes Quadrature Formulas

Newton–Cotes Quadrature Formulas are obtained by replacing the integrand $f(x)$ with its polynomial interpolant $P_n(x)$ over $n+1$ equispaced nodes $x_i = x_0 + ih$ with step size $h = \frac{b-a}{n}$.

Derivation via Forward Differences

Let $x = x_0 + uh \implies dx = h\,du$. Using Newton's forward difference formula for $f(x)$:

$$I = \int_{x_0}^{x_n} f(x)\,dx \approx \int_{x_0}^{x_0+nh} P_n(x)\,dx = h\int_0^n (1+\Delta)^u y_0\,du$$

Expanding $(1+\Delta)^u$ in binomial series and integrating term-by-term with respect to u:

$$I = h\int_0^n \left[y_0 + u\Delta y_0 + \frac{u(u-1)}{2}\Delta^2 y_0 + \frac{u(u-1)(u-2)}{6}\Delta^3 y_0 + \dots\right] du$$

Evaluating the definite integrals $\int_0^n u\,du = \frac{n^2}{2}$, $\int_0^n \frac{u^2-u}{2}\,du = \frac{n^3}{6} - \frac{n^2}{4} = \frac{n^2(2n-3)}{12}$, etc., yields the **General Newton–Cotes Quadrature Formula**:

$$\int_{x_0}^{x_n} f(x)\,dx = nh\left[y_0 + \frac{n}{2}\Delta y_0 + \frac{n(2n-3)}{12}\Delta^2 y_0 + \frac{n(n-2)^2}{24}\Delta^3 y_0 + \frac{n(6n^3 - 15n^2 + 40n - 30)}{720}\Delta^4 y_0 + \dots\right]$$

2.4.3 Trapezoidal Rule

Setting $n = 1$ in the General Newton–Cotes Formula approximates $f(x)$ linearly over a single subinterval $[x_0, x_1]$:

$$\int_{x_0}^{x_1} f(x)\,dx = (1)h\left[y_0 + \frac{1}{2}\Delta y_0\right] = h\left[y_0 + \frac{1}{2}(y_1 - y_0)\right] = \frac{h}{2}(y_0 + y_1)$$

Definition 2.4.9 (Composite Trapezoidal Rule). Subdividing $[a, b]$ into n equal subintervals of length $h = \frac{b-a}{n}$ and applying the simple trapezoidal rule to each subinterval $[x_{i-1}, x_i]$ yields the

Downloaded from – https://aijaz.vercel.app

Composite Trapezoidal Rule:

$$I_T = \frac{h}{2}\left[y_0 + 2\sum_{i=1}^{n-1} y_i + y_n\right] = \frac{h}{2}\left[y_0 + 2(y_1 + y_2 + \cdots + y_{n-1}) + y_n\right]$$

Theorem 2.4.10 (Trapezoidal Error Bound). *Let $f \in C^2[a,b]$. The truncation error E_T of the Composite Trapezoidal Rule is given by:*

$$E_T = -\frac{(b-a)h^2}{12} f''(\xi) = -\frac{(b-a)^3}{12n^2} f''(\xi) \quad \text{for some } \xi \in (a,b)$$

2.4.4 Simpson's 1/3 - Rule

Setting $n = 2$ in the General Newton–Cotes Formula approximates $f(x)$ quadratically over two subintervals $[x_0, x_2]$:

$$\int_{x_0}^{x_2} f(x)\,dx = 2h\left[y_0 + \frac{2}{2}\Delta y_0 + \frac{2(4-3)}{12}\Delta^2 y_0\right] = 2h\left[y_0 + (y_1 - y_0) + \frac{1}{6}(y_2 - 2y_1 + y_0)\right] = \frac{h}{3}(y_0 + 4y_1 + y_2)$$

Definition 2.4.11 (Composite Simpson's $\frac{1}{3}$ Rule). Subdividing $[a,b]$ into an **even number** of subintervals n (n must be even) of length $h = \frac{b-a}{n}$ yields the **Composite Simpson's $\frac{1}{3}$ Rule**:

$$I_{S1/3} = \frac{h}{3}\left[y_0 + 4\sum_{i=1,3,5\ldots}^{n-1} y_i + 2\sum_{i=2,4,6\ldots}^{n-2} y_i + y_n\right]$$

$$I_{S1/3} = \frac{h}{3}\left[(y_0 + y_n) + 4(y_1 + y_3 + \cdots + y_{n-1}) + 2(y_2 + y_4 + \cdots + y_{n-2})\right]$$

Theorem 2.4.12 (Simpson's $\frac{1}{3}$ Error Bound). *Let $f \in C^4[a,b]$. The truncation error $E_{S1/3}$ of Composite Simpson's $\frac{1}{3}$ Rule is given by:*

$$E_{S1/3} = -\frac{(b-a)h^4}{180} f^{(4)}(\xi) = -\frac{(b-a)^5}{2880n^4} f^{(4)}(\xi) \quad \text{for some } \xi \in (a,b)$$

2.4.5 Simpson's 3/8 - Rule

Setting $n = 3$ in the General Newton–Cotes Formula approximates $f(x)$ using a cubic polynomial over three subintervals $[x_0, x_3]$:

$$\int_{x_0}^{x_3} f(x)\,dx = \frac{3h}{8}(y_0 + 3y_1 + 3y_2 + y_3)$$

Definition 2.4.13 (Composite Simpson's $\frac{3}{8}$ Rule). Subdividing $[a,b]$ into n subintervals where n is a **multiple of 3** ($n = 3k$) with $h = \frac{b-a}{n}$ yields the **Composite Simpson's $\frac{3}{8}$ Rule**:

$$I_{S3/8} = \frac{3h}{8}\left[(y_0 + y_n) + 3(y_1 + y_2 + y_4 + y_5 + \cdots + y_{n-1}) + 2(y_3 + y_6 + \cdots + y_{n-3})\right]$$

Downloaded from – https://aijaz.vercel.app

Theorem 2.4.14 (Simpson's $\frac{3}{8}$ Error Bound). *Let $f \in C^4[a,b]$. The truncation error $E_{S3/8}$ of Composite Simpson's $\frac{3}{8}$ Rule is given by:*

$$E_{S3/8} = -\frac{(b-a)h^4}{80} f^{(4)}(\xi) = -\frac{3(b-a)^5}{80n^4} f^{(4)}(\xi) \quad \text{for some } \xi \in (a,b)$$

Remark 2.4.15 (Degree of Exactness Summary). The **degree of exactness** (or precision) of a quadrature rule is the highest degree polynomial for which the rule integrates exactly without error:

- **Trapezoidal Rule ($n = 1$):** Degree of exactness = 1 (exact for linear polynomials).
- **Simpson's $\frac{1}{3}$ Rule ($n = 2$):** Degree of exactness = 3 (exact for cubic polynomials due to symmetry cancelling the 3rd derivative error term).
- **Simpson's $\frac{3}{8}$ Rule ($n = 3$):** Degree of exactness = 3 (exact for cubic polynomials).

Example 2.4.16. Evaluate $I = \int_0^1 \frac{1}{1+x^2}\,dx$ using $n = 6$ subintervals for:

(i) Composite Trapezoidal Rule

(ii) Composite Simpson's $\frac{1}{3}$ Rule

(iii) Composite Simpson's $\frac{3}{8}$ Rule

Compare all numerical results with the exact analytical value $I = \frac{\pi}{4} \approx 0.785398$.

Solution. Here $a = 0, b = 1, n = 6 \implies h = \frac{1-0}{6} = \frac{1}{6}$. Tabulate nodes $x_i = \frac{i}{6}$ and $y_i = \frac{1}{1+x_i^2}$:

i	x_i	x_i^2	$y_i = \frac{1}{1+x_i^2}$
0	0	0	1.000000
1	1/6	1/36	$36/37 \approx 0.972973$
2	2/6	4/36	$36/40 = 0.900000$
3	3/6	9/36	$36/45 = 0.800000$
4	4/6	16/36	$36/52 \approx 0.692308$
5	5/6	25/36	$36/61 \approx 0.590164$
6	1	1	$1/2 = 0.500000$

Compute grouped sum combinations:

- Endpoints: $y_0 + y_6 = 1.000000 + 0.500000 = 1.500000$
- Odd interior sum: $y_1 + y_3 + y_5 = 0.972973 + 0.800000 + 0.590164 = 2.363137$
- Even interior sum: $y_2 + y_4 = 0.900000 + 0.692308 = 1.592308$
- Total interior sum: $\sum_{i=1}^{5} y_i = 2.363137 + 1.592308 = 3.955445$

Part (i): Composite Trapezoidal Rule:

$$I_T = \frac{h}{2}\left[(y_0 + y_6) + 2\sum_{i=1}^{5} y_i\right] = \frac{1/6}{2}\left[1.500000 + 2(3.955445)\right] = \frac{1}{12}[1.500000 + 7.910890] = \frac{9.410890}{12} \approx 0.784241$$

Downloaded from – https://aijaz.vercel.app

$$\text{Absolute Error } = |\frac{\pi}{4} - 0.784241| = |0.785398 - 0.784241| = 0.001157$$

Part (ii): Composite Simpson's $\frac{1}{3}$ Rule:

$$I_{S1/3} = \frac{h}{3}\left[(y_0 + y_6) + 4(y_1 + y_3 + y_5) + 2(y_2 + y_4)\right] = \frac{1/6}{3}\left[1.500000 + 4(2.363137) + 2(1.592308)\right]$$

$$I_{S1/3} = \frac{1}{18}[1.500000 + 9.452548 + 3.184616] = \frac{14.137164}{18} \approx 0.785398$$

$$\text{Absolute Error } = |0.785398 - 0.785398| = 0.000000 \quad \text{(Accurate to 6 decimal places)}$$

Part (iii): Composite Simpson's $\frac{3}{8}$ Rule: Grouping for $n = 6$ (multiples of 3 is y_3):

$$I_{S3/8} = \frac{3h}{8}\left[(y_0 + y_6) + 3(y_1 + y_2 + y_4 + y_5) + 2y_3\right]$$

$$y_1 + y_2 + y_4 + y_5 = 0.972973 + 0.900000 + 0.692308 + 0.590164 = 3.155445$$

$$I_{S3/8} = \frac{3(1/6)}{8}\left[1.500000 + 3(3.155445) + 2(0.800000)\right] = \frac{1}{16}[1.500000 + 9.466335 + 1.600000] = \frac{12.566335}{16} \approx 0.78$$

$$\text{Absolute Error } = |0.785398 - 0.785396| = 0.000002$$

■

Example 2.4.17. Determine the minimum number of subintervals n required to evaluate $I = \int_0^2 e^x\,dx$ such that the absolute truncation error is less than 10^{-4} using:

(i) Composite Trapezoidal Rule

(ii) Composite Simpson's $\frac{1}{3}$ Rule

Solution. Here $a = 0, b = 2$, so $b - a = 2$. For $f(x) = e^x$, all derivatives are $f^{(k)}(x) = e^x$. Since e^x is strictly increasing on $[0, 2]$:

$$M_2 = \max_{x\in[0,2]} |f''(x)| = e^2 \approx 7.389056, \quad M_4 = \max_{x\in[0,2]} |f^{(4)}(x)| = e^2 \approx 7.389056$$

Part (i): Composite Trapezoidal Rule: The error bound condition is:

$$|E_T| \le \frac{(b-a)^3}{12n^2} M_2 \le 10^{-4} \implies \frac{(2)^3}{12n^2}(7.389056) \le 10^{-4}$$

$$\frac{8 \times 7.389056}{12n^2} \le 10^{-4} \implies \frac{4.926037}{n^2} \le 10^{-4} \implies n^2 \ge 49260.37 \implies n \ge 221.95$$

Rounding up to the next integer, at least $n = 222$ subintervals are required.

Part (ii): Composite Simpson's $\frac{1}{3}$ Rule: The error bound condition is:

$$|E_{S1/3}| \le \frac{(b-a)^5}{2880n^4} M_4 \le 10^{-4} \implies \frac{(2)^5}{2880n^4}(7.389056) \le 10^{-4}$$

$$\frac{32 \times 7.389056}{2880n^4} \le 10^{-4} \implies \frac{0.082101}{n^4} \le 10^{-4} \implies n^4 \ge 821.01 \implies n \ge 5.35$$

Since n must be an **even integer** for Simpson's $\frac{1}{3}$ rule, rounding up to the next even integer gives $n = 6$ subintervals. ■

Example 2.4.18. The velocity $v(t)$ (in meters/second) of a rocket recorded at 10-second intervals is given below:

t (sec)	0	10	20	30	40	50	60
$v(t)$ (m/s)	0	120	280	410	540	680	810

Estimate the total distance $S = \int_0^{60} v(t)\,dt$ traveled by the rocket in 60 seconds using Simpson's $\frac{1}{3}$ Rule.

Solution. Step size $h = 10$ seconds, with $n = 6$ subintervals (which is even, so Simpson's $\frac{1}{3}$ rule applies). Function values:

$$y_0 = 0, \quad y_1 = 120, \quad y_2 = 280, \quad y_3 = 410, \quad y_4 = 540, \quad y_5 = 680, \quad y_6 = 810$$

Applying Composite Simpson's $\frac{1}{3}$ Rule:

$$S = \frac{h}{3}\left[(y_0 + y_6) + 4(y_1 + y_3 + y_5) + 2(y_2 + y_4)\right]$$

Substitute values:

$$S = \frac{10}{3}\left[(0 + 810) + 4(120 + 410 + 680) + 2(280 + 540)\right]$$

$$S = \frac{10}{3}\left[810 + 4(1210) + 2(820)\right] = \frac{10}{3}\left[810 + 4840 + 1640\right] = \frac{10}{3}[7290] = 24,300 \text{ meters}$$

The total distance covered by the rocket is 24,300 meters (24.3 km). ■

Example 2.4.19. Prove that Simpson's $\frac{1}{3}$ Rule integrates cubic polynomials $P_3(x) = a_3x^3 + a_2x^2 + a_1x + a_0$ exactly, despite being derived by interpolating quadratic polynomials.

Solution. By linearity of integration and quadrature operators, it suffices to prove that simple Simpson's $\frac{1}{3}$ rule on $[-h, h]$ ($x_0 = -h, x_1 = 0, x_2 = h$) integrates monomials $f(x) = x^k$ exactly for $k = 0, 1, 2, 3$.

Simple Simpson's $\frac{1}{3}$ formula on $[-h, h]$:

$$I_{\text{approx}} = \frac{h}{3}\left[f(-h) + 4f(0) + f(h)\right]$$

- **For $f(x) = 1$ ($k = 0$):**

$$I_{\text{exact}} = \int_{-h}^{h} 1\,dx = 2h; \quad I_{\text{approx}} = \frac{h}{3}(1 + 4 + 1) = 2h \quad \checkmark$$

- **For $f(x) = x$ ($k = 1$):**

$$I_{\text{exact}} = \int_{-h}^{h} x\,dx = 0; \quad I_{\text{approx}} = \frac{h}{3}(-h + 0 + h) = 0 \quad \checkmark$$

- **For $f(x) = x^2$ ($k = 2$):**

$$I_{\text{exact}} = \int_{-h}^{h} x^2\,dx = \frac{2h^3}{3}; \quad I_{\text{approx}} = \frac{h}{3}(h^2 + 0 + h^2) = \frac{2h^3}{3} \quad \checkmark$$

- **For $f(x) = x^3$ ($k = 3$):**

$$I_{\text{exact}} = \int_{-h}^{h} x^3\,dx = 0; \quad I_{\text{approx}} = \frac{h}{3}((-h)^3 + 0 + h^3) = \frac{h}{3}(-h^3 + h^3) = 0 \quad \checkmark$$

Since $I_{\text{exact}} = I_{\text{approx}}$ for $1, x, x^2$, and x^3, Simpson's $\frac{1}{3}$ rule integrates any 3rd-degree polynomial $P_3(x)$ with zero truncation error. ■

Downloaded from – https://aijaz.vercel.app

Exercise 2.4.20. Solve the following exercises.

1. Prove the following finite difference operator identities:
   (a) $\Delta - \nabla = \delta^2$
   (b) $\mu\delta = \frac{1}{2}(\Delta + \nabla)$
   (c) $\mu^2 = 1 + \frac{1}{4}\delta^2$

2. Establish the relationship connecting the differential operator D with the central difference operator δ:
$$hD = 2\sinh^{-1}\left(\frac{\delta}{2}\right)$$
Hence, expand hD in powers of δ.

3. Using the Fundamental Theorem of Finite Differences, evaluate the fourth-order forward difference:
$$\Delta^4(2x^4 - 3x^3 + 5x^2 - 7)$$
with a constant step size $h = 2$.

4. Construct a forward difference table for a second-degree polynomial $y = P_2(x)$ containing a missing entry M:

x	0	1	2	3	4
y	1	3	9	M	33

Using the property that $\Delta^3 y_i = 0$, determine the exact value of M.

5. Derive the finite difference formula for the first derivative in terms of forward differences:
$$f'(x_0) = \frac{1}{h}\left(\Delta y_0 - \frac{\Delta^2 y_0}{2} + \frac{\Delta^3 y_0}{3} - \frac{\Delta^4 y_0}{4} + \dots\right)$$
Use this formula to estimate $f'(1)$ from the dataset $(1,2), (2,9), (3,28), (4,65)$ with $h = 1$.

6. Prove the Fundamental Theorem of Finite Differences: If $P_n(x)$ is a polynomial of degree n with leading coefficient $a_n \neq 0$ and step size $h > 0$, then $\Delta^n P_n(x) = a_n n! h^n$ (a constant) and $\Delta^{n+1} P_n(x) = 0$.

7. Given the dataset:

x	0.0	0.1	0.2	0.3	0.4
$f(x)$	1.000	1.105	1.221	1.350	1.492

Use Newton's Forward Difference Interpolation Formula to estimate $f(0.05)$.

8. Using the dataset from Exercise 7, apply Newton's Backward Difference Interpolation Formula to estimate $f(0.38)$.

9. Let $f(x) = e^x$ be interpolated at equispaced nodes $x_0 = 0, x_1 = 0.2, x_2 = 0.4, x_3 = 0.6$. Use Newton's forward difference formula to estimate $e^{0.15}$, and compute a theoretical bound for the truncation error $|E_3(0.15)|$.

10. The population of a state (in lakhs) recorded in decennial censuses is given below:

Year (x)	1921	1931	1941	1951
Population (y)	46	66	81	93

Estimate the population in the year 1925 using Newton's Forward Difference Formula.

Downloaded from – https://aijaz.vercel.app

11. Using the census dataset from Exercise 10, estimate the population in the year 1948 using Newton's Backward Difference Formula.

12. Construct a **Natural Cubic Spline** $S(x)$ to interpolate the data points:

x_j	0	1	2
y_j	1	3	11

    Find the polynomials $S_0(x)$ on $[0,1]$ and $S_1(x)$ on $[1,2]$, and evaluate $S(0.5)$.

13. Set up the tridiagonal system for the interior second derivatives M_1, M_2 of a **Natural Cubic Spline** passing through the points $(0,1), (1,2), (2,1), (3,2)$. Solve for M_1 and M_2.

14. Construct a **Clamped Cubic Spline** on $[0,2]$ with knots $x_0 = 0, x_1 = 1, x_2 = 2$ and function values $y_0 = 1, y_1 = 2, y_2 = 5$, subject to clamped boundary derivative conditions $f'(0) = 0$ and $f'(2) = 4$.

15. Explain **Runge's Phenomenon** in global high-degree polynomial interpolation, and describe how Cubic Spline Interpolation avoids edge oscillations while guaranteeing C^2-continuity.

16. Use the **Method of Undetermined Coefficients** to derive a 3-point quadrature rule of the form:
$$\int_0^1 f(x)\,dx \approx w_0 f(0) + w_1 f\left(\frac{1}{2}\right) + w_2 f(1)$$
    Determine its weights and state its degree of exactness.

17. Evaluate $I = \int_0^2 \frac{1}{1+x}\,dx$ using the **Composite Midpoint Rule** with $n = 4$ subintervals. Compare the result with the exact analytical value $I = \ln 3 \approx 1.098612$.

18. Evaluate $I = \int_0^1 e^{-x}\,dx$ using the **Composite Trapezoidal Rule** with $n = 6$ subintervals. Compute the actual error against the exact value $I = 1 - e^{-1} \approx 0.632121$.

19. Evaluate $I = \int_0^1 \frac{1}{1+x^2}\,dx$ using **Composite Simpson's $\frac{1}{3}$ Rule** with $n = 6$ subintervals. Compare your result with $\frac{\pi}{4} \approx 0.785398$.

20. Evaluate $I = \int_0^3 \frac{1}{1+x}\,dx$ using **Composite Simpson's $\frac{3}{8}$ Rule** with $n = 6$ subintervals.

21. Determine the minimum number of subintervals n required to evaluate $I = \int_1^3 \ln x\,dx$ using the **Composite Trapezoidal Rule** such that the truncation error $|E_T| \leq 10^{-4}$.

22. Determine the minimum number of subintervals n required to evaluate $I = \int_0^1 e^x\,dx$ using **Composite Simpson's $\frac{1}{3}$ Rule** such that the truncation error $|E_{S1/3}| \leq 10^{-6}$.

23. Prove that Simpson's $\frac{1}{3}$ Rule has a **degree of exactness** equal to 3. Explain why it integrates cubic polynomials $P_3(x)$ exactly even though it is derived by interpolating quadratic polynomials.

CHAPTER

3

Linear Systems and Eigenvalue Problems

3.1 Direct Methods for Solving Linear Systems

Consider a system of n linear algebraic equations in n unknowns given by:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n, \end{aligned} \tag{3.1}$$

or in compact matrix notation,

$$A\mathbf{x} = \mathbf{b}, \tag{3.2}$$

where $A = (a_{ij}) \in \mathbb{R}^{n\times n}$ is the coefficient matrix, $\mathbf{x} = (x_1, x_2, \ldots, x_n)^T \in \mathbb{R}^n$ is the vector of unknowns, and $\mathbf{b} = (b_1, b_2, \ldots, b_n)^T \in \mathbb{R}^n$ is the right-hand side vector of constants.

Definition 3.1.1 (Direct Methods)**.** A numerical method for solving $A\mathbf{x} = \mathbf{b}$ is called a *direct method* if, in the theoretical absence of round-off errors, it yields the exact solution in a finite, pre-determined number of elementary arithmetic operations.

Direct methods manipulate the *augmented matrix* $\widetilde{A} = [A \mid \mathbf{b}] \in \mathbb{R}^{n\times(n+1)}$ using elementary row operations.

Definition 3.1.2 (Elementary Row Operations)**.** The three standard elementary row operations (EROs) on a matrix are:

i. **Row Interchange ($R_i \leftrightarrow R_j$):** Interchanging the i-th and j-th rows.

ii. **Row Scaling ($R_i \to cR_i$):** Multiplying the i-th row by a non-zero scalar $c \in \mathbb{R} \setminus \{0\}$.

iii. **Row Addition ($R_i \to R_i + cR_j$):** Adding c times the j-th row to the i-th row ($i \neq j$).

3.1.1 Gauss Elimination Method

The Gauss elimination method is a classic direct approach that reduces the augmented matrix $[A \mid \mathbf{b}]$ to an equivalent upper triangular matrix $[U \mid \mathbf{c}]$. The unknowns are then solved sequentially via *back substitution*.

Algorithm and Mathematical Formulation

The procedure consists of two sequential phases:

Downloaded from – https://aijaz.vercel.app

1. Forward Elimination Phase: The goal is to convert A into an upper triangular matrix U through $n-1$ elimination steps. At step k ($k = 1, 2, \ldots, n-1$), the element $a_{kk}^{(k)}$ is designated as the *pivot element*. Assuming $a_{kk}^{(k)} \neq 0$, we eliminate the entries $a_{ik}^{(k)}$ for all rows $i = k+1, \ldots, n$ beneath the pivot.

The explicit multipliers are computed as:

$$m_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, \quad \text{for } i = k+1, k+2, \ldots, n. \tag{3.3}$$

We then perform the elementary row operation $R_i \to R_i - m_{ik} R_k$, updating matrix entries and right-hand side values according to:

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik} a_{kj}^{(k)}, \quad \text{for } j = k+1, \ldots, n, \tag{3.4}$$

$$b_i^{(k+1)} = b_i^{(k)} - m_{ik} b_k^{(k)}, \quad \text{for } i = k+1, \ldots, n. \tag{3.5}$$

Upon completion of $n-1$ steps, the augmented system assumes upper triangular form:

$$\begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & a_{13}^{(1)} & \ldots & a_{1n}^{(1)} \\ 0 & a_{22}^{(2)} & a_{23}^{(2)} & \ldots & a_{2n}^{(2)} \\ 0 & 0 & a_{33}^{(3)} & \ldots & a_{3n}^{(3)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & a_{nn}^{(n)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1^{(1)} \\ b_2^{(2)} \\ b_3^{(3)} \\ \vdots \\ b_n^{(n)} \end{bmatrix}. \tag{3.6}$$

2. Back Substitution Phase: The transformed upper triangular system is solved starting from the bottom equation upwards:

$$x_n = \frac{b_n^{(n)}}{a_{nn}^{(n)}}, \tag{3.7}$$

and for $i = n-1, n-2, \ldots, 1$:

$$x_i = \frac{1}{a_{ii}^{(i)}} \left(b_i^{(i)} - \sum_{j=i+1}^{n} a_{ij}^{(i)} x_j \right). \tag{3.8}$$

Pivoting Strategies

Remark 3.1.3 (Need for Pivoting). Naïve Gauss elimination fails if any pivot element $a_{kk}^{(k)} = 0$. Furthermore, if $|a_{kk}^{(k)}|$ is very small relative to the remaining elements in its column, the multipliers m_{ik} become exceedingly large. This severely magnifies computational round-off errors, leading to severe numerical instability.

Definition 3.1.4 (Partial Pivoting)**.** To ensure numerical robustness, at step k, we search the current pivot column k on and below the diagonal for the element with the maximum absolute magnitude:

$$|a_{pk}^{(k)}| = \max_{k \leq i \leq n} |a_{ik}^{(k)}|. \tag{3.9}$$

If $p > k$, we perform a row interchange $R_k \leftrightarrow R_p$ prior to carrying out the elimination step.

Downloaded from – https://aijaz.vercel.app

Theorem 3.1.5 (Computational Complexity of Gauss Elimination). *For an $n \times n$ linear system, the forward elimination phase requires approximately $\frac{2}{3}n^3 + O(n^2)$ floating-point arithmetic operations (multiplications and divisions), while back substitution requires $O(n^2)$ operations. Thus, total operational cost is $O\left(\frac{2}{3}n^3\right)$.*

Example 3.1.6. Solve the following system of linear equations using the standard Gauss elimination method (without pivoting):

$$\begin{aligned} 2x_1 + x_2 - x_3 &= 1, \\ 4x_1 + 3x_2 + x_3 &= 11, \\ -2x_1 + x_2 + 2x_3 &= 3. \end{aligned}$$

Solution. **Step 1: Set up the Augmented Matrix.**

$$[A \mid \mathbf{b}] = \left[\begin{array}{ccc|c} 2 & 1 & -1 & 1 \\ 4 & 3 & 1 & 11 \\ -2 & 1 & 2 & 3 \end{array}\right].$$

Step 2: Forward Elimination.

- *Elimination in Column 1:* The pivot element is $a_{11} = 2$.

$$m_{21} = \frac{a_{21}}{a_{11}} = \frac{4}{2} = 2,$$
$$m_{31} = \frac{a_{31}}{a_{11}} = \frac{-2}{2} = -1.$$

  Apply row operations $R_2 \to R_2 - 2R_1$ and $R_3 \to R_3 - (-1)R_1$:

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 1 \\ 0 & 1 & 3 & 9 \\ 0 & 2 & 1 & 4 \end{array}\right].$$

- *Elimination in Column 2:* The pivot element is $a_{22} = 1$.

$$m_{32} = \frac{a_{32}}{a_{22}} = \frac{2}{1} = 2.$$

  Apply row operation $R_3 \to R_3 - 2R_2$:

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 1 \\ 0 & 1 & 3 & 9 \\ 0 & 0 & -5 & -14 \end{array}\right].$$

Step 3: Back Substitution.

1. From row 3:
$$-5x_3 = -14 \implies x_3 = \frac{14}{5} = 2.8.$$
2. From row 2:
$$x_2 + 3x_3 = 9 \implies x_2 = 9 - 3(2.8) = 9 - 8.4 = 0.6 = \frac{3}{5}.$$
3. From row 1:
$$2x_1 + x_2 - x_3 = 1 \implies 2x_1 = 1 - 0.6 + 2.8 = 3.2 \implies x_1 = 1.6 = \frac{8}{5}.$$

Hence, the exact solution is $\mathbf{x} = \begin{bmatrix} 1.6 & 0.6 & 2.8 \end{bmatrix}^T$. ■

Downloaded from – https://aijaz.vercel.app

Example 3.1.7. Solve the following linear system using Gauss elimination with **partial pivoting**:

$$\begin{aligned} 0x_1 + 2x_2 + x_3 &= 5, \\ x_1 + x_2 - 2x_3 &= -3, \\ 2x_1 + 4x_2 + x_3 &= 12. \end{aligned}$$

Solution. **Step 1: Set up the Augmented Matrix.**

$$[A \mid \mathbf{b}] = \left[\begin{array}{ccc|c} 0 & 2 & 1 & 5 \\ 1 & 1 & -2 & -3 \\ 2 & 4 & 1 & 12 \end{array}\right].$$

Step 2: Forward Elimination with Partial Pivoting.

- *Step 1 (Column 1):* Pivot search among $|a_{11}| = 0$, $|a_{21}| = 1$, $|a_{31}| = 2$. The maximum magnitude is 2 in Row 3. Swap $R_1 \leftrightarrow R_3$:

$$\left[\begin{array}{ccc|c} 2 & 4 & 1 & 12 \\ 1 & 1 & -2 & -3 \\ 0 & 2 & 1 & 5 \end{array}\right].$$

  Multipliers: $m_{21} = \frac{1}{2} = 0.5$, $m_{31} = \frac{0}{2} = 0$. Apply $R_2 \to R_2 - 0.5R_1$:

$$\left[\begin{array}{ccc|c} 2 & 4 & 1 & 12 \\ 0 & -1 & -2.5 & -9 \\ 0 & 2 & 1 & 5 \end{array}\right].$$

- *Step 2 (Column 2):* Pivot search among $|a_{22}| = |-1| = 1$ and $|a_{32}| = |2| = 2$. The maximum magnitude is 2 in Row 3. Swap $R_2 \leftrightarrow R_3$:

$$\left[\begin{array}{ccc|c} 2 & 4 & 1 & 12 \\ 0 & 2 & 1 & 5 \\ 0 & -1 & -2.5 & -9 \end{array}\right].$$

  Multiplier: $m_{32} = \frac{-1}{2} = -0.5$. Apply $R_3 \to R_3 - (-0.5)R_2 \implies R_3 \to R_3 + 0.5R_2$:

$$\left[\begin{array}{ccc|c} 2 & 4 & 1 & 12 \\ 0 & 2 & 1 & 5 \\ 0 & 0 & -2 & -6.5 \end{array}\right].$$

Step 3: Back Substitution.

1. From row 3:
$$-2x_3 = -6.5 \implies x_3 = 3.25 = \frac{13}{4}.$$
2. From row 2:
$$2x_2 + 3.25 = 5 \implies 2x_2 = 1.75 \implies x_2 = 0.875 = \frac{7}{8}.$$
3. From row 1:
$$2x_1 + 4(0.875) + 3.25 = 12 \implies 2x_1 + 3.5 + 3.25 = 12 \implies 2x_1 = 5.25 \implies x_1 = 2.625 = \frac{21}{8}.$$

Thus, the solution is $\mathbf{x} = \begin{bmatrix} 2.625 & 0.875 & 3.25 \end{bmatrix}^T$. ■

Downloaded from – https://aijaz.vercel.app

Example 3.1.8. Solve the following system using Gauss elimination:

$$\begin{aligned} x_1 + 2x_2 + 3x_3 &= 14, \\ 2x_1 + 5x_2 + 2x_3 &= 18, \\ 3x_1 + x_2 + 5x_3 &= 20. \end{aligned}$$

Solution. **Step 1: Form the Augmented Matrix.**

$$[A \mid \mathbf{b}] = \left[\begin{array}{ccc|c} 1 & 2 & 3 & 14 \\ 2 & 5 & 2 & 18 \\ 3 & 1 & 5 & 20 \end{array}\right].$$

Step 2: Forward Elimination.

- *Elimination in Column 1:* Pivot $a_{11} = 1$. Multipliers: $m_{21} = 2$, $m_{31} = 3$. Row operations: $R_2 \to R_2 - 2R_1$ and $R_3 \to R_3 - 3R_1$:

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 14 \\ 0 & 1 & -4 & -10 \\ 0 & -5 & -4 & -22 \end{array}\right].$$

- *Elimination in Column 2:* Pivot $a_{22} = 1$. Multiplier: $m_{32} = \frac{-5}{1} = -5$. Row operation: $R_3 \to R_3 - (-5)R_2 \implies R_3 \to R_3 + 5R_2$:

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 14 \\ 0 & 1 & -4 & -10 \\ 0 & 0 & -24 & -72 \end{array}\right].$$

Step 3: Back Substitution.

1. From row 3:
$$-24x_3 = -72 \implies x_3 = 3.$$
2. From row 2:
$$x_2 - 4(3) = -10 \implies x_2 = -10 + 12 = 2.$$
3. From row 1:
$$x_1 + 2(2) + 3(3) = 14 \implies x_1 + 4 + 9 = 14 \implies x_1 = 1.$$

The solution vector is $\mathbf{x} = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}^T$. ■

3.1.2 Gauss–Jordan Method

The Gauss–Jordan elimination method is an extension of the Gauss elimination method. Rather than terminating elimination once an upper triangular matrix is reached, operations are continued to reduce the coefficient matrix A directly into the identity matrix I (or Reduced Row Echelon Form - RREF). Consequently, the solution vector $\mathbf{x}$ is obtained directly without needing back substitution.

Downloaded from – https://aijaz.vercel.app

Algorithm and Matrix Inversion

1. System Solution Procedure: Given $[A \mid \mathbf{b}]$, elementary row operations are performed systematically across all rows: 1. At step k, normalize the k-th row by dividing by its pivot element $a_{kk}^{(k)}$ so that the diagonal entry becomes 1:

$$R_k \longrightarrow \frac{1}{a_{kk}^{(k)}} R_k. \tag{3.10}$$

2. Eliminate all non-zero off-diagonal entries in column k **both above and below** the pivot a_{kk}:

$$R_i \longrightarrow R_i - a_{ik} R_k, \quad \text{for all } i = 1, 2, \ldots, n \text{ with } i \neq k. \tag{3.11}$$

Upon completing this process for $k = 1, 2, \ldots, n$, the augmented matrix transforms directly as:

$$[A \mid \mathbf{b}] \xrightarrow{\text{Gauss–Jordan}} [I \mid \mathbf{x}]. \tag{3.12}$$

2. Matrix Inversion using Gauss–Jordan Method: The method provides an efficient framework for computing the matrix inverse A^{-1}. We construct the augmented block matrix $[A \mid I_n]$, where I_n is the $n \times n$ identity matrix.

By applying row operations to transform A into I_n, the same operations simultaneously map I_n into A^{-1}:

$$[A \mid I_n] \xrightarrow{\text{Gauss–Jordan}} [I_n \mid A^{-1}]. \tag{3.13}$$

Theorem 3.1.9 (Comparison of Computational Efficiency). *To solve a single system $A\mathbf{x} = \mathbf{b}$:*

- *Gauss Elimination requires approximately $\frac{2}{3}n^3 + O(n^2)$ operations.*
- *Gauss–Jordan Method requires approximately $\frac{3}{2}n^3 = O(n^3)$ operations.*

Thus, Gauss–Jordan elimination involves about 50% more floating-point operations than Gauss elimination for solving linear systems, making standard Gauss elimination preferred for single systems. However, Gauss–Jordan is extremely effective for matrix inversion.

Example 3.1.10. Solve the following linear system using the Gauss–Jordan method:

$$\begin{aligned} x_1 + 2x_2 + x_3 &= 8, \\ 2x_1 + 3x_2 + 4x_3 &= 20, \\ 4x_1 + 3x_2 + 2x_3 &= 16. \end{aligned}$$

Solution. **Step 1: Set up the Augmented Matrix.**

$$[A \mid \mathbf{b}] = \left[\begin{array}{ccc|c} 1 & 2 & 1 & 8 \\ 2 & 3 & 4 & 20 \\ 4 & 3 & 2 & 16 \end{array}\right].$$

Step 2: Forward and Backward Simultaneous Elimination.

- *Column 1 Pivot ($a_{11} = 1$):* Eliminate entries in R_2 and R_3 below pivot: $R_2 \to R_2 - 2R_1$ and $R_3 \to R_3 - 4R_1$:

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 8 \\ 0 & -1 & 2 & 4 \\ 0 & -5 & -2 & -16 \end{array}\right].$$

Downloaded from – https://aijaz.vercel.app

- *Column 2 Pivot ($a_{22} = -1$):* Normalize pivot row by $R_2 \to -1R_2$:

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 8 \\ 0 & 1 & -2 & -4 \\ 0 & -5 & -2 & -16 \end{array}\right].$$

  Now eliminate entries above and below a_{22}: $R_1 \to R_1 - 2R_2$ and $R_3 \to R_3 + 5R_2$:

$$\left[\begin{array}{ccc|c} 1 & 0 & 5 & 16 \\ 0 & 1 & -2 & -4 \\ 0 & 0 & -12 & -36 \end{array}\right].$$

- *Column 3 Pivot ($a_{33} = -12$):* Normalize pivot row by $R_3 \to -\frac{1}{12}R_3$:

$$\left[\begin{array}{ccc|c} 1 & 0 & 5 & 16 \\ 0 & 1 & -2 & -4 \\ 0 & 0 & 1 & 3 \end{array}\right].$$

  Eliminate entries above a_{33}: $R_1 \to R_1 - 5R_3$ and $R_2 \to R_2 + 2R_3$:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array}\right].$$

Step 3: Read Final Solution. Directly from the reduced matrix, $x_1 = 1$, $x_2 = 2$, and $x_3 = 3$. ■

Example 3.1.11. Find the inverse of the matrix A using the Gauss–Jordan method:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}.$$

Solution. **Step 1: Set up the Augmented Block Matrix $[A \mid I_3]$.**

$$[A \mid I_3] = \left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 4 & 0 & 1 & 0 \\ 5 & 6 & 0 & 0 & 0 & 1 \end{array}\right].$$

Step 2: Row Operations to obtain $[I_3 \mid A^{-1}]$.

- *Column 1:* Pivot $a_{11} = 1$. Eliminate below a_{11} via $R_3 \to R_3 - 5R_1$:

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 4 & 0 & 1 & 0 \\ 0 & -4 & -15 & -5 & 0 & 1 \end{array}\right].$$

- *Column 2:* Pivot $a_{22} = 1$. Eliminate entries above and below a_{22} via $R_1 \to R_1 - 2R_2$ and $R_3 \to R_3 + 4R_2$:

$$\left[\begin{array}{ccc|ccc} 1 & 0 & -5 & 1 & -2 & 0 \\ 0 & 1 & 4 & 0 & 1 & 0 \\ 0 & 0 & 1 & -5 & 4 & 1 \end{array}\right].$$

- *Column 3:* Pivot $a_{33} = 1$. Eliminate entries above a_{33} via $R_1 \to R_1 + 5R_3$ and $R_2 \to R_2 - 4R_3$:

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & -24 & 18 & 5 \\ 0 & 1 & 0 & 20 & -15 & -4 \\ 0 & 0 & 1 & -5 & 4 & 1 \end{array}\right].$$

Downloaded from – https://aijaz.vercel.app

Step 3: Extract Inverse Matrix.

$$A^{-1} = \begin{bmatrix} -24 & 18 & 5 \\ 20 & -15 & -4 \\ -5 & 4 & 1 \end{bmatrix}.$$

■

Example 3.1.12. Solve the system using Gauss–Jordan elimination with **partial pivoting**:

$$\begin{aligned} x_1 + x_2 + x_3 &= 6, \\ 2x_1 + 4x_2 + 2x_3 &= 16, \\ -x_1 + 2x_2 + 5x_3 &= 18. \end{aligned}$$

Solution. **Step 1: Set up the Augmented Matrix.**

$$[A \mid \mathbf{b}] = \left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 2 & 4 & 2 & 16 \\ -1 & 2 & 5 & 18 \end{array}\right].$$

Step 2: Elimination with Partial Pivoting.

- *Column 1 Pivoting:* In column 1, maximum absolute value is $|2|$ in Row 2. Swap $R_1 \leftrightarrow R_2$:

$$\left[\begin{array}{ccc|c} 2 & 4 & 2 & 16 \\ 1 & 1 & 1 & 6 \\ -1 & 2 & 5 & 18 \end{array}\right].$$

  Scale $R_1 \to \frac{1}{2}R_1$:

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 8 \\ 1 & 1 & 1 & 6 \\ -1 & 2 & 5 & 18 \end{array}\right].$$

  Eliminate below pivot: $R_2 \to R_2 - R_1$ and $R_3 \to R_3 + R_1$:

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 8 \\ 0 & -1 & 0 & -2 \\ 0 & 4 & 6 & 26 \end{array}\right].$$

- *Column 2 Pivoting:* In column 2 (rows 2, 3), maximum absolute value is $|4|$ in Row 3. Swap $R_2 \leftrightarrow R_3$:

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 8 \\ 0 & 4 & 6 & 26 \\ 0 & -1 & 0 & -2 \end{array}\right].$$

  Scale $R_2 \to \frac{1}{4}R_2$:

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 8 \\ 0 & 1 & 1.5 & 6.5 \\ 0 & -1 & 0 & -2 \end{array}\right].$$

  Eliminate above and below: $R_1 \to R_1 - 2R_2$ and $R_3 \to R_3 + R_2$:

$$\left[\begin{array}{ccc|c} 1 & 0 & -2 & -5 \\ 0 & 1 & 1.5 & 6.5 \\ 0 & 0 & 1.5 & 4.5 \end{array}\right].$$

Downloaded from – https://aijaz.vercel.app

- *Column 3 Pivot:* Scale $R_3 \to \frac{1}{1.5}R_3$:

$$\left[\begin{array}{ccc|c} 1 & 0 & -2 & -5 \\ 0 & 1 & 1.5 & 6.5 \\ 0 & 0 & 1 & 3 \end{array}\right].$$

Eliminate above pivot: $R_1 \to R_1 + 2R_3$ and $R_2 \to R_2 - 1.5R_3$:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array}\right].$$

Step 3: Solution. The solution is $x_1 = 1$, $x_2 = 2$, $x_3 = 3$. ■

3.2 Iterative Methods for Solving Linear Systems

In contrast to direct methods, which aim to obtain the exact solution of a linear system $A\mathbf{x} = \mathbf{b}$ in a finite number of deterministic operations, *iterative methods* construct a sequence of successive approximations $\{\mathbf{x}^{(k)}\}_{k=0}^{\infty}$ that converges to the exact solution $\mathbf{x}^*$ as $k \to \infty$.

Iterative techniques are particularly advantageous for large, sparse linear systems (such as those arising from the numerical solution of partial differential equations and finite element analysis), where direct methods become computationally expensive and memory-intensive due to matrix fill-in.

Definition 3.2.1 (Iterative Method)**.** An iterative method for solving $A\mathbf{x} = \mathbf{b}$ is a recurrence scheme of the form

$$\mathbf{x}^{(k+1)} = T\mathbf{x}^{(k)} + \mathbf{c}, \quad k = 0, 1, 2, \ldots \tag{3.14}$$

where $T \in \mathbb{R}^{n \times n}$ is called the *iteration matrix*, $\mathbf{c} \in \mathbb{R}^n$ is a constant vector derived from A and $\mathbf{b}$, and $\mathbf{x}^{(0)}$ is a chosen initial approximation vector.

3.2.1 Jacobi Iterative Method

The Jacobi iterative method, also known as the *method of simultaneous displacements*, is one of the foundational iterative algorithms for solving linear algebraic systems.

Derivation and Component-wise Formulation

Consider a linear system of n equations in n variables:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n. \end{aligned} \tag{3.15}$$

Assuming that all diagonal elements of A are non-zero ($a_{ii} \neq 0$ for $i = 1, 2, \ldots, n$), we can express the i-th variable x_i explicitly in terms of the remaining variables:

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^{n} a_{ij}x_j \right), \quad i = 1, 2, \ldots, n. \tag{3.16}$$

In the Jacobi iterative scheme, the values of the variables at iteration $k+1$ are calculated strictly using the values obtained at the previous iteration k.

Downloaded from – https://aijaz.vercel.app

Definition 3.2.2 (Jacobi Iteration Formula)**.** Given an initial approximation $\mathbf{x}^{(0)} = \left(x_1^{(0)}, x_2^{(0)}, \ldots, x_n^{(0)}\right)^T$, the Jacobi iterative update for $k = 0, 1, 2, \ldots$ is defined as:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^{n} a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \ldots, n. \tag{3.17}$$

Notice that each component $x_i^{(k+1)}$ at step $k+1$ is computed independently using only elements of the vector $\mathbf{x}^{(k)}$ from step k.

Matrix Formulation

To express the Jacobi method in matrix form, we split the coefficient matrix A into its diagonal (D), strictly lower triangular (L), and strictly upper triangular (U) components:

$$A = D + L + U, \tag{3.18}$$

where

$$D = \begin{bmatrix} a_{11} & 0 & \ldots & 0 \\ 0 & a_{22} & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & a_{nn} \end{bmatrix}, \quad L = \begin{bmatrix} 0 & 0 & \ldots & 0 \\ a_{21} & 0 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \ldots & 0 \end{bmatrix}, \quad U = \begin{bmatrix} 0 & a_{12} & \ldots & a_{1n} \\ 0 & 0 & \ldots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & 0 \end{bmatrix}. \tag{3.19}$$

Rewriting $A\mathbf{x} = \mathbf{b}$ as $(D + L + U)\mathbf{x} = \mathbf{b}$, we obtain:

$$D\mathbf{x} = \mathbf{b} - (L + U)\mathbf{x}. \tag{3.20}$$

Since $a_{ii} \neq 0$ for all i, D is invertible. Multiplying by D^{-1} yields:

$$\mathbf{x} = -D^{-1}(L + U)\mathbf{x} + D^{-1}\mathbf{b}. \tag{3.21}$$

This gives the matrix recurrence form of Jacobi iteration:

$$\mathbf{x}^{(k+1)} = T_J \mathbf{x}^{(k)} + \mathbf{c}_J, \quad k = 0, 1, 2, \ldots \tag{3.22}$$

where

$$T_J = -D^{-1}(L + U) \quad \text{and} \quad \mathbf{c}_J = D^{-1}\mathbf{b}. \tag{3.23}$$

The matrix T_J is known as the *Jacobi iteration matrix*.

Convergence Analysis

Definition 3.2.3 (Strictly Diagonally Dominant Matrix)**.** A matrix $A = (a_{ij}) \in \mathbb{R}^{n \times n}$ is said to be *strictly diagonally dominant* (SDD) if the magnitude of the diagonal entry in each row is strictly greater than the sum of the magnitudes of all off-diagonal entries in that row:

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^{n} |a_{ij}|, \quad \text{for all } i = 1, 2, \ldots, n. \tag{3.24}$$

Downloaded from – https://aijaz.vercel.app

Theorem 3.2.4 (Sufficient Condition for Convergence). *If the coefficient matrix A is strictly diagonally dominant, then for any initial guess $\mathbf{x}^{(0)}$, the sequence of Jacobi iterations $\{\mathbf{x}^{(k)}\}_{k=0}^{\infty}$ converges to the unique exact solution $\mathbf{x}^* = A^{-1}\mathbf{b}$.*

Theorem 3.2.5 (Necessary and Sufficient Condition for Convergence). *The Jacobi iterative scheme $\mathbf{x}^{(k+1)} = T_J\mathbf{x}^{(k)} + \mathbf{c}_J$ converges for any initial vector $\mathbf{x}^{(0)}$ if and only if the spectral radius of the iteration matrix T_J satisfies:*

$$\rho(T_J) < 1, \tag{3.25}$$

where $\rho(T_J) = \max\{|\lambda| : \lambda \text{ is an eigenvalue of } T_J\}$.

Stopping Criteria

In computational practice, the iterative loop is terminated when a specified error tolerance $\epsilon > 0$ is satisfied. Common stopping criteria include:

i. **Absolute Error Criterion:**

$$\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_\infty = \max_{1\le i\le n} |x_i^{(k+1)} - x_i^{(k)}| < \epsilon. \tag{3.26}$$

ii. **Relative Error Criterion:**

$$\frac{\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_\infty}{\|\mathbf{x}^{(k+1)}\|_\infty} < \epsilon. \tag{3.27}$$

iii. **Residual Norm Criterion:**

$$\|\mathbf{r}^{(k+1)}\|_\infty = \|\mathbf{b} - A\mathbf{x}^{(k+1)}\|_\infty < \epsilon. \tag{3.28}$$

Example 3.2.6. Solve the following strictly diagonally dominant linear system using the Jacobi iterative method, starting with the initial approximation $\mathbf{x}^{(0)} = \begin{bmatrix}0 & 0 & 0\end{bmatrix}^T$. Perform three iterations and compare the result with the exact solution.

$$\begin{aligned} 10x_1 + x_2 + 2x_3 &= 10,\\ x_1 + 10x_2 - x_3 &= 22,\\ -2x_1 + 2x_2 + 10x_3 &= -8. \end{aligned}$$

Solution. **Step 1: Check Diagonal Dominance.**

- Row 1: $|10| = 10 > |1| + |2| = 3$.
- Row 2: $|10| = 10 > |1| + |-1| = 2$.
- Row 3: $|10| = 10 > |-2| + |2| = 4$.

Since A is strictly diagonally dominant, convergence of the Jacobi method is guaranteed.

Step 2: Formulate the Jacobi Iteration Equations. Rearranging each equation to express x_1, x_2, x_3 yields:

$$\begin{aligned} x_1^{(k+1)} &= \frac{10 - x_2^{(k)} - 2x_3^{(k)}}{10} = 1 - 0.1x_2^{(k)} - 0.2x_3^{(k)},\\ x_2^{(k+1)} &= \frac{22 - x_1^{(k)} + x_3^{(k)}}{10} = 2.2 - 0.1x_1^{(k)} + 0.1x_3^{(k)},\\ x_3^{(k+1)} &= \frac{-8 + 2x_1^{(k)} - 2x_2^{(k)}}{10} = -0.8 + 0.2x_1^{(k)} - 0.2x_2^{(k)}. \end{aligned}$$

Downloaded from – https://aijaz.vercel.app

Step 3: Perform Iterations.
Iteration 1 ($k = 0$): Using $\mathbf{x}^{(0)} = (0, 0, 0)^T$:

$$\begin{aligned} x_1^{(1)} &= 1 - 0.1(0) - 0.2(0) = 1.0, \\ x_2^{(1)} &= 2.2 - 0.1(0) + 0.1(0) = 2.2, \\ x_3^{(1)} &= -0.8 + 0.2(0) - 0.2(0) = -0.8. \end{aligned}$$

So, $\mathbf{x}^{(1)} = \begin{bmatrix} 1.0 & 2.2 & -0.8 \end{bmatrix}^T$.
Iteration 2 ($k = 1$): Using $\mathbf{x}^{(1)} = (1.0, 2.2, -0.8)^T$:

$$\begin{aligned} x_1^{(2)} &= 1 - 0.1(2.2) - 0.2(-0.8) = 1 - 0.22 + 0.16 = 0.94, \\ x_2^{(2)} &= 2.2 - 0.1(1.0) + 0.1(-0.8) = 2.2 - 0.10 - 0.08 = 2.02, \\ x_3^{(2)} &= -0.8 + 0.2(1.0) - 0.2(2.2) = -0.8 + 0.20 - 0.44 = -1.04. \end{aligned}$$

So, $\mathbf{x}^{(2)} = \begin{bmatrix} 0.94 & 2.02 & -1.04 \end{bmatrix}^T$.
Iteration 3 ($k = 2$): Using $\mathbf{x}^{(2)} = (0.94, 2.02, -1.04)^T$:

$$\begin{aligned} x_1^{(3)} &= 1 - 0.1(2.02) - 0.2(-1.04) = 1 - 0.202 + 0.208 = 1.006, \\ x_2^{(3)} &= 2.2 - 0.1(0.94) + 0.1(-1.04) = 2.2 - 0.094 - 0.104 = 2.002, \\ x_3^{(3)} &= -0.8 + 0.2(0.94) - 0.2(2.02) = -0.8 + 0.188 - 0.404 = -1.016. \end{aligned}$$

So, $\mathbf{x}^{(3)} = \begin{bmatrix} 1.006 & 2.002 & -1.016 \end{bmatrix}^T$.
Summary of Results:

Iteration k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$
0	0.000	0.000	0.000
1	1.000	2.200	-0.800
2	0.940	2.020	-1.040
3	1.006	2.002	-1.016
Exact	1.000	2.000	-1.000

After 3 iterations, the numerical solution is very close to the exact solution $\mathbf{x}^* = \begin{bmatrix} 1 & 2 & -1 \end{bmatrix}^T$. ■

Example 3.2.7. Consider the linear system:

$$\begin{aligned} x_1 + 2x_2 + 8x_3 &= 28, \\ 5x_1 - x_2 + x_3 &= 12, \\ 2x_1 + 6x_2 - x_3 &= 7. \end{aligned}$$

Show that the system is not strictly diagonally dominant in its current form. Reorder the equations to make it strictly diagonally dominant, and perform three Jacobi iterations starting from $\mathbf{x}^{(0)} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T$.

Solution. **Step 1: Check Diagonal Dominance and Reorder System.** Examining the original rows:

- Row 1: $|1| < |2| + |8| = 10$ (Not dominant).
- Row 2: $|-1| < |5| + |1| = 6$ (Not dominant).

- Row 3: $|-1| < |2| + |6| = 8$ (Not dominant).

To establish strict diagonal dominance, we reorder the equations:

- Place $5x_1 - x_2 + x_3 = 12$ in Row 1 ($|5| > |-1| + |1| = 2$).
- Place $2x_1 + 6x_2 - x_3 = 7$ in Row 2 ($|6| > |2| + |-1| = 3$).
- Place $x_1 + 2x_2 + 8x_3 = 28$ in Row 3 ($|8| > |1| + |2| = 3$).

The reordered diagonally dominant system is:

$$\begin{aligned} 5x_1 - x_2 + x_3 &= 12, \\ 2x_1 + 6x_2 - x_3 &= 7, \\ x_1 + 2x_2 + 8x_3 &= 28. \end{aligned}$$

Step 2: Jacobi Iteration Formulas.

$$\begin{aligned} x_1^{(k+1)} &= \frac{12 + x_2^{(k)} - x_3^{(k)}}{5}, \\ x_2^{(k+1)} &= \frac{7 - 2x_1^{(k)} + x_3^{(k)}}{6}, \\ x_3^{(k+1)} &= \frac{28 - x_1^{(k)} - 2x_2^{(k)}}{8}. \end{aligned}$$

Step 3: Perform Iterations.
Iteration 1 ($k = 0$): With $\mathbf{x}^{(0)} = (0, 0, 0)^T$:

$$\begin{aligned} x_1^{(1)} &= \frac{12}{5} = 2.4, \\ x_2^{(1)} &= \frac{7}{6} \approx 1.1667, \\ x_3^{(1)} &= \frac{28}{8} = 3.5. \end{aligned}$$

Iteration 2 ($k = 1$): With $\mathbf{x}^{(1)} = (2.4, 1.1667, 3.5)^T$:

$$\begin{aligned} x_1^{(2)} &= \frac{12 + 1.1667 - 3.5}{5} = \frac{9.6667}{5} = 1.9333, \\ x_2^{(2)} &= \frac{7 - 2(2.4) + 3.5}{6} = \frac{5.7}{6} = 0.9500, \\ x_3^{(2)} &= \frac{28 - 2.4 - 2(1.1667)}{8} = \frac{23.2666}{8} = 2.9083. \end{aligned}$$

Iteration 3 ($k = 2$): With $\mathbf{x}^{(2)} = (1.9333, 0.9500, 2.9083)^T$:

$$\begin{aligned} x_1^{(3)} &= \frac{12 + 0.9500 - 2.9083}{5} = \frac{10.0417}{5} = 2.0083, \\ x_2^{(3)} &= \frac{7 - 2(1.9333) + 2.9083}{6} = \frac{6.0417}{6} = 1.0070, \\ x_3^{(3)} &= \frac{28 - 1.9333 - 2(0.9500)}{8} = \frac{24.1667}{8} = 3.0208. \end{aligned}$$

The iterates converge steadily toward the exact solution $\mathbf{x}^* = \begin{bmatrix} 2 & 1 & 3 \end{bmatrix}^T$. ■

Downloaded from – https://aijaz.vercel.app

Example 3.2.8. For the linear system

$$\begin{aligned} 4x_1 - x_2 &= 1, \\ -x_1 + 4x_2 &= 11, \end{aligned}$$

construct the explicit Jacobi iteration matrix T_J, compute its spectral radius $\rho(T_J)$ to analytically prove convergence, and execute four Jacobi iterations starting from $\mathbf{x}^{(0)} = \begin{bmatrix} 0 & 0 \end{bmatrix}^T$.

Solution. **Step 1: Construct the Matrix Formulation.** The system matrix A and vector $\mathbf{b}$ are:

$$A = \begin{bmatrix} 4 & -1 \\ -1 & 4 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 11 \end{bmatrix}.$$

Decomposing $A = D + L + U$:

$$D = \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix}, \quad L + U = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}.$$

The inverse of D is $D^{-1} = \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}$.

The Jacobi iteration matrix T_J and vector $\mathbf{c}_J$ are:

$$T_J = -D^{-1}(L+U) = -\begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}\begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0.25 \\ 0.25 & 0 \end{bmatrix},$$

$$\mathbf{c}_J = D^{-1}\mathbf{b} = \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}\begin{bmatrix} 1 \\ 11 \end{bmatrix} = \begin{bmatrix} 0.25 \\ 2.75 \end{bmatrix}.$$

Step 2: Calculate Spectral Radius $\rho(T_J)$. The characteristic equation of T_J is:

$$\det(T_J - \lambda I) = \det\begin{bmatrix} -\lambda & 0.25 \\ 0.25 & -\lambda \end{bmatrix} = \lambda^2 - 0.0625 = 0.$$

The eigenvalues are $\lambda_1 = 0.25$ and $\lambda_2 = -0.25$. Thus, the spectral radius is:

$$\rho(T_J) = \max\{|0.25|, |-0.25|\} = 0.25.$$

Since $\rho(T_J) = 0.25 < 1$, the Jacobi method is guaranteed to converge for any initial vector.

Step 3: Perform Iterations using Vector Recurrence $\mathbf{x}^{(k+1)} = T_J\mathbf{x}^{(k)} + \mathbf{c}_J$.

- $k = 0$:
$$\mathbf{x}^{(1)} = \begin{bmatrix} 0 & 0.25 \\ 0.25 & 0 \end{bmatrix}\begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.25 \\ 2.75 \end{bmatrix} = \begin{bmatrix} 0.25 \\ 2.75 \end{bmatrix}.$$
- $k = 1$:
$$\mathbf{x}^{(2)} = \begin{bmatrix} 0 & 0.25 \\ 0.25 & 0 \end{bmatrix}\begin{bmatrix} 0.25 \\ 2.75 \end{bmatrix} + \begin{bmatrix} 0.25 \\ 2.75 \end{bmatrix} = \begin{bmatrix} 0.6875 + 0.25 \\ 0.0625 + 2.75 \end{bmatrix} = \begin{bmatrix} 0.9375 \\ 2.8125 \end{bmatrix}.$$
- $k = 2$:
$$\mathbf{x}^{(3)} = \begin{bmatrix} 0 & 0.25 \\ 0.25 & 0 \end{bmatrix}\begin{bmatrix} 0.9375 \\ 2.8125 \end{bmatrix} + \begin{bmatrix} 0.25 \\ 2.75 \end{bmatrix} = \begin{bmatrix} 0.703125 + 0.25 \\ 0.234375 + 2.75 \end{bmatrix} = \begin{bmatrix} 0.953125 \\ 2.984375 \end{bmatrix}.$$
- $k = 3$:
$$\mathbf{x}^{(4)} = \begin{bmatrix} 0 & 0.25 \\ 0.25 & 0 \end{bmatrix}\begin{bmatrix} 0.953125 \\ 2.984375 \end{bmatrix} + \begin{bmatrix} 0.25 \\ 2.75 \end{bmatrix} = \begin{bmatrix} 0.746094 + 0.25 \\ 0.238281 + 2.75 \end{bmatrix} = \begin{bmatrix} 0.996094 \\ 2.988281 \end{bmatrix}.$$

The iterates rapidly approach the exact solution $\mathbf{x}^* = \begin{bmatrix} 1 & 3 \end{bmatrix}^T$. ■

Downloaded from – https://aijaz.vercel.app

3.2.2 Gauss–Seidel Iterative Method

The Gauss–Seidel iterative method, also known as the *method of successive displacements*, is a modification of the Jacobi iterative method. In the Jacobi method, all components of the new approximation vector $\mathbf{x}^{(k+1)}$ are computed using only the components from the previous vector $\mathbf{x}^{(k)}$. However, when computing the i-th component $x_i^{(k+1)}$ sequentially for $i = 1, 2, \ldots, n$, the newly computed updated values $x_1^{(k+1)}, x_2^{(k+1)}, \ldots, x_{i-1}^{(k+1)}$ are already available.

The core principle of the Gauss–Seidel method is to immediately utilize these updated values as soon as they are calculated within the current iteration step, thereby accelerating the rate of convergence.

Derivation and Component-wise Formulation

Consider the $n \times n$ linear system $A\mathbf{x} = \mathbf{b}$ with non-zero diagonal entries $a_{ii} \neq 0$ for $i = 1, 2, \ldots, n$:

$$\sum_{j=1}^{n} a_{ij} x_j = b_i, \quad i = 1, 2, \ldots, n. \tag{3.29}$$

Isolating x_i on the left-hand side and splitting the sum into terms preceding i and terms following i, we obtain:

$$a_{ii} x_i = b_i - \sum_{j=1}^{i-1} a_{ij} x_j - \sum_{j=i+1}^{n} a_{ij} x_j. \tag{3.30}$$

In the Gauss–Seidel method, the values $x_1, \ldots, x_{i-1}$ at step $k+1$ are replaced by their newly computed values $x_1^{(k+1)}, \ldots, x_{i-1}^{(k+1)}$, while $x_{i+1}, \ldots, x_n$ retain their values $x_{i+1}^{(k)}, \ldots, x_n^{(k)}$ from step k.

Definition 3.2.9 (Gauss–Seidel Iteration Formula)**.** Given an initial vector $\mathbf{x}^{(0)} = \left(x_1^{(0)}, x_2^{(0)}, \ldots, x_n^{(0)}\right)^T$, the Gauss–Seidel recurrence for $k = 0, 1, 2, \ldots$ is defined as:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \ldots, n. \tag{3.31}$$

Matrix Formulation

Decomposing the coefficient matrix A into $A = D + L + U$, where D is the diagonal matrix, L is the strictly lower triangular matrix, and U is the strictly upper triangular matrix:

$$(D + L + U)\mathbf{x} = \mathbf{b} \implies (D + L)\mathbf{x} = -U\mathbf{x} + \mathbf{b}. \tag{3.32}$$

Assigning iteration levels $k+1$ to the terms on the left and k to the terms on the right yields:

$$(D + L)\mathbf{x}^{(k+1)} = -U\mathbf{x}^{(k)} + \mathbf{b}. \tag{3.33}$$

Since $a_{ii} \neq 0$ for all i, the lower triangular matrix $(D + L)$ is non-singular and invertible. Multiplying by $(D + L)^{-1}$ gives the matrix form:

$$\mathbf{x}^{(k+1)} = T_{GS}\mathbf{x}^{(k)} + \mathbf{c}_{GS}, \quad k = 0, 1, 2, \ldots \tag{3.34}$$

where

$$T_{GS} = -(D + L)^{-1}U \quad \text{and} \quad \mathbf{c}_{GS} = (D + L)^{-1}\mathbf{b}. \tag{3.35}$$

The matrix T_{GS} is called the *Gauss–Seidel iteration matrix*.

Downloaded from – https://aijaz.vercel.app

Remark 3.2.10 (Computational and Memory Efficiency). In computer implementations, the Gauss–Seidel algorithm requires storing only a single vector array in memory. As each component $x_i^{(k+1)}$ is calculated, it directly overwrites $x_i^{(k)}$ in place. In contrast, the Jacobi method requires storing two distinct vector arrays, $\mathbf{x}^{(k)}$ and $\mathbf{x}^{(k+1)}$, simultaneously.

Example 3.2.11. Solve the following strictly diagonally dominant linear system using the Gauss–Seidel iterative method with initial guess $\mathbf{x}^{(0)} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T$. Perform three iterations and compare the convergence with the exact solution $\mathbf{x}^* = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}^T$.

$$\begin{aligned} 8x_1 + x_2 - x_3 &= 8, \\ 2x_1 + 10x_2 + x_3 &= 13, \\ x_1 - x_2 + 6x_3 &= 6. \end{aligned}$$

Solution. **Step 1: Verify Diagonal Dominance.**

- Row 1: $|8| = 8 > |1| + |-1| = 2$.
- Row 2: $|10| = 10 > |2| + |1| = 3$.
- Row 3: $|6| = 6 > |1| + |-1| = 2$.

A is strictly diagonally dominant, guaranteeing convergence.

Step 2: Formulate Gauss–Seidel Iterative Equations.

$$\begin{aligned} x_1^{(k+1)} &= \frac{8 - x_2^{(k)} + x_3^{(k)}}{8}, \\ x_2^{(k+1)} &= \frac{13 - 2x_1^{(k+1)} - x_3^{(k)}}{10}, \\ x_3^{(k+1)} &= \frac{6 - x_1^{(k+1)} + x_2^{(k+1)}}{6}. \end{aligned}$$

Step 3: Execute Iterations.

Iteration 1 ($k = 0$): Starting with $x_2^{(0)} = 0, x_3^{(0)} = 0$:

$$\begin{aligned} x_1^{(1)} &= \frac{8 - 0 + 0}{8} = 1.0, \\ x_2^{(1)} &= \frac{13 - 2(1.0) - 0}{10} = \frac{11}{10} = 1.1, \\ x_3^{(1)} &= \frac{6 - 1.0 + 1.1}{6} = \frac{6.1}{6} \approx 1.0167. \end{aligned}$$

Vector at end of step 1: $\mathbf{x}^{(1)} = \begin{bmatrix} 1.0000 & 1.1000 & 1.0167 \end{bmatrix}^T$.

Iteration 2 ($k = 1$): Using updated values as soon as available:

$$\begin{aligned} x_1^{(2)} &= \frac{8 - 1.1000 + 1.0167}{8} = \frac{7.9167}{8} \approx 0.9896, \\ x_2^{(2)} &= \frac{13 - 2(0.9896) - 1.0167}{10} = \frac{10.0041}{10} \approx 1.0004, \\ x_3^{(2)} &= \frac{6 - 0.9896 + 1.0004}{6} = \frac{6.0108}{6} = 1.0018. \end{aligned}$$

Downloaded from – https://aijaz.vercel.app

Vector at end of step 2: $\mathbf{x}^{(2)} = \begin{bmatrix} 0.9896 & 1.0004 & 1.0018 \end{bmatrix}^T$.

Iteration 3 ($k = 2$):

$$x_1^{(3)} = \frac{8 - 1.0004 + 1.0018}{8} = \frac{8.0014}{8} = 1.0002,$$
$$x_2^{(3)} = \frac{13 - 2(1.0002) - 1.0018}{10} = \frac{9.9978}{10} \approx 0.9998,$$
$$x_3^{(3)} = \frac{6 - 1.0002 + 0.9998}{6} = \frac{5.9996}{6} \approx 0.9999.$$

Vector at end of step 3: $\mathbf{x}^{(3)} = \begin{bmatrix} 1.0002 & 0.9998 & 0.9999 \end{bmatrix}^T$.

Summary Table:

Iteration k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$
0	0.0000	0.0000	0.0000
1	1.0000	1.1000	1.0167
2	0.9896	1.0004	1.0018
3	1.0002	0.9998	0.9999
Exact	1.0000	1.0000	1.0000

The method achieves 4-decimal-place accuracy in just 3 iterations. ■

Example 3.2.12. Reorder the following system to establish strict diagonal dominance, and apply the Gauss–Seidel method for three iterations starting with $\mathbf{x}^{(0)} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T$:

$$x_1 + 2x_2 + 10x_3 = 25,$$
$$10x_1 + x_2 + 2x_3 = 16,$$
$$x_1 + 10x_2 - x_3 = 19.$$

Solution. **Step 1: Reorder for Strict Diagonal Dominance.**

- Equation 2 has dominant coefficient for x_1 ($10 > 1 + 2$). Move to Row 1.
- Equation 3 has dominant coefficient for x_2 ($10 > 1 + 1$). Move to Row 2.
- Equation 1 has dominant coefficient for x_3 ($10 > 1 + 2$). Move to Row 3.

The reordered system is:

$$10x_1 + x_2 + 2x_3 = 16,$$
$$x_1 + 10x_2 - x_3 = 19,$$
$$x_1 + 2x_2 + 10x_3 = 25.$$

Step 2: Gauss–Seidel Recurrence Formulas.

$$x_1^{(k+1)} = \frac{16 - x_2^{(k)} - 2x_3^{(k)}}{10},$$
$$x_2^{(k+1)} = \frac{19 - x_1^{(k+1)} + x_3^{(k)}}{10},$$
$$x_3^{(k+1)} = \frac{25 - x_1^{(k+1)} - 2x_2^{(k+1)}}{10}.$$

Step 3: Perform Iterations.

Downloaded from – https://aijaz.vercel.app

Iteration 1 ($k = 0$): With $\mathbf{x}^{(0)} = (0, 0, 0)^T$:

$$x_1^{(1)} = \frac{16 - 0 - 0}{10} = 1.6000,$$
$$x_2^{(1)} = \frac{19 - 1.6000 + 0}{10} = 1.7400,$$
$$x_3^{(1)} = \frac{25 - 1.6000 - 2(1.7400)}{10} = \frac{19.9200}{10} = 1.9920.$$

Iteration 2 ($k = 1$): Using $\mathbf{x}^{(1)} = (1.6000, 1.7400, 1.9920)^T$:

$$x_1^{(2)} = \frac{16 - 1.7400 - 2(1.9920)}{10} = \frac{10.2760}{10} = 1.0276,$$
$$x_2^{(2)} = \frac{19 - 1.0276 + 1.9920}{10} = \frac{19.9644}{10} = 1.9964,$$
$$x_3^{(2)} = \frac{25 - 1.0276 - 2(1.9964)}{10} = \frac{19.9796}{10} = 1.9980.$$

Iteration 3 ($k = 2$): Using $\mathbf{x}^{(2)} = (1.0276, 1.9964, 1.9980)^T$:

$$x_1^{(3)} = \frac{16 - 1.9964 - 2(1.9980)}{10} = \frac{10.0076}{10} = 1.0008,$$
$$x_2^{(3)} = \frac{19 - 1.0008 + 1.9980}{10} = \frac{19.9972}{10} = 1.9997,$$
$$x_3^{(3)} = \frac{25 - 1.0008 - 2(1.9997)}{10} = \frac{19.9998}{10} = 2.0000.$$

The sequence converges smoothly to the exact solution $\mathbf{x}^* = \begin{bmatrix} 1 & 2 & 2 \end{bmatrix}^T$. ■

Example 3.2.13. Construct the explicit Gauss–Seidel iteration matrix T_{GS} for the linear system:

$$4x_1 - x_2 = 1,$$
$$-x_1 + 4x_2 = 11.$$

Compute its spectral radius $\rho(T_{GS})$, compare it directly with the spectral radius of the Jacobi iteration matrix $\rho(T_J) = 0.25$, and carry out two Gauss–Seidel iterations starting from $\mathbf{x}^{(0)} = \begin{bmatrix} 0 & 0 \end{bmatrix}^T$.

Solution. **Step 1: Construct Matrix Operators.** Decomposing matrix $A = \begin{bmatrix} 4 & -1 \\ -1 & 4 \end{bmatrix} = D + L + U$:

$$D + L = \begin{bmatrix} 4 & 0 \\ -1 & 4 \end{bmatrix}, \quad U = \begin{bmatrix} 0 & -1 \\ 0 & 0 \end{bmatrix}.$$

Computing $(D + L)^{-1}$:

$$(D + L)^{-1} = \begin{bmatrix} \frac{1}{4} & 0 \\ \frac{1}{16} & \frac{1}{4} \end{bmatrix} = \begin{bmatrix} 0.25 & 0 \\ 0.0625 & 0.25 \end{bmatrix}.$$

The Gauss–Seidel iteration matrix T_{GS} is:

$$T_{GS} = -(D + L)^{-1}U = -\begin{bmatrix} 0.25 & 0 \\ 0.0625 & 0.25 \end{bmatrix}\begin{bmatrix} 0 & -1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0.25 \\ 0 & 0.0625 \end{bmatrix}.$$

Downloaded from – https://aijaz.vercel.app

The constant vector $\mathbf{c}_{GS}$ is:

$$\mathbf{c}_{GS} = (D+L)^{-1}\mathbf{b} = \begin{bmatrix} 0.25 & 0 \\ 0.0625 & 0.25 \end{bmatrix} \begin{bmatrix} 1 \\ 11 \end{bmatrix} = \begin{bmatrix} 0.25 \\ 0.0625 + 2.75 \end{bmatrix} = \begin{bmatrix} 0.25 \\ 2.8125 \end{bmatrix}.$$

Step 2: Spectral Radius Comparison. The characteristic equation of T_{GS} is:

$$\det(T_{GS} - \lambda I) = \det \begin{bmatrix} -\lambda & 0.25 \\ 0 & 0.0625 - \lambda \end{bmatrix} = \lambda(\lambda - 0.0625) = 0.$$

The eigenvalues are $\lambda_1 = 0$ and $\lambda_2 = 0.0625$. Thus, the spectral radius is:

$$\rho(T_{GS}) = 0.0625.$$

Comparing with the Jacobi spectral radius $\rho(T_J) = 0.25$:

$$\rho(T_{GS}) = 0.0625 = (0.25)^2 = [\rho(T_J)]^2.$$

This demonstrates analytically that for this system, the Gauss–Seidel method converges four times faster per iteration than the Jacobi method.

Step 3: Execute Iterations.

- $k = 0$:

$$\mathbf{x}^{(1)} = T_{GS}\mathbf{x}^{(0)} + \mathbf{c}_{GS} = \begin{bmatrix} 0 & 0.25 \\ 0 & 0.0625 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.25 \\ 2.8125 \end{bmatrix} = \begin{bmatrix} 0.25 \\ 2.8125 \end{bmatrix}.$$

- $k = 1$:

$$\mathbf{x}^{(2)} = \begin{bmatrix} 0 & 0.25 \\ 0 & 0.0625 \end{bmatrix} \begin{bmatrix} 0.25 \\ 2.8125 \end{bmatrix} + \begin{bmatrix} 0.25 \\ 2.8125 \end{bmatrix} = \begin{bmatrix} 0.703125 + 0.25 \\ 0.175781 + 2.8125 \end{bmatrix} = \begin{bmatrix} 0.953125 \\ 2.988281 \end{bmatrix}.$$

After only two iterations, $\mathbf{x}^{(2)} \approx \begin{bmatrix} 0.9531 & 2.9883 \end{bmatrix}^T$, which is extremely close to the exact solution $\mathbf{x}^* = \begin{bmatrix} 1 & 3 \end{bmatrix}^T$. ■

3.2.3 Conditions for Convergence of Iterative Methods

To establish a general theoretical framework for iterative linear solvers, we analyze the convergence behavior of linear fixed-point iterations of the form:

$$\mathbf{x}^{(k+1)} = T\mathbf{x}^{(k)} + \mathbf{c}, \quad k = 0, 1, 2, \ldots \tag{3.36}$$

General Error Propagation and Spectral Radius Theorem

Let $\mathbf{x}^*$ denote the exact solution of $A\mathbf{x} = \mathbf{b}$, which satisfies the fixed-point equation:

$$\mathbf{x}^* = T\mathbf{x}^* + \mathbf{c}. \tag{3.37}$$

Definition 3.2.14 (Error Vector)**.** At iteration step k, the error vector is defined as:

$$\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^*. \tag{3.38}$$

Subtracting the exact solution equation from the iterative formula yields the error recurrence relation:

$$\mathbf{e}^{(k+1)} = \mathbf{x}^{(k+1)} - \mathbf{x}^* = (T\mathbf{x}^{(k)} + \mathbf{c}) - (T\mathbf{x}^* + \mathbf{c}) = T(\mathbf{x}^{(k)} - \mathbf{x}^*) = T\mathbf{e}^{(k)}. \tag{3.39}$$

By mathematical induction, the error after k steps is related to the initial error $\mathbf{e}^{(0)}$ by:

$$\mathbf{e}^{(k)} = T^k\mathbf{e}^{(0)}, \quad k = 1, 2, 3, \ldots \tag{3.40}$$

For the iterative scheme to converge for any arbitrary initial error $\mathbf{e}^{(0)}$, we require that:

$$\lim_{k\to\infty} \mathbf{e}^{(k)} = \mathbf{0} \iff \lim_{k\to\infty} T^k = 0. \tag{3.41}$$

Downloaded from – https://aijaz.vercel.app

Theorem 3.2.15 (Fundamental Spectral Radius Convergence Theorem). *The iterative process* $\mathbf{x}^{(k+1)} = T\mathbf{x}^{(k)} + \mathbf{c}$ *converges for any initial guess* $\mathbf{x}^{(0)}$ *if and only if the spectral radius of the iteration matrix* T *satisfies:*

$$\rho(T) < 1. \tag{3.42}$$

Theorem 3.2.16 (Matrix Norm Sufficient Condition). *If* $\|T\|$ *is any natural (induced) matrix norm such that* $\|T\| < 1$*, then* $\rho(T) \leq \|T\| < 1$*, and the iteration converges for any initial vector* $\mathbf{x}^{(0)}$*. Furthermore, the error bound satisfies:*

$$\|\mathbf{e}^{(k)}\| \leq \|T\|^k \|\mathbf{e}^{(0)}\|, \quad \text{and} \quad \|\mathbf{x}^{(k)} - \mathbf{x}^*\| \leq \frac{\|T\|^k}{1 - \|T\|} \|\mathbf{x}^{(1)} - \mathbf{x}^{(0)}\|. \tag{3.43}$$

Important Convergence Theorems for Standard Methods

Theorem 3.2.17 (Stein–Rosenberg Theorem). *Let the coefficient matrix* $A = D + L + U$ *have non-positive off-diagonal elements* ($a_{ij} \leq 0$ *for all* $i \neq j$*) and positive diagonal elements* ($a_{ii} > 0$*). Then for the Jacobi matrix* T_J *and Gauss–Seidel matrix* T_{GS}*, precisely one of the following statements holds:*

i. $\rho(T_J) = \rho(T_{GS}) = 0$.

ii. $0 < \rho(T_{GS}) < \rho(T_J) < 1$ *(both methods converge, but Gauss–Seidel converges faster).*

iii. $\rho(T_J) = \rho(T_{GS}) = 1$.

iv. $1 < \rho(T_J) < \rho(T_{GS})$ *(both methods diverge, and Gauss–Seidel diverges faster).*

Theorem 3.2.18 (Symmetric Positive Definite Matrices). *If* $A \in \mathbb{R}^{n \times n}$ *is a real symmetric and positive definite matrix (i.e.,* $A^T = A$ *and* $\mathbf{x}^T A\mathbf{x} > 0$ *for all* $\mathbf{x} \neq \mathbf{0}$*), then the Gauss–Seidel method converges for any initial guess* $\mathbf{x}^{(0)}$*.*

Example 3.2.19. An iterative method uses an iteration matrix $T = \begin{bmatrix} 0.5 & 0.2 \\ 0.1 & 0.4 \end{bmatrix}$. Determine its eigenvalues, compute its spectral radius $\rho(T)$, confirm convergence, and calculate the minimum number of iterations k required to guarantee that the norm of the initial error is reduced by a factor of at least 10^{-4}.

Solution. **Step 1: Compute Eigenvalues of** T**.** The characteristic equation is:

$$\det(T - \lambda I) = \det \begin{bmatrix} 0.5 - \lambda & 0.2 \\ 0.1 & 0.4 - \lambda \end{bmatrix} = 0.$$

Expanding the determinant:

$$(0.5 - \lambda)(0.4 - \lambda) - (0.2)(0.1) = \lambda^2 - 0.9\lambda + 0.20 - 0.02 = \lambda^2 - 0.9\lambda + 0.18 = 0.$$

Solving for λ using the quadratic formula:

$$\lambda = \frac{0.9 \pm \sqrt{(0.9)^2 - 4(1)(0.18)}}{2} = \frac{0.9 \pm \sqrt{0.81 - 0.72}}{2} = \frac{0.9 \pm \sqrt{0.09}}{2} = \frac{0.9 \pm 0.3}{2}.$$

Downloaded from – https://aijaz.vercel.app

The eigenvalues are $\lambda_1 = 0.6$ and $\lambda_2 = 0.3$.

Step 2: Determine Spectral Radius.

$$\rho(T) = \max\{|0.6|, |0.3|\} = 0.6.$$

Since $\rho(T) = 0.6 < 1$, the iteration process converges for any initial guess.

Step 3: Calculate a Guaranteed Number of Iterations. The error satisfies $\mathbf{e}^{(k)} = T^k \mathbf{e}^{(0)}$. A guarantee in the infinity norm follows from $\|T^k\|_\infty \le \|T\|_\infty^k$. Here

$$\|T\|_\infty = \max\{0.5 + 0.2,\ 0.1 + 0.4\} = 0.7.$$

Therefore $\|\mathbf{e}^{(k)}\|_\infty \le (0.7)^k \|\mathbf{e}^{(0)}\|_\infty$. To ensure a reduction by at least 10^{-4}, it suffices that

$$(0.7)^k \le 10^{-4} \quad \Longleftrightarrow \quad k \ge \frac{-4 \ln 10}{\ln 0.7} \approx 25.82.$$

Thus $k = 26$ is a sufficient number of iterations in the infinity norm. The estimate $0.6^k \le 10^{-4}$ uses only the spectral radius and does not, by itself, guarantee the norm reduction for an arbitrary initial error and an arbitrary matrix norm. ■

Example 3.2.20. For the linear system $A\mathbf{x} = \mathbf{b}$ given by

$$\begin{aligned} 5x_1 - 2x_2 + x_3 &= 10, \\ x_1 + 4x_2 - 2x_3 &= 3, \\ 2x_1 - x_2 + 6x_3 &= 12, \end{aligned}$$

construct the Jacobi iteration matrix T_J. Test for convergence using both the row-sum norm $\|T_J\|_\infty$ and column-sum norm $\|T_J\|_1$.

Solution. **Step 1: Construct T_J.** From system matrix $A = \begin{bmatrix} 5 & -2 & 1 \\ 1 & 4 & -2 \\ 2 & -1 & 6 \end{bmatrix}$, we extract diagonal matrix $D = \operatorname{diag}(5, 4, 6)$. The Jacobi iteration matrix $T_J = -D^{-1}(L + U)$ is:

$$T_J = -\begin{bmatrix} \frac{1}{5} & 0 & 0 \\ 0 & \frac{1}{4} & 0 \\ 0 & 0 & \frac{1}{6} \end{bmatrix} \begin{bmatrix} 0 & -2 & 1 \\ 1 & 0 & -2 \\ 2 & -1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & \frac{2}{5} & -\frac{1}{5} \\ -\frac{1}{4} & 0 & \frac{2}{4} \\ -\frac{2}{6} & \frac{1}{6} & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0.4000 & -0.2000 \\ -0.2500 & 0 & 0.5000 \\ -0.3333 & 0.1667 & 0 \end{bmatrix}.$$

Step 2: Evaluate Row-Sum Norm $\|T_J\|_\infty$. The maximum absolute row sum is:

$$\begin{aligned} \text{Row 1 sum:} \quad & |0| + |0.4000| + |-0.2000| = 0.6000, \\ \text{Row 2 sum:} \quad & |-0.2500| + |0| + |0.5000| = 0.7500, \\ \text{Row 3 sum:} \quad & |-0.3333| + |0.1667| + |0| = 0.5000. \end{aligned}$$

Thus, $\|T_J\|_\infty = \max\{0.6000, 0.7500, 0.5000\} = 0.7500$. Since $\|T_J\|_\infty = 0.7500 < 1$, convergence is guaranteed.

Step 3: Evaluate Column-Sum Norm $\|T_J\|_1$. The maximum absolute column sum is:

$$\begin{aligned} \text{Col 1 sum:} \quad & |0| + |-0.2500| + |-0.3333| = 0.5833, \\ \text{Col 2 sum:} \quad & |0.4000| + |0| + |0.1667| = 0.5667, \\ \text{Col 3 sum:} \quad & |-0.2000| + |0.5000| + |0| = 0.7000. \end{aligned}$$

Thus, $\|T_J\|_1 = \max\{0.5833, 0.5667, 0.7000\} = 0.7000$. Since $\|T_J\|_1 = 0.7000 < 1$, convergence is independently confirmed. ■

Downloaded from – https://aijaz.vercel.app

Example 3.2.21. Consider the 2×2 parameter-dependent system:

$$x_1 + \alpha x_2 = 1,$$
$$\alpha x_1 + x_2 = 1,$$

where $\alpha \in \mathbb{R}$ is a real parameter.

a. Derive the exact condition on α for which the Jacobi method converges.

b. Derive the exact condition on α for which the Gauss–Seidel method converges.

c. Compare their rates of convergence for $\alpha = 0.5$ and $\alpha = 2.0$.

Solution. **Part (a): Jacobi Method Convergence Condition.** The matrix is $A = \begin{bmatrix} 1 & \alpha \\ \alpha & 1 \end{bmatrix}$. Decomposing gives $D = I$ and $L + U = \begin{bmatrix} 0 & \alpha \\ \alpha & 0 \end{bmatrix}$. The Jacobi iteration matrix is:

$$T_J = -D^{-1}(L + U) = \begin{bmatrix} 0 & -\alpha \\ -\alpha & 0 \end{bmatrix}.$$

Characteristic equation: $\det(T_J - \lambda I) = \lambda^2 - \alpha^2 = 0 \implies \lambda = \pm\alpha$. The spectral radius is $\rho(T_J) = |\alpha|$. Therefore, the Jacobi method converges if and only if $|\alpha| < 1$.

Part (b): Gauss–Seidel Method Convergence Condition. Decomposing into $D + L = \begin{bmatrix} 1 & 0 \\ \alpha & 1 \end{bmatrix}$ and $U = \begin{bmatrix} 0 & \alpha \\ 0 & 0 \end{bmatrix}$. Inverting $D + L$:

$$(D + L)^{-1} = \begin{bmatrix} 1 & 0 \\ -\alpha & 1 \end{bmatrix}.$$

The Gauss–Seidel matrix is:

$$T_{GS} = -(D + L)^{-1}U = -\begin{bmatrix} 1 & 0 \\ -\alpha & 1 \end{bmatrix}\begin{bmatrix} 0 & \alpha \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -\alpha \\ 0 & \alpha^2 \end{bmatrix}.$$

Characteristic equation: $\det(T_{GS} - \lambda I) = \lambda(\lambda - \alpha^2) = 0 \implies \lambda_1 = 0, \lambda_2 = \alpha^2$. The spectral radius is $\rho(T_{GS}) = \alpha^2 = |\alpha|^2$. Therefore, the Gauss–Seidel method converges if and only if $\alpha^2 < 1 \iff |\alpha| < 1$.

Part (c): Comparison for Specific Values of α.

- For $\alpha = 0.5$:
$$\rho(T_J) = 0.5 < 1, \quad \rho(T_{GS}) = (0.5)^2 = 0.25 < 1.$$
Both methods converge, but Gauss–Seidel converges twice as fast per step since $\rho(T_{GS}) = [\rho(T_J)]^2$.

- For $\alpha = 2.0$:
$$\rho(T_J) = 2.0 > 1, \quad \rho(T_{GS}) = 4.0 > 1.$$
Both methods diverge, with Gauss–Seidel diverging faster.

■

3.3 Eigenvalues and Eigenvectors

Eigenvalue problems form one of the central pillars of linear algebra and numerical analysis. They arise naturally in quantum mechanics, structural dynamics, stability analysis of differential equations, vibration analysis, and network analysis.

Downloaded from – https://aijaz.vercel.app

Definition 3.3.1 (Eigenvalue and Eigenvector)**.** Let $A \in \mathbb{C}^{n\times n}$ be a square matrix. A non-zero vector $\mathbf{v} \in \mathbb{C}^n \setminus \{\mathbf{0}\}$ is called an *eigenvector* of A if there exists a scalar $\lambda \in \mathbb{C}$ such that

$$A\mathbf{v} = \lambda\mathbf{v}. \tag{3.44}$$

The scalar λ is called the *eigenvalue* associated with the eigenvector $\mathbf{v}$.

Rewriting Equation (3.44) in homogeneous form yields:

$$(A - \lambda I)\mathbf{v} = \mathbf{0}, \tag{3.45}$$

where I is the $n \times n$ identity matrix. For a non-trivial vector $\mathbf{v} \neq \mathbf{0}$ to exist, the coefficient matrix $(A - \lambda I)$ must be singular.

Definition 3.3.2 (Characteristic Equation and Polynomial)**.** The matrix equation $\det(A - \lambda I) = 0$ is called the *characteristic equation* of A. The function

$$P(\lambda) = \det(A - \lambda I) = (-1)^n\lambda^n + c_{n-1}\lambda^{n-1} + \cdots + c_1\lambda + c_0 \tag{3.46}$$

is a polynomial of degree n in λ, known as the *characteristic polynomial* of A.

According to the Fundamental Theorem of Algebra, an $n \times n$ matrix A has exactly n eigenvalues (counting algebraic multiplicities) in the complex field $\mathbb{C}$.

Definition 3.3.3 (Spectrum and Spectral Radius)**.** The set of all eigenvalues of A, denoted by $\sigma(A) = \{\lambda_1, \lambda_2, \ldots, \lambda_n\}$, is called the *spectrum* of A. The *spectral radius* of A is defined as:

$$\rho(A) = \max_{1\leq i\leq n} |\lambda_i|. \tag{3.47}$$

Theorem 3.3.4 (Fundamental Properties of Eigenvalues)**.** *For any square matrix $A \in \mathbb{C}^{n\times n}$ with eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$:*

i. ***Trace Relation:*** *The sum of the eigenvalues equals the trace of A:*

$$\sum_{i=1}^{n} \lambda_i = tr(A) = \sum_{i=1}^{n} a_{ii}. \tag{3.48}$$

ii. ***Determinant Relation:*** *The product of the eigenvalues equals the determinant of A:*

$$\prod_{i=1}^{n} \lambda_i = \det(A). \tag{3.49}$$

iii. ***Shifted Eigenvalues:*** *For any scalar $c \in \mathbb{C}$, the matrix $(A - cI)$ has eigenvalues $\lambda_i - c$.*

iv. ***Inverse Eigenvalues:*** *If A is non-singular ($\det(A) \neq 0$), the matrix A^{-1} has eigenvalues $\frac{1}{\lambda_i}$.*

Theorem 3.3.5 (Gerschgorin Circle Theorem)**.** *Let $A = (a_{ij}) \in \mathbb{C}^{n\times n}$. Define the row sums of the*

Downloaded from – https://aijaz.vercel.app

off-diagonal magnitudes as:

$$R_i = \sum_{\substack{j=1 \\ j \neq i}}^{n} |a_{ij}|, \quad i = 1, 2, \ldots, n. \tag{3.50}$$

Every eigenvalue λ of A lies within at least one of the closed discs $D(a_{ii}, R_i)$ in the complex plane, defined by:

$$D(a_{ii}, R_i) = \{z \in \mathbb{C} : |z - a_{ii}| \leq R_i\}. \tag{3.51}$$

Consequently, $\sigma(A) \subseteq \bigcup_{i=1}^{n} D(a_{ii}, R_i)$.

Example 3.3.6. Find all eigenvalues and their corresponding eigenvectors for the 2×2 matrix:

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}.$$

Solution. **Step 1: Compute the Characteristic Polynomial.**

$$\det(A - \lambda I) = \det \begin{bmatrix} 4-\lambda & 1 \\ 2 & 3-\lambda \end{bmatrix} = (4-\lambda)(3-\lambda) - (2)(1) = \lambda^2 - 7\lambda + 10.$$

Setting $\det(A - \lambda I) = 0$:

$$\lambda^2 - 7\lambda + 10 = (\lambda - 5)(\lambda - 2) = 0.$$

Thus, the eigenvalues are $\lambda_1 = 5$ and $\lambda_2 = 2$.

Step 2: Find the Eigenvector for $\lambda_1 = 5$. Solve $(A - 5I)\mathbf{v}_1 = \mathbf{0}$:

$$\begin{bmatrix} 4-5 & 1 \\ 2 & 3-5 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

This gives the equation $-v_{11} + v_{12} = 0 \implies v_{11} = v_{12}$. Choosing $v_{12} = 1$:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Step 3: Find the Eigenvector for $\lambda_2 = 2$. Solve $(A - 2I)\mathbf{v}_2 = \mathbf{0}$:

$$\begin{bmatrix} 4-2 & 1 \\ 2 & 3-2 \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

This gives $2v_{21} + v_{22} = 0 \implies v_{22} = -2v_{21}$. Choosing $v_{21} = 1$:

$$\mathbf{v}_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

■

Example 3.3.7. Find the eigenvalues and an orthonormal basis of eigenvectors for the symmetric matrix:

$$A = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 1 & 0 \\ 0 & 0 & -1 \end{bmatrix}.$$

Downloaded from – https://aijaz.vercel.app

Solution. **Step 1: Compute Eigenvalues.**

$$\det(A-\lambda I)=\det\begin{bmatrix}1-\lambda & 2 & 0\\ 2 & 1-\lambda & 0\\ 0 & 0 & -1-\lambda\end{bmatrix}=(-1-\lambda)\left[(1-\lambda)^2-4\right]=0.$$

Solving $(-1-\lambda)(\lambda^2-2\lambda-3)=-(\lambda+1)(\lambda-3)(\lambda+1)=0$:

$$-(\lambda+1)^2(\lambda-3)=0.$$

The eigenvalues are $\lambda_1=3$ (multiplicity 1) and $\lambda_2=-1$ (multiplicity 2).

Step 2: Eigenvector for $\lambda_1=3$. Solve $(A-3I)\mathbf{v}_1=\mathbf{0}$:

$$\begin{bmatrix}-2 & 2 & 0\\ 2 & -2 & 0\\ 0 & 0 & -4\end{bmatrix}\begin{bmatrix}x\\ y\\ z\end{bmatrix}=\begin{bmatrix}0\\ 0\\ 0\end{bmatrix}\implies -2x+2y=0,\quad -4z=0.$$

Thus $x=y$ and $z=0$. Normalizing $\mathbf{v}_1=\begin{bmatrix}1 & 1 & 0\end{bmatrix}^T$ gives $\mathbf{u}_1=\begin{bmatrix}\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0\end{bmatrix}^T$.

Step 3: Eigenspace for $\lambda_2=-1$. Solve $(A+I)\mathbf{v}=\mathbf{0}$:

$$\begin{bmatrix}2 & 2 & 0\\ 2 & 2 & 0\\ 0 & 0 & 0\end{bmatrix}\begin{bmatrix}x\\ y\\ z\end{bmatrix}=\begin{bmatrix}0\\ 0\\ 0\end{bmatrix}\implies x+y=0.$$

Here z is free. A linearly independent basis for this eigenspace is:

$$\mathbf{v}_2=\begin{bmatrix}1\\ -1\\ 0\end{bmatrix},\quad \mathbf{v}_3=\begin{bmatrix}0\\ 0\\ 1\end{bmatrix}.$$

Normalizing yields $\mathbf{u}_2=\begin{bmatrix}\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0\end{bmatrix}^T$ and $\mathbf{u}_3=\begin{bmatrix}0 & 0 & 1\end{bmatrix}^T$. The vectors $\{\mathbf{u}_1,\mathbf{u}_2,\mathbf{u}_3\}$ form an orthonormal set. ■

Example 3.3.8. Apply the Gerschgorin Circle Theorem to bound the eigenvalues of the matrix:

$$A=\begin{bmatrix}4 & 1 & 1\\ 0.2 & 3 & 0.5\\ 1 & 0.5 & 5\end{bmatrix}.$$

Solution. **Step 1: Compute Gerschgorin Discs for Each Row.**

- *Row 1:* Center $c_1=a_{11}=4$. Radius $R_1=|a_{12}|+|a_{13}|=1+1=2$.
$$D_1=\{z\in\mathbb{C}:|z-4|\le 2\}\implies z\in[2,6].$$
- *Row 2:* Center $c_2=a_{22}=3$. Radius $R_2=|a_{21}|+|a_{23}|=0.2+0.5=0.7$.
$$D_2=\{z\in\mathbb{C}:|z-3|\le 0.7\}\implies z\in[2.3,3.7].$$
- *Row 3:* Center $c_3=a_{33}=5$. Radius $R_3=|a_{31}|+|a_{32}|=1+0.5=1.5$.
$$D_3=\{z\in\mathbb{C}:|z-5|\le 1.5\}\implies z\in[3.5,6.5].$$

Step 2: Determine Global Bounds. Combining the intervals, all eigenvalues of A lie within the union $\bigcup_{i=1}^{3}D_i$:

$$\sigma(A)\subset[2.3,3.7]\cup[2,6.5]=[2.0,6.5].$$

Furthermore, since all discs lie entirely in the right half of the complex plane ($z>0$), A is non-singular and all real eigenvalues are strictly positive. ■

Downloaded from – https://aijaz.vercel.app

3.3.1 Power Method for Dominant Eigenvalue

The Power Method is an iterative numerical technique used to determine the *dominant eigenvalue* (i.e., the eigenvalue with the largest absolute magnitude) and its associated eigenvector.

Derivation and Computational Algorithm

Assume $A \in \mathbb{R}^{n\times n}$ has n linearly independent eigenvectors $\{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n\}$ corresponding to eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$ sorted in strictly order of magnitude:

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|. \tag{3.52}$$

The eigenvalue λ_1 is called the *dominant eigenvalue*.

Any arbitrary initial vector $\mathbf{x}^{(0)} \in \mathbb{R}^n$ can be expressed as a linear combination of the eigenvectors:

$$\mathbf{x}^{(0)} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_n\mathbf{v}_n, \quad \text{assuming } c_1 \neq 0. \tag{3.53}$$

Multiplying $\mathbf{x}^{(0)}$ repeatedly by A:

$$A^k\mathbf{x}^{(0)} = c_1\lambda_1^k\mathbf{v}_1 + c_2\lambda_2^k\mathbf{v}_2 + \cdots + c_n\lambda_n^k\mathbf{v}_n = \lambda_1^k\left[c_1\mathbf{v}_1 + \sum_{i=2}^{n} c_i\left(\frac{\lambda_i}{\lambda_1}\right)^k\mathbf{v}_i\right]. \tag{3.54}$$

Since $|\lambda_1| > |\lambda_i|$ for $i \geq 2$, we have $\lim_{k\to\infty}\left(\frac{\lambda_i}{\lambda_1}\right)^k = 0$. Consequently:

$$\lim_{k\to\infty}\frac{A^k\mathbf{x}^{(0)}}{\lambda_1^k} = c_1\mathbf{v}_1. \tag{3.55}$$

Thus, as k increases, the vector $A^k\mathbf{x}^{(0)}$ becomes aligned with the dominant eigenvector $\mathbf{v}_1$.

Definition 3.3.9 (Power Method Algorithm)**.** To prevent overflow or underflow during repeated matrix multiplication, the vector is normalized at each iteration:

1. Choose an initial non-zero guess vector $\mathbf{x}^{(0)}$ (typically $\mathbf{x}^{(0)} = \begin{bmatrix}1 & 1 & \ldots & 1\end{bmatrix}^T$).
2. For $k = 0, 1, 2, \ldots$:

$$\mathbf{y}^{(k+1)} = A\mathbf{x}^{(k)}, \tag{3.56}$$

$$m_{k+1} = \|\mathbf{y}^{(k+1)}\|_\infty = \max_{1\leq i\leq n}|y_i^{(k+1)}|, \tag{3.57}$$

$$\mathbf{x}^{(k+1)} = \frac{\mathbf{y}^{(k+1)}}{m_{k+1}}. \tag{3.58}$$

As $k \to \infty$, $m_{k+1} \to |\lambda_1|$ and $\mathbf{x}^{(k+1)}$ converges to the normalized dominant eigenvector.

Remark 3.3.10 (Rayleigh Quotient for Symmetric Matrices). If A is a real symmetric matrix ($A = A^T$), an exceptionally accurate estimate for the dominant eigenvalue is provided by the *Rayleigh Quotient*:

$$q(\mathbf{x}^{(k)}) = \frac{(\mathbf{x}^{(k)})^T A\mathbf{x}^{(k)}}{(\mathbf{x}^{(k)})^T\mathbf{x}^{(k)}} \approx \lambda_1. \tag{3.59}$$

Downloaded from – https://aijaz.vercel.app

Example 3.3.11. Apply the Power Method to find the dominant eigenvalue and corresponding eigenvector of

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix},$$

starting with $\mathbf{x}^{(0)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. Perform four iterations.

Solution. **Iteration 1 ($k = 0$):**

$$\mathbf{y}^{(1)} = A\mathbf{x}^{(0)} = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}.$$

Scaling factor $m_1 = \max(|4|, |2|) = 4$.

$$\mathbf{x}^{(1)} = \frac{1}{4} \begin{bmatrix} 4 \\ 2 \end{bmatrix} = \begin{bmatrix} 1.0000 \\ 0.5000 \end{bmatrix}.$$

Iteration 2 ($k = 1$):

$$\mathbf{y}^{(2)} = A\mathbf{x}^{(1)} = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 1.0000 \\ 0.5000 \end{bmatrix} = \begin{bmatrix} 4.5000 \\ 3.5000 \end{bmatrix}.$$

Scaling factor $m_2 = \max(|4.5|, |3.5|) = 4.5$.

$$\mathbf{x}^{(2)} = \frac{1}{4.5} \begin{bmatrix} 4.5000 \\ 3.5000 \end{bmatrix} = \begin{bmatrix} 1.0000 \\ 0.7778 \end{bmatrix}.$$

Iteration 3 ($k = 2$):

$$\mathbf{y}^{(3)} = A\mathbf{x}^{(2)} = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 1.0000 \\ 0.7778 \end{bmatrix} = \begin{bmatrix} 4.7778 \\ 4.3334 \end{bmatrix}.$$

Scaling factor $m_3 = \max(|4.7778|, |4.3334|) = 4.7778$.

$$\mathbf{x}^{(3)} = \frac{1}{4.7778} \begin{bmatrix} 4.7778 \\ 4.3334 \end{bmatrix} = \begin{bmatrix} 1.0000 \\ 0.9070 \end{bmatrix}.$$

Iteration 4 ($k = 3$):

$$\mathbf{y}^{(4)} = A\mathbf{x}^{(3)} = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 1.0000 \\ 0.9070 \end{bmatrix} = \begin{bmatrix} 4.9070 \\ 4.7210 \end{bmatrix}.$$

Scaling factor $m_4 = 4.9070$.

$$\mathbf{x}^{(4)} = \frac{1}{4.9070} \begin{bmatrix} 4.9070 \\ 4.7210 \end{bmatrix} = \begin{bmatrix} 1.0000 \\ 0.9621 \end{bmatrix}.$$

The sequence $m_k \to 5$ and $\mathbf{x}^{(k)} \to \begin{bmatrix} 1 & 1 \end{bmatrix}^T$. ■

Example 3.3.12. Perform three iterations of the Power Method on the 3×3 matrix:

$$A = \begin{bmatrix} 6 & 2 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 1 \end{bmatrix},$$

starting with $\mathbf{x}^{(0)} = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}^T$.

Downloaded from – https://aijaz.vercel.app

Solution. **Iteration 1 ($k = 0$):**

$$\mathbf{y}^{(1)} = A\mathbf{x}^{(0)} = \begin{bmatrix} 6 & 2 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 9 \\ 6 \\ 3 \end{bmatrix}.$$

Scaling factor $m_1 = 9$.

$$\mathbf{x}^{(1)} = \frac{1}{9} \begin{bmatrix} 9 \\ 6 \\ 3 \end{bmatrix} = \begin{bmatrix} 1.0000 \\ 0.6667 \\ 0.3333 \end{bmatrix}.$$

Iteration 2 ($k = 1$):

$$\mathbf{y}^{(2)} = A\mathbf{x}^{(1)} = \begin{bmatrix} 6 & 2 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1.0000 \\ 0.6667 \\ 0.3333 \end{bmatrix} = \begin{bmatrix} 7.6667 \\ 4.3333 \\ 2.0000 \end{bmatrix}.$$

Scaling factor $m_2 = 7.6667$.

$$\mathbf{x}^{(2)} = \frac{1}{7.6667} \begin{bmatrix} 7.6667 \\ 4.3333 \\ 2.0000 \end{bmatrix} = \begin{bmatrix} 1.0000 \\ 0.5652 \\ 0.2609 \end{bmatrix}.$$

Iteration 3 ($k = 2$):

$$\mathbf{y}^{(3)} = A\mathbf{x}^{(2)} = \begin{bmatrix} 6 & 2 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1.0000 \\ 0.5652 \\ 0.2609 \end{bmatrix} = \begin{bmatrix} 7.3913 \\ 3.9565 \\ 1.8261 \end{bmatrix}.$$

Scaling factor $m_3 = 7.3913$.

$$\mathbf{x}^{(3)} = \frac{1}{7.3913} \begin{bmatrix} 7.3913 \\ 3.9565 \\ 1.8261 \end{bmatrix} = \begin{bmatrix} 1.0000 \\ 0.5353 \\ 0.2471 \end{bmatrix}.$$

Estimated dominant eigenvalue is $\lambda_1 \approx 7.3913$. ■

Example 3.3.13. Given the symmetric matrix $A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$ and an approximate eigenvector $\mathbf{x} = \begin{bmatrix} 0.55 \\ 1.00 \end{bmatrix}$, use the Rayleigh Quotient to compute a refined estimate of the dominant eigenvalue. Compare it with the exact value.

Solution. **Step 1: Compute Matrix-Vector Product $A\mathbf{x}$.**

$$A\mathbf{x} = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 0.55 \\ 1.00 \end{bmatrix} = \begin{bmatrix} 2(0.55) + 1.00 \\ 1(0.55) + 3(1.00) \end{bmatrix} = \begin{bmatrix} 2.10 \\ 3.55 \end{bmatrix}.$$

Step 2: Evaluate Rayleigh Quotient $q(\mathbf{x})$.

$$\mathbf{x}^T A\mathbf{x} = \begin{bmatrix} 0.55 & 1.00 \end{bmatrix} \begin{bmatrix} 2.10 \\ 3.55 \end{bmatrix} = 0.55(2.10) + 1.00(3.55) = 1.155 + 3.55 = 4.705.$$

$$\mathbf{x}^T\mathbf{x} = (0.55)^2 + (1.00)^2 = 0.3025 + 1.0000 = 1.3025.$$

$$q(\mathbf{x}) = \frac{\mathbf{x}^T A\mathbf{x}}{\mathbf{x}^T\mathbf{x}} = \frac{4.705}{1.3025} \approx 3.6123.$$

Step 3: Compare with Exact Value. Characteristic equation: $\det(A - \lambda I) = (2 - \lambda)(3 - \lambda) - 1 = \lambda^2 - 5\lambda + 5 = 0$. The exact dominant eigenvalue is:

$$\lambda_1 = \frac{5 + \sqrt{5}}{2} \approx 3.6180.$$

The Rayleigh Quotient estimate (3.6123) is accurate to two decimal places. ■

Downloaded from – https://aijaz.vercel.app

3.3.2 Introduction to Inverse Power Method

While the standard Power Method computes the largest eigenvalue of a matrix in absolute magnitude, many practical applications require finding the *smallest eigenvalue* in magnitude, or an eigenvalue closest to a specific target shift μ. The *Inverse Power Method* addresses this requirement.

Theoretical Foundations and Methodologies

1. Basic Inverse Power Method (Smallest Eigenvalue): Suppose $A \in \mathbb{R}^{n\times n}$ has non-zero eigenvalues sorted in order of magnitude:

$$|\lambda_1| \geq |\lambda_2| \geq \cdots > |\lambda_n| > 0. \tag{3.60}$$

The matrix inverse A^{-1} exists and possesses eigenvalues:

$$\mu_i = \frac{1}{\lambda_i}, \quad \text{with } \left|\frac{1}{\lambda_n}\right| > \left|\frac{1}{\lambda_{n-1}}\right| \geq \cdots \geq \left|\frac{1}{\lambda_1}\right|. \tag{3.61}$$

Thus, the smallest eigenvalue λ_n of A corresponds directly to the dominant eigenvalue $\frac{1}{\lambda_n}$ of A^{-1}. Applying the Power Method to A^{-1} calculates λ_n.

To avoid explicitly computing the matrix inverse A^{-1} (which is computationally expensive and numerically undesirable), at each iteration step we solve a system of linear equations:

$$A\mathbf{y}^{(k+1)} = \mathbf{x}^{(k)}. \tag{3.62}$$

2. Shifted Inverse Power Method (Intermediate Eigenvalues): To find an eigenvalue λ closest to a target scalar value $\mu \in \mathbb{R}$, we consider the shifted matrix $(A - \mu I)$. Its eigenvalues are $\lambda_i - \mu$.

The inverse matrix $(A - \mu I)^{-1}$ has eigenvalues $\frac{1}{\lambda_i - \mu}$. The dominant eigenvalue of $(A - \mu I)^{-1}$ corresponds to the eigenvalue λ_j closest to μ, since $|\lambda_j - \mu|$ is minimized, making $\frac{1}{|\lambda_j - \mu|}$ maximized.

Definition 3.3.14 (Shifted Inverse Power Method Algorithm)**.** 1. Select a target shift μ and an initial vector $\mathbf{x}^{(0)}$. Factorize $(A - \mu I) = LU$ once to solve linear systems efficiently. 2. For $k = 0, 1, 2, \ldots$:

i. Solve $(A - \mu I)\mathbf{y}^{(k+1)} = \mathbf{x}^{(k)}$ using the LU factors.

ii. Find the scale factor $m_{k+1} = \|\mathbf{y}^{(k+1)}\|_\infty = \max_i |y_i^{(k+1)}|$.

iii. Normalize $\mathbf{x}^{(k+1)} = \frac{\mathbf{y}^{(k+1)}}{m_{k+1}}$.

iv. Estimate the eigenvalue: $\lambda \approx \mu + \frac{1}{m_{k+1}}$.

Example 3.3.15. Use the Inverse Power Method to compute the smallest eigenvalue of

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix},$$

starting with $\mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. Perform two iterations.

Solution. **Iteration 1 ($k = 0$):** Solve $A\mathbf{y}^{(1)} = \mathbf{x}^{(0)}$:

$$\begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Downloaded from – https://aijaz.vercel.app

Using row elimination: $4y_1 + y_2 = 0 \implies y_2 = -4y_1$. Substituting into $2y_1 + 3(-4y_1) = 1 \implies -10y_1 = 1 \implies y_1 = -0.1, y_2 = 0.4$.

$$\mathbf{y}^{(1)} = \begin{bmatrix} -0.1 \\ 0.4 \end{bmatrix}.$$

Scale factor $m_1 = 0.4$.

$$\mathbf{x}^{(1)} = \frac{1}{0.4}\begin{bmatrix} -0.1 \\ 0.4 \end{bmatrix} = \begin{bmatrix} -0.25 \\ 1.00 \end{bmatrix}.$$

Estimated eigenvalue $\lambda_{min} \approx \frac{1}{m_1} = \frac{1}{0.4} = 2.50$.

Iteration 2 ($k = 1$): Solve $A\mathbf{y}^{(2)} = \mathbf{x}^{(1)}$:

$$\begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} -0.25 \\ 1.00 \end{bmatrix}.$$

Row 1 gives $y_2 = -0.25 - 4y_1$. Row 2: $2y_1 + 3(-0.25 - 4y_1) = 1.00 \implies -10y_1 - 0.75 = 1.00 \implies y_1 = -0.175$. Then $y_2 = -0.25 - 4(-0.175) = 0.45$.

$$\mathbf{y}^{(2)} = \begin{bmatrix} -0.175 \\ 0.450 \end{bmatrix}.$$

Scale factor $m_2 = 0.450$.

$$\mathbf{x}^{(2)} = \frac{1}{0.450}\begin{bmatrix} -0.175 \\ 0.450 \end{bmatrix} = \begin{bmatrix} -0.3889 \\ 1.0000 \end{bmatrix}.$$

Estimated eigenvalue $\lambda_{min} \approx \frac{1}{m_2} = \frac{1}{0.450} \approx 2.2222$. (Exact smallest eigenvalue is $\lambda_2 = 2$). ■

Example 3.3.16. Perform two iterations of the Inverse Power Method on

$$A = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix},$$

starting from $\mathbf{x}^{(0)} = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}^T$.

Solution. **Iteration 1 ($k = 0$):** Solve $A\mathbf{y}^{(1)} = \mathbf{x}^{(0)}$:

$$\begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix}\begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

From symmetry, $y_1 = y_3$. Row 1: $2y_1 + y_2 = 1 \implies y_2 = 1 - 2y_1$. Row 2: $y_1 + 2(1 - 2y_1) + y_1 = 1 \implies 2 - 2y_1 = 1 \implies y_1 = 0.5$. Then $y_2 = 0, y_3 = 0.5$.

$$\mathbf{y}^{(1)} = \begin{bmatrix} 0.5 \\ 0.0 \\ 0.5 \end{bmatrix}.$$

Scale factor $m_1 = 0.5$.

$$\mathbf{x}^{(1)} = \begin{bmatrix} 1.0 \\ 0.0 \\ 1.0 \end{bmatrix}.$$

Iteration 2 ($k = 1$): Solve $A\mathbf{y}^{(2)} = \mathbf{x}^{(1)}$:

$$\begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}.$$

From symmetry, $y_1 = y_3$. Row 1: $2y_1 + y_2 = 1 \implies y_2 = 1 - 2y_1$. Row 2: $y_1 + 2(1 - 2y_1) + y_1 = 0 \implies 2 - 2y_1 = 0 \implies y_1 = 1.0$. Then $y_2 = -1.0, y_3 = 1.0$.

$$\mathbf{y}^{(2)} = \begin{bmatrix} 1.0 \\ -1.0 \\ 1.0 \end{bmatrix}.$$

Scale factor $m_2 = -1.0$. Normalized vector $\mathbf{x}^{(2)} = \begin{bmatrix} 1.0 & -1.0 & 1.0 \end{bmatrix}^T$. Using the Rayleigh quotient on $\mathbf{x}^{(2)}$ yields $\lambda \approx 0.6667$ (approaching exact value $2 - \sqrt{2} \approx 0.5858$). ■

Example 3.3.17. Apply the Shifted Inverse Power Method with target shift $\mu = 2.0$ to find the eigenvalue closest to 2.0 for the matrix:

$$A = \begin{bmatrix} 5 & 1 \\ 1 & 2 \end{bmatrix},$$

starting with $\mathbf{x}^{(0)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Perform two iterations.

Solution. **Step 1: Construct Shifted Matrix $(A - 2I)$.**

$$(A - 2I) = \begin{bmatrix} 5 - 2 & 1 \\ 1 & 2 - 2 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 0 \end{bmatrix}.$$

Iteration 1 ($k = 0$): Solve $(A - 2I)\mathbf{y}^{(1)} = \mathbf{x}^{(0)}$:

$$\begin{bmatrix} 3 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

From Row 2: $y_1 = 1$. From Row 1: $3(1) + y_2 = 1 \implies y_2 = -2$.

$$\mathbf{y}^{(1)} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

Max component $m_1 = -2$.

$$\mathbf{x}^{(1)} = \frac{1}{-2} \begin{bmatrix} 1 \\ -2 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 1.0 \end{bmatrix}.$$

Estimated eigenvalue: $\lambda \approx \mu + \frac{1}{m_1} = 2.0 + \frac{1}{-2} = 1.5000$.

Iteration 2 ($k = 1$): Solve $(A - 2I)\mathbf{y}^{(2)} = \mathbf{x}^{(1)}$:

$$\begin{bmatrix} 3 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 1.0 \end{bmatrix}.$$

From Row 2: $y_1 = 1.0$. From Row 1: $3(1.0) + y_2 = -0.5 \implies y_2 = -3.5$.

$$\mathbf{y}^{(2)} = \begin{bmatrix} 1.0 \\ -3.5 \end{bmatrix}.$$

Downloaded from – https://aijaz.vercel.app

Max component $m_2 = -3.5$.

$$\mathbf{x}^{(2)} = \frac{1}{-3.5}\begin{bmatrix} 1.0 \\ -3.5 \end{bmatrix} = \begin{bmatrix} -0.2857 \\ 1.0000 \end{bmatrix}.$$

Estimated eigenvalue: $\lambda \approx 2.0 + \frac{1}{-3.5} = 2.0 - 0.2857 = 1.7143$. (Exact eigenvalue is $\frac{7-\sqrt{13}}{2} \approx 1.6972$). ■

Exercise 3.3.18. Solve the following exercise

1. Solve the following system of linear equations using the Gauss elimination method without pivoting:
$$\begin{aligned} x_1 + 2x_2 + x_3 &= 4, \\ 3x_1 + 8x_2 + 7x_3 &= 20, \\ 2x_1 + 7x_2 + 9x_3 &= 23. \end{aligned}$$

2. Solve the linear system using the Gauss elimination method with partial pivoting:
$$\begin{aligned} x_1 + x_2 + 2x_3 &= 8, \\ -x_1 - 2x_2 + 3x_3 &= 1, \\ 3x_1 - 7x_2 + 4x_3 &= 10. \end{aligned}$$

3. Solve the following 4×4 linear system using Gauss elimination:
$$\begin{aligned} 2x_1 + x_2 + x_3 + x_4 &= 7, \\ x_1 + 3x_2 + x_3 + x_4 &= 8, \\ x_1 + x_2 + 4x_3 + x_4 &= 9, \\ x_1 + x_2 + x_3 + 5x_4 &= 10. \end{aligned}$$

4. Apply the Gauss–Jordan elimination method to solve the system of equations:
$$\begin{aligned} x_1 + 3x_2 + 2x_3 &= 5, \\ 2x_1 + 4x_2 - x_3 &= -1, \\ x_1 - 2x_2 + 3x_3 &= 6. \end{aligned}$$

5. Compute the inverse A^{-1} using the Gauss–Jordan matrix inversion method for:
$$A = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 3 & 2 \\ 1 & 0 & 0 \end{bmatrix}.$$

6. Find the matrix inverse B^{-1} using Gauss–Jordan elimination with partial pivoting:
$$B = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 0 & 3 \\ 4 & -3 & 8 \end{bmatrix}.$$

7. Verify that the following system is strictly diagonally dominant, and carry out four Jacobi iterations starting from the initial guess $\mathbf{x}^{(0)} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T$:
$$\begin{aligned} 8x_1 + x_2 - x_3 &= 8, \\ x_1 + 5x_2 + 2x_3 &= 16, \\ 2x_1 + x_2 + 6x_3 &= 18. \end{aligned}$$

Downloaded from – https://aijaz.vercel.app

8. Reorder the system to make it strictly diagonally dominant, then perform three Jacobi iterations starting with $\mathbf{x}^{(0)} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T$:

$$\begin{aligned} x_1 + 3x_2 + 10x_3 &= 24, \\ 10x_1 + 2x_2 + x_3 &= 14, \\ 2x_1 + 8x_2 + 2x_3 &= 20. \end{aligned}$$

9. Construct the explicit Jacobi iteration matrix T_J and constant vector $\mathbf{c}_J$ for the system:

$$\begin{aligned} 6x_1 - 2x_2 &= 10, \\ -x_1 + 5x_2 &= 9. \end{aligned}$$

   Perform two iterations starting with $\mathbf{x}^{(0)} = \begin{bmatrix} 0 & 0 \end{bmatrix}^T$.

10. Perform three Gauss–Seidel iterations for the following system starting from the initial guess $\mathbf{x}^{(0)} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T$:

$$\begin{aligned} 10x_1 - 2x_2 - x_3 &= 3, \\ -2x_1 + 10x_2 - x_3 &= 15, \\ -x_1 - x_2 + 10x_3 &= 27. \end{aligned}$$

11. Reorder the following system to establish strict diagonal dominance, and execute three iterations of the Gauss–Seidel method using $\mathbf{x}^{(0)} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T$:

$$\begin{aligned} x_1 + 2x_2 + 6x_3 &= 22, \\ 5x_1 - x_2 + x_3 &= 10, \\ 2x_1 + 8x_2 - x_3 &= 19. \end{aligned}$$

12. Formulate the Gauss–Seidel iteration matrix T_{GS} for:

$$\begin{aligned} 3x_1 + x_2 &= 5, \\ x_1 + 4x_2 &= 9. \end{aligned}$$

    Calculate its spectral radius $\rho(T_{GS})$ and compare it with the Jacobi spectral radius $\rho(T_J)$.

13. An iteration matrix T is given by:

$$T = \begin{bmatrix} 0.4 & 0.1 \\ 0.2 & 0.3 \end{bmatrix}.$$

    Compute the eigenvalues of T, determine its spectral radius $\rho(T)$, and obtain a sufficient number of iterations to guarantee, using the induced infinity norm, that the initial error norm is reduced by a factor of at least 10^{-5}.

14. Test whether the Jacobi iteration converges for the system $A\mathbf{x} = \mathbf{b}$ by evaluating both the row-sum norm $\|T_J\|_\infty$ and the column-sum norm $\|T_J\|_1$, where:

$$A = \begin{bmatrix} 4 & 1 & -1 \\ 2 & 5 & 1 \\ 1 & 1 & 4 \end{bmatrix}.$$

Downloaded from – https://aijaz.vercel.app

15. Determine all eigenvalues and corresponding eigenvectors analytically for the matrix:

$$A = \begin{bmatrix} 3 & 2 & 0 \\ 2 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix}.$$

16. Apply the Gerschgorin Circle Theorem to sketch the discs containing the eigenvalues of the matrix:

$$A = \begin{bmatrix} 6 & 1 & 2 \\ 1 & 4 & 1 \\ 0.5 & 1 & 8 \end{bmatrix}.$$

Deduce the real interval within which all eigenvalues must lie.

17. Apply the Power Method to approximate the dominant eigenvalue and its associated eigenvector for:

$$A = \begin{bmatrix} 5 & 2 \\ 2 & 2 \end{bmatrix},$$

starting with $\mathbf{x}^{(0)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. Perform four iterations.

18. Perform three iterations of the Power Method on the 3×3 matrix:

$$A = \begin{bmatrix} 4 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{bmatrix},$$

starting with $\mathbf{x}^{(0)} = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}^T$. Refine the eigenvalue estimate using the Rayleigh Quotient after the third iteration.

19. Apply the Inverse Power Method to approximate the smallest eigenvalue of the matrix:

$$A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix},$$

starting with $\mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$. Perform three iterations.

CHAPTER

4

Numerical Solutions of Differential Equations

4.1 Numerical Methods for Ordinary Differential Equations (ODEs)

Many physical and engineering phenomena are modeled by differential equations. While simple first-order ordinary differential equations (ODEs) can be solved analytically, many non-linear or complex ODEs cannot be solved in closed form. In such cases, numerical methods provide approximate solutions at discrete grid points.

Consider a first-order Initial Value Problem (IVP) defined on an interval $[x_0, b]$:

$$\frac{dy}{dx} = f(x, y), \quad y(x_0) = y_0, \tag{4.1}$$

where $f(x, y)$ is a continuous function satisfying a Lipschitz condition to guarantee the existence and uniqueness of a solution $y(x)$.

To compute numerical approximations, we discretize the domain $[x_0, b]$ into N subintervals of uniform step size $h = \frac{b-x_0}{N}$, generating mesh points:

$$x_k = x_0 + kh, \quad k = 0, 1, 2, \ldots, N. \tag{4.2}$$

The numerical value approximating the exact solution $y(x_k)$ at the grid point x_k is denoted by y_k.

4.1.1 Euler's Method

Euler's method, named after Leonhard Euler, is the most fundamental explicit single-step numerical scheme for solving initial value problems.

Geometric and Mathematical Derivations

Euler's method can be derived through two distinct mathematical approaches:

1. Geometric Derivation via Tangent Line Approximation: At the initial point (x_0, y_0), the slope of the solution curve $y(x)$ is given by the differential equation $y'(x_0) = f(x_0, y_0)$. Approximating the solution curve near x_0 by its tangent line at (x_0, y_0):

$$y(x) \approx y_0 + (x - x_0) f(x_0, y_0). \tag{4.3}$$

Evaluating this tangent line approximation at $x_1 = x_0 + h$ yields the estimate:

$$y_1 = y_0 + h f(x_0, y_0). \tag{4.4}$$

Repeating this process sequentially from (x_1, y_1) to (x_2, y_2) and onward establishes the general iterative formula.

Downloaded from – https://aijaz.vercel.app

2. Derivation via Taylor Series Expansion: Expanding the exact solution $y(x_{k+1}) = y(x_k + h)$ in a Taylor series about x_k:

$$y(x_{k+1}) = y(x_k) + hy'(x_k) + \frac{h^2}{2!}y''(\xi_k), \quad \text{where } \xi_k \in (x_k, x_{k+1}). \tag{4.5}$$

Substituting $y'(x_k) = f(x_k, y(x_k))$ and truncating the series after the first-derivative term gives:

Definition 4.1.1 (Euler's Explicit Formula)**.** Given x_0, y_0, and step size h, the forward Euler iteration scheme is defined as:

$$y_{k+1} = y_k + hf(x_k, y_k), \quad x_{k+1} = x_k + h, \quad k = 0, 1, 2, \ldots \tag{4.6}$$

Error Analysis and Stability

Definition 4.1.2 (Local Truncation Error)**.** The Local Truncation Error (LTE) is the error committed in a single step assuming that the previous value $y_k = y(x_k)$ is exact:

$$\tau_{k+1} = y(x_{k+1}) - [y(x_k) + hf(x_k, y(x_k))] = \frac{h^2}{2}y''(\xi_k) = O(h^2). \tag{4.7}$$

Theorem 4.1.3 (Global Truncation Error and Order of Convergence)**.** *The Global Truncation Error (GTE) accumulated over $N = O(1/h)$ steps from x_0 to b is bounded by:*

$$E_N = |y(b) - y_N| \leq Ch = O(h), \tag{4.8}$$

*where C is a constant independent of h. Consequently, Euler's method is a **first-order accurate** method.*

Remark 4.1.4 (Absolute Stability). When applied to the test equation $y' = \lambda y$ with $\mathrm{Re}(\lambda) < 0$, Euler's formula becomes $y_{k+1} = (1 + h\lambda)y_k$. For numerical stability (preventing error amplification), we require $|1 + h\lambda| < 1$, which imposes the step size restriction:

$$h < \frac{2}{|\lambda|}. \tag{4.9}$$

Example 4.1.5. Consider the initial value problem:

$$\frac{dy}{dx} = x + y, \quad y(0) = 1.$$

Use Euler's method with step size $h = 0.1$ to approximate $y(x)$ at $x = 0.1, 0.2, 0.3, 0.4$. Compare each numerical result with the exact analytical solution $y(x) = 2e^x - x - 1$, and calculate the absolute error at each step.

Solution. Here $f(x, y) = x + y$, $x_0 = 0$, $y_0 = 1$, and $h = 0.1$. The Euler recurrence relation is:

$$y_{k+1} = y_k + 0.1(x_k + y_k).$$

Step 1 ($k = 0$): $x_0 = 0.0, y_0 = 1.0$.

$$f(x_0, y_0) = 0.0 + 1.0 = 1.0,$$
$$y_1 = 1.0 + 0.1(1.0) = 1.100000.$$

Exact solution at $x_1 = 0.1$: $y(0.1) = 2e^{0.1} - 0.1 - 1 = 1.110342$. Absolute Error: $|1.110342 - 1.100000| = 0.010342$.

Step 2 ($k = 1$): $x_1 = 0.1, y_1 = 1.100000$.

$$f(x_1, y_1) = 0.1 + 1.100000 = 1.200000,$$
$$y_2 = 1.100000 + 0.1(1.200000) = 1.220000.$$

Exact solution at $x_2 = 0.2$: $y(0.2) = 2e^{0.2} - 0.2 - 1 = 1.242806$. Absolute Error: $|1.242806 - 1.220000| = 0.022806$.

Step 3 ($k = 2$): $x_2 = 0.2, y_2 = 1.220000$.

$$f(x_2, y_2) = 0.2 + 1.220000 = 1.420000,$$
$$y_3 = 1.220000 + 0.1(1.420000) = 1.362000.$$

Exact solution at $x_3 = 0.3$: $y(0.3) = 2e^{0.3} - 0.3 - 1 = 1.399718$. Absolute Error: $|1.399718 - 1.362000| = 0.037718$.

Step 4 ($k = 3$): $x_3 = 0.3, y_3 = 1.362000$.

$$f(x_3, y_3) = 0.3 + 1.362000 = 1.662000,$$
$$y_4 = 1.362000 + 0.1(1.662000) = 1.528200.$$

Exact solution at $x_4 = 0.4$: $y(0.4) = 2e^{0.4} - 0.4 - 1 = 1.583649$. Absolute Error: $|1.583649 - 1.528200| = 0.055449$.

Summary Table:

k	x_k	Euler y_k	Exact $y(x_k)$	Absolute Error
0	0.0	1.000000	1.000000	0.000000
1	0.1	1.100000	1.110342	0.010342
2	0.2	1.220000	1.242806	0.022806
3	0.3	1.362000	1.399718	0.037718
4	0.4	1.528200	1.583649	0.055449

■

Example 4.1.6. Solve the non-linear initial value problem using Euler's method with step size $h = 0.2$ on $[0, 0.6]$:

$$\frac{dy}{dx} = -2xy^2, \quad y(0) = 1.$$

Compare the results with the exact analytical solution $y(x) = \frac{1}{1+x^2}$.

Solution. Here $f(x, y) = -2xy^2$, $x_0 = 0$, $y_0 = 1$, and $h = 0.2$.

Step 1 ($k = 0$): $x_0 = 0.0, y_0 = 1.0$.

$$f(x_0, y_0) = -2(0.0)(1.0)^2 = 0.0,$$
$$y_1 = 1.0 + 0.2(0.0) = 1.000000.$$

Exact value at $x_1 = 0.2$: $y(0.2) = \frac{1}{1+(0.2)^2} = \frac{1}{1.04} \approx 0.961538$. Error: $|0.961538 - 1.000000| = 0.038462$.

Downloaded from – https://aijaz.vercel.app

Step 2 ($k = 1$): $x_1 = 0.2, y_1 = 1.000000$.

$$f(x_1, y_1) = -2(0.2)(1.0)^2 = -0.400000,$$
$$y_2 = 1.000000 + 0.2(-0.400000) = 0.920000.$$

Exact value at $x_2 = 0.4$: $y(0.4) = \frac{1}{1+(0.4)^2} = \frac{1}{1.16} \approx 0.862069$. Error: $|0.862069 - 0.920000| = 0.057931$.

Step 3 ($k = 2$): $x_2 = 0.4, y_2 = 0.920000$.

$$f(x_2, y_2) = -2(0.4)(0.920000)^2 = -0.8(0.846400) = -0.677120,$$
$$y_3 = 0.920000 + 0.2(-0.677120) = 0.784576.$$

Exact value at $x_3 = 0.6$: $y(0.6) = \frac{1}{1+(0.6)^2} = \frac{1}{1.36} \approx 0.735294$. Error: $|0.735294 - 0.784576| = 0.049282$. ■

Example 4.1.7. A liquid at temperature $T_0 = 100°\text{C}$ cools in an environment held at constant ambient temperature $T_A = 20°\text{C}$. According to Newton's Law of Cooling, the rate of temperature change is governed by:

$$\frac{dT}{dt} = -0.1(T - 20), \quad T(0) = 100.$$

Use Euler's method with a step size of $h = 2$ minutes to estimate the liquid temperature at $t = 2, 4, 6$ minutes. Compare the numerical estimates with the exact solution $T(t) = 20 + 80e^{-0.1t}$.

Solution. Here $f(t, T) = -0.1(T - 20)$, $t_0 = 0$, $T_0 = 100$, and $h = 2$.

Step 1 ($k = 0$): $t_0 = 0, T_0 = 100.0$.

$$f(t_0, T_0) = -0.1(100.0 - 20) = -8.0,$$
$$T_1 = 100.0 + 2(-8.0) = 84.0000°\text{C}.$$

Exact $T(2) = 20 + 80e^{-0.2} = 85.4985°\text{C}$. Error: $|85.4985 - 84.0000| = 1.4985°\text{C}$.

Step 2 ($k = 1$): $t_1 = 2, T_1 = 84.0$.

$$f(t_1, T_1) = -0.1(84.0 - 20) = -6.4,$$
$$T_2 = 84.0 + 2(-6.4) = 71.2000°\text{C}.$$

Exact $T(4) = 20 + 80e^{-0.4} = 73.6256°\text{C}$. Error: $|73.6256 - 71.2000| = 2.4256°\text{C}$.

Step 3 ($k = 2$): $t_2 = 4, T_2 = 71.2$.

$$f(t_2, T_2) = -0.1(71.2 - 20) = -5.12,$$
$$T_3 = 71.2 + 2(-5.12) = 60.9600°\text{C}.$$

Exact $T(6) = 20 + 80e^{-0.6} = 63.9049°\text{C}$. Error: $|63.9049 - 60.9600| = 2.9449°\text{C}$. ■

4.1.2 Modified Euler's Method

The forward Euler method is limited by its first-order accuracy ($O(h)$ global error), which necessitates small step sizes to keep local truncation errors under control. To construct higher-order single-step schemes without computing higher analytical derivatives of $f(x, y)$, we consider numerical quadrature over the step interval.

Integrating the initial value problem $y' = f(x, y)$ across $[x_n, x_{n+1}]$ gives:

$$y(x_{n+1}) = y(x_n) + \int_{x_n}^{x_{n+1}} f(x, y(x))\, dx. \tag{4.10}$$

Approximating the integral by the Trapezoidal Rule yields the implicit formula:

$$y_{n+1} = y_n + \frac{h}{2}\left[f(x_n, y_n) + f(x_{n+1}, y_{n+1})\right]. \tag{4.11}$$

To make this scheme explicit, we estimate the unknown y_{n+1} on the right-hand side using an initial forward Euler prediction $y_{n+1}^{(0)} = y_n + hf(x_n, y_n)$. This formulation is known as the *Modified Euler Method* (or Heun's second-order method).

Definition 4.1.8 (Modified Euler Predictor-Corrector Method)**.** For a step size $h = x_{n+1} - x_n$, the Modified Euler method advances the solution from (x_n, y_n) to x_{n+1} via the two-stage process:

$$\text{Predictor:} \quad y_{n+1}^{(0)} = y_n + hf(x_n, y_n), \tag{4.12}$$

$$\text{Corrector:} \quad y_{n+1} = y_n + \frac{h}{2}\left[f(x_n, y_n) + f(x_{n+1}, y_{n+1}^{(0)})\right]. \tag{4.13}$$

Theorem 4.1.9 (Accuracy of Modified Euler Method)**.** *By expanding $f(x_{n+1}, y_{n+1}^{(0)})$ in a bivariate Taylor series about (x_n, y_n), it can be shown that the local truncation error satisfies $\tau_{n+1} = O(h^3)$. Thus, the global error accumulated over a fixed interval $[x_0, b]$ is $O(h^2)$, making the method second-order accurate.*

Example 4.1.10. Apply the Modified Euler method with $h = 0.1$ to approximate the solution of

$$\frac{dy}{dx} = x + y, \quad y(0) = 1,$$

at $x = 0.1$ and $x = 0.2$. Compare the numerical values with the exact solution $y(x) = 2e^x - x - 1$.

Solution. We have $f(x, y) = x + y$, $x_0 = 0$, $y_0 = 1$, and $h = 0.1$.

For the first step ($n = 0$, $x_1 = 0.1$): The predictor gives:

$$y_1^{(0)} = y_0 + hf(x_0, y_0) = 1 + 0.1(0 + 1) = 1.100000.$$

Evaluating the slopes at both endpoints: $f(x_0, y_0) = 1.000000$ and $f(x_1, y_1^{(0)}) = 0.1 + 1.100000 = 1.200000$. Applying the corrector step:

$$y_1 = 1 + \frac{0.1}{2}(1.000000 + 1.200000) = 1.110000.$$

The exact value is $y(0.1) = 2e^{0.1} - 0.1 - 1 \approx 1.110342$, giving an absolute error of 3.42×10^{-4}.

For the second step ($n = 1$, $x_2 = 0.2$): Using $x_1 = 0.1$ and $y_1 = 1.110000$, the predictor is:

$$y_2^{(0)} = 1.110000 + 0.1(0.1 + 1.110000) = 1.231000.$$

The corrected value becomes:

$$y_2 = 1.110000 + 0.05\left[(0.1 + 1.110000) + (0.2 + 1.231000)\right] = 1.242050.$$

Comparing with the exact solution $y(0.2) = 2e^{0.2} - 0.2 - 1 \approx 1.242806$, the absolute error is 7.56×10^{-4}. ■

Downloaded from – https://aijaz.vercel.app

Example 4.1.11. Compute the numerical solution to the non-linear initial value problem

$$y' = -2xy^2, \quad y(0) = 1,$$

for $x \in \{0.2, 0.4\}$ using $h = 0.2$. Compare the results with $y(x) = (1 + x^2)^{-1}$.

Solution. Here $f(x, y) = -2xy^2$ with $x_0 = 0$, $y_0 = 1$, and $h = 0.2$.

Step $n = 0$ ($x_1 = 0.2$):

$$\begin{aligned} f(x_0, y_0) &= -2(0)(1)^2 = 0, \\ y_1^{(0)} &= 1 + 0.2(0) = 1.000000, \\ f(x_1, y_1^{(0)}) &= -2(0.2)(1.000000)^2 = -0.400000, \\ y_1 &= 1 + \frac{0.2}{2}(0 - 0.400000) = 0.960000. \end{aligned}$$

The exact analytical value is $y(0.2) = (1 + 0.04)^{-1} \approx 0.961538$ (error $= 1.54 \times 10^{-3}$).

Step $n = 1$ ($x_2 = 0.4$):

$$\begin{aligned} f(x_1, y_1) &= -2(0.2)(0.960000)^2 = -0.368640, \\ y_2^{(0)} &= 0.960000 + 0.2(-0.368640) = 0.886272, \\ f(x_2, y_2^{(0)}) &= -2(0.4)(0.886272)^2 = -0.628382, \\ y_2 &= 0.960000 + 0.1(-0.368640 - 0.628382) = 0.860298. \end{aligned}$$

The exact value is $y(0.4) = (1 + 0.16)^{-1} \approx 0.862069$ (error $= 1.77 \times 10^{-3}$). ■

Example 4.1.12. A heated body cools according to $\frac{dT}{dt} = -0.1(T - 20)$ with $T(0) = 100°\text{C}$. Use the Modified Euler method with $h = 2$ minutes to approximate $T(2)$ and $T(4)$.

Solution. Let $f(t, T) = -0.1(T - 20)$, $t_0 = 0$, $T_0 = 100$, and $h = 2$.

At $t_1 = 2$:

$$\begin{aligned} f(t_0, T_0) &= -0.1(100 - 20) = -8.0000, \\ T_1^{(0)} &= 100 + 2(-8.0000) = 84.0000, \\ f(t_1, T_1^{(0)}) &= -0.1(84.0000 - 20) = -6.4000, \\ T_1 &= 100 + \frac{2}{2}(-8.0000 - 6.4000) = 85.6000°\text{C}. \end{aligned}$$

At $t_2 = 4$:

$$\begin{aligned} f(t_1, T_1) &= -0.1(85.6000 - 20) = -6.5600, \\ T_2^{(0)} &= 85.6000 + 2(-6.5600) = 72.4800, \\ f(t_2, T_2^{(0)}) &= -0.1(72.4800 - 20) = -5.2480, \\ T_2 &= 85.6000 + 1.0(-6.5600 - 5.2480) = 73.7920°\text{C}. \end{aligned}$$

Exact values $T(2) = 20 + 80e^{-0.2} \approx 85.4985°\text{C}$ and $T(4) = 20 + 80e^{-0.4} \approx 73.6256°\text{C}$ confirm errors under $0.17°\text{C}$. ■

Downloaded from – https://aijaz.vercel.app

4.1.3 Runge–Kutta Method of Order 2 (RK2)

Runge–Kutta methods generate higher-order single-step approximations by sampling $f(x, y)$ at multiple stage points within each step interval $[x_n, x_{n+1}]$, avoiding the need for analytical differentiation.

General Second-Order Scheme

An explicit two-stage Runge–Kutta (RK2) formula updates y_n according to:

$$k_1 = hf(x_n, y_n), \tag{4.14}$$
$$k_2 = hf(x_n + c_2 h, y_n + a_{21} k_1), \tag{4.15}$$
$$y_{n+1} = y_n + w_1 k_1 + w_2 k_2. \tag{4.16}$$

Expanding k_2 in a Taylor series about (x_n, y_n) and equating the expression for y_{n+1} to the second-order Taylor polynomial of $y(x_n + h)$ yields three algebraic parameter constraints:

$$w_1 + w_2 = 1, \quad w_2 c_2 = \frac{1}{2}, \quad w_2 a_{21} = \frac{1}{2}. \tag{4.17}$$

Because there are four free parameters (w_1, w_2, c_2, a_{21}) and only three constraint equations, RK2 represents an underdetermined family of methods parameterized by $c_2 \neq 0$.

Definition 4.1.13 (Standard RK2 Variants)**.** Selecting specific values for c_2 produces well-known second-order methods:

i. **Heun's Method ($c_2 = 1$):** $a_{21} = 1$, $w_1 = \frac{1}{2}$, $w_2 = \frac{1}{2}$.

$$y_{n+1} = y_n + \frac{1}{2}(k_1 + k_2), \quad k_1 = hf(x_n, y_n), \quad k_2 = hf(x_n + h, y_n + k_1). \tag{4.18}$$

ii. **Midpoint Method ($c_2 = \frac{1}{2}$):** $a_{21} = \frac{1}{2}$, $w_1 = 0$, $w_2 = 1$.

$$y_{n+1} = y_n + k_2, \quad k_1 = hf(x_n, y_n), \quad k_2 = hf\left(x_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right). \tag{4.19}$$

iii. **Ralston's Method ($c_2 = \frac{2}{3}$):** $a_{21} = \frac{2}{3}$, $w_1 = \frac{1}{4}$, $w_2 = \frac{3}{4}$ (minimizes the local truncation error bound).

$$y_{n+1} = y_n + \frac{1}{4}k_1 + \frac{3}{4}k_2, \quad k_1 = hf(x_n, y_n), \quad k_2 = hf\left(x_n + \frac{2h}{3}, y_n + \frac{2k_1}{3}\right). \tag{4.20}$$

Example 4.1.14. Solve $y' = x + y$, $y(0) = 1$ over $x \in [0, 0.2]$ with $h = 0.1$ using the **Midpoint Method**.

Solution. For $f(x, y) = x + y$, $x_0 = 0$, $y_0 = 1$, $h = 0.1$:
Step 1 ($n = 0$, $x_1 = 0.1$):

$$k_1 = 0.1 f(0, 1) = 0.1(0 + 1) = 0.100000,$$
$$k_2 = 0.1 f\left(0 + 0.05, 1 + \frac{0.100000}{2}\right) = 0.1 f(0.05, 1.050000) = 0.110000,$$
$$y_1 = 1 + 0.110000 = 1.110000.$$

Step 2 ($n = 1$, $x_2 = 0.2$):

$$\begin{aligned} k_1 &= 0.1 f(0.1, 1.110000) = 0.121000, \\ k_2 &= 0.1 f\,(0.15, 1.110000 + 0.060500) = 0.1 f(0.15, 1.170500) = 0.132050, \\ y_2 &= 1.110000 + 0.132050 = 1.242050. \end{aligned}$$

Exact values: $y(0.1) \approx 1.110342$ and $y(0.2) \approx 1.242806$. ■

Example 4.1.15. Apply **Heun's RK2 Method** with $h = 0.2$ to solve $y' = y - x^2 + 1$, $y(0) = 0.5$ on $[0, 0.4]$.

Solution. Here $f(x, y) = y - x^2 + 1$, $x_0 = 0$, $y_0 = 0.5$, $h = 0.2$.

Step 1 ($x_1 = 0.2$):

$$\begin{aligned} k_1 &= 0.2 f(0, 0.5) = 0.2(0.5 - 0 + 1) = 0.300000, \\ k_2 &= 0.2 f(0.2, 0.5 + 0.300000) = 0.2(0.800000 - 0.04 + 1) = 0.352000, \\ y_1 &= 0.5 + \frac{1}{2}(0.300000 + 0.352000) = 0.826000. \end{aligned}$$

Step 2 ($x_2 = 0.4$):

$$\begin{aligned} k_1 &= 0.2 f(0.2, 0.826000) = 0.2(0.826000 - 0.04 + 1) = 0.357200, \\ k_2 &= 0.2 f(0.4, 0.826000 + 0.357200) = 0.2(1.183200 - 0.16 + 1) = 0.404640, \\ y_2 &= 0.826000 + \frac{1}{2}(0.357200 + 0.404640) = 1.206920. \end{aligned}$$

Exact solution $y(x) = (x + 1)^2 - 0.5e^x$ gives $y(0.2) \approx 0.829299$ and $y(0.4) \approx 1.214088$. ■

Example 4.1.16. Use **Ralston's Method** with $h = 0.1$ to find $y(0.1)$ for $y' = -y$, $y(0) = 1$.

Solution. With $f(x, y) = -y$, $x_0 = 0$, $y_0 = 1$, $h = 0.1$:

$$\begin{aligned} k_1 &= 0.1 f(0, 1) = -0.100000, \\ k_2 &= 0.1 f\left(\frac{0.2}{3}, 1 + \frac{2(-0.100000)}{3}\right) = 0.1\left(-\frac{2.8}{3}\right) \approx -0.093333, \\ y_1 &= 1 + \frac{1}{4}(-0.100000) + \frac{3}{4}(-0.093333) = 0.905000. \end{aligned}$$

Comparing with $e^{-0.1} \approx 0.904837$, the error is 1.63×10^{-4}. ■

4.1.4 Runge–Kutta Method of Order 4 (RK4)

The classical fourth-order Runge–Kutta method (RK4) is one of the most effective and widely used single-step schemes for initial value problems. It evaluates four intermediate slopes per step, providing fourth-order global accuracy ($O(h^4)$) without evaluating higher derivatives.

Algorithm and Derivation Overview

By matching the Taylor expansion of $y(x_n + h)$ up to terms in h^4, Kutta (1901) established the classic four-stage explicit RK scheme:

Downloaded from – https://aijaz.vercel.app

Definition 4.1.17 (Classical RK4 Scheme). Given the current state (x_n, y_n) and step size h, the solution is updated to $x_{n+1} = x_n + h$ via:

$$k_1 = hf(x_n, y_n), \tag{4.21}$$

$$k_2 = hf\left(x_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right), \tag{4.22}$$

$$k_3 = hf\left(x_n + \frac{h}{2}, y_n + \frac{k_2}{2}\right), \tag{4.23}$$

$$k_4 = hf\left(x_n + h, y_n + k_3\right), \tag{4.24}$$

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4). \tag{4.25}$$

The coefficients $\frac{1}{6}, \frac{2}{6}, \frac{2}{6}, \frac{1}{6}$ correspond to Simpson's $1/3$ rule weights across $[x_n, x_{n+1}]$.

Theorem 4.1.18 (Order of Accuracy). *The local truncation error of classical RK4 is $O(h^5)$, and the accumulated global truncation error is $O(h^4)$.*

Example 4.1.19. Compute $y(0.1)$ for $y' = x + y$, $y(0) = 1$ using classical **RK4** with $h = 0.1$.

Solution. Here $f(x, y) = x + y$, $x_0 = 0$, $y_0 = 1$, $h = 0.1$.

Stage evaluations:

$$\begin{aligned} k_1 &= 0.1(0 + 1) = 0.10000000, \\ k_2 &= 0.1(0.05 + 1.05000000) = 0.11000000, \\ k_3 &= 0.1(0.05 + 1.05500000) = 0.11050000, \\ k_4 &= 0.1(0.10 + 1.11050000) = 0.12105000. \end{aligned}$$

Combining the stage values:

$$y_1 = 1 + \frac{1}{6}\left[0.10000000 + 2(0.11000000) + 2(0.11050000) + 0.12105000\right] = 1.11034167.$$

The exact value is $2e^{0.1} - 1.1 \approx 1.11034184$ (error $= 1.7 \times 10^{-7}$). ■

Example 4.1.20. Apply classical **RK4** with $h = 0.2$ to approximate $y(0.2)$ for $y' = -2xy^2$, $y(0) = 1$.

Solution. With $f(x, y) = -2xy^2$, $x_0 = 0$, $y_0 = 1$, $h = 0.2$:

$$\begin{aligned} k_1 &= 0.2\left[-2(0)(1)^2\right] = 0.00000000, \\ k_2 &= 0.2\left[-2(0.1)(1.0)^2\right] = -0.04000000, \\ k_3 &= 0.2\left[-2(0.1)(0.98000000)^2\right] = -0.03841600, \\ k_4 &= 0.2\left[-2(0.2)(0.96158400)^2\right] = -0.07397150. \end{aligned}$$

Updating the solution:

$$y_1 = 1 + \frac{1}{6}\left[0 + 2(-0.04000000) + 2(-0.03841600) - 0.07397150\right] = 0.96153275.$$

Comparing with $y(0.2) = 1/1.04 \approx 0.96153846$, the error is 5.71×10^{-6}. ■

Downloaded from – https://aijaz.vercel.app

Example 4.1.21. Using $h = 2$, compute $T(2)$ via **RK4** for the cooling model $\frac{dT}{dt} = -0.1(T - 20)$, $T(0) = 100°\text{C}$.

Solution. For $f(t, T) = -0.1(T - 20)$, $t_0 = 0$, $T_0 = 100$, $h = 2$:

$$\begin{aligned} k_1 &= 2[-0.1(100 - 20)] = -16.000000, \\ k_2 &= 2[-0.1(92 - 20)] = -14.400000, \\ k_3 &= 2[-0.1(92.8 - 20)] = -14.560000, \\ k_4 &= 2[-0.1(85.44 - 20)] = -13.088000. \end{aligned}$$

Weighted average update:

$$T_1 = 100 + \frac{1}{6}\left[-16.000000 - 28.800000 - 29.120000 - 13.088000\right] = 85.498667°\text{C}.$$

The exact value $T(2) = 20 + 80e^{-0.2} \approx 85.498316°\text{C}$ demonstrates four-decimal-place agreement (3.51×10^{-4} error). ■

4.2 Finite Difference Method for Boundary Value Problems

Boundary value problems (BVPs) for ordinary differential equations involve conditions specified at two or more distinct boundary points. A standard second-order linear boundary value problem defined on an interval $[a, b]$ is formulated as:

$$y'' + p(x)y' + q(x)y = r(x), \quad x \in (a, b), \tag{4.26}$$

subject to Dirichlet boundary conditions:

$$y(a) = \alpha, \quad y(b) = \beta. \tag{4.27}$$

To approximate the continuous solution $y(x)$ numerically, we divide $[a, b]$ into N equal subintervals of step size $h = \frac{b-a}{N}$, creating grid points:

$$x_i = a + ih, \quad i = 0, 1, 2, \ldots, N. \tag{4.28}$$

Finite Difference Approximations

Replacing the derivatives at interior nodes x_i ($i = 1, 2, \ldots, N - 1$) with central finite difference quotients derived from Taylor series expansions:

$$y'(x_i) = \frac{y(x_{i+1}) - y(x_{i-1})}{2h} - \frac{h^2}{6}y'''(\xi_i), \tag{4.29}$$

$$y''(x_i) = \frac{y(x_{i+1}) - 2y(x_i) + y(x_{i-1})}{h^2} - \frac{h^2}{12}y^{(4)}(\eta_i). \tag{4.30}$$

Truncating the $O(h^2)$ truncation error terms and substituting $y_i \approx y(x_i)$, $p_i = p(x_i)$, $q_i = q(x_i)$, $r_i = r(x_i)$ into Equation (4.26) gives the algebraic difference equation at node x_i:

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} + p_i\frac{y_{i+1} - y_{i-1}}{2h} + q_i y_i = r_i. \tag{4.31}$$

Rearranging terms in order of node indices yields:

$$\left(1 - \frac{h}{2}p_i\right)y_{i-1} + \left(-2 + h^2 q_i\right)y_i + \left(1 + \frac{h}{2}p_i\right)y_{i+1} = h^2 r_i, \quad i = 1, 2, \ldots, N - 1. \tag{4.32}$$

Downloaded from – https://aijaz.vercel.app

Matrix System Formulation

Setting $y_0 = \alpha$ and $y_N = \beta$, Equation (4.32) forms a tridiagonal linear system $A\mathbf{y} = \mathbf{d}$ of dimension $(N-1) \times (N-1)$:

$$\begin{bmatrix} d_1 & c_1 & 0 & \dots & 0 \\ a_2 & d_2 & c_2 & \dots & 0 \\ 0 & a_3 & d_3 & \dots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \dots & a_{N-1} & d_{N-1} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{N-1} \end{bmatrix} = \begin{bmatrix} h^2 r_1 - a_1 \alpha \\ h^2 r_2 \\ h^2 r_3 \\ \vdots \\ h^2 r_{N-1} - c_{N-1} \beta \end{bmatrix}, \tag{4.33}$$

where $a_i = 1 - \frac{h}{2} p_i$, $d_i = -2 + h^2 q_i$, and $c_i = 1 + \frac{h}{2} p_i$. The tridiagonal system is efficiently solved using the Thomas algorithm in $O(N)$ operations.

Example 4.2.1. Solve the boundary value problem

$$y'' + y = 0, \quad x \in \left[0, \frac{\pi}{2}\right], \quad y(0) = 0, \quad y\left(\frac{\pi}{2}\right) = 1,$$

using the finite difference method with step size $h = \frac{\pi}{6}$. Compare numerical results with the exact solution $y(x) = \sin x$.

Solution. Dividing $[0, \pi/2]$ into $N = 3$ subintervals with $h = \pi/6 \approx 0.523599$:

$$x_0 = 0, \quad x_1 = \frac{\pi}{6}, \quad x_2 = \frac{\pi}{3}, \quad x_3 = \frac{\pi}{2}.$$

Boundary conditions fix $y_0 = 0$ and $y_3 = 1$. Here $p(x) = 0$, $q(x) = 1$, and $r(x) = 0$. The finite difference equation simplifies to:

$$y_{i-1} - (2 - h^2) y_i + y_{i+1} = 0.$$

Evaluating $h^2 = (\pi/6)^2 \approx 0.274156$, we have $2 - h^2 \approx 1.725844$.

At node $i = 1$ ($x_1 = \pi/6$):

$$y_0 - 1.725844 y_1 + y_2 = 0 \implies -1.725844 y_1 + y_2 = 0.$$

At node $i = 2$ ($x_2 = \pi/3$):

$$y_1 - 1.725844 y_2 + y_3 = 0 \implies y_1 - 1.725844 y_2 = -1.$$

Substituting $y_2 = 1.725844 y_1$ into the second equation:

$$y_1 - (1.725844)^2 y_1 = -1 \implies -1.978537 y_1 = -1 \implies y_1 \approx 0.505424.$$

Then $y_2 = 1.725844(0.505424) \approx 0.872283$.

Comparing with the analytical values $y(\pi/6) = \sin(\pi/6) = 0.500000$ and $y(\pi/3) = \sin(\pi/3) \approx 0.866025$:

$$|y(\pi/6) - y_1| = |0.500000 - 0.505424| = 0.005424,$$
$$|y(\pi/3) - y_2| = |0.866025 - 0.872283| = 0.006258.$$

■

Downloaded from – https://aijaz.vercel.app

Example 4.2.2. Use finite differences with $h = 0.25$ to approximate the solution of

$$y'' - y = x, \quad y(0) = 0, \quad y(1) = 0.$$

Compare the approximations with the exact solution $y(x) = \frac{\sinh x}{\sinh 1} - x$.

Solution. Setting $N = 4$ gives $h = 0.25$ and grid points $x_0 = 0, x_1 = 0.25, x_2 = 0.50, x_3 = 0.75, x_4 = 1.0$. Boundary values are $y_0 = 0$ and $y_4 = 0$. The central difference equation at x_i is:

$$y_{i-1} - (2 + h^2)y_i + y_{i+1} = h^2 x_i.$$

With $h^2 = 0.0625$, $2 + h^2 = 2.0625$. The system for interior nodes $i = 1, 2, 3$ is:

$$\begin{aligned} -2.0625y_1 + y_2 &= 0.0625(0.25) = 0.015625, \\ y_1 - 2.0625y_2 + y_3 &= 0.0625(0.50) = 0.031250, \\ y_2 - 2.0625y_3 &= 0.0625(0.75) = 0.046875. \end{aligned}$$

Solving this symmetric tridiagonal system: From equation 1, $y_2 = 2.0625y_1 + 0.015625$. From equation 3, $y_2 = 2.0625y_3 + 0.046875$. Substituting into equation 2 yields:

$$y_1 \approx -0.034885, \quad y_2 \approx -0.056326, \quad y_3 \approx -0.050037.$$

Evaluating the exact solution $y(x) = \frac{\sinh x}{\sinh 1} - x$:

$$\begin{aligned} y(0.25) &= \frac{\sinh 0.25}{\sinh 1} - 0.25 \approx -0.035048 \quad (\text{error} = 1.63 \times 10^{-4}), \\ y(0.50) &= \frac{\sinh 0.50}{\sinh 1} - 0.50 \approx -0.056591 \quad (\text{error} = 2.65 \times 10^{-4}), \\ y(0.75) &= \frac{\sinh 0.75}{\sinh 1} - 0.75 \approx -0.050275 \quad (\text{error} = 2.38 \times 10^{-4}). \end{aligned}$$

■

Example 4.2.3. Solve $y'' - 2y' + y = 0$ on $[0, 1]$ with $y(0) = 1, y(1) = 0$ using $h = \frac{1}{3}$. Compare with exact solution $y(x) = (1 - x)e^x$.

Solution. For $N = 3$, $h = 1/3 \approx 0.333333$. Interior nodes are $x_1 = 1/3$ and $x_2 = 2/3$. Here $p(x) = -2, q(x) = 1, r(x) = 0$. The finite difference equation is:

$$(1 + h)\, y_{i-1} - \left(2 - h^2\right) y_i + (1 - h)\, y_{i+1} = 0.$$

Substituting $h = 1/3$: $1 + h = 4/3$, $2 - h^2 = 17/9$, $1 - h = 2/3$. Multiplying by 9 gives $12y_{i-1} - 17y_i + 6y_{i+1} = 0$.

At $i = 1$ ($x_1 = 1/3, y_0 = 1$):

$$12(1) - 17y_1 + 6y_2 = 0 \implies -17y_1 + 6y_2 = -12.$$

At $i = 2$ ($x_2 = 2/3, y_3 = 0$):

$$12y_1 - 17y_2 + 6(0) = 0 \implies y_2 = \frac{12}{17}y_1.$$

Substituting y_2 into the first equation:

$$-17y_1 + 6\left(\frac{12}{17}y_1\right) = -12 \implies \frac{-217}{17}y_1 = -12 \implies y_1 = \frac{204}{217} \approx 0.940092.$$

Then $y_2 = \frac{12}{17}\left(\frac{204}{217}\right) = \frac{144}{217} \approx 0.663594$.

Exact values: $y(1/3) = \frac{2}{3}e^{1/3} \approx 0.930408$ and $y(2/3) = \frac{1}{3}e^{2/3} \approx 0.649245$. ■

Downloaded from – https://aijaz.vercel.app

4.3 Introduction to Partial Differential Equations (PDEs)

Partial differential equations govern physical systems involving multi-variable continuous fields, such as heat conduction, wave propagation, and fluid flow. A second-order linear PDE in two independent variables (x, y) takes the general form:

$$Au_{xx} + Bu_{xy} + Cu_{yy} + Du_x + Eu_y + Fu = G(x, y), \tag{4.34}$$

where A, B, C, D, E, F are functions of (x, y).

Definition 4.3.1 (Classification of Second-Order PDEs)**.** The mathematical nature and boundary conditions of Equation (4.34) are determined by the sign of the discriminant $\Delta = B^2 - 4AC$:

i. **Hyperbolic ($\Delta > 0$):** Models wave motion and convective transport (e.g., 1D Wave equation $u_{tt} = c^2 u_{xx}$).

ii. **Parabolic ($\Delta = 0$):** Models dissipative and evolutionary transport (e.g., 1D Heat equation $u_t = \alpha u_{xx}$).

iii. **Elliptic ($\Delta < 0$):** Models equilibrium states and potential fields (e.g., 2D Laplace equation $u_{xx} + u_{yy} = 0$).

4.3.1 Basic Finite Difference Schemes for Heat Equation

Consider the 1D transient heat conduction equation for temperature $u(x, t)$ in a rod of length L:

$$\frac{\partial u}{\partial t} = c^2 \frac{\partial^2 u}{\partial x^2}, \quad 0 < x < L, \quad t > 0, \tag{4.35}$$

subject to initial condition $u(x, 0) = f(x)$ and Dirichlet boundary conditions $u(0, t) = g_1(t), u(L, t) = g_2(t)$.

Discretizing the domain into spatial grid spacing $\Delta x = L/N$ ($x_i = i\Delta x$) and temporal step size Δt ($t_j = j\Delta t$), we denote the numerical solution at (x_i, t_j) by $u_{i,j}$.

Define the mesh Fourier ratio parameter:

$$r = \frac{c^2 \Delta t}{(\Delta x)^2}. \tag{4.36}$$

1. Explicit FTCS Scheme (Forward Time, Centered Space)

Approximating u_t by a forward difference and u_{xx} by a central difference at time level j:

$$\frac{u_{i,j+1} - u_{i,j}}{\Delta t} = c^2 \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{(\Delta x)^2}. \tag{4.37}$$

Solving explicitly for $u_{i,j+1}$:

$$u_{i,j+1} = ru_{i-1,j} + (1 - 2r)u_{i,j} + ru_{i+1,j}. \tag{4.38}$$

Theorem 4.3.2 (Stability of Explicit FTCS Scheme)**.** *By von Neumann stability analysis, the explicit FTCS scheme is conditionally stable and converges if and only if $r \leq \frac{1}{2}$, requiring $\Delta t \leq \frac{(\Delta x)^2}{2c^2}$.*

Downloaded from – https://aijaz.vercel.app

2. Implicit BTCS Scheme (Backward Time, Centered Space)

Approximating u_t by a backward difference at time level $j+1$:

$$\frac{u_{i,j+1}-u_{i,j}}{\Delta t} = c^2 \frac{u_{i+1,j+1}-2u_{i,j+1}+u_{i-1,j+1}}{(\Delta x)^2}, \tag{4.39}$$

which yields the implicit tridiagonal system:

$$-ru_{i-1,j+1} + (1+2r)u_{i,j+1} - ru_{i+1,j+1} = u_{i,j}. \tag{4.40}$$

The BTCS scheme is unconditionally stable for all $r > 0$.

3. Crank–Nicolson Scheme

Averaging the spatial derivatives at time levels j and $j+1$ gives a second-order accurate scheme in both space and time ($O((\Delta x)^2 + (\Delta t)^2)$):

$$\frac{u_{i,j+1}-u_{i,j}}{\Delta t} = \frac{c^2}{2}\left[\frac{u_{i+1,j+1}-2u_{i,j+1}+u_{i-1,j+1}}{(\Delta x)^2} + \frac{u_{i+1,j}-2u_{i,j}+u_{i-1,j}}{(\Delta x)^2}\right]. \tag{4.41}$$

Rearranging into implicit system form:

$$-\frac{r}{2}u_{i-1,j+1} + (1+r)u_{i,j+1} - \frac{r}{2}u_{i+1,j+1} = \frac{r}{2}u_{i-1,j} + (1-r)u_{i,j} + \frac{r}{2}u_{i+1,j}. \tag{4.42}$$

The Crank–Nicolson scheme is unconditionally stable for all $r > 0$.

Example 4.3.3. Solve $u_t = u_{xx}$ for $0 < x < 1, t > 0$ with $u(0,t) = 0, u(1,t) = 0$ and $u(x,0) = \sin(\pi x)$. Use the explicit FTCS scheme with $\Delta x = 0.25$ and $\Delta t = 0.025$ to compute solutions at $t = 0.025$ and $t = 0.050$.

Solution. Here $c^2 = 1$, $\Delta x = 0.25$, and $\Delta t = 0.025$. The mesh ratio is:

$$r = \frac{\Delta t}{(\Delta x)^2} = \frac{0.025}{(0.25)^2} = \frac{0.025}{0.0625} = 0.4 \leq 0.5 \quad \text{(stable)}.$$

The FTCS update formula simplifies to:

$$u_{i,j+1} = 0.4u_{i-1,j} + 0.2u_{i,j} + 0.4u_{i+1,j}.$$

Spatial grid points: $x_0 = 0, x_1 = 0.25, x_2 = 0.50, x_3 = 0.75, x_4 = 1.0$. Boundary values: $u_{0,j} = 0$ and $u_{4,j} = 0$ for all j.

Initial values at $j = 0$ ($t = 0$):

$$\begin{aligned} u_{1,0} &= \sin(\pi/4) \approx 0.707107, \\ u_{2,0} &= \sin(\pi/2) = 1.000000, \\ u_{3,0} &= \sin(3\pi/4) \approx 0.707107. \end{aligned}$$

First time step $j = 1$ ($t = 0.025$):

$$\begin{aligned} u_{1,1} &= 0.4(0) + 0.2(0.707107) + 0.4(1.000000) = 0.541421, \\ u_{2,1} &= 0.4(0.707107) + 0.2(1.000000) + 0.4(0.707107) = 0.765685, \\ u_{3,1} &= 0.541421. \end{aligned}$$

Downloaded from – https://aijaz.vercel.app

Second time step $j = 2$ ($t = 0.050$):

$$\begin{aligned} u_{1,2} &= 0.4(0) + 0.2(0.541421) + 0.4(0.765685) = 0.414558, \\ u_{2,2} &= 0.4(0.541421) + 0.2(0.765685) + 0.4(0.541421) = 0.586273, \\ u_{3,2} &= 0.414558. \end{aligned}$$

Exact values $u(x,t) = e^{-\pi^2 t}\sin(\pi x)$ at $t = 0.050$ yield $u(0.25, 0.050) \approx 0.431685$ and $u(0.50, 0.050) \approx 0.610495$. ■

Example 4.3.4. Apply the Crank–Nicolson scheme with $\Delta x = 0.25$ and $\Delta t = 0.0625$ ($r = 1.0$) to solve $u_t = u_{xx}$ for $0 < x < 1$ subject to $u(0,t) = 0, u(1,t) = 0$ and $u(x,0) = 100\sin(\pi x)$ for one time step.

Solution. With $\Delta x = 0.25, \Delta t = 0.0625$, $r = \frac{0.0625}{0.0625} = 1.0$. The Crank–Nicolson scheme for $r = 1.0$ becomes:

$$-0.5u_{i-1,1} + 2.0u_{i,1} - 0.5u_{i+1,1} = 0.5u_{i-1,0} + 0.5u_{i+1,0}.$$

Initial values at $j = 0$:

$$u_{1,0} = 100\sin(\pi/4) \approx 70.7107, \quad u_{2,0} = 100\sin(\pi/2) = 100.0000, \quad u_{3,0} = 70.7107.$$

Evaluating the right-hand side for interior nodes $i = 1, 2, 3$:

$$\begin{aligned} i = 1: &\quad 0.5(0) + 0.5(100.0000) = 50.0000, \\ i = 2: &\quad 0.5(70.7107) + 0.5(70.7107) = 70.7107, \\ i = 3: &\quad 0.5(100.0000) + 0.5(0) = 50.0000. \end{aligned}$$

Setting up the system with $u_{0,1} = 0$ and $u_{4,1} = 0$:

$$\begin{aligned} 2u_{1,1} - 0.5u_{2,1} &= 50.0000, \\ -0.5u_{1,1} + 2u_{2,1} - 0.5u_{3,1} &= 70.7107, \\ -0.5u_{2,1} + 2u_{3,1} &= 50.0000. \end{aligned}$$

By symmetry, $u_{1,1} = u_{3,1}$. From equation 1, $u_{2,1} = 4u_{1,1} - 100.0000$. Substituting into equation 2:

$$-0.5u_{1,1} + 2(4u_{1,1} - 100.0000) - 0.5u_{1,1} = 70.7107 \implies 7u_{1,1} = 270.7107 \implies u_{1,1} \approx 38.6730.$$

Then $u_{2,1} = 4(38.6730) - 100.0000 = 54.6920$ and $u_{3,1} = 38.6730$. Exact values are $u(0.25, 0.0625) \approx 38.1585$ and $u(0.50, 0.0625) \approx 53.9642$. ■

Example 4.3.5. Solve $u_t = u_{xx}$ for $0 < x < 1$ with $u(0,t) = 0, u(1,t) = 0$ and $u(x,0) = \sin(\pi x)$ for one step of the implicit BTCS scheme using $\Delta x = 0.25$ and $\Delta t = 0.125$ ($r = 2.0$).

Solution. Here $r = \frac{0.125}{(0.25)^2} = 2.0$. The BTCS equation for $r = 2.0$ is:

$$-2u_{i-1,1} + 5u_{i,1} - 2u_{i+1,1} = u_{i,0}.$$

Initial values at $j = 0$:

$$u_{1,0} = \sin(\pi/4) \approx 0.707107, \quad u_{2,0} = 1.000000, \quad u_{3,0} = 0.707107.$$

Downloaded from – https://aijaz.vercel.app

System for $j = 1$ ($u_{0,1} = 0, u_{4,1} = 0$):

$$\begin{aligned} 5u_{1,1} - 2u_{2,1} &= 0.707107, \\ -2u_{1,1} + 5u_{2,1} - 2u_{3,1} &= 1.000000, \\ -2u_{2,1} + 5u_{3,1} &= 0.707107. \end{aligned}$$

Using symmetry $u_{1,1} = u_{3,1}$, equation 1 gives $u_{2,1} = 2.5u_{1,1} - 0.353554$. Substituting into equation 2:

$$-4u_{1,1} + 5(2.5u_{1,1} - 0.353554) = 1.000000 \implies 8.5u_{1,1} = 2.767767 \implies u_{1,1} \approx 0.325620.$$

Then $u_{2,1} = 2.5(0.325620) - 0.353554 = 0.460496$ and $u_{3,1} = 0.325620$. ■

Exercise 4.3.6. Solve the following exercise

1. Use Euler's method with step size $h = 0.1$ to approximate the solution of the initial value problem
$$\frac{dy}{dx} = x + 2y, \quad y(0) = 1,$$
for $x \in [0, 0.3]$. Calculate the values y_1, y_2, y_3.

2. Apply Euler's method with $h = 0.2$ to solve $y' = -xy$ with $y(0) = 2$ on the interval $[0, 0.6]$. Compare the numerical result at $x = 0.6$ with the exact solution $y(x) = 2e^{-x^2/2}$.

3. The voltage $V(t)$ across a capacitor in a charging RC circuit satisfies the differential equation
$$\frac{dV}{dt} = -0.2(V - 12), \quad V(0) = 0.$$
Use Euler's method with a time step $h = 2$ seconds to estimate the voltage at $t = 2, 4$, and 6 seconds.

4. Apply the Modified Euler method with step size $h = 0.1$ to compute $y(0.1)$ and $y(0.2)$ for the initial value problem:
$$\frac{dy}{dx} = x^2 + y, \quad y(0) = 1.$$

5. Solve the non-linear initial value problem $y' = -y^2$ with $y(0) = 1$ using the Modified Euler predictor-corrector method with $h = 0.2$ for two steps ($x = 0.2, 0.4$). Compare the numerical approximations with the exact solution $y(x) = \frac{1}{1+x}$.

6. A population $P(t)$ grows according to the logistic model
$$\frac{dP}{dt} = 0.1P\left(1 - \frac{P}{1000}\right), \quad P(0) = 100.$$
Use the Modified Euler method with a step size $h = 5$ days to approximate the population at $t = 5$ and $t = 10$ days.

7. Use Heun's second-order Runge–Kutta method (RK2) with $h = 0.1$ to solve the initial value problem
$$\frac{dy}{dx} = x - y, \quad y(0) = 1,$$
to estimate $y(0.1)$ and $y(0.2)$. Compare with the exact solution $y(x) = 2e^{-x} + x - 1$.

Downloaded from – https://aijaz.vercel.app

8. Apply the Midpoint RK2 scheme with step size $h = 0.2$ to compute $y(0.2)$ and $y(0.4)$ for:

$$\frac{dy}{dx} = 1 + y^2, \quad y(0) = 0.$$

Compare the approximations with the exact analytical solution $y(x) = \tan x$.

9. Solve the non-linear initial value problem using the classical fourth-order Runge–Kutta method (RK4) with $h = 0.1$ for one step ($x = 0.1$):

$$\frac{dy}{dx} = x + y^2, \quad y(0) = 0.$$

10. Solve the boundary value problem

$$y'' + 4y = 0, \quad x \in [0, 1], \quad y(0) = 0, \quad y(1) = 1,$$

using the finite difference method with step size $h = 0.25$. Set up and solve the resulting tridiagonal system.

11. Approximate the solution of the non-homogeneous boundary value problem

$$y'' - 2y' + y = x, \quad x \in [0, 1], \quad y(0) = 0, \quad y(1) = 2,$$

using central finite differences with step size $h = 0.25$.

12. Set up the finite difference linear system for the boundary value problem

$$y'' + xy' - y = 0, \quad x \in [0, 1], \quad y(0) = 1, \quad y(1) = 3,$$

using step size $h = \frac{1}{3}$. Solve for the interior values y_1 and y_2.

13. Classify each of the following second-order partial differential equations as hyperbolic, parabolic, or elliptic:

    a. $u_{xx} + 4u_{xy} + 4u_{yy} = 0$,
    b. $u_{xx} + 2u_{xy} + 5u_{yy} = 0$,
    c. $3u_{xx} - 5u_{xy} + 2u_{yy} = 0$.

14. Solve the 1D heat equation $u_t = u_{xx}$ for $0 < x < 1, t > 0$, with boundary conditions $u(0, t) = 0, u(1, t) = 0$ and initial condition $u(x, 0) = 100 \sin(\pi x)$. Use the explicit FTCS scheme with spatial step $\Delta x = 0.25$ and time step $\Delta t = 0.02$ ($r = 0.32$) to calculate temperature values at $t = 0.02$ and $t = 0.04$.

15. Apply the Crank–Nicolson implicit scheme with $\Delta x = 0.25$ and $\Delta t = 0.0625$ ($r = 1.0$) to solve the heat equation $u_t = u_{xx}$ on $0 < x < 1$ with $u(0, t) = 0, u(1, t) = 0$ and initial profile $u(x, 0) = 50 \sin(2\pi x)$ for one time step ($t = 0.0625$).

16. Formulate and solve the implicit BTCS system for $u_t = 2u_{xx}$ on $0 < x < 1$ with boundary conditions $u(0, t) = 0, u(1, t) = 0$ and initial condition $u(x, 0) = \sin(\pi x)$, using $\Delta x = 0.25$ and $\Delta t = 0.0625$ ($r = 2.0$) for one time step ($t = 0.0625$).

Downloaded from – **https://aijaz.vercel.app**

CHAPTER 5

Python Programming: Basics and Elementary Numerical Methods

5.1 Introduction to Python Programming

Python is a high-level, interpreted programming language widely adopted in scientific computing, numerical analysis, and data engineering due to its clear syntax, dynamic typing, and vast ecosystem of computational libraries. In numerical analysis, Python serves both as an algorithmic prototyping tool and an execution platform for mathematical models.

5.1.1 Output, Variables, and Basic Arithmetic

Scientific computation requires precise data representation, manipulation, and display. Python provides intuitive primitives for output generation, dynamic variable assignment, and arithmetic operations.

The primary mechanism for displaying results in Python is the built-in `print()` function. It converts expressions into their textual representations and writes them to the standard output stream.

To present numerical results with mathematical precision (e.g., controlling floating-point decimal places), formatted string literals (commonly known as *f-strings*) are utilized.

Definition 5.1.1 (Formatted String Literal)**.** An *f-string* in Python is prefixed with `f` or `F`, allowing expressions enclosed in curly braces `{expr:spec}` to be evaluated dynamically and formatted according to a format specifier `spec`.

For example, to format a floating-point number $x \in \mathbb{R}$ to k decimal places, the specifier `.kf` is appended to the variable inside the brace.

```
# Example of standard and formatted output
pi_approx = 3.141592653589793
print("Unformatted output:", pi_approx)
print(f"Formatted to 4 decimal places: {pi_approx:.4f}")
print(f"Scientific notation: {pi_approx:.6e}")
```

Variables and Data Types

Unlike statically typed languages (such as C++ or Fortran), Python uses *dynamic typing*. A variable name acts as a reference (or pointer) to an object stored in memory, rather than a fixed memory location with a predefined type.

Definition 5.1.2 (Primitive Numerical Types)**.** Python natively supports four primary mathematical data types:

(i) **Integers** (`int`): Represents elements of the set of integers $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ with arbitrary precision.

(ii) **Floating-Point Numbers** (`float`): Represents real numbers $\mathbb{R}$ using double-precision 64-bit IEEE 754 binary floating-point format.

(iii) **Complex Numbers** (`complex`): Represents complex numbers $z = a + bj \in \mathbb{C}$, where $a, b \in \mathbb{R}$ and $j = \sqrt{-1}$.

(iv) **Booleans** (`bool`): Represents truth values from the set $\{\text{True}, \text{False}\}$, mapped internally to $\{1, 0\}$.

```
# Variable assignments and type inspection
a = 42              # int
x = 3.14159         # float
z = 2.0 + 3.5j      # complex
flag = True         # bool

print(f"a: {type(a)}, x: {type(x)}, z: {type(z)}, flag: {type(flag)}")
```

Basic Arithmetic Operators

Python provides a full suite of mathematical operators mapping directly to standard algebraic operations on $\mathbb{R}$ and $\mathbb{C}$.

Operator	Operation	Mathematical Syntax	Python Syntax
`+`	Addition	$a + b$	`a + b`
`-`	Subtraction	$a - b$	`a - b`
`*`	Multiplication	$a \cdot b$	`a * b`
`/`	True Division	$a/b \quad (b \neq 0)$	`a / b`
`//`	Floor Division	$\lfloor a/b \rfloor$	`a // b`
`%`	Modulo (Remainder)	$a \pmod b$	`a % b`
`**`	Exponentiation	a^b	`a ** b`

Table 5.1: Basic arithmetic operators in Python.

Example 5.1.3 (Polynomial Evaluation via Arithmetic Operations)**.** Consider evaluating the cubic polynomial

$$P(x) = 3x^3 - 5x^2 + 2x - 7$$

at $x = 2.5$. Horner's method optimizes this computation by rewriting $P(x)$ as:

$$P(x) = ((3x - 5)x + 2)x - 7$$

We implement both standard expansion and Horner's method in Python.

```
x = 2.5

# Direct evaluation
P_direct = 3 * (x**3) - 5 * (x**2) + 2 * x - 7
```

```
# Horner's scheme evaluation
P_horner = ((3 * x - 5) * x + 2) * x - 7

print(f"Direct Evaluation : {P_direct}")
print(f"Horner's Scheme   : {P_horner}")
```

Both yield the output $P(2.5) = 13.625$.

Remark 5.1.4 (Floating-Point Representation and Round-off Errors). Because floats are stored in binary IEEE 754 format, numbers such as $0.1 = \frac{1}{10}$ cannot be represented exactly in binary floating-point. This induces a fundamental limitation:

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

In numerical algorithms, direct equality comparisons (==) between floating-point numbers must be avoided in favor of tolerance checks $|a - b| < \varepsilon$, where $\varepsilon > 0$ is a small threshold.

5.1.2 Control Structures: `if` Conditions, `while` and `for` Loops

Algorithms in numerical analysis are intrinsically logical and iterative. Control structures govern the flow of execution based on mathematical conditions and convergence criteria.

Indentation and Block Structure

Python replaces traditional block delimiters (such as braces `{}` or `begin...end`) with strict structural *indentation*. Standard convention requires 4 spaces per indentation level.

Conditional Branching (`if`, `elif`, `else`)

Conditional statements execute specific code blocks contingent on the evaluation of Boolean expressions.

Definition 5.1.5 (Relational and Logical Operators)**.** Conditional expressions rely on relational operators (==, !=, <, >, <=, >=) combined with logical operators (`and`, `or`, `not`).

Example 5.1.6 (Piecewise Function Evaluation)**.** Consider the piecewise continuous function $f : \mathbb{R} \to \mathbb{R}$ defined by:

$$f(x) = \begin{cases} x^2 + 1 & \text{if } x < 0, \\ 2x & \text{if } 0 \leq x \leq 5, \\ 3x - 5 & \text{if } x > 5. \end{cases}$$

We implement this function using an `if-elif-else` construct.

```
def evaluate_f(x):
    if x < 0:
        y = x**2 + 1
```

```
    elif 0 <= x <= 5:
        y = 2 * x
    else:
        y = 3 * x - 5
    return y

# Testing values
for val in [-2.0, 3.0, 6.0]:
    print(f"f({val}) = {evaluate_f(val)}")
```

Indefinite Iteration (`while` Loops)

A `while` loop repeatedly executes a body of instructions as long as a specified Boolean condition remains `True`. This structure is ubiquitous in numerical methods where the exact number of iterations is unknown *a priori*, and execution continues until an error criterion satisfies a given tolerance ε.

Example 5.1.7 (Computing Square Roots via Babylonian Iteration)**.** To compute $\sqrt{S}$ for $S > 0$, we use the Babylonian (Newton-Raphson) iteration:

$$x_{k+1} = \frac{1}{2}\left(x_k + \frac{S}{x_k}\right), \quad k = 0, 1, 2, \ldots$$

Iterating continues until the absolute difference satisfies $|x_{k+1} - x_k| < \varepsilon$.

```
S = 25.0
x = S / 2.0      # Initial estimate x_0
tol = 1e-8       # Error tolerance epsilon
max_iter = 100   # Safeguard against infinite loops
iteration = 0

diff = tol + 1.0 # Initialize variable to enter the loop

while diff >= tol and iteration < max_iter:
    x_new = 0.5 * (x + S / x)
    diff = abs(x_new - x)
    x = x_new
    iteration += 1
    print(f"Iter {iteration}: x = {x:.10f}, diff = {diff:.2e}")

print(f"\nCalculated sqrt({S}) = {x:.8f}")
```

Definite Iteration (`for` Loops and `range()`)

A `for` loop iterates over a fixed sequence of values. It is commonly paired with the built-in `range()` function to execute a block a predefined number of times.

Definition 5.1.8 (The `range()` Sequence Generator)**.** The function `range(start, stop, step)` generates an arithmetic progression of integers from `start` up to (but excluding) `stop`, incrementing by `step`. Default values are `start = 0` and `step = 1`.

Downloaded from – https://aijaz.vercel.app

Example 5.1.9 (Partial Sum of the Basel Problem)**.** The Basel problem considers the infinite series sum:

$$S = \sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6} \approx 1.6449340668$$

We approximate S using a finite partial sum $S_N = \sum_{n=1}^{N} \frac{1}{n^2}$ with $N = 1000$.

```
N = 1000
partial_sum = 0.0

for n in range(1, N + 1):
    partial_sum += 1.0 / (n**2)

import math
exact_val = (math.pi**2) / 6.0
abs_error = abs(exact_val - partial_sum)

print(f"Partial Sum (N={N}): {partial_sum:.10f}")
print(f"Exact Value        : {exact_val:.10f}")
print(f"Absolute Error     : {abs_error:.4e}")
```

Exercise 5.1.10. Write a Python script using a `for` loop to compute the Taylor series approximation of e^x given by:

$$e^x \approx \sum_{k=0}^{N} \frac{x^k}{k!}$$

Evaluate the approximation for $x = 1.5$ with $N = 10$ and compute the absolute error relative to Python's built-in `math.exp(1.5)`.

Solution. We compute $e^{1.5}$ by maintaining a running sum and accumulating the factorial iteratively or using `math.factorial`.

```
import math

x = 1.5
N = 10
approx = 0.0

for k in range(N + 1):
    term = (x**k) / math.factorial(k)
    approx += term

exact = math.exp(x)
error = abs(exact - approx)

print(f"Approximation: {approx:.10f}")
print(f"Exact Value  : {exact:.10f}")
print(f"Abs Error    : {error:.4e}")
```

The output shows that for $N = 10$, $e^{1.5} \approx 4.4816890703$, achieving machine-level precision with an absolute error on the order of 10^{-11}. ■

Downloaded from – https://aijaz.vercel.app

5.2 Python Implementations

Numerical analysis focuses on constructing algorithms that approximate continuous mathematical processes using finite discrete operations. This section details Python implementations of foundational numerical methods.

5.2.1 Computation of Absolute and Relative Errors

In numerical computation, approximations are inevitable due to truncation of infinite processes and finite-precision floating-point arithmetic. Quantifying the discrepancy between exact and approximate values is essential for evaluating algorithm accuracy.

Definition 5.2.1 (Errors in Numerical Analysis)**.** Let $x \in \mathbb{R}$ be the true (exact) value and $\tilde{x} \in \mathbb{R}$ be an approximate value.

(i) **Absolute Error** (E_a): The magnitude of the difference between the exact and approximate values:

$$E_a = |x - \tilde{x}|$$

(ii) **Relative Error** (E_r): The absolute error scaled by the magnitude of the exact value ($x \neq 0$):

$$E_r = \frac{|x - \tilde{x}|}{|x|} = \frac{E_a}{|x|}$$

(iii) **Percentage Error** (E_p): The relative error expressed as a percentage:

$$E_p = E_r \times 100\%$$

Remark 5.2.2. While absolute error measures the absolute scale of discrepancy, relative error provides a scale-invariant metric. For instance, an error of 0.01 is severe if $x = 0.02$ ($E_r = 50\%$), but negligible if $x = 10^6$ ($E_r = 10^{-6}\%$).

Below is the general Python implementation to calculate these errors:

```
def compute_errors(true_val, approx_val):
    abs_err = abs(true_val - approx_val)
    rel_err = abs_err / abs(true_val) if true_val != 0 else float('nan')
    pct_err = rel_err * 100.0
    return abs_err, rel_err, pct_err
```

Example 5.2.3 (Approximation of π by Rational Fraction)**.** Compute the absolute, relative, and percentage errors when approximating π by Archimedes' rational fraction $\frac{22}{7}$.

Solution. Given exact $x = \pi \approx 3.1415926536$ and approximate $\tilde{x} = \frac{22}{7} \approx 3.1428571429$:

$$\begin{aligned} E_a &= |3.1415926536 - 3.1428571429| = 0.0012644893 \\ E_r &= \frac{0.0012644893}{3.1415926536} \approx 0.0004024994 \\ E_p &= 0.0004024994 \times 100\% = 0.04025\% \end{aligned}$$

Python Implementation:

```
import math

true_pi = math.pi
approx_pi = 22 / 7

ea, er, ep = compute_errors(true_pi, approx_pi)
print(f"Absolute Error  : {ea:.8f}")
print(f"Relative Error  : {er:.8e}")
print(f"Percentage Error: {ep:.5f}%")
```

■

Example 5.2.4 (Truncation Error in Euler's Constant e)**.** Euler's constant $e \approx 2.7182818285$ is rounded to 3 decimal places as $\tilde{x} = 2.718$. Calculate the resulting absolute, relative, and percentage errors.

Solution. Given $x = e \approx 2.7182818285$ and $\tilde{x} = 2.718$:

$$E_a = |2.7182818285 - 2.718| = 0.0002818285$$
$$E_r = \frac{0.0002818285}{2.7182818285} \approx 0.0001036789$$
$$E_p = 0.01037\%$$

Python Implementation:

```
import math

true_e = math.e
approx_e = 2.718

ea, er, ep = compute_errors(true_e, approx_e)
print(f"Absolute Error  : {ea:.8f}")
print(f"Relative Error  : {er:.8e}")
print(f"Percentage Error: {ep:.5f}%")
```

■

Example 5.2.5 (Truncation Error in Maclaurin Series for Cosine)**.** The cosine function can be approximated by its second-order Taylor polynomial around $x = 0$:

$$\cos(x) \approx 1 - \frac{x^2}{2}$$

Evaluate E_a, E_r, and E_p at $x = 0.5$ radians.

Solution. Exact value: $x = \cos(0.5) \approx 0.8775825619$.
Approximate value: $\tilde{x} = 1 - \frac{0.5^2}{2} = 1 - 0.125 = 0.875$.

$$E_a = |0.8775825619 - 0.875| = 0.0025825619$$
$$E_r = \frac{0.0025825619}{0.8775825619} \approx 0.0029428135$$
$$E_p = 0.29428\%$$

Python Implementation:

Downloaded from – https://aijaz.vercel.app

```
import math

x_val = 0.5
true_cos = math.cos(x_val)
approx_cos = 1.0 - (x_val**2) / 2.0

ea, er, ep = compute_errors(true_cos, approx_cos)
print(f"Absolute Error  : {ea:.8f}")
print(f"Relative Error  : {er:.8e}")
print(f"Percentage Error: {ep:.5f}%")
```

■

5.2.2 Bisection Method (Manual 2–3 Iterations)

The Bisection Method is a root-finding algorithm based on the Intermediate Value Theorem.

Theorem 5.2.6 (Intermediate Value Theorem). *If $f \in C[a,b]$ and $f(a) \cdot f(b) < 0$, then there exists at least one root $\alpha \in (a,b)$ such that $f(\alpha) = 0$.*

Algorithm Steps

Given an interval $[a_0, b_0]$ with $f(a_0)f(b_0) < 0$:

1. Compute the midpoint $c_k = \frac{a_k + b_k}{2}$.
2. Evaluate $f(c_k)$. If $f(c_k) = 0$, c_k is the exact root.
3. If $f(a_k)f(c_k) < 0$, set $a_{k+1} = a_k$ and $b_{k+1} = c_k$.
4. If $f(a_k)f(c_k) > 0$, set $a_{k+1} = c_k$ and $b_{k+1} = b_k$.

The maximum error after n iterations is bounded by $\frac{b-a}{2^n}$.

Example 5.2.7 (Cubic Polynomial Root). Find the root of $f(x) = x^3 - x - 2 = 0$ on $[1,2]$ performing 3 manual iterations.

Solution. $f(1) = 1 - 1 - 2 = -2 < 0$, $f(2) = 8 - 2 - 2 = 4 > 0$. Since $f(1)f(2) < 0$, a root exists in $[1,2]$.

- **Iteration 1**: $c_1 = \frac{1+2}{2} = 1.5$.
  $f(1.5) = (1.5)^3 - 1.5 - 2 = 3.375 - 3.5 = -0.125 < 0$.
  Since $f(1.5)f(2) < 0$, new interval is $[1.5, 2]$.
- **Iteration 2**: $c_2 = \frac{1.5+2}{2} = 1.75$.
  $f(1.75) = (1.75)^3 - 1.75 - 2 = 5.359375 - 3.75 = 1.609375 > 0$.
  Since $f(1.5)f(1.75) < 0$, new interval is $[1.5, 1.75]$.
- **Iteration 3**: $c_3 = \frac{1.5+1.75}{2} = 1.625$.
  $f(1.625) = (1.625)^3 - 1.625 - 2 = 4.291016 - 3.625 = 0.666016 > 0$.
  New interval is $[1.5, 1.625]$.

Python Implementation:

```
def f1(x):
    return x**3 - x - 2

a, b = 1.0, 2.0
print(f"{'Iter':<5}{'a':<10}{'b':<10}{'c':<10}{'f(c)':<12}")
print("-" * 50)
for i in range(1, 4):
    c = (a + b) / 2.0
    fc = f1(c)
    print(f"{i:<5}{a:<10.4f}{b:<10.4f}{c:<10.4f}{fc:<12.6f}")
    if f1(a) * fc < 0:
        b = c
    else:
        a = c
```

■

Example 5.2.8 (Transcendental Equation involving Exponential). Find a root of $f(x) = xe^x - 1 = 0$ in $[0, 1]$ using 3 iterations of the Bisection Method.

Solution. $f(0) = -1 < 0$, $f(1) = 1 \cdot e^1 - 1 \approx 1.71828 > 0$.

- **Iteration 1**: $c_1 = \frac{0+1}{2} = 0.5$.
  $f(0.5) = 0.5e^{0.5} - 1 \approx 0.5(1.64872) - 1 = -0.175639 < 0$.
  Since $f(0.5)f(1) < 0$, interval becomes $[0.5, 1]$.
- **Iteration 2**: $c_2 = \frac{0.5+1}{2} = 0.75$.
  $f(0.75) = 0.75e^{0.75} - 1 \approx 0.75(2.11700) - 1 = 0.58775 > 0$.
  Since $f(0.5)f(0.75) < 0$, interval becomes $[0.5, 0.75]$.
- **Iteration 3**: $c_3 = \frac{0.5+0.75}{2} = 0.625$.
  $f(0.625) = 0.625e^{0.625} - 1 \approx 0.625(1.86825) - 1 = 0.167655 > 0$.
  New interval is $[0.5, 0.625]$.

Python Implementation:

```
import math

def f2(x):
    return x * math.exp(x) - 1

a, b = 0.0, 1.0
print(f"{'Iter':<5}{'a':<10}{'b':<10}{'c':<10}{'f(c)':<12}")
print("-" * 50)
for i in range(1, 4):
    c = (a + b) / 2.0
    fc = f2(c)
    print(f"{i:<5}{a:<10.4f}{b:<10.4f}{c:<10.4f}{fc:<12.6f}")
    if f2(a) * fc < 0:
        b = c
    else:
        a = c
```

■

Downloaded from – https://aijaz.vercel.app

Example 5.2.9 (Trigonometric Equation)**.** Find the root of $f(x) = \cos(x) - x = 0$ in $[0, 1]$ across 3 iterations.

Solution. $f(0) = \cos(0) - 0 = 1 > 0$, $f(1) = \cos(1) - 1 \approx 0.540302 - 1 = -0.459698 < 0$.

- **Iteration 1**: $c_1 = \frac{0+1}{2} = 0.5$.
  $f(0.5) = \cos(0.5) - 0.5 \approx 0.877583 - 0.5 = 0.377583 > 0$.
  New interval is $[0.5, 1]$.
- **Iteration 2**: $c_2 = \frac{0.5+1}{2} = 0.75$.
  $f(0.75) = \cos(0.75) - 0.75 \approx 0.731689 - 0.75 = -0.018311 < 0$.
  New interval is $[0.5, 0.75]$.
- **Iteration 3**: $c_3 = \frac{0.5+0.75}{2} = 0.625$.
  $f(0.625) = \cos(0.625) - 0.625 \approx 0.810965 - 0.625 = 0.185965 > 0$.
  New interval is $[0.625, 0.75]$.

Python Implementation:

```
import math

def f3(x):
    return math.cos(x) - x

a, b = 0.0, 1.0
print(f"{'Iter':<5}{'a':<10}{'b':<10}{'c':<10}{'f(c)':<12}")
print("-" * 50)
for i in range(1, 4):
    c = (a + b) / 2.0
    fc = f3(c)
    print(f"{i:<5}{a:<10.4f}{b:<10.4f}{c:<10.4f}{fc:<12.6f}")
    if f3(a) * fc < 0:
        b = c
    else:
        a = c
```

■

5.2.3 Evaluating Polynomials using Interpolation Formulas

Polynomial interpolation seeks a unique polynomial $P_n(x)$ of degree at most n passing through $n + 1$ distinct data points $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$.

Definition 5.2.10 (Lagrange Interpolation Formula)**.** For arbitrarily spaced points, the Lagrange interpolating polynomial is:

$$P_n(x) = \sum_{i=0}^{n} y_i L_i(x), \quad \text{where } L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

Definition 5.2.11 (Newton Forward Difference Formula)**.** For equally spaced points with step size

Downloaded from – https://aijaz.vercel.app

$h = x_{i+1} - x_i$, letting $u = \frac{x - x_0}{h}$:

$$P_n(x) = y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \cdots + \frac{u(u-1)\ldots(u-n+1)}{n!}\Delta^n y_0$$

where $\Delta y_k = y_{k+1} - y_k$ and $\Delta^m y_k = \Delta^{m-1} y_{k+1} - \Delta^{m-1} y_k$.

Example 5.2.12 (Lagrange Interpolation on Unequally Spaced Data)**.** Given data points $(1, 2)$, $(2, 3)$, and $(4, 11)$, construct the Lagrange polynomial to estimate $f(3)$.

Solution. Here $n = 2$, $x = 3$. The basis polynomials are:

$$L_0(3) = \frac{(3-2)(3-4)}{(1-2)(1-4)} = \frac{(1)(-1)}{(-1)(-3)} = -\frac{1}{3}$$
$$L_1(3) = \frac{(3-1)(3-4)}{(2-1)(2-4)} = \frac{(2)(-1)}{(1)(-2)} = 1$$
$$L_2(3) = \frac{(3-1)(3-2)}{(4-1)(4-2)} = \frac{(2)(1)}{(3)(2)} = \frac{1}{3}$$

Evaluating $P_2(3)$:

$$P_2(3) = 2\left(-\frac{1}{3}\right) + 3(1) + 11\left(\frac{1}{3}\right) = -\frac{2}{3} + 3 + \frac{11}{3} = 6$$

Python Implementation:

```
def lagrange_interpolate(x_nodes, y_nodes, x_eval):
    n = len(x_nodes)
    result = 0.0
    for i in range(n):
        term = y_nodes[i]
        for j in range(n):
            if i != j:
                term *= (x_eval - x_nodes[j]) / (x_nodes[i] - x_nodes[j])
        result += term
    return result

x_pts = [1.0, 2.0, 4.0]
y_pts = [2.0, 3.0, 11.0]
val = lagrange_interpolate(x_pts, y_pts, 3.0)
print(f"f(3) via Lagrange Interpolation = {val:.4f}")
```

■

Example 5.2.13 (Newton Forward Difference Formula)**.** Given equally spaced data $(0, 1)$, $(1, 2)$, $(2, 9)$, and $(3, 28)$, compute $f(1.5)$ using Newton's forward difference formula.

Solution. Here $h = 1$, $x_0 = 0$, $x = 1.5 \implies u = \frac{1.5-0}{1} = 1.5$. We compute the forward difference table:

x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$
0	1	1	6	6
1	2	7	12	
2	9	19		
3	28			

Downloaded from – https://aijaz.vercel.app

Applying Newton's forward formula:

$$\begin{aligned} P_3(1.5) &= y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \frac{u(u-1)(u-2)}{3!}\Delta^3 y_0 \\ &= 1 + 1.5(1) + \frac{1.5(0.5)}{2}(6) + \frac{1.5(0.5)(-0.5)}{6}(6) \\ &= 1 + 1.5 + 2.25 - 0.375 = 4.375 \end{aligned}$$

Python Implementation:

```
import math

def newton_forward(x_nodes, y_nodes, x_eval):
    n = len(x_nodes)
    h = x_nodes[1] - x_nodes[0]
    u = (x_eval - x_nodes[0]) / h

    # Construct Difference Table
    diff_table = [[0.0] * n for _ in range(n)]
    for i in range(n):
        diff_table[i][0] = y_nodes[i]

    for j in range(1, n):
        for i in range(n - j):
            diff_table[i][j] = diff_table[i+1][j-1] - diff_table[i][j-1]

    # Interpolation
    res = diff_table[0][0]
    u_product = 1.0
    for j in range(1, n):
        u_product *= (u - (j - 1))
        res += (u_product * diff_table[0][j]) / math.factorial(j)

    return res

x_data = [0.0, 1.0, 2.0, 3.0]
y_data = [1.0, 2.0, 9.0, 28.0]
print(f"f(1.5) via Newton Forward = {newton_forward(x_data, y_data, 1.5):.4f}")
```

■

Example 5.2.14 (Lagrange Interpolation with Four Nodes). Interpolate the point $x = 1$ given dataset: $x = [-1, 0, 2, 5]$, $y = [7, 1, 1, 31]$.

Downloaded from – https://aijaz.vercel.app

Solution. Given $x = 1$:

$$L_0(1) = \frac{(1-0)(1-2)(1-5)}{(-1-0)(-1-2)(-1-5)} = \frac{4}{-18} = -\frac{2}{9}$$
$$L_1(1) = \frac{(1+1)(1-2)(1-5)}{(0+1)(0-2)(0-5)} = \frac{8}{10} = \frac{4}{5}$$
$$L_2(1) = \frac{(1+1)(1-0)(1-5)}{(2+1)(2-0)(2-5)} = \frac{-8}{-18} = \frac{4}{9}$$
$$L_3(1) = \frac{(1+1)(1-0)(1-2)}{(5+1)(5-0)(5-2)} = \frac{-2}{90} = -\frac{1}{45}$$

Summing the terms:

$$P_3(1) = 7\left(-\frac{2}{9}\right) + 1\left(\frac{4}{5}\right) + 1\left(\frac{4}{9}\right) + 31\left(-\frac{1}{45}\right) = -1.0$$

Python Implementation:

```
x_pts = [-1.0, 0.0, 2.0, 5.0]
y_pts = [7.0, 1.0, 1.0, 31.0]
val = lagrange_interpolate(x_pts, y_pts, 1.0)
print(f"f(1) via Lagrange Interpolation = {val:.4f}")
```

■

5.2.4 Trapezoidal Rule

The Trapezoidal Rule approximates a definite integral $\int_a^b f(x)\,dx$ by partitioning the domain into linear segments.

Definition 5.2.15 (Composite Trapezoidal Rule)**.** Divide $[a, b]$ into n equal subintervals of width $h = \frac{b-a}{n}$ with nodes $x_i = a + ih$.

$$I_T = \frac{h}{2}\left[f(x_0) + 2\sum_{i=1}^{n-1} f(x_i) + f(x_n)\right]$$

Theorem 5.2.16 (Trapezoidal Error Bound)**.** *The truncation error of the Composite Trapezoidal Rule is:*

$$E_T = -\frac{(b-a)h^2}{12} f''(\xi), \quad \xi \in (a, b)$$

Below is the implementation of the composite trapezoidal rule:

```
def trapezoidal(f, a, b, n):
    h = (b - a) / n
    s = 0.5 * (f(a) + f(b))
    for i in range(1, n):
        s += f(a + i * h)
    return s * h
```

Downloaded from – https://aijaz.vercel.app

Example 5.2.17 (Integration of a Quadratic Polynomial)**.** Compute $\int_0^1 x^2\,dx$ using the Trapezoidal Rule with $n = 4$ subintervals. Compare with the exact answer.

Solution. Here $a = 0, b = 1, n = 4 \implies h = \frac{1-0}{4} = 0.25$. Nodes: $x = [0, 0.25, 0.5, 0.75, 1.0]$. Function values $f(x) = x^2$: $y = [0, 0.0625, 0.25, 0.5625, 1.0]$.

$$\begin{aligned} I_T &= \frac{0.25}{2}\left[0 + 2(0.0625 + 0.25 + 0.5625) + 1.0\right] \\ &= 0.125\left[2(0.875) + 1.0\right] = 0.125 \times 2.75 = 0.34375 \end{aligned}$$

Exact value: $\int_0^1 x^2 dx = \left[\frac{x^3}{3}\right]_0^1 = \frac{1}{3} \approx 0.333333$.
Absolute Error: $|0.333333 - 0.34375| = 0.010417$.

Python Implementation:

```
def f1(x):
    return x**2

res = trapezoidal(f1, 0.0, 1.0, 4)
exact = 1.0 / 3.0
print(f"Trapezoidal Result : {res:.6f}")
print(f"Exact Value        : {exact:.6f}")
print(f"Absolute Error     : {abs(exact - res):.6f}")
```

■

Example 5.2.18 (Integration of Trigonometric Function)**.** Evaluate $\int_0^{\pi/2} \sin(x)\,dx$ using $n = 4$ subintervals.

Solution. Here $a = 0, b = \frac{\pi}{2}, n = 4 \implies h = \frac{\pi/2}{4} = \frac{\pi}{8} \approx 0.392699$.
Nodes $x_i = \frac{i\pi}{8}$ for $i = 0, 1, 2, 3, 4$:

$$\begin{aligned} f(x_0) &= \sin(0) = 0 \\ f(x_1) &= \sin(\pi/8) \approx 0.382683 \\ f(x_2) &= \sin(\pi/4) \approx 0.707107 \\ f(x_3) &= \sin(3\pi/8) \approx 0.923880 \\ f(x_4) &= \sin(\pi/2) = 1.0 \end{aligned}$$

$$\begin{aligned} I_T &= \frac{0.392699}{2}\left[0 + 2(0.382683 + 0.707107 + 0.923880) + 1.0\right] \\ &= 0.196350 \times 5.027340 \approx 0.987116 \end{aligned}$$

Exact value is 1.0. Absolute Error: $|1.0 - 0.987116| = 0.012884$.

Python Implementation:

```
import math

res = trapezoidal(math.sin, 0.0, math.pi / 2.0, 4)
exact = 1.0
print(f"Trapezoidal Result : {res:.6f}")
print(f"Absolute Error     : {abs(exact - res):.6f}")
```

■

Example 5.2.19 (Logarithmic Integral). Compute $\int_1^2 \frac{1}{x}\,dx$ using $n = 4$ subintervals.

Solution. $a = 1, b = 2, n = 4 \implies h = 0.25$.
Nodes: $x = [1.0, 1.25, 1.5, 1.75, 2.0]$.
Function values $f(x) = \frac{1}{x}$: $y = [1.0, 0.8, 0.666667, 0.571429, 0.5]$.

$$\begin{aligned} I_T &= \frac{0.25}{2}\left[1.0 + 2(0.8 + 0.666667 + 0.571429) + 0.5\right] \\ &= 0.125\left[1.5 + 4.076192\right] = 0.125 \times 5.576192 = 0.697024 \end{aligned}$$

Exact value: $\ln(2) \approx 0.693147$. Absolute Error: 0.003877.

Python Implementation:

```
import math

def f_reciprocal(x):
    return 1.0 / x

res = trapezoidal(f_reciprocal, 1.0, 2.0, 4)
exact = math.log(2.0)
print(f"Trapezoidal Result : {res:.6f}")
print(f"Exact Value        : {exact:.6f}")
print(f"Absolute Error     : {abs(exact - res):.6f}")
```

■

5.2.5 Simpson's 1/3 - Rule

Simpson's 1/3 Rule approximates integrals using piecewise quadratic interpolating polynomials.

Definition 5.2.20 (Composite Simpson's 1/3 Rule). Let n be an **even integer**. Partition $[a, b]$ into n equal subintervals of width $h = \frac{b-a}{n}$.

$$I_S = \frac{h}{3}\left[f(x_0) + 4\sum_{\substack{i=1 \\ i \text{ odd}}}^{n-1} f(x_i) + 2\sum_{\substack{i=2 \\ i \text{ even}}}^{n-2} f(x_i) + f(x_n)\right]$$

Theorem 5.2.21 (Simpson's Rule Error Bound). *The truncation error of the Composite Simpson's 1/3 Rule is:*

$$E_S = -\frac{(b-a)h^4}{180} f^{(4)}(\xi), \quad \xi \in (a, b)$$

Since E_S depends on $f^{(4)}$, Simpson's 1/3 Rule is exact for polynomials up to degree 3.

Below is the Python implementation:

```
def simpson13(f, a, b, n):
    if n % 2 != 0:
        raise ValueError("Subintervals 'n' must be an even integer.")
    h = (b - a) / n
    s = f(a) + f(b)
```

```
    for i in range(1, n):
        weight = 4 if i % 2 != 0 else 2
        s += weight * f(a + i * h)
    return s * (h / 3.0)
```

Example 5.2.22 (Exact Integration of Cubic Polynomial)**.** Evaluate $\int_0^1 x^3\,dx$ using Simpson's 1/3 Rule with $n = 2$ subintervals.

Solution. $a = 0, b = 1, n = 2 \implies h = 0.5$.
Nodes: $x = [0, 0.5, 1.0]$. Function values $f(x) = x^3$: $y = [0, 0.125, 1.0]$.

$$I_S = \frac{0.5}{3}\left[0 + 4(0.125) + 1.0\right] = \frac{0.5}{3}[0.5 + 1.0] = \frac{0.75}{3} = 0.25$$

Exact value: $\int_0^1 x^3 dx = \left[\frac{x^4}{4}\right]_0^1 = 0.25$. As expected, the error is 0.

Python Implementation:

```
def f_cubic(x):
    return x**3

res = simpson13(f_cubic, 0.0, 1.0, 2)
print(f"Simpson's 1/3 Result : {res:.6f}")
print(f"Exact Value          : 0.250000")
```

■

Example 5.2.23 (Integration of Sine across Half Period)**.** Evaluate $\int_0^{\pi} \sin(x)\,dx$ using Simpson's 1/3 Rule with $n = 4$.

Solution. $a = 0, b = \pi, n = 4 \implies h = \frac{\pi}{4} \approx 0.785398$.
Nodes: $x = [0, \pi/4, \pi/2, 3\pi/4, \pi]$.
Values: $y = [0, \frac{\sqrt{2}}{2}, 1.0, \frac{\sqrt{2}}{2}, 0]$.

$$\begin{aligned} I_S &= \frac{\pi/4}{3}\left[0 + 4\left(\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}\right) + 2(1.0) + 0\right] \\ &= \frac{\pi}{12}\left[4\sqrt{2} + 2\right] \approx 0.261799 \times 7.656854 \approx 2.004560 \end{aligned}$$

Exact value: $\int_0^{\pi} \sin(x) dx = 2.0$. Absolute error: 0.004560.

Python Implementation:

```
import math

res = simpson13(math.sin, 0.0, math.pi, 4)
exact = 2.0
print(f"Simpson's 1/3 Result : {res:.6f}")
print(f"Exact Value          : {exact:.6f}")
print(f"Absolute Error     : {abs(exact - res):.6f}")
```

■

Example 5.2.24 (Exponential Function Integration)**.** Compute $\int_0^1 e^x\,dx$ using Simpson's 1/3 Rule with $n = 4$.

Solution. $a = 0, b = 1, n = 4 \implies h = 0.25$.
Nodes $x = [0.0, 0.25, 0.50, 0.75, 1.00]$.
Function values:

$$f(0.0) = 1.0, \quad f(0.25) \approx 1.284025, \quad f(0.50) \approx 1.648721$$
$$f(0.75) \approx 2.117000, \quad f(1.0) \approx 2.718282$$

Applying formula:

$$\begin{aligned} I_S &= \frac{0.25}{3}\left[1.0 + 4(1.284025 + 2.117000) + 2(1.648721) + 2.718282\right] \\ &= \frac{0.25}{3}\left[1.0 + 13.604100 + 3.297442 + 2.718282\right] \\ &= \frac{0.25}{3} \times 20.619824 \approx 1.718319 \end{aligned}$$

Exact value: $e - 1 \approx 1.718282$. Absolute error: 0.000037.

Python Implementation:

```
import math

res = simpson13(math.exp, 0.0, 1.0, 4)
exact = math.e - 1.0
print(f"Simpson's 1/3 Result : {res:.6f}")
print(f"Exact Value          : {exact:.6f}")
print(f"Absolute Error     : {abs(exact - res):.6f}")
```

■

Exercise 5.2.25. Solve the following Exercises

1. **Python Basics and Formatting:** Write a Python script that calculates the real roots of the quadratic equation $ax^2 + bx + c = 0$ for user-provided coefficients $a = 1, b = -5, c = 6$. Format the output using an f-string to display the roots rounded to 4 decimal places.

2. **Error Analysis (Square Root Approximation):** The actual value of $\sqrt{3}$ is approximately 1.7320508075. If an engineer uses the rational approximation $\tilde{x} = \frac{7}{4} = 1.75$, calculate the absolute error, relative error, and percentage error.

3. **Truncation Error in Taylor Series:** Using the third-order Maclaurin series expansion $e^x \approx 1 + x + \frac{x^2}{2} + \frac{x^3}{6}$, compute the absolute error and relative error when approximating $e^{0.4}$. Compare this with Python's exact `math.exp(0.4)`.

4. **Control Structures (`while` Loop):** Write a Python program using a `while` loop to find the smallest positive integer N such that the sum of cubes

$$S_N = \sum_{k=1}^{N} k^3$$

exceeds 10,000. Print the value of N and the corresponding sum S_N.

Downloaded from – https://aijaz.vercel.app

5. **Bisection Method (Algebraic Root):** Use the Bisection Method to find an approximate root of $f(x) = x^3 - 4x - 9 = 0$ in the interval $[2, 3]$. Perform 3 manual iterations, listing a_k, b_k, c_k, and $f(c_k)$ at each step.

6. **Bisection Method (Transcendental Root):** Show that $f(x) = e^{-x} - x = 0$ has a root in the interval $[0, 1]$. Execute 3 iterations of the Bisection Method to locate the root.

7. **Bisection Convergence Estimate:** For the equation $x^3 + x - 1 = 0$ on the interval $[0, 1]$, determine theoretically the minimum number of iterations n required by the Bisection Method to guarantee that the root approximation has an error bounded by $\varepsilon = 10^{-4}$.

8. **Lagrange Interpolation (Unequal Spacing):** Given the dataset $(-2, -15)$, $(0, 1)$, $(1, 3)$, and $(3, 25)$, construct the Lagrange interpolating polynomial $P_3(x)$ and estimate the functional value $f(2)$.

9. **Newton Forward Difference Interpolation:** Construct the forward difference table for the equally spaced data points $(1, 1)$, $(2, 8)$, $(3, 27)$, and $(4, 64)$, and compute $f(2.5)$ using Newton's forward difference interpolation formula.

10. **Trapezoidal Rule (Reciprocal Square Function):** Evaluate the definite integral

$$I = \int_1^3 \frac{1}{x^2}\, dx$$

using the Composite Trapezoidal Rule with $n = 4$ subintervals. Calculate the exact analytical value and determine the absolute error.

11. **Simpson's 1/3 Rule (Radical Integrand):** Estimate the integral

$$I = \int_0^{\pi/2} \sqrt{1 + \sin(x)}\, dx$$

using Simpson's $1/3$ Rule with $n = 4$ subintervals. Express your answer rounded to 6 decimal places.

12. **Comparative Quadrature:** Evaluate the integral

$$I = \int_0^2 e^{-x^2}\, dx$$

using both the Composite Trapezoidal Rule and Simpson's $1/3$ Rule with $n = 4$ subintervals. Compare the two numerical approximations.

13. **Python Integration Function:** Write a complete Python program containing a function `compare_quadrature(f, a, b, n)` that accepts an integrable function f, bounds a and b, and an even integer n. The program should compute and display the integral estimates using both the Composite Trapezoidal Rule and Simpson's $1/3$ Rule, alongside their absolute discrepancy $|I_T - I_S|$. Test your script on $f(x) = \frac{1}{1+x^2}$ over $[0, 1]$ with $n = 6$.

CHAPTER

6

Python Programming: Advanced Numerical Techniques

6.1 Linear Systems and Iterative Methods

Systems of linear equations arise across all branches of computational science and engineering. A general $n \times n$ linear system is expressed in matrix-vector form as:

$$A\mathbf{x} = \mathbf{b}$$

where $A \in \mathbb{R}^{n \times n}$ is a square coefficient matrix, $\mathbf{x} \in \mathbb{R}^n$ is the unknown vector, and $\mathbf{b} \in \mathbb{R}^n$ is the right-hand side constant vector. Solution techniques are broadly categorized into:

(i) **Direct Methods:** Obtain the exact theoretical solution (up to round-off error) in a finite number of deterministic operations (e.g., Cramer's Rule, Gaussian Elimination, LU Decomposition).

(ii) **Iterative Methods:** Generate a sequence of successive approximations $\{\mathbf{x}^{(k)}\}_{k=0}^{\infty}$ starting from an initial guess $\mathbf{x}^{(0)}$, converging to the true solution as $k \to \infty$ (e.g., Jacobi, Gauss-Seidel, SOR Methods).

6.1.1 Solving 2x2 Linear Systems via Direct Formula

For small systems, direct algebraic formulas provide immediate closed-form solutions without requiring complex matrix decompositions.

Theoretical Formulation

Consider a general 2×2 system of linear equations:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 = b_1 \\ a_{21}x_1 + a_{22}x_2 = b_2 \end{cases} \iff \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

Definition 6.1.1 (Determinant and Solvability)**.** The determinant $D = \det(A)$ of the coefficient matrix A is:

$$D = a_{11}a_{22} - a_{12}a_{21}$$

The system has a unique solution if and only if $D \neq 0$. If $D = 0$, the system is singular and either possesses infinitely many solutions (consistent) or no solution (inconsistent).

When $D \neq 0$, matrix inversion or Cramer's Rule yields the closed-form direct solution:

$$A^{-1} = \frac{1}{D} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix}$$

Downloaded from – https://aijaz.vercel.app

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \frac{1}{D} \begin{bmatrix} a_{22}b_1 - a_{12}b_2 \\ -a_{21}b_1 + a_{11}b_2 \end{bmatrix}$$

Thus, the explicit coordinate expressions are:

$$x_1 = \frac{b_1 a_{22} - b_2 a_{12}}{D}, \qquad x_2 = \frac{a_{11}b_2 - a_{21}b_1}{D} \tag{6.1}$$

Remark 6.1.2 (Numerical Ill-Conditioning). If $|D|$ is extremely close to zero, the matrix is ill-conditioned. In floating-point arithmetic, dividing by a near-zero determinant causes severe catastrophic cancellation and amplifies measurement noise.

Python Implementation

The following function computes the direct solution of a 2×2 system while validating the non-singularity condition:

```
def solve_2x2(A, b, tol=1e-12):
    """
    Solves A x = b for a 2x2 system using direct formula.
    A: 2x2 list or array [[a11, a12], [a21, a22]]
    b: 1D list [b1, b2]
    """
    a11, a12 = A[0][0], A[0][1]
    a21, a22 = A[1][0], A[1][1]
    b1, b2 = b[0], b[1]

    det = a11 * a22 - a12 * a21

    if abs(det) < tol:
        raise ValueError(f"System is singular or ill-conditioned (det = {det}).")

    x1 = (b1 * a22 - b2 * a12) / det
    x2 = (a11 * b2 - a21 * b1) / det

    return [x1, x2]
```

Example 6.1.3 (Standard Well-Conditioned 2x2 System)**.** Solve the linear system directly:

$$\begin{cases} 2x_1 + 3x_2 = 8 \\ 4x_1 - x_2 = 2 \end{cases}$$

Solution. Here $a_{11} = 2$, $a_{12} = 3$, $a_{21} = 4$, $a_{22} = -1$, $b_1 = 8$, $b_2 = 2$. Compute the determinant:

$$D = (2)(-1) - (3)(4) = -2 - 12 = -14 \neq 0$$

Applying equation (6.1):

$$x_1 = \frac{(8)(-1) - (2)(3)}{-14} = \frac{-8 - 6}{-14} = \frac{-14}{-14} = 1.0$$
$$x_2 = \frac{(2)(2) - (4)(8)}{-14} = \frac{4 - 32}{-14} = \frac{-28}{-14} = 2.0$$

Python Implementation:

Downloaded from – https://aijaz.vercel.app

```
A = [[2.0, 3.0], [4.0, -1.0]]
b = [8.0, 2.0]

x = solve_2x2(A, b)
print(f"Solution: x1 = {x[0]:.4f}, x2 = {x[1]:.4f}")
```

■

Example 6.1.4 (Singular System Detection)**.** Attempt to solve the system:

$$\begin{cases} 3x_1 + 6x_2 = 9 \\ 2x_1 + 4x_2 = 5 \end{cases}$$

Solution. Computing the determinant:

$$D = (3)(4) - (6)(2) = 12 - 12 = 0$$

Since $D = 0$, the coefficient matrix is singular, and no unique solution exists.

Python Implementation:

```
A_sing = [[3.0, 6.0], [2.0, 4.0]]
b_sing = [9.0, 5.0]

try:
    x = solve_2x2(A_sing, b_sing)
except ValueError as e:
    print(f"Handled Error: {e}")
```

■

Example 6.1.5 (Ill-Conditioned System Sensitivity)**.** Solve the nearly singular system:

$$\begin{cases} 1.000x_1 + 1.000x_2 = 2.000 \\ 1.000x_1 + 1.001x_2 = 2.001 \end{cases}$$

Solution. Compute the determinant:

$$D = (1.000)(1.001) - (1.000)(1.000) = 1.001 - 1.000 = 0.001$$

Applying the direct formula:

$$x_1 = \frac{(2.000)(1.001) - (2.001)(1.000)}{0.001} = \frac{2.002 - 2.001}{0.001} = 1.0$$
$$x_2 = \frac{(1.000)(2.001) - (1.000)(2.000)}{0.001} = \frac{0.001}{0.001} = 1.0$$

Python Implementation:

```
A_ill = [[1.000, 1.000], [1.000, 1.001]]
b_ill = [2.000, 2.001]

x = solve_2x2(A_ill, b_ill)
print(f"Ill-conditioned Solution: x1 = {x[0]:.4f}, x2 = {x[1]:.4f}")
```

■

Downloaded from – https://aijaz.vercel.app

6.1.2 Single Iteration of Jacobi Method

When dealing with large sparse linear systems $A\mathbf{x} = \mathbf{b}$, direct methods become computationally expensive ($O(n^3)$ operations). Iterative methods provide scalable approximations.

Decompose the matrix A into $A = D + L + U$, where:

- $D = \text{diag}(a_{11}, a_{22}, \ldots, a_{nn})$ is the main diagonal.
- L is the strictly lower triangular part.
- U is the strictly upper triangular part.

Assuming $a_{ii} \neq 0$ for all $i = 1, \ldots, n$, the Jacobi iteration decomposes $A\mathbf{x} = \mathbf{b}$ as:

$$D\mathbf{x} + (L + U)\mathbf{x} = \mathbf{b} \implies D\mathbf{x} = \mathbf{b} - (L + U)\mathbf{x}$$

Multiplying by D^{-1} defines the Jacobi iterative recurrence relation:

$$\mathbf{x}^{(k+1)} = D^{-1}\left(\mathbf{b} - (L + U)\mathbf{x}^{(k)}\right)$$

Definition 6.1.6 (Component-Wise Jacobi Iteration)**.** For an initial guess $\mathbf{x}^{(0)} = [x_1^{(0)}, x_2^{(0)}, \ldots, x_n^{(0)}]^T$, a single Jacobi iteration updates all components simultaneously:

$$x_i^{(1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{\substack{j=1 \\ j\neq i}}^{n} a_{ij}x_j^{(0)}\right), \qquad i = 1, 2, \ldots, n \tag{6.2}$$

Theorem 6.1.7 (Sufficient Condition for Convergence)**.** *The Jacobi method converges to the unique solution* $\mathbf{x}^*$ *from any initial guess* $\mathbf{x}^{(0)}$ *if* A *is* ***strictly diagonally dominant****, i.e.,*

$$|a_{ii}| > \sum_{\substack{j=1 \\ j\neq i}}^{n} |a_{ij}| \quad \textit{for all } i = 1, 2, \ldots, n$$

Python Implementation

The following function executes exactly one iteration of the Jacobi method:

```
def jacobi_single_step(A, b, x0):
    """
    Executes a single iteration of the Jacobi method.
    A: n x n list/array
    b: n dimensional vector
    x0: initial guess vector
    """
    n = len(b)
    x1 = [0.0] * n

    for i in range(n):
        s = sum(A[i][j] * x0[j] for j in range(n) if j != i)
        if A[i][i] == 0:
```

Downloaded from – https://aijaz.vercel.app

```
            raise ZeroDivisionError(f"Zero diagonal element at index {i}.")
        x1[i] = (b[i] - s) / A[i][i]

    return x1
```

Example 6.1.8 (Single Jacobi Step on 2x2 System)**.** Perform a single iteration of the Jacobi method on the system:

$$\begin{cases} 4x_1 + x_2 = 9 \\ x_1 + 5x_2 = 7 \end{cases}$$

starting from the initial guess $\mathbf{x}^{(0)} = [0, 0]^T$.

Solution. Check diagonal dominance: $|4| > |1|$ and $|5| > |1|$ (Satisfied).
Express component equations:

$$x_1^{(1)} = \frac{9 - x_2^{(0)}}{4}, \qquad x_2^{(1)} = \frac{7 - x_1^{(0)}}{5}$$

Substitute $\mathbf{x}^{(0)} = [0, 0]^T$:

$$x_1^{(1)} = \frac{9 - 0}{4} = 2.25$$
$$x_2^{(1)} = \frac{7 - 0}{5} = 1.40$$

After 1 iteration, $\mathbf{x}^{(1)} = [2.25, 1.40]^T$ (Exact solution is $[2.0, 1.0]^T$).

Python Implementation:

```
A1 = [[4.0, 1.0], [1.0, 5.0]]
b1 = [9.0, 7.0]
x0_1 = [0.0, 0.0]

x1_step = jacobi_single_step(A1, b1, x0_1)
print(f"After 1 Jacobi step: x1 = {x1_step[0]:.4f}, x2 = {x1_step[1]:.4f}")
```

■

Example 6.1.9 (Single Jacobi Step on 3x3 System)**.** Perform a single Jacobi iteration for the 3×3 system:

$$\begin{cases} 10x_1 - x_2 + 2x_3 = 6 \\ -x_1 + 11x_2 - x_3 = 25 \\ 2x_1 - x_2 + 10x_3 = -11 \end{cases}$$

with initial guess $\mathbf{x}^{(0)} = [0, 0, 0]^T$.

Solution. Verify diagonal dominance:

- Row 1: $|10| > |-1| + |2| = 3$
- Row 2: $|11| > |-1| + |-1| = 2$
- Row 3: $|10| > |2| + |-1| = 3$

Downloaded from – https://aijaz.vercel.app

Component equations for iteration 1:

$$x_1^{(1)} = \frac{6 - (-x_2^{(0)} + 2x_3^{(0)})}{10} = \frac{6-0}{10} = 0.6000$$
$$x_2^{(1)} = \frac{25 - (-x_1^{(0)} - x_3^{(0)})}{11} = \frac{25-0}{11} \approx 2.2727$$
$$x_3^{(1)} = \frac{-11 - (2x_1^{(0)} - x_2^{(0)})}{10} = \frac{-11-0}{10} = -1.1000$$

Thus, $\mathbf{x}^{(1)} = [0.6000, 2.2727, -1.1000]^T$.

Python Implementation:

```
A2 = [
    [10.0, -1.0, 2.0],
    [-1.0, 11.0, -1.0],
    [2.0, -1.0, 10.0]
]
b2 = [6.0, 25.0, -11.0]
x0_2 = [0.0, 0.0, 0.0]

x2_step = jacobi_single_step(A2, b2, x0_2)
print("3x3 Single Step Solution:")
for i, val in enumerate(x2_step, 1):
    print(f"  x{i}^(1) = {val:.4f}")
```

■

Example 6.1.10 (Jacobi Step with Non-Zero Initial Guess)**.** Perform one Jacobi iteration on the system:

$$\begin{cases} 5x_1 - 2x_2 + x_3 = 4 \\ x_1 + 6x_2 - 2x_3 = 9 \\ 2x_1 + x_2 + 7x_3 = 17 \end{cases}$$

given a non-zero initial estimate $\mathbf{x}^{(0)} = [1.0, 1.0, 1.0]^T$.

Solution. Component update formulas evaluated at $\mathbf{x}^{(0)} = [1.0, 1.0, 1.0]^T$:

$$x_1^{(1)} = \frac{4 - (-2(1.0) + 1.0)}{5} = \frac{4 - (-1.0)}{5} = \frac{5.0}{5} = 1.0000$$
$$x_2^{(1)} = \frac{9 - (1.0(1.0) - 2.0(1.0))}{6} = \frac{9 - (-1.0)}{6} = \frac{10.0}{6} \approx 1.6667$$
$$x_3^{(1)} = \frac{17 - (2.0(1.0) + 1.0(1.0))}{7} = \frac{17 - 3.0}{7} = \frac{14.0}{7} = 2.0000$$

The state vector after one step is $\mathbf{x}^{(1)} = [1.0000, 1.6667, 2.0000]^T$.

Python Implementation:

```
A3 = [
    [5.0, -2.0, 1.0],
    [1.0, 6.0, -2.0],
    [2.0, 1.0, 7.0]
]
b3 = [4.0, 9.0, 17.0]
```

Downloaded from – https://aijaz.vercel.app

```
x0_3 = [1.0, 1.0, 1.0]

x3_step = jacobi_single_step(A3, b3, x0_3)
print("Non-zero initial guess single step solution:")
for i, val in enumerate(x3_step, 1):
    print(f"  x{i}^(1) = {val:.4f}")
```

■

6.2 Differential Equations and Data Storage

Ordinary Differential Equations (ODEs) model dynamic systems across physics, biology, and engineering. A first-order Initial Value Problem (IVP) is formulated as:

$$\frac{dy}{dt} = f(t, y), \qquad y(t_0) = y_0 \tag{6.3}$$

where $f : [t_0, T] \times \mathbb{R} \to \mathbb{R}$ is a continuous function, t_0 is the initial time, and y_0 is the given initial state. Numerical integration computes discrete approximations $y_n \approx y(t_n)$ at time nodes $t_n = t_0 + nh$, where $h > 0$ represents the step size.

6.2.1 Single Step Euler's Method

Euler's Method is the foundational single-step numerical scheme for approximating solutions to initial value problems.

Consider the Taylor series expansion of the exact solution $y(t)$ around t_n:

$$y(t_{n+1}) = y(t_n + h) = y(t_n) + hy'(t_n) + \frac{h^2}{2!}y''(\xi_n), \quad \xi_n \in (t_n, t_{n+1})$$

Substituting $y'(t_n) = f(t_n, y(t_n))$ and truncating terms of order $O(h^2)$ yields the **Forward Euler Formula**:

$$y_{n+1} = y_n + hf(t_n, y_n) \tag{6.4}$$

Definition 6.2.1 (Truncation Error Metrics). (i) **Local Truncation Error (LTE):** The error committed in a single step assuming previous values are exact:

$$\tau_{n+1} = \frac{h^2}{2}y''(\xi_n) = O(h^2)$$

(ii) **Global Truncation Error (GTE):** The accumulated error over $N = \frac{T-t_0}{h}$ steps, satisfying $\text{GTE} = O(h)$. Thus, Euler's method is a **first-order accurate** scheme.

Python Implementation

The single-step Euler operator is implemented as follows:

```
def euler_single_step(f, t0, y0, h):
    """
    Executes a single step of Forward Euler's method.
    f : Right-hand side function f(t, y)
    t0: Current time node
    y0: Current state approximation
    h : Step size
```

Downloaded from – https://aijaz.vercel.app

```
    """
    return y0 + h * f(t0, y0)
```

Example 6.2.2 (Linear Decay ODE). Apply a single step of Euler's method to $\frac{dy}{dt} = -2y$ with $y(0) = 1.0$ and step size $h = 0.1$. Compute the absolute error relative to the exact solution $y(t) = e^{-2t}$.

Solution. Here $t_0 = 0$, $y_0 = 1.0$, $h = 0.1$, and $f(t, y) = -2y$.

$$f(0, 1.0) = -2(1.0) = -2.0$$

Applying equation (6.4):

$$y_1 = y_0 + hf(t_0, y_0) = 1.0 + 0.1(-2.0) = 1.0 - 0.2 = 0.8000$$

Exact value: $y(0.1) = e^{-2(0.1)} = e^{-0.2} \approx 0.818731$.
Absolute Error: $|0.818731 - 0.800000| = 0.018731$.

Python Implementation:

```
import math

def f_decay(t, y):
    return -2.0 * y

t0, y0, h = 0.0, 1.0, 0.1
y1 = euler_single_step(f_decay, t0, y0, h)
exact = math.exp(-2.0 * (t0 + h))

print(f"Euler Step Result : {y1:.6f}")
print(f"Exact Solution    : {exact:.6f}")
print(f"Absolute Error    : {abs(exact - y1):.6f}")
```

■

Example 6.2.3 (Non-Linear Non-Autonomous ODE). Compute a single Euler step for $\frac{dy}{dt} = t + y$ with $y(0) = 1.0$ and step size $h = 0.2$.

Solution. Given $t_0 = 0$, $y_0 = 1.0$, $h = 0.2$, $f(t, y) = t + y$:

$$f(0, 1.0) = 0 + 1.0 = 1.0$$

$$y_1 = 1.0 + 0.2(1.0) = 1.2000$$

The analytical solution is $y(t) = 2e^t - t - 1$. At $t = 0.2$:

$$y(0.2) = 2e^{0.2} - 0.2 - 1 \approx 2(1.221403) - 1.2 = 1.242806$$

Absolute Error: $|1.242806 - 1.200000| = 0.042806$.

Python Implementation:

```
def f_linear(t, y):
    return t + y

t0, y0, h = 0.0, 1.0, 0.2
```

Downloaded from – https://aijaz.vercel.app

```
y1 = euler_single_step(f_linear, t0, y0, h)
exact = 2.0 * math.exp(0.2) - 0.2 - 1.0

print(f"Euler Step Result : {y1:.6f}")
print(f"Exact Solution    : {exact:.6f}")
print(f"Absolute Error    : {abs(exact - y1):.6f}")
```

■

Example 6.2.4 (Non-Linear Quadratic ODE)**.** Evaluate a single Euler step for $\frac{dy}{dt} = y^2 - t^2$ with initial state $y(0) = 1.0$ and $h = 0.1$.

Solution. Here $t_0 = 0, y_0 = 1.0, h = 0.1, f(t, y) = y^2 - t^2$:

$$f(0, 1.0) = 1.0^2 - 0^2 = 1.0$$

$$y_1 = 1.0 + 0.1(1.0) = 1.1000$$

Python Implementation:

```
def f_quad(t, y):
    return y**2 - t**2

y1 = euler_single_step(f_quad, 0.0, 1.0, 0.1)
print(f"Euler Step Result : {y1:.4f}")
```

■

6.2.2 Single Step Runge–Kutta Method (RK2/RK4)

Runge–Kutta methods achieve higher-order accuracy by evaluating the function $f(t, y)$ at intermediate stage points within the step interval $[t_n, t_n + h]$, eliminating the need to calculate higher-order derivatives of f.

Second-Order Runge–Kutta (RK2 - Heun's Method)

RK2 averages slope estimates at the start and end of the interval.

Definition 6.2.5 (Heun's RK2 Formula)**.**

$$\begin{aligned} k_1 &= f(t_n, y_n) \\ k_2 &= f(t_n + h, y_n + hk_1) \\ y_{n+1} &= y_n + \frac{h}{2}(k_1 + k_2) \end{aligned}$$

RK2 has a local truncation error of $O(h^3)$ and a global order of accuracy $O(h^2)$.

Fourth-Order Runge–Kutta (Classical RK4)

The classical RK4 method uses a weighted average of four intermediate stage slopes.

Definition 6.2.6 (Classical RK4 Scheme)**.**

$$
\begin{aligned}
k_1 &= f(t_n, y_n) \\
k_2 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \\
k_3 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \\
k_4 &= f(t_n + h, y_n + hk_3) \\
y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)
\end{aligned}
$$

RK4 has a local error of $O(h^5)$ and a global order of accuracy $O(h^4)$.

Python Implementation

Functions for executing single steps of RK2 and RK4 are defined below:

```
def rk2_single_step(f, t0, y0, h):
    k1 = f(t0, y0)
    k2 = f(t0 + h, y0 + h * k1)
    return y0 + (h / 2.0) * (k1 + k2)

def rk4_single_step(f, t0, y0, h):
    k1 = f(t0, y0)
    k2 = f(t0 + 0.5 * h, y0 + 0.5 * h * k1)
    k3 = f(t0 + 0.5 * h, y0 + 0.5 * h * k2)
    k4 = f(t0 + h, y0 + h * k3)
    return y0 + (h / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4)
```

Example 6.2.7 (RK2 Step on Exponential Decay)**.** Compute a single step of Heun's RK2 method for $\frac{dy}{dt} = -2y$ with $y(0) = 1.0$ and $h = 0.1$.

Solution. Given $t_0 = 0$, $y_0 = 1.0$, $h = 0.1$:

$$
\begin{aligned}
k_1 &= f(0, 1.0) = -2(1.0) = -2.0 \\
k_2 &= f(0.1, 1.0 + 0.1(-2.0)) = f(0.1, 0.8) = -2(0.8) = -1.6 \\
y_1 &= 1.0 + \frac{0.1}{2}(-2.0 - 1.6) = 1.0 + 0.05(-3.6) = 0.8200
\end{aligned}
$$

Exact value: $y(0.1) \approx 0.818731$. Absolute error is $|0.818731 - 0.820000| = 0.001269$ (over $14\times$ more accurate than Euler's method).

Python Implementation:

```
y1_rk2 = rk2_single_step(f_decay, 0.0, 1.0, 0.1)
print(f"RK2 Single Step Result : {y1_rk2:.6f}")
```

■

Example 6.2.8 (RK4 Step on Non-Autonomous ODE)**.** Compute a single RK4 step for $\frac{dy}{dt} = t + y$ with $y(0) = 1.0$ and $h = 0.2$.

Solution. Given $t_0 = 0, y_0 = 1.0, h = 0.2, f(t, y) = t + y$:

$$\begin{aligned}
k_1 &= f(0, 1.0) = 0 + 1.0 = 1.0000 \\
k_2 &= f(0.1, 1.0 + 0.1(1.0)) = f(0.1, 1.1) = 0.1 + 1.1 = 1.2000 \\
k_3 &= f(0.1, 1.0 + 0.1(1.2)) = f(0.1, 1.12) = 0.1 + 1.12 = 1.2200 \\
k_4 &= f(0.2, 1.0 + 0.2(1.22)) = f(0.2, 1.244) = 0.2 + 1.244 = 1.4440
\end{aligned}$$

Applying RK4 combination:

$$\begin{aligned}
y_1 &= 1.0 + \frac{0.2}{6}\left[1.0 + 2(1.2) + 2(1.22) + 1.444\right] \\
&= 1.0 + \frac{0.2}{6}\left[1.0 + 2.4 + 2.44 + 1.444\right] = 1.0 + \frac{0.2}{6}(7.284) = 1.242800
\end{aligned}$$

Exact value: $y(0.2) \approx 1.242806$. Absolute Error: 5.5×10^{-6}.

Python Implementation:

```
y1_rk4 = rk4_single_step(f_linear, 0.0, 1.0, 0.2)
print(f"RK4 Single Step Result : {y1_rk4:.6f}")
```

■

Example 6.2.9 (RK4 Step on Polynomial ODE)**.** Evaluate a single RK4 step for $\frac{dy}{dt} = y - t^2 + 1$ with $y(0) = 0.5$ and $h = 0.2$.

Solution. Given $t_0 = 0, y_0 = 0.5, h = 0.2$:

$$\begin{aligned}
k_1 &= f(0, 0.5) = 0.5 - 0^2 + 1 = 1.500000 \\
k_2 &= f(0.1, 0.5 + 0.1(1.5)) = f(0.1, 0.65) = 0.65 - (0.1)^2 + 1 = 1.640000 \\
k_3 &= f(0.1, 0.5 + 0.1(1.64)) = f(0.1, 0.664) = 0.664 - 0.01 + 1 = 1.654000 \\
k_4 &= f(0.2, 0.5 + 0.2(1.654)) = f(0.2, 0.8308) = 0.8308 - (0.2)^2 + 1 = 1.790800
\end{aligned}$$

Evaluating RK4 state sum:

$$\begin{aligned}
y_1 &= 0.5 + \frac{0.2}{6}\left[1.5 + 2(1.64) + 2(1.654) + 1.7908\right] \\
&= 0.5 + \frac{0.2}{6}\left[9.8788\right] = 0.5 + 0.329293 = 0.829293
\end{aligned}$$

Python Implementation:

```
def f_poly(t, y):
    return y - t**2 + 1.0

y1_rk4_poly = rk4_single_step(f_poly, 0.0, 0.5, 0.2)
print(f"RK4 Single Step Result : {y1_rk4_poly:.6f}")
```

■

6.2.3 Basic Usage of Lists and Arrays for Data Storage

When integrating ODEs over multi-step time intervals $[t_0, T]$, storing generated trajectory time points t_n and state values y_n efficiently is essential for post-processing, tabular display, and plotting.

Downloaded from – https://aijaz.vercel.app

Feature	Python List (`list`)	NumPy Array (`numpy.ndarray`)
Memory Layout	Pointers to dynamic objects	Contiguous block of homogeneous memory
Element Types	Heterogeneous types permitted	Strictly homogeneous (e.g., `float64`)
Growth	Dynamic resizing via `.append()`	Fixed size; pre-allocation recommended
Operations	Element iteration / comprehensions	Vectorized SIMD operations
Efficiency	General purpose / higher overhead	High performance / numerical analysis

Table 6.1: Comparison of native Python lists and NumPy arrays for data storage.

Built-in Python Lists vs. NumPy Arrays

Data Accumulation Patterns for Numerical Trajectories

There are two primary computational patterns used to record time-series state data:

1. **Dynamic Appending (Python Lists):** Useful when total steps N are unknown beforehand (e.g., adaptive step-size solvers).

```
t_list = [t0]
y_list = [y0]

t_curr, y_curr = t0, y0
for _ in range(N):
    y_curr = rk4_single_step(f, t_curr, y_curr, h)
    t_curr += h
    t_list.append(t_curr)
    y_list.append(y_curr)
```

2. **Static Pre-Allocation (NumPy Arrays):** Preferred for fixed-step integration due to memory efficiency and fast indexing.

```
import numpy as np

t_array = np.linspace(t0, T, N + 1)
y_array = np.zeros(N + 1)
y_array[0] = y0

for n in range(N):
    y_array[n + 1] = rk4_single_step(f, t_array[n], y_array[n], h)
```

Complete Code Example: Trajectory Storage and Analysis

The script below demonstrates complete ODE integration trajectory generation, storing results using both structures, and extracting numerical metrics:

```
import math
import numpy as np

# Problem setup: dy/dt = -2y, y(0) = 1, t in [0, 1]
def f_sys(t, y):
    return -2.0 * y

t0, T, y0 = 0.0, 1.0, 1.0
```

```
N = 10
h = (T - t0) / N

# Strategy 1: Pre-allocated NumPy arrays
t_np = np.linspace(t0, T, N + 1)
y_np = np.zeros(N + 1)
y_np[0] = y0

for n in range(N):
    y_np[n + 1] = rk4_single_step(f_sys, t_np[n], y_np[n], h)

# Strategy 2: Vectorized error computation across all stored steps
exact_trajectory = np.exp(-2.0 * t_np)
max_abs_error = np.max(np.abs(exact_trajectory - y_np))

print(f"{'Time (t)':<10}{'RK4 y(t)':<15}{'Exact y(t)':<15}{'Abs Error':<15}")
print("-" * 55)
for i in range(len(t_np)):
    err = abs(exact_trajectory[i] - y_np[i])
    print(f"{t_np[i]:<10.2f}{y_np[i]:<15.8f}{exact_trajectory[i]:<15.8f}{err:<15.2e}")

print(f"\nMaximum Trajectory Error across [0, 1]: {max_abs_error:.4e}")
```

Exercise 6.2.10. Solve the following Exercises

1. **Direct Solution of 2x2 Linear System:** Solve the following 2×2 system directly using the closed-form determinant formula:
$$\begin{cases} 3x_1 - 2x_2 = 7 \\ 5x_1 + 4x_2 = 8 \end{cases}$$
Verify your solution by calculating the determinant D and evaluating equations for x_1 and x_2.

2. **Singularity Analysis:** Determine all values of $k \in \mathbb{R}$ for which the following 2×2 system is singular (has no unique solution):
$$\begin{cases} kx_1 + 4x_2 = 12 \\ 2x_1 + kx_2 = 6 \end{cases}$$

3. **Ill-Conditioning and Sensitivity:** Consider the linear system:
$$\begin{cases} x_1 + x_2 = 3 \\ x_1 + 1.0001x_2 = 3.0001 \end{cases}$$
Calculate its determinant D. Find the exact solution $\mathbf{x} = [x_1, x_2]^T$. Then, perturb the second equation's right-hand side to 3.0002 and re-solve. Explain why small perturbations cause large variations in the solution.

4. **Strict Diagonal Dominance Condition:** Test whether the matrix A is strictly diagonally dominant:
$$A = \begin{bmatrix} 6 & 2 & -1 \\ 1 & 8 & 2 \\ 3 & -1 & 5 \end{bmatrix}$$

Downloaded from – https://aijaz.vercel.app

Does the Jacobi method guarantee convergence for any initial guess $\mathbf{x}^{(0)}$ on this system?

5. **Single Jacobi Step (2x2 System):** Perform a single iteration of the Jacobi method for the system:
$$\begin{cases} 5x_1 - 2x_2 = 9 \\ x_1 + 4x_2 = 6 \end{cases}$$
starting from the zero initial guess $\mathbf{x}^{(0)} = [0, 0]^T$.

6. **Single Jacobi Step with Non-Zero Initial Guess:** Given the 3×3 system:
$$\begin{cases} 8x_1 - x_2 + x_3 = 8 \\ 2x_1 + 10x_2 - x_3 = 11 \\ x_1 - x_2 + 6x_3 = 12 \end{cases}$$
and initial guess $\mathbf{x}^{(0)} = [1.0, 1.0, 1.0]^T$, execute one step of the Jacobi method to compute $\mathbf{x}^{(1)} = [x_1^{(1)}, x_2^{(1)}, x_3^{(1)}]^T$.

7. **Multi-Step Forward Euler Integration:** Perform two manual steps of Forward Euler's method with step size $h = 0.1$ to approximate $y(0.2)$ for the initial value problem:
$$\frac{dy}{dt} = y - t^2, \qquad y(0) = 1.0$$

8. **Effect of Step Size in Euler's Method:** Consider the IVP $\frac{dy}{dt} = -3y$ with $y(0) = 1.0$.
   (i) Compute $y(0.2)$ using a single Euler step with $h = 0.2$.
   (ii) Compute $y(0.2)$ using two Euler steps with $h = 0.1$.
   (iii) Compare both numerical approximations to the exact analytical solution $y(0.2) = e^{-0.6} \approx 0.548812$ by calculating their respective absolute errors.

9. **Heun's RK2 Single Step:** Compute a single step of Heun's second-order Runge–Kutta method (RK2) with $h = 0.2$ to solve:
$$\frac{dy}{dt} = t\sqrt{y}, \qquad y(0) = 4.0$$
Find the numerical value $y_1 \approx y(0.2)$.

10. **Classical RK4 Stage Calculation:** For the initial value problem:
$$\frac{dy}{dt} = y - t, \qquad y(0) = 2.0$$
calculate all four classical RK4 stage values (k_1, k_2, k_3, k_4) for $h = 0.1$, and determine $y_1 \approx y(0.1)$.

11. **RK2 vs. RK4 Truncation Error Analysis:** Explain why the classical RK4 method requires four stage evaluations per step to achieve a local truncation error of $O(h^5)$, whereas Heun's RK2 method requires two evaluations per step for a local error of $O(h^3)$.

12. **Python Direct Solver for 2x2 Systems:** Write a Python function `solve_system_2x2(A, b)` that accepts a 2×2 nested list `A` and a 2-element list `b`. The function must:

Downloaded from – https://aijaz.vercel.app

(i) Compute the determinant $D = a_{11}a_{22} - a_{12}a_{21}$.

(ii) Raise a `ValueError` if $|D| < 10^{-10}$.

(iii) Return the direct solution vector $[x_1, x_2]$ using the analytical determinant formula.

13. **Python Pre-allocation for Iterative Trajectories:** Write a Python function `jacobi_trajectory_storage(A, b, x0, num_steps)` that pre-allocates a 2D NumPy array of shape `(num_steps + 1, n)` to store state vectors $\mathbf{x}^{(k)}$ across $k = 0, 1, \ldots,$ num_steps using the Jacobi iterative scheme.

Downloaded from – **https://aijaz.vercel.app**

Bibliography

[1] B. Bradie, *A Friendly Introduction to Numerical Analysis*, Pearson Education, India, 2007.

[2] S. S. Sastry, *Introductory Methods of Numerical Analysis*, PHI Learning Pvt. Ltd., New Delhi, 2012.

[3] R. L. Burden and J. D. Faires, *Numerical Analysis*, Brooks/Cole, Cengage Learning, 2011.

[4] K. E. Atkinson and W. Han, *Elementary Numerical Analysis*, Wiley India Pvt. Ltd., New Delhi, 2009.

[5] Q. Kong, T. Siauw, and A. Bayen, *Python Programming and Numerical Methods: A Guide for Engineers and Scientists*, Elsevier, 2020.

[6] Y. Liu, *First Semester in Numerical Analysis with Python*, Auraria Institutional Repository, 2020.

Index

A

Absolute Error, 3, 132
Absolute Stability (Euler's method), 110
Approximation, 10
Arithmetic Operators (Python), 128
Asymptotic Error Constant, 12
Averaging Operator (μ), 44

B

Backward Difference Operator (∇), 43
Backward Difference Table, 55
Babylonian Iteration, 130
Basel Problem, 131
Basic Quadrature Rules, 64
Bisection Method, 18
    Algorithmic Procedure, 19
    Convergence & Error Bound, 19
    Iteration Bound Formula, 20
    Manual Iterations, 20–23, 135–136
    Python Implementation, 134–135
Boundary Value Problems (BVPs), 118
    Finite Difference Method, 118–120
    Tridiagonal System Formulation, 119
BTCS (Backward Time, Centered Space) Scheme, 122

C

Central Difference Operator (δ), 43
Characteristic Equation / Polynomial, 98
Clamped Cubic Spline, 61, 66–68
Complex Numbers (Python `complex`), 128
Composite Quadrature Rules:
    Composite Midpoint Rule, 65
    Composite Simpson's 1/3 Rule, 69, 141
    Composite Simpson's 3/8 Rule, 70
    Composite Trapezoidal Rule, 69, 139
Condition Number / Ill-Conditioning, 76, 146
Contraction Mapping Theorem, 24
Control Structures (Python), 129–130
Convergence:
    Cubic Convergence, 12
    Definition of Sequence Convergence, 10
    Linear Convergence, 12
    Order and Rate of Convergence, 12
    Quadratic Convergence, 12
    Superlinear Convergence, 12
Crank–Nicolson Scheme, 122
Cubic Spline Interpolation, 59
    Mathematical Definition, 59
    Tridiagonal System Derivation, 60

D

Data Storage Techniques (Python), 155–156
    Dynamic Appending (`list.append`), 155
    Lists vs. NumPy Arrays, 155
    Static Pre-Allocation (`np.zeros`), 156
Degree of Exactness / Precision, 65, 70
Determinant, 33, 75, 98, 145
Difference Tables, 45, 55
Differential Operator (D), 44
    Relation to Shift Operator ($E = e^{hD}$), 45, 50
Direct Methods for Linear Systems, 75, 145
    Solving 2×2 Systems via Direct Formula, 145–147
Dirichlet Boundary Conditions, 118, 121
Division-Free Reciprocal Formula, 31
Dominant Eigenvalue, 100
Dynamic Typing (Python), 127

E

Eigenvalues and Eigenvectors, 98
    Determinant Relation, 98
    Gerschgorin Circle Theorem, 98–99
    Inverse Eigenvalues, 98
    Properties, 98
    Shifted Eigenvalues, 98
    Spectrum and Spectral Radius, 98
    Trace Relation, 98
Elementary Row Operations (EROs), 75
Error Analysis, 3–10
    Absolute Error, 3, 132
    Inherent (Input) Errors, 6
    Percentage Error, 3, 132
    Propagation of Errors, 4–6

Downloaded from – https://aijaz.vercel.app

Relative Error, 3, 132
Round-off Errors, 6, 129
Sources of Errors, 6
Truncation Errors, 6, 132
Euler's Method (ODEs), 109, 151
Geometric Derivation, 109
Global Truncation Error (GTE), 110, 151
Local Truncation Error (LTE), 110, 151
Python Implementation, 151
Taylor Series Derivation, 110, 151
Explicit FTCS Scheme, 122

F

Finite Difference Operators, 43–45
Averaging Operator (μ), 44
Backward Difference Operator (∇), 43
Central Difference Operator (δ), 43
Differential Operator (D), 44
Forward Difference Operator (Δ), 43
Operator Identities & Relations, 44, 46
Shift Operator (E), 44
Fixed-Point Iteration Method, 24
Contraction Mapping Theorem, 24
Order of Convergence, 25
Sufficient Condition for Linear Convergence, 15
Floating-Point Numbers (Python `float`), 128
Floating-Point Representation (IEEE 754), 6, 128, 129
Format Specifiers / f-strings (Python), 127
Forward Difference Operator (Δ), 43
Forward Difference Table, 45
FTCS (Forward Time, Centered Space) Scheme, 122
Fundamental Theorem of Finite Differences, 45

G

Gauss Elimination Method, 75
Back Substitution Phase, 76
Computational Complexity, 76
Forward Elimination Phase, 76
Need for Pivoting, 76
Gauss–Jordan Elimination Method, 79
Matrix Inversion, 80, 85
Gauss–Seidel Iterative Method, 89
Matrix Formulation, 89
Stopping Criteria, 85
Gerschgorin Circle Theorem, 97–99
Global Truncation Error (GTE), 110, 151

H

Heat Equation (1D Transient), 121
Crank–Nicolson Scheme, 122
Implicit BTCS Scheme, 122
Explicit FTCS Scheme, 122
Heun's Method (RK2 / Predictor–Corrector), 112, 115, 153
Horner's Scheme (Polynomial Evaluation), 128

I

IEEE 754 Floating-Point Standard, 6, 128, 129
Ill-Conditioned Systems, 76, 146
Implicit BTCS Scheme, 122
Indentation and Block Structure (Python), 129
Inherent Errors, 6
Initial Value Problem (IVP), 109, 151
Integers (Python `int`), 128
Intermediate Value Theorem (IVT), 18, 134
Interpolation:
Cubic Spline Interpolation, 59
Inverse Lagrange Interpolation, 36
Lagrange Interpolation, 33, 137
Newton's Backward Difference Interpolation, 54
Newton's Divided Difference Interpolation, 37
Newton's Forward Difference Interpolation, 50, 137
Inverse Lagrange Interpolation, 36
Inverse Power Method, 103
Shifted Inverse Power Method, 103
Iterative Methods for Linear Systems, 83, 145, 148
Gauss–Seidel Method, 89
Jacobi Method, 83, 148
Stein–Rosenberg Theorem, 94
Stopping Criteria, 85

J

Jacobi Iterative Method, 83
Component-Wise Formulation, 83, 148
Matrix Formulation, 84
Single Step Implementation, 148
Spectral Radius & Convergence, 84
Strict Diagonal Dominance (SDD), 84, 148

K

Knots (Cubic Splines), 59
Kronecker Delta Property, 33

L

Lagrange Basis Polynomials, 33
Kronecker Delta Property, 33

Downloaded from – https://aijaz.vercel.app

Partition of Unity, 33
Lagrange Interpolation, 33, 137
Error Formula, 34
Inverse Interpolation, 36
Python Implementation, 137
Left Rectangle Rule, 64
Linear Convergence, 12
Linear Systems:
2×2 Direct Formula, 145
Direct Methods, 75
Gauss Elimination, 75
Gauss–Jordan Method, 79
Iterative Solvers, 83, 148
Lipschitz Condition, 109
Lists vs. NumPy Arrays (Python), 155
Local Truncation Error (LTE), 110, 151, 153

M

Maclaurin Series Expansion, 6, 133
Matrix Inversion (Gauss–Jordan), 80
Method of Undetermined Coefficients, 65
Midpoint Method (RK2), 115
Midpoint Rule (Numerical Quadrature), 64
Error Bound, 65
Modified Euler's Method, 112

N

Natural Cubic Spline, 61, 65
Newton–Cotes Quadrature Formulas, 68
Newton's Backward Difference Interpolation, 54
Error Formula, 55
Newton's Divided Difference Interpolation, 37
Divided Differences, 37
Incremental Property, 40
Table Structure, 38
Newton's Forward Difference Interpolation, 50, 137
Error Formula, 51
Python Implementation, 138
Newton–Raphson Method, 27
Analytic Derivation, 28
Division-Free Reciprocal Formula, 31
Geometric Derivation, 28
Limitations and Pitfalls, 29
Multiple Roots Degeneracy, 29
Quadratic Convergence Proof, 28
NumPy Arrays (`numpy.ndarray`), 155
Static Pre-allocation, 156
Vectorized Operations, 155, 156

O

Order of Accuracy (Big-$\mathcal{O}$ Notation), 15
Order of Convergence, 12
Ordinary Differential Equations (ODEs), 109, 151
Euler's Method, 109, 151
Modified Euler's Method, 112
Runge–Kutta Methods, 115, 116, 153

P

Partial Differential Equations (PDEs), 121
Classification (Elliptic, Hyperbolic, Parabolic), 121
Heat Equation Schemes, 121–123
Partial Pivoting, 76
Partition of Unity, 33
Percentage Error, 3, 132
Periodic Cubic Spline, 61
Pivoting Strategies, 76
Polynomial Interpolation, 32
Power Method, 100
Dominant Eigenvalue, 100
Rayleigh Quotient Refinement, 101
`print()` Function (Python), 127
Propagation of Errors, 4
General Error Propagation Formula, 4

Q

Quadratic Convergence, 12, 28
Quadrature (Numerical Integration), 64
Degree of Exactness, 65
Weights and Nodes, 64

R

Ralston's Method (RK2), 115
`range()` Sequence Generator (Python), 130
Rate of Convergence, 12
Rayleigh Quotient, 101
Relative Error, 3, 132
Right Rectangle Rule, 64
Round-off Errors, 6, 129
Runge's Phenomenon, 59, 63
Runge–Kutta Methods, 115, 116, 153
Classical RK4 Scheme, 116, 153
Heun's RK2 Scheme, 115, 153
Midpoint RK2 Scheme, 115
Ralston's RK2 Scheme, 115
Single Step Python Functions, 154

S

Shift Operator (E), 44
Shifted Inverse Power Method, 103

Downloaded from – https://aijaz.vercel.app

Significant Digits, 4
    Relative Error Bounds, 4
Simpson's 1/3 – Rule, 69, 141
    Composite Simpson's 1/3 Rule, 69, 141
    Error Bound, 69, 141
    Python Implementation, 142
Simpson's 3/8 – Rule, 70
    Composite Simpson's 3/8 Rule, 70
    Error Bound, 70
Sources of Errors, 6
Spectral Radius, 84, 94, 98
Spline Interpolation, 59
    Cubic Splines, 59
    Tridiagonal Linear System, 60
Stein–Rosenberg Theorem, 94
Stopping Criteria (Iterative Solvers), 85
Strict Diagonal Dominance (SDD), 84, 148
Superlinear Convergence, 12
Symmetric Positive Definite Matrices, 94

T

Taylor's Remainder Theorem, 6
Thomas Algorithm, 119
Trapezoidal Rule, 69, 139
    Composite Trapezoidal Rule, 69, 139
    Error Bound, 69, 139
    Python Implementation, 139
Tridiagonal Linear Systems, 60, 119
Truncation Error, 6, 110, 132, 139, 141, 151, 153

U

Undetermined Coefficients, Method of, 65

V

Vandermonde Matrix, 33
Variables and Data Types (Python), 127–128
Vectorized Operations (NumPy), 155, 156
von Neumann Stability Analysis, 122

W

`while` Loop (Python), 130

A Concluding Note to Students

"Mathematics teaches us to seek exact answers, while numerical analysis teaches us how to approach them when exact solutions remain beyond our reach. Its true strength lies not merely in approximation, but in understanding how close we are to the truth, how errors arise, and how successive computations bring us nearer to a reliable solution."

DEAR STUDENTS,

Congratulations on completing Numerical Analysis and Computational Methods!

As we reach the end of this mathematical journey, take a moment to appreciate the remarkable ideas and techniques we have explored together. We began by understanding approximation, errors, and convergence, learning that the reliability of a numerical solution is just as important as the solution itself. We then developed systematic methods for solving nonlinear equations, constructing interpolation polynomials, and approximating definite integrals.

Our journey continued through systems of linear equations, iterative techniques, eigenvalues, and eigenvectors, revealing how seemingly complicated problems can be approached through carefully designed algorithms. Finally, we explored numerical methods for ordinary and partial differential equations, discovering how mathematical models of physical processes can be transformed into practical computational solutions.

Through the computational exercises, we also learned that mathematical reasoning and programming complement one another. An algorithm is not simply a sequence of instructions; it is a mathematical idea translated into a form that can be implemented, tested, and refined.

What we have gained from this course is far more than the ability to perform numerical calculations. We have developed a **computational mindset**—the ability to approximate intelligently, analyze errors critically, understand convergence, and recognize the limitations of numerical procedures.

Numerical analysis provides essential tools for scientific computing, engineering, mathematical modelling, artificial intelligence, data science, and the simulation of complex physical systems. As you advance in mathematics, remember that a meaningful numerical answer must be supported by sound reasoning, careful computation, and an understanding of its accuracy.

Keep your curiosity alive! Do not be discouraged when an exact solution seems impossible. Mathematics often advances through successive approximations, and every careful step brings us closer to understanding.

"Precision is important, but understanding the error is what makes an approximation trustworthy."

DR. AIJAZ
Assistant Professor of Mathematics
JK Higher Education Department
`https://www.aijaz.vercel.app`